

Crypto Guide

Cryptographic primitives from scratch

m@rek.onl

Abstract

This volume of *The Zcash Arboretum* assembles the cryptographic toolbox: the primitives from which a shielded protocol is built, each defined, constructed, and proved against a named enemy. It assumes the *Math Guide*, whose groups, finite fields, elliptic curves, probability, and model of computation are used throughout without re-derivation.

Every guarantee here is a claim about an adversary, and every security definition is a game: a challenger, an efficient adversary, and an advantage that must be negligible. A proof is a reduction that turns any winning adversary into a solver for a long-attacked computational problem, with the reduction's loss accounted for down to a concrete bit-security budget. Under that discipline the volume denies, in turn: the cryptanalyst who would recover the exponent behind a public group element, and with it every key; the forger by coincidence, who would collide two documents into one digest or replay a digest outside its role; the peeker and the equivocator on either side of a sealed envelope, who would read a commitment early or open it to a different value; the oracle distinguisher, who would tell a short secret key from blind chance; the owner of the channel, who would read, splice, or counterfeit what passes over it; the woman in the middle, who would terminate both ends of a “secure” conversation at her own desk; the forger of signatures, who would exhibit an authorisation the signer never made; the cheating prover and the curious verifier, who would certify a falsehood or squeeze a transcript for the witness it hides; and the teller of lies about sets, who would prove membership in a collection that never recorded it. What survives their attentions— hash functions, commitments, pseudorandom functions, authenticated encryption, key agreement, signatures, zero-knowledge arguments, and Merkle trees—is the toolbox the deployed protocol spends.

Contents

1	The provable-security model	4
1.1	Adversaries and the security parameter	4
1.2	Distribution ensembles and indistinguishability	6
1.3	The hybrid argument	8
1.4	Security as a game	9
1.5	Reductions	10
1.6	The standard model and the random oracle model	13
1.7	Concrete security and the bit-security budget	14
2	Hardness assumptions	16
2.1	The discrete logarithm problem	16
2.2	Diffie–Hellman: computational and decisional	17
2.3	The assumption hierarchy	18
2.4	Generic algorithms and the square-root barrier	19
2.5	Pohlig–Hellman and the necessity of large prime order	21
2.6	Instantiation on elliptic curves; the Pasta curves	23
2.7	The quantum caveat: Shor’s algorithm	25
2.8	Standing assumptions	26
3	Hash functions and the random oracle model	26
3.1	Syntax	27
3.2	Security notions: preimage, second-preimage, and collision resistance	28
3.3	The birthday bound on generic collision finding	30
3.4	The random oracle model	31
3.5	Domain separation and personalisation	32
3.6	Hashing to a field element	33
3.7	Hashing to a curve point	35
3.8	Arithmetisation-friendly hashing: Poseidon	37
3.9	The transparent layer’s hashes	38
4	Commitment schemes	39
4.1	The sealed envelope	40
4.2	Syntax	40
4.3	Hiding and binding	41
4.4	Incompatibility of perfect hiding and perfect binding	42
4.5	Hash-based commitments	43
4.6	The Pedersen commitment	44
4.7	Pedersen vector commitments	47
4.8	Sinsemilla: an algebraic hash-based commitment	48
4.9	Polynomial commitment schemes	51
4.10	Secret sharing	55
5	Pseudorandom functions, block ciphers, and format-preserving encryption	57
5.1	Pseudorandom functions and permutations	57
5.2	Block ciphers and AES	60
5.3	PRFs from hash functions: length extension and keyed BLAKE2	61
5.4	Format-preserving encryption and FF1	62

6	Symmetric encryption, AEAD, and key derivation	66
6.1	Stream ciphers and ChaCha20	66
6.2	Chosen-plaintext security	67
6.3	Message authentication codes	68
6.4	Authenticated encryption with associated data	71
6.5	The ChaCha20-Poly1305 AEAD	72
6.6	Key-derivation functions	74
7	Key agreement	76
7.1	The Diffie–Hellman protocol	77
7.2	Static and ephemeral keys	78
7.3	The key-agreement scheme abstraction	78
7.4	Passive security and the Diffie–Hellman assumptions	79
7.5	Active attacks and the role of authentication	81
7.6	Elliptic-curve Diffie–Hellman	82
7.7	Hybrid public-key encryption	83
7.8	Key privacy	86
7.9	Shielded-note delivery as deployed hybrid encryption	88
8	Digital signatures	89
8.1	Signatures versus MACs	89
8.2	Syntax and security goal	90
8.3	The hash-and-sign paradigm	91
8.4	The Schnorr identification protocol	91
8.5	From identification to signature via the Fiat–Shamir transform	93
8.6	Security in the random oracle model and the forking lemma	94
8.7	RedDSA: re-randomisable Schnorr for Orchard	97
8.8	Key re-randomisation and unlinkability	98
8.9	Binding signatures as a proof of knowledge of a discrete logarithm	99
8.10	The transparent layer’s schemes: ECDSA and BIP-340 over secp256k1	102
9	Interactive proofs, zero knowledge, and SNARKs	104
9.1	Three questions	104
9.2	Languages, relations, and witnesses	104
9.3	Interactive protocols, proofs, and arguments	105
9.4	Knowledge soundness and extractors	107
9.5	The simulation paradigm and zero knowledge	109
9.6	Sigma-protocols	110
9.7	The Fiat–Shamir transform: from interactive to non-interactive	113
9.8	SNARKs: succinct non-interactive arguments of knowledge	115
10	Merkle trees and commitments to sets	120
10.1	Two statements about a set	121
10.2	The compression function and collision resistance	121
10.3	Binary Merkle trees	122
10.4	Authentication paths and membership proofs	123
10.5	Soundness: forging a path implies a collision	125
10.6	Layer and domain separation	126
10.7	Append-only and incrementally updatable trees	128
10.8	Vector commitments	130
10.9	Privacy-preserving membership via zero-knowledge paths	131
10.10	A shielded spend, end to end	132

1 The provable-security model

Every claim this volume makes ends, once unwound, in a sentence about an enemy. No forger produces a valid signature on a message the signer never signed; no observer tells an encryption of m_0 from an encryption of m_1 ; no prover opens one commitment to two different values. Before any such sentence can be a theorem, the enemy must be a mathematical object: something with a precisely delimited repertoire, about which a universally quantified statement can be proved. The preceding volume supplied the algebraic raw material—groups in which discrete logarithms appear hard, finite fields, elliptic curves—but an object is not yet a guarantee. This section builds the adversary, and around it the framework in which “the scheme is secure” becomes a claim with mathematical content.

The central methodological idea is the following. One almost never knows how to prove unconditionally that a cryptographic scheme is secure; such a proof would typically imply $P \neq NP$ —that problems whose solutions can be checked in polynomial time are not all solvable in polynomial time—and a great deal more. Cryptography therefore adopts a *relativised* standard of proof. One isolates a small number of computational problems—factoring, discrete logarithm, decisional Diffie–Hellman, and the like—that have resisted decades of attack, and *reduces* the security of the schemes to the conjectured hardness of those problems. A proof of security is then a proof of an implication: *if* the underlying problem is hard, *then* the scheme is secure. Equivalently, in its contrapositive and more operationally useful form: *any* adversary that breaks the scheme can be mechanically transformed into an algorithm that solves the underlying hard problem. Since no such algorithm is believed to exist, one concludes that no such adversary exists.

To render any of this rigorous, we must pin down four things: what counts as a “scheme”, parametrised by a notion of input size; what counts as an “adversary”; what it means for an adversary to “break” a scheme, measured by a quantity called its *advantage*; and what it means for one problem to reduce to another. We develop these in turn, followed by the two idealisations under which proofs proceed (the standard model and the random oracle model), and finally the translation between the clean asymptotic language and the concrete “ n -bit security” numbers that practitioners actually quote.

1.1 Adversaries and the security parameter

The vocabulary of efficiency and smallness is the *Math Guide*’s, fixed in its closing section; we recall it here in the form this volume uses and extend it with the refinements that are specifically cryptographic.

The *security parameter* $\lambda \in \mathbb{N}$ is a positive integer that controls the sizes of all objects in a cryptographic scheme—key lengths, group orders, output lengths—and is conventionally supplied to every algorithm in unary, written 1^λ , the string of λ ones, so that an algorithm running in time polynomial in the length of its input automatically receives time polynomial in λ (*Math Guide*, §“The security parameter”). Every scheme in this monograph is accordingly not a single object but an infinite *family* of objects indexed by λ .

Two questions immediately arise: why a family, and why unary.

The reason for a *family* is that security is inherently asymptotic. There is no such thing as an absolutely unbreakable scheme with fixed finite key length: an adversary with enough time can always enumerate a finite key space. What we can hope for is that as λ grows, the resources required to break the scheme grow far faster than the resources required to use it. The honest parties’ costs (key generation, encryption, signing) should grow *slowly* in λ —polynomially—while the best attack’s cost should grow *quickly*—super-polynomially, ideally exponentially. The security parameter is the dial that separates these two growth rates, and security is a statement about the limit $\lambda \rightarrow \infty$.

The reason for *unary* encoding, 1^λ rather than the $\lceil \log_2 \lambda \rceil$ -bit binary representation of λ , is a book-keeping convenience that aligns two notions of “polynomial time”. Complexity theory measures running time as a function of input *length*. If λ came in binary it would have length $\Theta(\log \lambda)$, and an al-

gorithm “polynomial in its input length” would get only $\text{poly}(\log \lambda)$ steps—not even enough to write down a λ -bit key. Padding the parameter to length λ ensures that “polynomial in the input length” coincides with the cryptographically meaningful “polynomial in λ ”. This is purely a normalisation; nothing of substance depends on it.

Remark 1.1. Because everything is indexed by λ , every quantity we discuss is really a *function* of λ : the adversary’s success probability, its running time, the key length. A single number—“the advantage is at most 2^{-128} ”—either fixes a concrete λ or describes the function’s behaviour for the λ of interest. Keeping the dependence on λ explicit, at least mentally, prevents nearly every confusion in this subject.

Having fixed how scheme sizes scale, we fix what an attacker may do. An algorithm $\mathcal{A}$ is *probabilistic polynomial-time* (PPT) if it is a probabilistic Turing machine—one equipped with a read-only tape of independent uniformly random bits, its *random tape*—and there is a polynomial p such that on every input x , *and for every setting of the random tape*, $\mathcal{A}$ halts within $p(|x|)$ steps: the strict, worst-case notion, exactly as fixed in the *Math Guide* (§“Algorithms, running time, and PPT”). Write $y \leftarrow \mathcal{A}(x)$ for the random variable obtained by running $\mathcal{A}$ on x with freshly sampled randomness, and $y := \mathcal{A}(x; r)$ for the deterministic output when the random tape is fixed to the string r . The honest algorithms and the adversary alike are modelled as PPT machines.

Two of the modelling choices folded into that sentence warrant comment.

Why polynomial time? Polynomial time is the standard mathematical abstraction of “feasible computation”. It is robust—closed under composition, and invariant across all reasonable deterministic computational models (the strong Church–Turing thesis)—which makes it a clean class to quantify over. Crucially, we require the *honest* parties to run in polynomial time too; a scheme is useful only if it can be used efficiently. The security claim is then an asymmetry between a polynomial-time honest world and a quantified statement that *no* polynomial-time attack succeeds.

Why probabilistic? Randomness is not a luxury but a necessity in cryptography. Deterministic encryption leaks whether two ciphertexts encrypt the same message; key generation must sample secrets unpredictably. We therefore grant the adversary randomness too, both for fairness and because a deterministic adversary is the special case where the random tape is ignored.

The adversary class itself admits a strengthening that the *Math Guide* mentions (§“Algorithms, running time, and PPT”) and this volume adopts as standard.

Definition 1.2 (Non-uniform PPT). A *non-uniform* PPT adversary is a family $\{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$ of probabilistic circuits (or, equivalently, Turing machines each supplied with an *advice string* z_λ of length $\text{poly}(\lambda)$) such that a single polynomial in λ bounds the size of $\mathcal{A}_\lambda$. The advice may depend arbitrarily on λ but not on the particular inputs drawn at run time.

Remark 1.3. The distinction between uniform adversaries (a single machine that reads 1^λ and works for all λ) and non-uniform ones (a separate circuit per λ , or a uniform machine with per- λ advice) is a genuine one. Non-uniformity grants the adversary free precomputed advice tailored to each parameter size—for example, a short hint about the group of the day. Security against non-uniform adversaries is the stronger and now-standard requirement, partly because non-uniform models compose more cleanly under reduction (one may “hard-wire” a fixed good choice of randomness or auxiliary input). All definitions below can be read in either flavour; we state hardness assumptions against non-uniform adversaries when it matters and otherwise leave the choice implicit.

Remark 1.4 (Expected versus strict polynomial time, and quantum adversaries). Two refinements recur. First, some definitions allow *expected* polynomial-time adversaries, bounding the average rather than worst-case running time; this matters for certain extraction arguments in zero-knowledge, and we flag it where it arises. Second, for post-quantum security one replaces PPT by *quantum* polynomial-time (QPT)—polynomial-size circuits of quantum gates followed by a measurement, a model this volume does not formalise; the consequences for the assumptions this volume relies on are taken up under “The quantum caveat: Shor’s algorithm” in the next section. The structural definitions of

this section (advantage, indistinguishability, reduction) are agnostic to which class of adversaries one quantifies over; only the underlying hardness assumptions change.

1.2 Distribution ensembles and indistinguishability

The most basic thing an adversary can attempt is to *tell two situations apart*—a real protocol transcript from a simulated one, an encryption of m_0 from an encryption of m_1 , a pseudorandom string from a truly random one. The most basic security goal is that it cannot. To formalise “look the same” we first formalise the two situations as *ensembles* of probability distributions, then define three grades of sameness.

Definition 1.5 (Probability ensemble). A *probability ensemble* is a sequence $X = \{X_\lambda\}_{\lambda \in \mathbb{N}}$ of probability distributions (equivalently, random variables), one for each value of the security parameter, where X_λ is supported on binary strings whose length is bounded by a polynomial in λ . There is often an additional index—an input a —written $\{X_\lambda(a)\}$; for clarity we suppress it unless needed.

The natural metric on a single pair of distributions is the statistical distance of the *Math Guide* (§“Statistical distance”), which we recall in the slightly wider generality this volume needs: supports there are finite, while ensembles as just defined live on countable sets of strings.

Definition 1.6 (Statistical distance). Let X, Y be discrete random variables over a common finite or countable set S . Their *statistical distance* (or total variation distance) is

$$\Delta(X, Y) := \frac{1}{2} \sum_{s \in S} |\Pr[X = s] - \Pr[Y = s]|.$$

Proposition 1.7 (Variational characterisation). For X, Y over S ,

$$\Delta(X, Y) = \max_{T \subseteq S} (\Pr[X \in T] - \Pr[Y \in T]) = \max_D (\Pr[D(X) = 1] - \Pr[D(Y) = 1]),$$

where the last maximum ranges over all (even computationally unbounded, deterministic) decision functions $D : S \rightarrow \{0, 1\}$. In particular, no procedure whatsoever—efficient or not—can distinguish X from Y with advantage exceeding $\Delta(X, Y)$.

Proof. The set form is the variational characterisation in the statistical-distance theorem of the *Math Guide* (§“Statistical distance”), whose argument is unchanged over a countable S . Only the reformulation over decision functions is new: a deterministic D is the indicator of the set $T = D^{-1}(1)$, so the maximum over D equals the maximum over T ; a randomised D is a convex combination of deterministic ones and cannot do better. $\square$

Proposition 1.8 (Properties of statistical distance). The *statistical distance* is a metric on probability distributions: $0 \leq \Delta(X, Y) \leq 1$, it is symmetric, $\Delta(X, Y) = 0$ iff X and Y are identically distributed, and it satisfies the triangle inequality $\Delta(X, Z) \leq \Delta(X, Y) + \Delta(Y, Z)$. Moreover it never increases under (possibly randomised) post-processing: for any function—indeed any randomised algorithm— f ,

$$\Delta(f(X), f(Y)) \leq \Delta(X, Y).$$

Proof. These are the metric and data-processing parts of the statistical-distance theorem of the *Math Guide* (§“Statistical distance”), again with the argument unchanged over a countable S . $\square$

The weakest of the three grades below needs the asymptotic vocabulary of the *Math Guide* (§“Polynomial, exponential, and negligible functions”), which we recall in one breath. A function $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible*, written $f = \text{negl}(\lambda)$, if for every $c \in \mathbb{N}$ there exists λ_0 such that $f(\lambda) < \lambda^{-c}$ for all $\lambda \geq \lambda_0$ —it decays faster than the reciprocal of every polynomial. A function is *non-negligible* if it is not negligible; it is *noticeable* if there exists c with $f(\lambda) \geq \lambda^{-c}$ for all sufficiently large λ (noticeable functions are non-negligible, but not conversely); it is *overwhelming* if $1 - f$ is negligible. The class of negligible functions is closed under addition and under multiplication by any polynomial—the closure property, proved there, on which the hybrid argument of the next subsection depends.

Definition 1.9 (Perfect, statistical, and computational indistinguishability). Let $X = \{X_\lambda\}$ and $Y = \{Y_\lambda\}$ be probability ensembles.

- The ensembles X and Y are *perfectly indistinguishable*, written $X \equiv Y$, if X_λ and Y_λ are *identically distributed* for every λ ; equivalently $\Delta(X_\lambda, Y_\lambda) = 0$ for all λ .
- The ensembles X and Y are *statistically indistinguishable*, written $X \approx_s Y$, if their statistical distance is negligible:

$$\Delta(X_\lambda, Y_\lambda) = \text{negl}(\lambda).$$

- The ensembles X and Y are *computationally indistinguishable*, written $X \approx_c Y$, if for every PPT algorithm D (the *distinguisher*), the function

$$\text{Adv}_{X,Y}^{\text{dist}}(D, \lambda) := \left| \Pr[D(1^\lambda, X_\lambda) = 1] - \Pr[D(1^\lambda, Y_\lambda) = 1] \right|$$

is negligible in λ . The probabilities are over the draw from the ensemble and over D ’s internal randomness.

These three notions sit in a strict hierarchy. The implications are immediate from the definitions and the variational characterisation; standard examples witness the strictness. The second example needs two primitives, which we define here in the form the volume uses throughout; write U_n for the uniform distribution on $\{0, 1\}^n$. A *one-way function* is a polynomial-time computable $h : \{0, 1\}^* \rightarrow \{0, 1\}^*$ that no PPT algorithm inverts on a random input: for every PPT $\mathcal{A}$,

$$\Pr[h(\mathcal{A}(1^\lambda, h(x))) = h(x)] = \text{negl}(\lambda) \quad (x \leftarrow U_\lambda).$$

A *pseudorandom generator* (PRG) is a polynomial-time computable $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ whose output on a uniform seed is computationally indistinguishable from uniform, $\{G(U_\lambda)\} \approx_c \{U_{2\lambda}\}$; the doubling stretch is a normalisation. Whether one-way functions exist is open, which is why the second strictness below is stated conditionally.

Proposition 1.10 (Hierarchy). *For all ensembles X, Y :*

$$X \equiv Y \implies X \approx_s Y \implies X \approx_c Y.$$

The first implication is strict: there exist ensembles that are statistically but not perfectly indistinguishable. The second is strict provided one-way functions exist: under that assumption there exist ensembles that are computationally but not statistically indistinguishable.

Proof. If $X \equiv Y$ then $\Delta(X_\lambda, Y_\lambda) = 0$, which is negligible, so $X \approx_s Y$. If $X \approx_s Y$, then for any—in particular any PPT—distinguisher D , Proposition 1.7 gives $\text{Adv}_{X,Y}^{\text{dist}}(D, \lambda) \leq \Delta(X_\lambda, Y_\lambda) = \text{negl}(\lambda)$, so $X \approx_c Y$.

For strictness of the first implication, let X_λ be uniform on $\{0, 1\}^\lambda$ and let Y_λ be uniform on $\{0, 1\}^\lambda \setminus \{0^\lambda\}$. They are not identically distributed (the point 0^λ has probabilities $2^{-\lambda}$ and 0), so not perfectly indistinguishable, yet $\Delta(X_\lambda, Y_\lambda) = 2^{-\lambda} = \text{negl}(\lambda)$, so they are statistically indistinguishable.

For strictness of the second, assume one-way functions exist; then a PRG exists—the Håstad–Impagliazzo–Levin–Luby theorem, together with the routine lemma that extends any stretch to the doubling one, both used without proof—and furnishes an ensemble: let G be a PRG, let $X_\lambda := G(U_\lambda)$, and let $Y_\lambda := U_{2\lambda}$. By definition of a PRG, $X \approx_c Y$. But X_λ is supported on at most 2^λ strings out of $2^{2\lambda}$, so a set T equal to the image of G has $\Pr[X_\lambda \in T] = 1$ while $\Pr[Y_\lambda \in T] \leq 2^\lambda / 2^{2\lambda} = 2^{-\lambda}$; hence $\Delta(X_\lambda, Y_\lambda) \geq 1 - 2^{-\lambda}$, which is not negligible. Thus $X \approx_c Y$ but $X \not\approx_s Y$. $\square$

Remark 1.11 (The price and the prize of computational indistinguishability). The strictness example is instructive. Statistically the PRG output and a random string are almost as far apart as two distributions can be—distance nearly 1—because the PRG image is a vanishing fraction of the codomain. An unbounded distinguisher simply checks membership in the image and wins. The entire content of computational indistinguishability is that *finding* that image membership test is infeasible. This is the bargain at the heart of cryptography: we trade the impossible (information-theoretic security with short keys) for the merely conjectured-hard (computational security), and in exchange we obtain the ability to encrypt long messages under short keys, build public-key primitives, and so on. Statistical and perfect security, when attainable, are unconditional and need no hardness assumption; computational security buys vastly more functionality at the cost of resting on an assumption.

1.3 The hybrid argument

Indistinguishability would be of little use if every application had to be argued from scratch. Two lemmas make it composable: efficient post-processing cannot create a distinguishing edge, and a polynomially long chain of pairwise-indistinguishable steps cannot accumulate one. The first is a reduction in miniature—the template for every reduction in this volume; the second, the hybrid argument, is the workhorse of the entire proof literature.

Lemma 1.12 (Closure under efficient post-processing). *If $X \approx_c Y$ and M is any PPT algorithm, then the ensembles $\{M(1^\lambda, X_\lambda)\}$ and $\{M(1^\lambda, Y_\lambda)\}$ satisfy $\{M(X_\lambda)\} \approx_c \{M(Y_\lambda)\}$.*

Proof. Suppose some PPT distinguisher D separates $\{M(X_\lambda)\}$ from $\{M(Y_\lambda)\}$ with advantage $\delta(\lambda)$. Define $D' := D \circ M$, i.e. $D'(1^\lambda, z) := D(1^\lambda, M(1^\lambda, z))$. Since M and D are both PPT, so is D' . Then

$$\text{Adv}_{X,Y}^{\text{dist}}(D', \lambda) = |\Pr[D(M(X_\lambda)) = 1] - \Pr[D(M(Y_\lambda)) = 1]| = \delta(\lambda).$$

By $X \approx_c Y$ the left side is negligible, so δ is negligible. $\square$

Many proofs must compare two ensembles that are far apart but connected by a chain of pairwise-close intermediate ensembles. The hybrid argument turns such a chain into a single conclusion. It is essentially the triangle inequality, but with a refinement—the uniformity hypothesis—that explains why the number of hybrids must be polynomial and that careless statements omit.

Lemma 1.13 (Hybrid lemma). *Let $t = t(\lambda)$ be polynomially bounded, and let $H_\lambda^0, H_\lambda^1, \dots, H_\lambda^t$ be a sequence of distributions (the hybrids). Suppose that for every PPT distinguisher D the neighbouring advantages*

$$\alpha_i(D, \lambda) := |\Pr[D(H_\lambda^i) = 1] - \Pr[D(H_\lambda^{i+1}) = 1]|$$

are uniformly negligible: a single negligible function ν (depending on D) satisfies $\alpha_i(D, \lambda) \leq \nu(\lambda)$ for all $i \in \{0, \dots, t(\lambda) - 1\}$ simultaneously. Equivalently, $\max_{0 \leq i < t(\lambda)} \alpha_i(D, \lambda) = \text{negl}(\lambda)$, i.e. $\alpha_{i(\lambda)}(D, \lambda)$ is negligible along every index function $i(\lambda)$ —arbitrary, not merely efficiently computable, since the maximising index need not be efficient. Then $\{H_\lambda^0\}$ and $\{H_\lambda^t\}$ are computationally indistinguishable, i.e. for every PPT D , $|\Pr[D(H_\lambda^0) = 1] - \Pr[D(H_\lambda^t) = 1]| = \text{negl}(\lambda)$.

Proof. Fix a PPT distinguisher D and let ν be the uniform bound the hypothesis supplies for D . By the triangle inequality applied to the telescoping sum,

$$|\Pr[D(H_\lambda^0) = 1] - \Pr[D(H_\lambda^t) = 1]| = \left| \sum_{i=0}^{t-1} (\Pr[D(H_\lambda^i) = 1] - \Pr[D(H_\lambda^{i+1}) = 1]) \right| \leq \sum_{i=0}^{t-1} \alpha_i(D, \lambda),$$

and the hypothesis bounds the sum by $t(\lambda) \cdot \nu(\lambda)$, which is negligible since t is a polynomial (closure of the negligible class under polynomial multiplication). Hence $\{H^0\} \approx_c \{H^t\}$. $\square$

Uniformity in i is essential, and no argument derives it from the weaker hypothesis that each α_i is negligible for every *fixed* i : there the threshold beyond which the i -th term is small can depend on i , and the dependence cannot be removed. A countermodel makes this concrete: take $t(\lambda) = \lambda + 1$ and let H_λ^i be the point mass at 0 if $i \leq \lambda$ and the point mass at 1 otherwise. For every fixed i the advantage $\alpha_i(D, \lambda)$ is nonzero only at $\lambda = i$, hence negligible; yet H_λ^0 and $H_\lambda^{t(\lambda)}$ are the point masses at 0 and 1, perfectly distinguishable.

Applications usually obtain the conclusion by a stronger, structural route. Suppose the chain arises from a *single* underlying assumption $X \approx_c Y$: neighbouring hybrids H_λ^i and H_λ^{i+1} differ only in that one designated component is drawn from X_λ in the former and from Y_λ in the latter, and each hybrid is efficiently samplable given $(1^\lambda, i)$ and a sample of that component. Then the *random-index* distinguisher D' for the pair (X, Y) , on challenge z , samples i uniformly from $\{0, \dots, t-1\}$, builds the hybrid with z in the designated position, and runs D ; writing $p_i := \Pr[D(H_\lambda^i) = 1]$, its distinguishing advantage equals $|\frac{1}{t} \sum_{i=0}^{t-1} (p_i - p_{i+1})| = |p_0 - p_t|/t$. Since D' is a single PPT machine, the assumption makes its advantage negligible, and $|p_0 - p_t| \leq t(\lambda) \cdot \text{Adv}(D') = \text{negl}(\lambda)$. Note that this argument bounds D 's total advantage directly, bypassing the lemma's uniform per-step hypothesis rather than discharging it; the lemma itself covers the general case in which no such common structure is available.

Remark 1.14. The hybrid argument is precisely where the “polynomially many” constraint earns its keep. A negligible per-step advantage multiplied by a polynomial number of steps remains negligible; a negligible per-step advantage multiplied by, say, 2^λ steps need not. This is why constructions that would require exponentially many hybrids do not yield security proofs, and why the closure of negligibility under polynomial factors is not a technicality but the load-bearing wall of the whole edifice.

1.4 Security as a game

We can now state what it means to “break” a scheme. The modern idiom expresses every security definition as a *game* (or experiment) played between a *challenger* and an adversary $\mathcal{A}$. The challenger, following a fixed protocol, sets up the scheme, answers whatever queries the adversary may pose, and at the end declares the adversary a winner or loser by outputting a single bit. Security says that no PPT adversary wins by more than it could by trivial guessing.

Definition 1.15 (Security game and advantage). A *security game* (or *experiment*) G is a PPT interactive algorithm, the challenger, that takes 1^λ , interacts with an adversary $\mathcal{A}$, and finally outputs a bit $G^\mathcal{A}(\lambda) \in \{0, 1\}$, where 1 denotes “adversary wins”. The *advantage* of $\mathcal{A}$ is a measure of how much better than trivial its winning probability is; its precise form depends on the type of game:

- *Distinguishing (indistinguishability) games*, where the challenger secretly chooses a uniform bit $b \in \{0, 1\}$ and $\mathcal{A}$ outputs a guess b' ; the adversary wins iff $b' = b$. Here

$$\text{Adv}_G(\mathcal{A}, \lambda) := \left| \Pr[G^\mathcal{A}(\lambda) = 1] - \frac{1}{2} \right| = \frac{1}{2} |\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]|.$$

Trivial guessing achieves winning probability $1/2$, hence advantage 0.

- *Search (unforgeability/inversion) games*, where the adversary must produce a specific kind of object—a forgery, a preimage, a discrete logarithm—and wins iff it succeeds. Here the baseline winning probability is (essentially) 0, and one simply sets

$$\text{Adv}_G(\mathcal{A}, \lambda) := \Pr[G^{\mathcal{A}}(\lambda) = 1].$$

Definition 1.16 (Security relative to a game). A scheme is *secure* with respect to the game G if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_G(\mathcal{A}, \lambda) = \text{negl}(\lambda).$$

Two remarks on the shape of this definition are in order. First, the quantifier order is essential: *for every* PPT adversary, the advantage is negligible. For each fixed adversary there may be a different negligible bound; the claim is universal over the entire class. Second, the use of negl on the right is what makes the definition asymptotic and clean: it is robust to the polynomial slop that reductions introduce, and it captures “the adversary does no better than the trivial strategy, up to an amount that vanishes faster than any inverse polynomial”.

Example 1.17 (Pseudorandomness, as a game). To anchor the abstraction with machinery already on the table, consider the distinguishing game for a pseudorandom generator $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ —the primitive that witnessed strictness in Proposition 1.10:

1. The challenger samples $b \leftarrow \{0, 1\}$ uniformly. If $b = 0$ it samples a seed $s \leftarrow \{0, 1\}^\lambda$ and sends $z := G(s)$; if $b = 1$ it sends a uniform $z \leftarrow \{0, 1\}^{2\lambda}$.
2. The adversary $\mathcal{A}$ outputs a guess b' ; the game outputs 1 iff $b' = b$.

The generator is secure iff $\text{Adv}(\mathcal{A}, \lambda) = |\Pr[b' = b] - \frac{1}{2}|$ is negligible for every PPT $\mathcal{A}$. Here the game packaging and the ensemble packaging coincide: the two situations are the fixed ensembles $\{G(U_\lambda)\}$ and $\{U_{2\lambda}\}$, and the displayed identity in Definition 1.15 shows the game advantage of $\mathcal{A}$ is exactly half its distinguishing advantage in the sense of Definition 1.9—so the game is secure iff the ensembles are computationally indistinguishable. The game form earns its keep beyond this case: when the adversary chooses part of the challenge or queries oracles adaptively, the interaction no longer collapses to a comparison of two fixed ensembles, and the game is the more general packaging—the shape the security definitions of the later sections take.

Remark 1.18 (Game-hopping). Once we phrase security as a game, proofs become *sequences of games*. One starts from the real security game and rewrites it through a series of games $G_0, G_1, \dots, G_k$, each differing from the last by a small, justifiable change, until reaching a final game in which the adversary provably has zero (or trivially negligible) advantage. Each hop is justified by exactly one of three moves: the two games are identical unless some negligible-probability “bad” event occurs, so their winning probabilities differ by at most $\Pr[\text{bad}]$ (on the complement of that event the two runs coincide); or distinguishing the two games would break a hardness assumption (a reduction, below); or the change is purely syntactic. Bounding the total advantage is then a telescoping sum across hops—a structured cousin of the hybrid argument (Lemma 1.13). This discipline keeps otherwise sprawling proofs honest and auditable.

1.5 Reductions

The engine of provable security is the reduction. A *reduction* is an efficient transformation that converts a successful adversary against a scheme into a successful algorithm against a problem believed to be hard. Its existence proves the implication “ Π is hard $\Rightarrow$ X is secure” by establishing the contrapositive “ X is broken $\Rightarrow$ Π is solved”.

Definition 1.19 (Computational problem and hardness). A *computational problem* Π consists of a PPT instance generator $\text{Inst}(1^\lambda)$ producing a problem instance together with any side information, and a relation deciding when a candidate output *solves* the instance. The *success probability* of an algorithm $\mathcal{B}$ is

$$\text{Succ}_\Pi(\mathcal{B}, \lambda) := \Pr[\mathcal{B}(1^\lambda, \text{Inst}(1^\lambda)) \text{ solves the instance}].$$

The problem Π is *hard* if $\text{Succ}_\Pi(\mathcal{B}, \lambda) = \text{negl}(\lambda)$ for every PPT $\mathcal{B}$ (for a search problem), or if every PPT $\mathcal{B}$ decides the instance with at most negligible advantage, $\text{Adv}(\mathcal{B}, \lambda) = \text{negl}(\lambda)$ in the sense of Definition 1.15 (for a decision problem).

Definition 1.20 (Black-box reduction). Let X be a scheme with security game G_X and Π a computational problem. A (*black-box*) *reduction* from breaking X to solving Π is a PPT oracle algorithm $\mathcal{R}$ with the following per-adversary guarantee: for every adversary $\mathcal{A}$, writing $\varepsilon_X(\lambda) := \text{Adv}_{G_X}(\mathcal{A}, \lambda)$ for its advantage, the algorithm $\mathcal{R}^\mathcal{A}$ —which runs $\mathcal{A}$ as a subroutine, simulating $\mathcal{A}$ ’s game internally—solves Π with success probability

$$\text{Succ}_\Pi(\mathcal{R}^\mathcal{A}, \lambda) \geq f(\varepsilon_X(\lambda), \lambda),$$

where f is a function fixed by the reduction (not by $\mathcal{A}$) that maps every non-negligible ε_X to a non-negligible bound. The running time of $\mathcal{R}$ is measured with each call to $\mathcal{A}$ counted as one step, so “PPT” bounds $\mathcal{R}$ ’s own work, and hence its number of calls to $\mathcal{A}$, by a polynomial in λ ; the running time of $\mathcal{R}^\mathcal{A}$ is then polynomial whenever $\mathcal{A}$ ’s is, however many times $\mathcal{R}$ runs $\mathcal{A}$. How far f degrades ε_X is the reduction’s *loss*, discussed below. The reduction is *black-box* because $\mathcal{R}$ uses $\mathcal{A}$ only through its input/output interface, making no use of $\mathcal{A}$ ’s code.

The structure of a reduction is always the same and is worth stating as a recipe. To prove “if Π is hard then X is secure”:

1. Assume toward the contrapositive a PPT adversary $\mathcal{A}$ that breaks X with non-negligible advantage ε_X .
2. Construct a PPT algorithm $\mathcal{R}$ that, given a random instance of Π , *embeds* that instance into a simulated security game for $\mathcal{A}$. The simulation must be faithful: the view $\mathcal{R}$ presents to $\mathcal{A}$ must follow (perfectly, statistically, or computationally) the distribution of the real game, so that $\mathcal{A}$ ’s advantage carries over.
3. Run $\mathcal{A}$. From whatever $\mathcal{A}$ does to win its game, $\mathcal{R}$ *extracts* a solution to the Π -instance.
4. Conclude that $\mathcal{R}$ solves Π with non-negligible probability, contradicting the hardness of Π . Therefore no such $\mathcal{A}$ exists, and X is secure.

A complete, genuine reduction illustrates the point.

Theorem 1.21 (Hard-core security of the Diffie–Hellman-style key, schematically). Let $\Pi = \text{CDH}$ be the computational Diffie–Hellman problem in a cyclic group $\mathbb{G} = \langle g \rangle$ of prime order q , the setting of the Math Guide (§“The discrete logarithm problem”): given (g, g^a, g^b) for uniform $a, b \in \mathbb{Z}/q\mathbb{Z}$, compute g^{ab} . Consider the “key-recovery” game G_X for a key-exchange scheme in which the adversary, given the public transcript (g^a, g^b) , wins iff it outputs the shared key g^{ab} . If CDH is hard, then no PPT adversary wins G_X with noticeable probability.

Proof. Suppose $\mathcal{A}$ is a PPT adversary that, on input a transcript (g^a, g^b) , outputs g^{ab} with probability $\varepsilon_X(\lambda) = \text{Adv}_{G_X}(\mathcal{A}, \lambda)$. Construct $\mathcal{R}$ solving CDH. On input a CDH instance $(g, A, B) = (g, g^a, g^b)$:

1. The reduction $\mathcal{R}$ forms the simulated transcript (A, B) and feeds it to $\mathcal{A}$ as the public transcript of G_X .
2. Because (a, b) are uniform in the CDH instance *and* in the real game, the view of $\mathcal{A}$ follows *identically* the distribution of its view in G_X (the simulation is perfect; no loss in faithfulness).
3. When $\mathcal{A}$ outputs a candidate key K , $\mathcal{R}$ outputs K as its CDH answer.

Since the simulated and real views are identically distributed, $\mathcal{A}$ outputs $K = g^{ab}$ with exactly probability $\varepsilon_X(\lambda)$; whenever it does, $\mathcal{R}$ returns the correct CDH solution. Hence

$$\text{Succ}_{\text{CDH}}(\mathcal{R}, \lambda) = \varepsilon_X(\lambda).$$

The running time of $\mathcal{R}$ is that of $\mathcal{A}$ plus a constant amount of group bookkeeping, hence polynomial. If ε_X were noticeable, $\mathcal{R}$ would solve CDH with noticeable probability, contradicting its hardness. Therefore $\varepsilon_X(\lambda)$ is not noticeable, as claimed. In fact, since $\mathcal{R}$ is PPT and CDH is hard (Definition 1.19, search branch), $\varepsilon_X(\lambda) = \text{Succ}_{\text{CDH}}(\mathcal{R}, \lambda) = \text{negl}(\lambda)$. $\square$

This example is deliberately *tight*: the reduction loses nothing, calls the adversary once, and converts advantage ε_X into success probability exactly ε_X . Most reductions are not so fortunate, which leads to the quantitative anatomy of a reduction.

Definition 1.22 (Reduction loss and tightness). Consider a reduction $\mathcal{R}$ that transforms an adversary running in time t with advantage ε into a Π -solver running in time $t' = t + t_{\mathcal{R}}$ with success probability ε' . The *security loss* is the factor

$$L(\lambda) := \frac{\varepsilon/t}{\varepsilon'/t'} = \frac{\varepsilon}{\varepsilon'} \cdot \frac{t'}{t},$$

the ratio of the solver's work-per-success to the adversary's work-per-success. A reduction is *tight* if $L(\lambda) = O(1)$ (a small constant, ideally 1), and *loose* if $L(\lambda)$ grows with λ (e.g. $L = q_H$, the number of random-oracle queries (Definition 1.26 below), or $L = q_s$, the number of signature queries, or $L = \lambda$). Two common sources of loss are: an *advantage gap* $\varepsilon' \ll \varepsilon$ (the solver succeeds far less often than the adversary, e.g. because the reduction must *guess* where to embed its instance among q queries, succeeding only with probability $1/q$); and a *time blow-up* $t' \gg t$ (the reduction runs the adversary many times, as in rewinding/forking arguments).

Remark 1.23 (Why loss matters concretely). Asymptotically, loss is invisible: a polynomial loss factor turns a negligible advantage into a negligible advantage and a hard problem into security, without qualification. Concretely, however, loss carries a cost. Suppose a scheme reduces to a problem believed to offer 128 bits of security, but the reduction loses a factor $L = 2^{40}$ (not unusual: $q_H \approx 2^{60}$ random-oracle queries times a forking-lemma square-root loss can be much worse). Then the *proof* guarantees only $128 - 40 = 88$ bits of security for the scheme. To restore a 128-bit guarantee one must enlarge the group so that the *underlying* problem offers 168 bits—larger keys, slower operations. A tight reduction lets the scheme inherit the full strength of the assumption at the smallest parameters; a loose one imposes a permanent cost on every user. This is why tightness is a first-class design goal in modern cryptography, not a theoretical nicety.

Remark 1.24 (Rewinding and the forking lemma, in brief). A recurrent source of large loss is *rewinding*: running the adversary, recording its behaviour, then restarting it from a saved state with fresh randomness on some branch to obtain a second, correlated transcript. From two transcripts that share a prefix but diverge afterward one can often extract a secret (e.g. two valid signatures on the same commitment yield the signing key, or two accepting proof transcripts yield a witness). The *forking lemma*, stated as the general forking lemma under “Security in the random oracle model and the forking lemma”, quantifies the success probability of obtaining such a pair: if the adversary succeeds

with probability ε after at most q random-oracle queries whose answers are drawn from a set of size h , the forked run succeeds with probability at least $\varepsilon(\varepsilon/q - 1/h)$, i.e. on the order of ε^2/q . The square in ε is a quadratic security loss: to keep ε^2/q noticeable one needs ε noticeably large, roughly halving the bit-security extracted. Such arguments—central to the analyses of Schnorr-type and Fiat–Shamir signatures that this volume builds toward—are the archetype of loose but still valid reductions.

1.6 The standard model and the random oracle model

Reductions reduce security to assumptions. *Which* assumptions one is willing to make defines the *model* in which a proof resides. Two models predominate.

Definition 1.25 (Standard model). A proof resides in the *standard model* if it assumes only the computational hardness of explicitly stated problems (factoring, discrete logarithm, the decisional Diffie–Hellman problem of the next section, and so on) and grants the adversary no idealised access to any component of the scheme. Every primitive, including every hash function, is a concrete algorithm whose code the adversary may inspect and exploit.

Definition 1.26 (Random oracle model). A proof resides in the *random oracle model* (ROM) if it idealises one or more hash functions as a *random oracle*: a publicly accessible function $\mathcal{O}$ that, on each distinct input, returns an *independent, uniformly random* output of the appropriate length, consistently answering repeated queries with the same value. The oracle is available to the honest parties, to the adversary, and—crucially—to the reduction. No party holds the “code” of $\mathcal{O}$; it can only be queried.

The random oracle model is an idealisation: real hash functions are fixed, public algorithms, not truly random functions. The motivation for adopting it is that the reduction gains two capabilities that the standard model withholds and that render many otherwise-intractable proofs tractable.

Remark 1.27 (The powers of the random oracle: observability and programmability). In the ROM the reduction *simulates* the oracle for the adversary, and this yields two additional capabilities. *Observability*: the reduction sees every query the adversary makes to $\mathcal{O}$. Since the adversary, to use a hash value meaningfully, must query it, the reduction learns the adversary’s intermediate computations—for instance the preimage the adversary is about to hash. *Programmability*: the reduction may choose each oracle answer on the fly, subject only to the answers appearing uniform and consistent. It can therefore *plant* its hard-problem instance inside an oracle response—e.g. set $\mathcal{O}(x) := B$ where B is part of a CDH challenge—so that the adversary’s success necessarily reveals the solution. Neither capability exists for a concrete standard-model hash function, which fixes its outputs in advance and hides its internal queries.

Example 1.28 (The benefit of the ROM: full-domain hash, sketched). In the “full domain hash” signature, the signer signs a message m as $\sigma = H(m)^d \bmod N$, where (N, d) is an RSA private key—a modulus $N = pq$ with secret prime factors and a secret exponent d paired with a public e so that $x^{ed} \equiv x \pmod{N}$ for x coprime to N , by Euler’s theorem (*Math Guide*, §“Fermat’s little theorem and Euler’s theorem”); thus $x \mapsto x^e$ is a permutation of the unit group $(\mathbb{Z}/N\mathbb{Z})^\times$ that only the holder of d can invert—and H is a hash function mapping messages into $(\mathbb{Z}/N\mathbb{Z})^\times$. (Restricting to units costs nothing: a nonzero non-unit of $\mathbb{Z}/N\mathbb{Z}$ shares a factor with N , so stumbling on one factors N . Hash functions appear here informally, as this sketch needs them; their formal syntax and security games are the business of the next-but-one section.) To prove unforgeability one reduces to RSA inversion: given a challenge y (find y^d), the reduction programs the oracle so that for a random subset of messages $H(m) := y \cdot r_m^e$ (embedding the challenge) and for the rest $H(m) := r_m^e$ (where it knows the “signature” r_m); with each r_m drawn uniformly from $(\mathbb{Z}/N\mathbb{Z})^\times$, the programmed values are distributed

exactly as honest oracle answers on H 's range. When the forger outputs a forgery on a message where the reduction embedded the challenge, the reduction extracts y^d . This proof has *no known standard-model analogue* for the plain scheme: it relies essentially on programming H . The cost is a security loss (the reduction must guess which message the forger attacks, incurring a factor q_H unless cleverly optimised)—connecting back to Definition 1.22.

Remark 1.29 (The status of the ROM: heuristic, not theorem). A proof in the ROM is a proof about an *idealised* world: deploying the scheme instantiates $\mathcal{O}$ with a concrete hash function, and the proof does *not* formally transfer—a ROM proof rules out all attacks that treat the hash as a black box, while offering no guarantee against attacks on the specific structure of the chosen hash. The case for trusting it runs in three steps: prove the scheme secure against a perfect random oracle; instantiate $\mathcal{O}$ with a hash believed to have no exploitable structure; and observe that any remaining attack must exploit structure the random oracle lacks, so would be a direct break of the presumed structure-free hash. The gap is nonetheless a theorem: Canetti, Goldreich, and Halevi exhibit contrived schemes provably secure in the ROM yet insecure under *every* efficiently computable instantiation, so no concrete hash soundly instantiates the oracle for all schemes. No natural deployed scheme is known to suffer this fate, and the working stance of this volume is the practical one: a ROM proof is strong positive evidence, not a guarantee, and a standard-model proof is preferred when one is available at comparable cost. One caveat looks forward. A quantum adversary (Remark 1.4) queries the oracle *in superposition*, a single query addressing a weighted combination of all inputs at once; in the resulting quantum random oracle model (QROM) the two powers of Remark 1.27 become subtler and classical ROM theorems do not automatically carry over, so post-quantum claims for arguments made non-interactive by hashing the transcript (the Fiat–Shamir transform, developed under “The Fiat–Shamir transform: from interactive to non-interactive”) must be made in that model.

Remark 1.30 (Relatives: CRS, generic-group, and algebraic-group models). Between and beyond these poles lie further idealisations relevant to the proof systems this volume targets. The *common reference string* (CRS) model posits a trusted public string sampled from a prescribed distribution, used by many succinct proof systems. The *generic group model* (GGM) idealises the group dual to the manner in which the ROM idealises a hash: the adversary may only perform group operations through an oracle on opaque “handles”, never exploiting the bit representation of group elements; it is the natural setting for lower bounds on discrete-log-type problems. The *algebraic group model* (AGM) is an intermediate, milder idealisation requiring the adversary to “explain” every group element it outputs as a known combination of its inputs. These models trade realism for provability along the same axis as the ROM, and the constructions ahead invoke them by name where they require them.

1.7 Concrete security and the bit-security budget

The asymptotic language—“negligible”, “polynomial”, “for all sufficiently large λ ”—is mathematically clean but operationally silent. It establishes that a scheme is eventually secure; it does not establish whether *this* deployment with *these* parameters resists an adversary with *that* budget. Concrete (or exact) security supplies the missing numbers.

Definition 1.31 (Concrete security: (t, ϵ) -security). Fix the security parameter to a concrete value (so all sizes are fixed). A scheme is (t, ϵ) -secure for a game G if every adversary running in time at most t has advantage $\text{Adv}_G(\mathcal{A}) \leq \epsilon$. More refined statements bound the advantage as an explicit function of the adversary’s resources, e.g. time t , number of queries q , and memory s : $\text{Adv}_G(\mathcal{A}) \leq f(t, q, s)$.

Definition 1.32 (n -bit security). A scheme (or problem) is commonly said to offer n bits of security

when the cheapest known attack achieving constant success probability (say at least $\frac{1}{2}$) costs on the order of 2^n elementary operations. A proof may justify the stronger, explicit claim that every adversary running in time t has advantage at most about $t/2^n$, but that linear advantage bound is not implied merely by the absence of a faster known attack. Thus “128-bit security” is a concrete estimate, relative to a specified attack model and state of knowledge, that breaking the scheme costs roughly 2^{128} work.

The relationship to the asymptotic picture is the following: one chooses the security parameter λ *precisely so that* the resulting concrete scheme delivers the desired number of bits of security. In the cleanest case the two coincide, $n = \lambda$, but in general one must account for the actual cost of the best attack on the underlying problem.

Example 1.33 (Why $\lambda \neq n$ for discrete logarithm). For a generic-group or hash-based primitive, doubling the output length doubles the bit security: a hash with $2n$ -bit output offers n -bit collision resistance—by the birthday bound (*Math Guide*, §“The union bound and a birthday calculation”)—and $2n$ -bit preimage resistance, so the parameter and the security level track simply. For discrete-logarithm-based primitives the accounting differs and instructs. In a group of prime order $q \approx 2^\ell$, generic attacks (baby-step/giant-step, Pollard’s rho) solve discrete logarithm in time $\approx \sqrt{q} = 2^{\ell/2}$ (*Math Guide*, §“The discrete logarithm problem”). Hence a prime-order elliptic-curve group whose order is 256 bits offers only about 128 bits of security—one halves the bit length of the group order to obtain the generic security level. For discrete logarithm in a finite-field multiplicative group, sub-exponential index-calculus attacks additionally force the ambient field modulus into the thousands of bits for 128-bit security. This is precisely why elliptic-curve groups, for which no comparable sub-exponential classical attack is known, are preferred: they deliver n -bit security at $\ell \approx 2n$ rather than thousands of bits. The single number n conceals a problem-specific translation from all parameter sizes to the security level. The working groups of this volume are priced by exactly this arithmetic: the Pasta curves assembled in the *Math Guide* (§“Pallas and Vesta assembled”) have prime order close to 2^{254} , so the square-root rule prices generic discrete logarithm there at about 2^{127} operations; the next section audits them criterion by criterion under “Instantiation on elliptic curves; the Pasta curves”.

Remark 1.34 (Reading a bit-security budget end to end). Combining the preceding components yields the practitioner’s parameter-selection recipe. Decide the target security level n (commonly 128). Identify the underlying hard problem and the cost of its best attack as a function of the parameter (e.g. $2^{\ell/2}$ for elliptic-curve discrete log), and set the parameter so the problem itself offers n bits (here $\ell = 2n = 256$). Then subtract the reduction’s security loss (Definition 1.22): if the proof loses a factor $L = 2^k$ (that is, k bits), the problem must offer $n + k$ bits for the *scheme* to offer n , so one enlarges the parameter accordingly—or, with a tight reduction, one need not. Finally, sanity-check against the ROM caveat (Remark 1.29): a ROM proof bounds only black-box attacks on the hash, so the concrete hash must additionally exhibit no exploitable structure. The clean asymptotic theorem “the scheme is secure if the problem is hard” thus unfolds, through the quantitative machinery of this section, into a concrete choice of bytes.

Remark 1.35 (A note on what security does and does not promise). A caution that frames everything above is in order. A proof of security is a proof *relative to a model*: a definition of the goal (the game), a class of adversaries (PPT, possibly non-uniform or quantum), and a set of assumptions (the hard problems, and any idealisation such as the ROM). It guarantees nothing outside that model. Side-channel leakage (timing, power), faulty randomness, implementation bugs, broken assumptions, and goals the game did not capture all lie beyond its reach. The value of the framework is not that it renders schemes unconditionally safe—nothing can—but that it renders the *conditions* of safety explicit, falsifiable, and modular, so that progress on assumptions and attacks translates directly into recalibrated, auditable guarantees. Everything this monograph builds rests on this discipline.

2 Hardness assumptions

Every secret in this volume is an exponent. A spending key, the trapdoor of a commitment, the nonce of a signature—each lives in $\mathbb{Z}/q\mathbb{Z}$ and faces the world only through a group element g^x . The adversary of this section is the patient cryptanalyst who collects those elements from the public record and asks for nothing more than the exponent behind one of them: an adversary who computes discrete logarithms recovers signing keys from verification keys, opens and re-opens commitments at will, and strips the blinding from every rerandomised key the later sections construct. What she can *forge* is everything; what she must be *denied* is a single inversion. This section assembles the family of assumptions that deny it: the discrete logarithm problem itself, the ladder of Diffie–Hellman variants above it, the generic-attack bounds that calibrate how hard the problem can possibly be, the composite-order failure mode that squanders that hardness, the elliptic curves on which the remainder of the volume instantiates the family, and the quantum caveat under which the whole edifice stands. Throughout, the lodestar is a single number: the security parameter λ , read as the bit-length of security demanded. The recurring conclusion is that to obtain λ bits of security against the best generic attacks the group must have *prime* order q with $\log_2 q \geq 2\lambda$. Notation is fixed once: $G = \langle g \rangle$ denotes a cyclic group of prime order q , written multiplicatively until §2.6 switches to the additive notation of elliptic curves.

2.1 The discrete logarithm problem

The stage is a finite cyclic group. Recall from the *Math Guide* (§“The discrete logarithm problem”) that a group G is *cyclic* if $G = \langle g \rangle = \{g^k : k \in \mathbb{Z}\}$ for some *generator* g ; written additively, as we shall write elliptic curves, this reads $G = \{kg : k \in \mathbb{Z}\}$. If $|G| = q$ then $\mathbb{Z}/q\mathbb{Z} \cong G$ via $k \mapsto g^k$, an isomorphism trivial to evaluate forwards—repeated squaring costs $O(\log k)$ group operations—but conjecturally infeasible to invert. That asymmetry *is* the discrete logarithm problem.

Definition 2.1 (Discrete logarithm). Let $G = \langle g \rangle$ be a cyclic group of order q , written multiplicatively. For $h \in G$, the *discrete logarithm* of h to the base g , denoted $\log_g h$, is the unique residue $x \in \mathbb{Z}/q\mathbb{Z}$ with $g^x = h$. Existence follows from $G = \langle g \rangle$ and $\text{ord}(g) = q$; uniqueness because $g^x = g^{x'}$ forces $g^{x-x'} = 1$, whence $q \mid x - x'$.

Definition 2.2 (Discrete logarithm problem, DLP). Fix a family $\{G_\lambda\}$ of cyclic groups, where $G_\lambda = \langle g_\lambda \rangle$ has *prime* order q_λ with $\log_2 q_\lambda = \Theta(\lambda)$, together with a sampler that on input 1^λ outputs a description of $(G_\lambda, g_\lambda, q_\lambda)$. The game $\text{DLOG}^\mathcal{A}(\lambda)$ runs as follows.

1. The challenger runs the sampler, draws $x \xleftarrow{\$} \mathbb{Z}/q_\lambda\mathbb{Z}$ uniformly, and sends $(G_\lambda, g_\lambda, q_\lambda, h = g_\lambda^x)$ to the adversary $\mathcal{A}$.
2. The adversary outputs $x' \in \mathbb{Z}/q_\lambda\mathbb{Z}$.
3. The game outputs 1 iff $g_\lambda^{x'} = h$.

The *DLP assumption* for the family states that for every PPT $\mathcal{A}$,

$$\Pr[\text{DLOG}^\mathcal{A}(\lambda) = 1] \in \text{negl}(\lambda).$$

The order is restricted to be prime for reasons that §2.5 makes emphatic: in prime order every non-identity element is a generator, there are no proper nontrivial subgroups through which information can leak, and the problem cannot be split across factors of the order. For now we record the convenience that prime order affords immediately.

Proposition 2.3 (Random self-reducibility of DLP). *In a cyclic group $G = \langle g \rangle$ of prime order q , the discrete logarithm problem is random self-reducible: an algorithm solving a fraction ε of instances, over uniformly random h , converts into one solving any fixed instance with probability ε per trial. Consequently worst-case and average-case hardness coincide.*

Proof. Let $\mathcal{A}$ succeed on a uniform target with probability ε . Given a fixed challenge $h = g^x$, draw $r \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ and form $h' = h \cdot g^r = g^{x+r}$. Addition modulo q is a bijection and r is uniform, so h' is uniform in G and independent of h ; hence $\mathcal{A}(h')$ returns $x + r \bmod q$ with probability ε , and subtracting r recovers x . Each candidate is checked by testing $g^x \stackrel{?}{=} h$, so failures are recognised. Fresh randomness r per trial makes trials independent: t of them succeed with probability $1 - (1 - \varepsilon)^t$, overwhelming once $t = \omega(\varepsilon^{-1} \log \lambda)$. $\square$

Random self-reducibility is the formal licence to speak of “the” hardness of DLP in a group: an average-case solver also solves every fixed instance after rerandomisation. This does not literally make every instance difficult—the logarithm of the identity, for one, is immediate.

2.2 Diffie–Hellman: computational and decisional

The discrete logarithm problem asks to invert exponentiation. Many protocols—the Diffie–Hellman key exchange and ElGamal encryption are the historical motivations—need only a weaker-looking guarantee about the *product* of two exponents. Suppose Alice publishes g^a and Bob publishes g^b ; their shared secret is g^{ab} , which each computes from the other’s public value and their own exponent. The eavesdropper sees g, g^a, g^b and wants g^{ab} .

Definition 2.4 (Computational Diffie–Hellman, CDH). With notation as in Definition 2.2, the game $\text{CDH}^\mathcal{A}(\lambda)$ runs as follows: the challenger draws $a, b \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ independently and sends (g, g^a, g^b) ; the adversary outputs $z \in G$; the game outputs 1 iff $z = g^{ab}$. The *CDH assumption* states that for every PPT $\mathcal{A}$,

$$\Pr_{a,b}[\text{CDH}^\mathcal{A}(\lambda) = 1] \in \text{negl}(\lambda).$$

The computational guarantee delivers the shared key but not its secrecy: an adversary may be unable to *produce* g^{ab} yet perfectly able to *distinguish* it from a random group element, and that alone already breaks, say, the indistinguishability of an ElGamal ciphertext. The decisional variant excludes it.

Definition 2.5 (Decisional Diffie–Hellman, DDH). The *decisional Diffie–Hellman problem* is to distinguish two distributions over G^3 ,

$$\mathcal{D}_{\text{dh}} = (g^a, g^b, g^{ab}), \quad \mathcal{D}_{\text{rand}} = (g^a, g^b, g^c),$$

with a, b, c independent and uniform in $\mathbb{Z}/q\mathbb{Z}$; this is the distinguishing game of Definition 1.15, the challenger handing over a sample of one or the other according to its hidden bit. The *DDH assumption* states that every PPT distinguisher D has negligible advantage

$$\text{Adv}_{\text{ddh}}(D) = \left| \Pr_{a,b}[D(g^a, g^b, g^{ab}) = 1] - \Pr_{a,b,c}[D(g^a, g^b, g^c) = 1] \right| \in \text{negl}(\lambda).$$

A triple (g^a, g^b, g^c) with $c \equiv ab \pmod{q}$ is a *Diffie–Hellman triple*; the assumption says no efficient distinguisher recognises such triples.

Remark 2.6 (DDH admits a random self-reduction). Rerandomisation works for DDH as it did for DLP, though the correct map takes some care. Given a target triple $(u, v, w) = (g^a, g^b, g^c)$, choose $\alpha, \gamma \xleftarrow{\$} (\mathbb{Z}/q\mathbb{Z})^\times$ and $\beta, \delta \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$, and form

$$(u', v', w') = (u^\alpha g^\beta, v^\gamma g^\delta, w^{\alpha\gamma} u^{\alpha\delta} v^{\beta\gamma} g^{\beta\delta}).$$

Writing $A = \alpha a + \beta$, $B = \gamma b + \delta$, and $E = c - ab$, the new exponents are A , B , and $AB + \alpha\gamma E$, with A and B uniform and independent. A DH triple ($E = 0$) therefore becomes a fresh uniform DH triple; a non-DH triple becomes uniform conditional on being non-DH, because $\alpha\gamma E$ is uniform in $(\mathbb{Z}/q\mathbb{Z})^\times$. The latter distribution lies at statistical distance (Definition 1.6) $1/q$ from $\mathcal{D}_{\text{rand}}$, whose third component completes a DH triple by accident with probability $1/q$. A noticeable distinguishing advantage on random instances thus transfers to every fixed triple, up to this negligible term.

2.3 The assumption hierarchy

The three problems form a chain. Write $A \leq B$ for “ A reduces to B ”: an oracle solving B yields an efficient solver for A ; equivalently, B is at least as hard as A , so that if B is easy then A is easy; equivalently again, the assumption “ A is hard” is the *stronger* commitment, since it implies “ B is hard”. The reductions of this subsection give

$$\text{DDH} \leq \text{CDH} \leq \text{DLP},$$

ordering the problems by increasing hardness and the assumptions by decreasing strength: assuming DDH hard is the strongest commitment, assuming DLP hard the weakest.

Theorem 2.7 (DLP is at least as hard as CDH). *If DLP is solvable in PPT, then CDH is solvable in PPT. Equivalently, the CDH assumption implies the DLP assumption.*

Proof. Suppose $\mathcal{A}$ solves DLP. Given a CDH instance (g, g^a, g^b) , run $\mathcal{A}$ on (g, g^a) to recover a , then compute $(g^b)^a = g^{ab}$ by fast exponentiation. The cost is one DLP call plus $O(\log q)$ group operations, and only one of the two exponents ever needed inverting. $\square$

Theorem 2.8 (CDH is at least as hard as DDH). *If CDH is solvable in PPT with non-negligible success probability, then DDH is solvable in PPT with non-negligible advantage; if the success probability is noticeable, rerandomisation drives the advantage overwhelmingly close to 1. Equivalently, the DDH assumption implies the CDH assumption.*

Proof. Suppose $\mathcal{A}$ solves CDH with probability ε . Build a distinguisher D : on input (g^a, g^b, z) , call $\mathcal{A}(g, g^a, g^b)$ to obtain a candidate w , and output 1 iff $w = z$. On a Diffie–Hellman triple, $z = g^{ab}$, so D outputs 1 whenever $\mathcal{A}$ succeeds—probability ε . On a random triple, $z = g^c$ with c uniform and independent of a, b , while w depends only on (g^a, g^b) ; the event $w = z$ therefore has probability exactly $1/q$. Hence

$$\text{Adv}_{\text{ddh}}(D) \geq \varepsilon - \frac{1}{q},$$

non-negligible whenever ε is, since $1/q$ is negligible.

For the amplified form let ε be noticeable. The distinguisher D' rerandomises its input t times independently by the map of Remark 2.6, runs D with fresh coins on each copy, and outputs 1 iff *some* copy answered 1. A DH input becomes t independent uniform DH triples, each accepted with probability ε , so D' accepts with probability at least $1 - (1 - \varepsilon)^t$. A sample of $\mathcal{D}_{\text{rand}}$ fails to be a DH triple except with probability $1/q$, and then each copy’s third component is uniform over the $q - 1$ non-completing values given the first two, so each copy is accepted with probability at most $1/(q - 1)$ and D' accepts with probability at most $1/q + t/(q - 1)$. With $t = \omega(\varepsilon^{-1} \log \lambda)$, polynomial because ε is noticeable, the advantage of D' is $1 - \text{negl}(\lambda)$. $\square$

Example 2.9 (DDH can be easy while CDH and DLP are hard). Some elliptic-curve groups carry an efficiently computable, non-degenerate *bilinear pairing* $e: G \times G \rightarrow G_T$ with $e(g^a, g^b) = e(g, g)^{ab}$. On such a group DDH is *easy*: to test whether (g^a, g^b, z) is a DH triple, check

$$e(g^a, g^b) \stackrel{?}{=} e(g, z),$$

which holds iff $\log_g z \equiv ab \pmod{q}$. Yet CDH—and a fortiori DLP—may remain hard, because the pairing reveals g^{ab} only inside the *target* group G_T , never as an element of G . Such groups, called *gap groups*, separate DDH from CDH and found pairing-based cryptography. The moral is that DDH is a genuinely stronger assumption: choosing a “DDH group” means choosing a group that rules out pairings of this convenient kind on G itself.

Remark 2.10 (The converses are open). The reverse reductions are not known in general. Whether CDH implies DLP—whether the ability to compute g^{ab} can be leveraged to extract a —is the long-standing Diffie–Hellman versus discrete log question; partial results of den Boer and Maurer establish equivalence when a suitable auxiliary group of *smooth* order—an order that is a product of small primes—is available, but no unconditional equivalence is known. In the other direction DDH may be strictly easier than CDH, as Example 2.9 shows. Practice therefore assumes exactly the strength a protocol requires: signatures and key exchange often need only CDH, while ElGamal-style encryption needs DDH for its ciphertexts to hide the plaintext.

2.4 Generic algorithms and the square-root barrier

How hard can the discrete logarithm problem possibly be? In a group with no exploitable structure beyond the group law, the best known algorithms are *generic*: they manipulate group elements only through multiplication, inversion, and equality testing, never inspecting the bit-representation of an element. Shoup’s idealisation makes the class precise: a random injective *labelling* $\sigma: \mathbb{Z}/q\mathbb{Z} \rightarrow S$ encodes the element g^k as the opaque string $\sigma(k)$, and the algorithm may only ask an oracle for $\sigma(i+j)$ given $\sigma(i)$ and $\sigma(j)$, and compare labels for equality. Two generic algorithms achieve $O(\sqrt{q})$ group operations; a matching lower bound then shows that, generically, nothing does better.

Baby-step giant-step

Proposition 2.11 (Baby-step giant-step, BSGS). *In a cyclic group of order q , any discrete logarithm can be computed in $O(\sqrt{q})$ group operations and $O(\sqrt{q})$ stored group elements, deterministically and unconditionally.*

Proof. Let $m = \lceil \sqrt{q} \rceil$. Since $m^2 \geq q$, every $x \in \{0, \dots, q-1\}$ decomposes as

$$x = i \cdot m + j, \quad 0 \leq i < m, \quad 0 \leq j < m,$$

and the target relation $g^x = h$ becomes $g^j = h \cdot (g^{-m})^i$. *Baby steps*: compute and store the table $\{(j, g^j) : 0 \leq j < m\}$, indexed by group element in a hash map— $m-1$ multiplications. *Giant steps*: set $u = g^{-m}$ (one inversion plus one exponentiation, $O(\log q)$ operations) and for $i = 0, 1, 2, \dots$ compute $h \cdot u^i$ incrementally, looking each value up in the table. The decomposition guarantees some pair (i, j) matches, yielding $x = im + j$ after at most m giant steps. Total: $O(\sqrt{q})$ operations and $O(\sqrt{q})$ memory. $\square$

Remark 2.12 (Memory is the bottleneck). The $O(\sqrt{q})$ *memory* of BSGS, not its time, is what bites in practice: at the 128-bit security level no attacker can store $\sqrt{q} \approx 2^{128}$ table entries. Pollard’s rho method matches the time bound with negligible memory, which is why rho, rather than BSGS, is the realistic generic attack.

Pollard's rho

Proposition 2.13 (Pollard's rho, sketch). *In a cyclic group of prime order q there is a randomised algorithm computing discrete logarithms in expected $O(\sqrt{q})$ group operations and $O(1)$ storage.*

Proof. Sketch. Walk pseudo-randomly on G , keeping with each visited element its known representation $g^\alpha h^\beta$. Partition G into three roughly equal sets S_1, S_2, S_3 by an easily computed function of the element's label, and from $P = g^\alpha h^\beta$ step to

$$f(P) = \begin{cases} h \cdot P = g^\alpha h^{\beta+1}, & P \in S_1, \\ P^2 = g^{2\alpha} h^{2\beta}, & P \in S_2, \\ g \cdot P = g^{\alpha+1} h^\beta, & P \in S_3, \end{cases}$$

each step updating $(\alpha, \beta) \in (\mathbb{Z}/q\mathbb{Z})^2$ explicitly at $O(1)$ cost. The sequence eventually cycles—a tail leading into a loop, the shape of the letter ρ . Modelling f as a random function on q elements, the birthday bound (*Math Guide*, §“The union bound and a birthday calculation”) produces a collision $P_i = P_j$, $i \neq j$, within $O(\sqrt{q})$ steps in expectation. A collision gives $g^{\alpha_i} h^{\beta_i} = g^{\alpha_j} h^{\beta_j}$, hence $g^{\alpha_i - \alpha_j} = h^{\beta_j - \beta_i}$; writing $h = g^x$,

$$\alpha_i - \alpha_j \equiv x(\beta_j - \beta_i) \pmod{q},$$

and if $\beta_j \not\equiv \beta_i$ the difference is invertible modulo the *prime* q , so $x \equiv (\alpha_i - \alpha_j)(\beta_j - \beta_i)^{-1} \pmod{q}$. Floyd's tortoise-and-hare cycle detection, or Brent's improvement, finds the collision with $O(1)$ memory, never storing the trajectory. The exceptional case $\beta_j \equiv \beta_i$ occurs with probability $O(1/q)$; re-randomising the start point handles it. $\square$

Remark 2.14 (Cost, not time, and constant factors). Pollard rho parallelises almost perfectly via the van Oorschot–Wiener method of *distinguished points*: each of P processors walks independently and reports to a shared table only the points whose label satisfies a cheap predicate, say a run of leading zero bits; two walks that ever meet coincide from then on and are detected at the next distinguished point, so the processors achieve $\sqrt{q}/P$ wall-clock time with modest communication. The security-relevant measure is therefore the *cost* of the attack—the total operation count $\sqrt{q}$, read as the attacker's budget—not the time on any single machine. A further constant-factor saving applies in groups admitting the $\pm$ automorphism $P \mapsto -P$, as elliptic curves do: identifying P with $-P$ halves the effective search space and lowers the expected count from $\sqrt{\pi q/2}$ to $\sqrt{\pi q/4}$, a factor $\sqrt{2}$ that leaves the $\Theta(\sqrt{q})$ scaling untouched. The constant is the birthday integral: a random walk on N points is still collision-free after k steps with probability $\prod_{i < k} (1 - i/N) \approx e^{-k^2/2N}$ (the product in the birthday calculation cited above), and summing over k gives an expected first-collision index of $\int_0^\infty e^{-k^2/2N} dk = \sqrt{\pi N/2}$.

The generic lower bound

The two attacks are not merely the best known; up to constants they are the best *possible* for any generic algorithm. This is Shoup's theorem, the formal statement of the square-root barrier.

Theorem 2.15 (Shoup's generic lower bound). *Let q be prime. Any generic algorithm that makes at most n queries to the group oracle outputs the discrete logarithm of a uniformly random challenge with probability at most*

$$\frac{\binom{n+2}{2} + 1}{q} = O\left(\frac{n^2}{q}\right).$$

In particular, constant success probability requires $n = \Omega(\sqrt{q})$ queries.

Proof. Sketch—the linear-polynomial technique. In the idealisation, the challenge logarithm is an indeterminate X drawn uniformly from $\mathbb{Z}/q\mathbb{Z}$. By induction on the queries, every element the algorithm forms is σ of a *known linear polynomial* $a_i + b_i X \bmod q$ whose coefficients the algorithm chose. Treat X formally: as long as the polynomials produced are pairwise distinct as functions of X , the labels seen are consistent with X taking any value, so the algorithm has learnt nothing and can only guess, succeeding with probability $1/q$. Information leaks only through an accidental collision: two distinct polynomials agreeing at the secret x , that is, $(a_i - a_j) + (b_i - b_j)x \equiv 0 \pmod{q}$. For a fixed pair this nonzero linear equation has at most one root modulo the prime q —the degree-one case of the Schwartz–Zippel lemma (*Math Guide*, §“The Schwartz–Zippel lemma”)—so it holds with probability at most $1/q$ over uniform x . With n queries plus the two inputs g and h there are at most $\binom{n+2}{2}$ pairs, and the union bound (*Math Guide*, §“The union bound and a birthday calculation”) caps the collision probability at $\binom{n+2}{2}/q$. Adding the $1/q$ guess yields the bound; solving $n^2/q = \Omega(1)$ gives $n = \Omega(\sqrt{q})$. $\square$

Corollary 2.16 (Why $\log_2 q \geq 2\lambda$). *To force every generic attacker to spend at least 2^λ group operations— λ bits of security against generic attacks—the prime group order must satisfy*

$$\sqrt{q} \gtrsim 2^\lambda \iff \log_2 q \geq 2\lambda.$$

Proof. Propositions 2.11 and 2.13 show that $\Theta(\sqrt{q})$ operations suffice for a generic attacker, and Theorem 2.15 shows no generic attacker does asymptotically better. Equating the attacker’s cost $\sqrt{q}$ with the target work factor 2^λ gives $q \approx 2^{2\lambda}$; demanding at least this much is $\log_2 q \geq 2\lambda$. $\square$

Example 2.17 (The 128-bit target). For $\lambda = 128$ the corollary demands a prime group order with $\log_2 q \geq 256$. This explains the standardised 256-bit elliptic curves: NIST P-256 (secp256r1) has a 256-bit field and a 256-bit group order, so $\sqrt{q} \approx 2^{128.0}$. The Pasta curves of §2.6, with 255-bit orders just above 2^{254} and about 126 bits of security, sit just below this class. Contrast the multiplicative group $\mathbb{F}_p^\times$: there the sub-exponential index calculus of Remark 2.18 forces $\log_2 p$ into the thousands of bits for the same λ . Elliptic curves earn their keep precisely because no sub-exponential, generic-beating algorithm is known for them.

Remark 2.18 (Why the barrier is special to elliptic curves). The square-root barrier governs *generic* groups only; a concrete representation can leak structure that beats it. The multiplicative group $\mathbb{F}_p^\times$ is the cautionary case: *index calculus* exploits the factorisation of integers into small primes—smooth numbers—to solve DLP in sub-exponential time

$$\exp(O((\log p)^{1/3}(\log \log p)^{2/3})),$$

far below $\sqrt{p}$. No analogue is known for the points of a well-chosen elliptic curve over $\mathbb{F}_p$: there is no useful notion of a “small” point over which to factor. This non-generic gap is why elliptic-curve groups can live at 256 bits where $\mathbb{F}_p^\times$ must reach about 3072 bits for the same security. It also means the rule $\log_2 q \geq 2\lambda$ protects against generic attacks *only*; a curve must additionally be screened against the special-purpose attacks of Definition 2.24.

2.5 Pohlig–Hellman and the necessity of large prime order

The square-root barrier priced the attack at $\sqrt{q}$ for *prime* q . If the group order factors, the discrete logarithm splits into independent pieces the size of the factors, and the difficulty collapses to that of the *largest prime* dividing the order. This is the Pohlig–Hellman reduction—the reason Definition 2.2 insisted on prime order.

Theorem 2.19 (Pohlig–Hellman). *Let $G = \langle g \rangle$ have order $N = \prod_{i=1}^r p_i^{e_i}$ with the p_i distinct primes. Then a discrete logarithm in G can be computed in*

$$O\left(\sum_{i=1}^r e_i (\log N + \sqrt{p_i})\right)$$

group operations. The term $\sqrt{p_{\max}}$ dominates, where $p_{\max}$ is the largest prime dividing N .

Proof. We compute $x = \log_g h$ modulo each prime power $p_i^{e_i}$ and recombine by the Chinese Remainder Theorem, the moduli being pairwise coprime with product N .

Reduction to prime-power order. Fix $p = p_i$, $e = e_i$, and put $g_i = g^{N/p^e}$, $h_i = h^{N/p^e}$. Then g_i has order exactly p^e , and $\log_{g_i} h_i \equiv x \pmod{p^e}$: raising to N/p^e kills the part of x coprime to p 's power and leaves $x \bmod p^e$. It therefore suffices to solve DLP in $\langle g_i \rangle$ of order p^e .

Reduction to prime order, digit by digit. Write the unknown in base p :

$$x \equiv x_0 + x_1 p + \cdots + x_{e-1} p^{e-1} \pmod{p^e}, \quad 0 \leq x_k < p.$$

Let $\gamma = g_i^{p^{e-1}}$, an element of order exactly p . Raising h_i to the power p^{e-1} gives

$$h_i^{p^{e-1}} = g_i^{x p^{e-1}} = \gamma^{x_0},$$

because every higher digit contributes a multiple of p^e to the exponent and annihilates γ . Thus x_0 is a single discrete logarithm in a group of prime order p , solvable in $O(\sqrt{p})$ operations by BSGS or rho (Propositions 2.11 and 2.13). Having x_0 , set $h_i^{(1)} = h_i \cdot g_i^{-x_0}$; its logarithm is divisible by p , and the same step at exponent p^{e-2} yields x_1 . Iterating recovers all e digits with e prime-order DLPs of cost $O(\sqrt{p})$ each, plus $O(e \log N)$ operations of exponentiation. Summing over i and applying the CRT gives the stated bound. $\square$

Example 2.20 (A composite-order trap). Suppose, contrary to good practice, a group has order $N = 2^{256} - 1$. The order factors completely into

$$\begin{aligned} 2^{256} - 1 = & 3 \cdot 5 \cdot 17 \cdot 257 \cdot 641 \cdot 65537 \cdot 274177 \cdot 6700417 \cdot 67280421310721 \\ & \cdot 59649589127497217 \cdot 5704689200685129054721, \end{aligned}$$

a product of small primes—it is *smooth*. Despite N having 256 bits, the largest prime factor is only about $2^{72.3}$, far below 2^{128} , so Pohlig–Hellman with rho in the largest subgroup solves DLP in roughly $\sqrt{p_{\max}} \approx 2^{36.1}$ operations—about 2^{36} , nowhere near the apparent 128-bit level. A 256-bit order is necessary but *not* sufficient: it must be (almost) prime.

Corollary 2.21 (Design rule for the group order). *For DLP-based security at level λ , the group order must be divisible by a prime q with $\log_2 q \geq 2\lambda$, and the cryptographic group must be, or be restricted to, the subgroup of order q . When the ambient group has order $N = h \cdot q$ with small cofactor h and large prime q , one works in the order- q subgroup; clearing the cofactor—multiplying incoming elements by h , or checking subgroup membership—prevents small-subgroup attacks that would otherwise leak $x \bmod h$ via Pohlig–Hellman on the order- h part.*

Remark 2.22 (Prime order, revisited). Here is the deeper justification for “prime order” in Definition 2.2. Prime order q makes Pohlig–Hellman vacuous (the only factor is q itself), makes every non-identity element a generator (no element lands in a weak proper subgroup), and removes cofactor bookkeeping entirely. The Pasta curves below are engineered to have prime order: each group is its own prime-order subgroup, with cofactor 1.

2.6 Instantiation on elliptic curves; the Pasta curves

We now fix the concrete groups the remainder of the volume uses. Recall from the *Math Guide* (§“Elliptic curves”, with the group law in §“The group structure of $E(\mathbb{F}_p)$ ”) that an elliptic curve over $\mathbb{F}_p$, for $p > 3$ prime, in short Weierstrass form is

$$E: y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p, \quad 4a^3 + 27b^2 \neq 0,$$

and that its set of $\mathbb{F}_p$ -points,

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\},$$

equipped with chord-and-tangent addition and the point at infinity $\mathcal{O}$ as identity, forms a finite abelian group. The discrete logarithm problem in this group—given P and $Q = [x]P$, find x —is the *elliptic-curve discrete logarithm problem* (ECDLP), stated in its own habitat in the *Math Guide* (§“The elliptic-curve discrete logarithm problem”); CDH and DDH transcribe verbatim with multiplicative notation replaced by the additive scalar multiplication $x \mapsto [x]P$, giving ECCDH and ECDDH. The group size is pinned down by the Hasse bound, likewise recalled.

Theorem 2.23 (Hasse bound, recalled). *For E over $\mathbb{F}_p$ (Math Guide, §“The group structure of $E(\mathbb{F}_p)$ ”),*

$$|\#E(\mathbb{F}_p) - (p + 1)| \leq 2\sqrt{p}, \quad \text{i.e.} \quad \#E(\mathbb{F}_p) = p + 1 - t, \quad |t| \leq 2\sqrt{p},$$

where t is the trace of Frobenius. The group order is thus $p + 1$ up to a deviation of order $\sqrt{p}$.

The parameter recipe for λ -bit security follows: choose p of about 2λ bits, so that $\#E(\mathbb{F}_p) \approx p \approx 2^{2\lambda}$, and insist by Corollary 2.21 that $\#E(\mathbb{F}_p)$ be prime—cofactor 1. One then assumes ECDLP, EC-CDH, and ECDDH at the level Corollary 2.16 predicts, *provided* the curve avoids the known structural attacks that beat the generic bound. The screening criteria are as follows.

Definition 2.24 (Secure-curve criteria). A curve $E/\mathbb{F}_p$ of prime order $q = \#E(\mathbb{F}_p)$ is admissible for λ -bit DLP security if:

1. (*Size / generic.*) The order satisfies $\log_2 q \geq 2\lambda$, so Pollard rho costs at least 2^λ operations (Corollary 2.16).
2. (*Prime order.*) The order q is prime, so Pohlig–Hellman is vacuous and the cofactor is 1 (Corollary 2.21).
3. (*Anti-MOV / large embedding degree.*) The multiplicative order k of p modulo q —the *embedding degree*—is large. The MOV/Frey–Rück attack uses a pairing to embed $E(\mathbb{F}_p)$ into $\mathbb{F}_{p^k}^\times$ and runs the sub-exponential index calculus of Remark 2.18 there; the threat is real only for small k , as on *supersingular* curves—over a prime field $p > 3$, those with trace $t = 0$, so that $\#E(\mathbb{F}_p) = p + 1$ divides $p^2 - 1$ and $k \leq 2$. A curve with $t \neq 0$ is *ordinary*.
4. (*Anti-Smart / not anomalous.*) The order avoids $q = p$: on an *anomalous* curve, $\#E(\mathbb{F}_p) = p$ with trace $t = 1$, the Smart–Sato–Araki–Semaev attack solves ECDLP in polynomial time.
5. (*Twist security.*) The *quadratic twist* E' —the curve $dy^2 = x^3 + ax + b$ for a fixed nonsquare $d \in \mathbb{F}_p$; every $x \in \mathbb{F}_p$ is the x -coordinate of a point of E or of E' —should also have large prime order, so that computation cannot be pushed onto a weak twist by a fault or an invalid-point injection.

Definition 2.25 (The Pasta curves). The *Pasta* curves are a pair, *Pallas* and *Vesta*, in short Weierstrass form

$$y^2 = x^3 + 5$$

(so $a = 0$, $b = 5$, a j -invariant-0 shape), defined over two 255-bit primes p and q arranged in a cycle:

$$\#Pallas = q \text{ (Pallas defined over } \mathbb{F}_p), \quad \#Vesta = p \text{ (Vesta defined over } \mathbb{F}_q).$$

The order of each curve equals the field of definition of the other. Both primes are congruent to 1 mod 2^{32} , giving their multiplicative groups large 2-adic valuation for fast FFTs, and each curve has cofactor 1: the whole group is of prime order. The primes themselves are assembled digit by digit in the *Math Guide* (§“Pallas and Vesta assembled”), and the field/scalar bookkeeping of the cycle in §“Base fields, scalar fields, and the Pasta cycle” there.

Remark 2.26 (Why a cycle, and what it buys). The defining property— $\#Pallas = q$, the base field of Vesta, and $\#Vesta = p$, the base field of Pallas—makes (Pallas, Vesta) an *amicable*, or cyclic, pair: one curve’s scalar field is the other’s coordinate field. A proof system can therefore use Pallas-arithmetic to verify statements about Vesta-arithmetic and vice versa without non-native field emulation; this is the mechanism behind recursive proof composition in Halo 2, whose recursion alternates between the two curves. The cycle is an *efficiency* property, not a security one: it does not enter the hardness assumptions, and each curve is screened independently against Definition 2.24.

Proposition 2.27 (The Pasta curves against the criteria). *The curves Pallas and Vesta satisfy criteria (1)–(4) of Definition 2.24 at $\lambda \approx 126$. Criterion (5) fails for both curves—neither twist has prime order, and the Pallas twist falls below the SafeCurves 2^{100} twist-attack cost threshold—but the failure is rendered moot by mandatory point validation.*

Proof. Sketch, criterion by criterion.

(1) Each order is a 255-bit prime just above 2^{254} , so $\log_2 q \approx 254.0$ and the bare generic cost is $\sqrt{q} \approx 2^{127.0}$. Plain Pollard rho expects about $\sqrt{\pi q/2} \approx 2^{127.3}$ operations; the $\pm$ automorphism speedup of Remark 2.14 lowers this to $\sqrt{\pi q/4} \approx 2^{126.8}$; and the order-6 automorphism group of these j -invariant-0 curves—the factor $\sqrt{m}$ for an order- m automorphism group of Remark 2.28, here an extra $\sqrt{3}$ —lowers it to $\sqrt{\pi q/12} \approx 2^{126.0}$. Hence “about 126 bits” of security, the same range as the 128-bit curves of Example 2.17.

(2) Both orders are prime by construction: cofactor 1, Pohlig–Hellman vacuous.

(3) Both curves are ordinary: the trace $t = p + 1 - q$ is odd, so $t \neq 0$. The embedding degree k of each is enormous. For Pallas, $q - 1 = 2^{32} \cdot 3^2 \cdot 1709 \cdot 24859 \cdot C$ with C a 194-bit composite, and $p^{(q-1)/32} \not\equiv 1$ and $p^{(q-1)/\ell} \not\equiv 1 \pmod{q}$ for $\ell \in \{3, 1709, 24859\}$, so the order k of p modulo q is a multiple of $2^{28} \cdot 3^2 \cdot 1709 \cdot 24859 > 2^{56}$; for Vesta, $p - 1 = 2^{32} \cdot 3 \cdot 463 \cdot C'$, and $q^{(p-1)/4} \not\equiv 1$, $q^{(p-1)/\ell} \not\equiv 1 \pmod{p}$ for $\ell \in \{3, 463\}$ make k a multiple of $2^{31} \cdot 3 \cdot 463 > 2^{41}$. Either extension field is astronomically beyond any feasible index-calculus target; MOV/Frey–Rück does not apply.

(4) Neither curve is anomalous: $\#Pallas = q \neq p$ and $\#Vesta = p \neq q$, so the trace $t = p + 1 - q \neq 1$ and the Smart attack does not apply.

(5) Twist security is the criterion the pair does not meet. The Pasta search deliberately ignored it—the curves were generated with the search flag `--ignoretwist` of the `amicable.sage` search recorded in the README of the `zcash/pasta` repository—and neither twist has prime order. Each $x \in \mathbb{F}_p$ supplies two affine points in total, on E or on E' (one on each when $x^3 + b = 0$), so $\#E + \#E' = 2(p + 1)$ and $\#E' = 2(p + 1) - \#E = p + 1 + t$; hence $\#Pallas' = 2(p + 1) - q$ and $\#Vesta' = 2(q + 1) - p$,

which factor as

$$\begin{aligned}\#Pallas' &= 3^2 \cdot 7 \cdot 8191 \cdot 85021 \cdot 11540602760389 \cdot P_{176}, \\ \#Vesta' &= 3^3 \cdot 7 \cdot 13 \cdot 2851 \cdot 30097 \cdot P_{217},\end{aligned}$$

with P_{176} a 176-bit prime and P_{217} a 217-bit prime. The best attack on the Pallas twist therefore costs about $\sqrt{P_{176}} \approx 2^{87.6}$, below the SafeCurves 2^{100} twist-security threshold; the Vesta twist clears the threshold at about $2^{108.2}$. The failure is moot in our setting: twist attacks require an implementation that accepts an attacker-supplied x -coordinate without validation (an x -only ladder) or that skips its on-curve checks, and the deployed arithmetic does neither. The `pasta_curves` crate carries points in full coordinates and has no x -only code path; its deserialisation (`from_bytes` in `src/curves.rs`) reconstructs y by taking the square root of $x^3 + 5$, failing exactly when x lies on the twist, where $x^3 + 5$ is a non-square, and even `from_bytes_unchecked` delegates to the checked path because a compressed encoding cannot skip the curve check, while points built from raw coordinates are gated on `is_on_curve`. This is the behaviour protocol specification § 5.4.9.6 mandates: its decoding map returns $\perp$ (failure) when $x^3 + b$ has no square root, and an implementation must not assume that the root exists or that the encoding lies on the curve. Orchard deserialises its points through this routine (e.g. the `orchard` crate, `src/value.rs`, `ValueCommitment::from_bytes`, and the key material of `src/keys.rs`), and cofactor 1 leaves no small subgroup on the curve itself. Invalid-curve and twist-point injection are therefore excluded by *validation*, not by twist structure.

The standard generic analysis governs, and the security level is about 126 bits. $\square$

Remark 2.28 (j -invariant 0 and endomorphisms). The shape $y^2 = x^3 + b$ gives j -invariant 0, and with it an extra automorphism of order 3 (*Math Guide*, §“The j -invariant and the GLV endomorphism”): for ω a primitive cube root of unity in $\mathbb{F}_p$, the cheap endomorphism $\phi(x, y) = (\omega x, y)$ acts on the group as multiplication by a fixed scalar ζ with $\zeta^2 + \zeta + 1 \equiv 0 \pmod{q}$ (the λ of the *Math Guide*’s proposition), enabling the GLV speedup for honest scalar multiplication. The same endomorphism hands the attacker a slightly larger automorphism group, and with it the constant-factor Pollard-rho speedup used in Proposition 2.27—a factor $\sqrt{m}$ for an order- m automorphism group—but that is a small constant and does not change the $\Theta(\sqrt{q})$ scaling. Beyond that constant, the extra automorphism is not known to weaken ECDLP; the danger lies only in small embedding degree and in anomalousness, both excluded above.

2.7 The quantum caveat: Shor’s algorithm

Every assumption above holds the adversary to classical PPT computation. Against a large fault-tolerant quantum computer the picture changes qualitatively, and honesty requires stating exactly what breaks and what survives.

Theorem 2.29 (Shor, informal; stated without proof). *There is a quantum algorithm that, given a cyclic group $G = \langle g \rangle$ of order q with efficient group operations and a target $h = g^x$, outputs x in time polynomial in $\log q$, succeeding with overwhelming probability. Consequently DLP—and with it CDH and DDH—falls in quantum polynomial time in any group, elliptic-curve groups included.*

We do not prove the theorem; the shape of the algorithm is nevertheless worth recording. Discrete logarithm is an instance of the *hidden subgroup problem* over $(\mathbb{Z}/q\mathbb{Z})^2$. The function $f(\alpha, \beta) = g^\alpha h^{-\beta} = g^{\alpha - x\beta}$ is constant exactly on the cosets of

$$H = \{(\alpha, \beta) : \alpha \equiv x\beta \pmod{q}\} = \langle (x, 1) \rangle \subseteq (\mathbb{Z}/q\mathbb{Z})^2,$$

a line of slope x . The algorithm prepares a uniform superposition over $(\mathbb{Z}/q\mathbb{Z})^2$, evaluates f into a second register, and applies the quantum Fourier transform—the reversible change of basis into the characters of $(\mathbb{Z}/q\mathbb{Z})^2$, the homomorphisms $\chi_{u,v}(\alpha, \beta) = e^{2\pi i(u\alpha + v\beta)/q}$ into the unit circle; measurement

then samples characters trivial on H , i.e. pairs (u, v) with $ux + v \equiv 0 \pmod{q}$, and a handful of samples reveal the slope x by classical linear algebra modulo q . The whole procedure uses $O(\text{poly}(\log q))$ qubits and gates, and it touches the group only through the black-box map $(\alpha, \beta) \mapsto g^\alpha h^{-\beta}$, so it is agnostic to representation: elliptic-curve DLP falls exactly as $\mathbb{F}_p^\times$ DLP does, and integer factoring falls to the order-finding variant.

Remark 2.30 (No contradiction with the square-root barrier). Shoup’s lower bound (Theorem 2.15) assumes the generic model for *classical* algorithms making sequential, individual oracle queries. Shor’s algorithm is non-generic in exactly the relevant sense: it queries the group function in superposition and exploits the periodicity the Fourier transform exposes, abilities the classical generic model does not grant. The two results coexist: the barrier bounds classical generic attackers; Shor is a quantum, non-generic attacker.

Remark 2.31 (What Shor breaks and what survives). Shor’s algorithm breaks the entire discrete-log family and integer factoring in polynomial time: RSA, finite-field Diffie–Hellman, and elliptic-curve cryptography are all quantum-broken. What it does *not* break:

- *Symmetric primitives and hash functions.* Grover’s algorithm searches a space of size 2^n in about $2^{n/2}$ quantum queries—a quadratic, not exponential, speedup—so a cipher with a 2λ -bit key, or an $n = 2\lambda$ -bit hash against preimages, retains λ bits of *quantum* security. Generic quantum collision finding instead costs $\Theta(2^{n/3})$ queries, so in the query metric an output of about 3λ bits is needed for λ -bit quantum collision security—though the algorithm needs comparably much quantum memory, and whether it beats classical parallel collision search in realistic cost models is debated. Doubling therefore suffices for key search and preimages, but for collisions only under that contested cost reading.
- *Post-quantum assumptions.* Hardness assumptions based on neither discrete logarithms nor factoring are not known to fall to Shor and underlie post-quantum cryptography.

The discrete-log assumptions of this volume are therefore adopted under the standing premise that no cryptographically relevant quantum computer exists. Within that premise, and against all classical attacks, the Pasta curves deliver the approximately 126-bit security of Proposition 2.27—the ground on which the Halo 2 constructions to come are built.

2.8 Standing assumptions

Remark 2.32 (Standing assumptions for the sequel). We collect the premises the rest of the volume relies on. For each Pasta curve $C \in \{\text{Pallas}, \text{Vesta}\}$ with prime order q_C and generator g_C :

1. ECDLP is hard in $\langle g_C \rangle$: no classical PPT adversary finds x from $[x]g_C$ with non-negligible probability—concretely, every known attack costs at least 2^{126} operations (Proposition 2.27).
2. ECCDH and, where needed, ECDDH hold in $\langle g_C \rangle$ (Definitions 2.4 and 2.5, transcribed additively).
3. The adversary is classical and probabilistic polynomial-time (Remark 2.31).

By the hierarchy of §2.3, assuming ECDDH is the strongest commitment and implies the rest; each later construction invokes, by name, the weakest assumption it requires.

3 Hash functions and the random oracle model

The adversary of this section forges by *coincidence*. She prepares two contracts—one benign, one ruinous—whose digests agree, has the benign one signed, and produces the ruinous one in court; or she takes a digest minted for one purpose, say a key-derivation output, and replays it in another role, say a commitment, exploiting the fact that both are interchangeable bit strings; or she simply walks a

pseudo-random path through the digest space until, square-root-of-the-space steps later, two inputs collide. Denying her these coincidences is the business of the hash function: a function compressing arbitrary data into a short fixed-length tag. It underlies digital signatures, commitment schemes, message authentication, key derivation, proof-of-work, Merkle trees, and the Fiat–Shamir transform behind Halo 2’s non-interactive arguments—so many purposes for one primitive because an *ideal* hash acts like a deterministic source of fresh randomness keyed by its input. This section builds the primitive from the ground up: its syntax; the three classical security notions and the generic birthday attack that bounds them; the random oracle model; domain separation; hashing into a finite field and onto an elliptic curve; the arithmetisation-friendly hash Poseidon that Orchard evaluates inside its circuit; and finally a register of the transparent layer’s SHA-256-derived hashes.

3.1 Syntax

Notation for strings is fixed once. The set of all finite binary strings is $\{0, 1\}^*$; the strings of length exactly n form $\{0, 1\}^n$; the length of a string x is $|x|$; and concatenation is written $x \parallel y$.

Definition 3.1 (Hash function). A *hash function* with output length $n \in \mathbb{N}$ is a function

$$H : \{0, 1\}^* \longrightarrow \{0, 1\}^n.$$

Elements of the codomain are called *digests*, *hashes*, or *tags*. The function is *compressing* on a domain $D \subseteq \{0, 1\}^*$ if D contains strings longer than n ; when the domain is all of $\{0, 1\}^*$, the function is overwhelmingly many-to-one.

A single fixed function such as SHA-256 is what practice deploys, but a foundational wrinkle appears the moment one tries to say what its security *means*. For any fixed compressing H there exist inputs $x \neq x'$ with $H(x) = H(x')$ —the pigeonhole principle guarantees it—and therefore a tiny program that simply outputs one such hardcoded pair also exists. The asymptotic statement “no efficient adversary finds a collision” is consequently *false* for every fixed compressing function: the hardcoding program is efficient and always succeeds. (Writing that program down is another matter—nobody knows a collision for SHA-256—but existence suffices to falsify the universally quantified claim.) The standard remedy indexes the function by a key.

Definition 3.2 (Keyed hash family). A *keyed hash family* (or *hash function ensemble*) is a function

$$H : \mathcal{K} \times \{0, 1\}^* \longrightarrow \{0, 1\}^n,$$

where $\mathcal{K}$ is a finite key space. A key $s \in \mathcal{K}$ is sampled *publicly* at random, fixed, and we write $H_s(\cdot) := H(s, \cdot)$ for the family member in force.

The key is *not* secret. Its role is to index which member of the family the adversary must attack: a hardcoded collision for one member is useless against a freshly sampled other, so “no efficient adversary finds a collision for a random member” is a meaningful asymptotic statement.

Remark 3.3 (Convention: keyed definitions, unkeyed deployments). The convention of this volume follows the standard resolution of the mismatch between theory and practice. Security notions are stated for the keyed family, where the definitions are clean; every concrete construction we deploy is unkeyed; and the key is mentally identified with “the choice of standard”—the sampling of s happened, once and in public, when the function was designed. No formal theorem bridges the two readings; the gap is the same one the random oracle model of §3.4 inhabits, and we flag it rather than hide it.

Definition 3.4 (Extendable-output function). An *extendable-output function* (XOF) is a primitive producing a digest of caller-chosen length: syntactically a function

$$H : \{0, 1\}^* \times \mathbb{N} \longrightarrow \{0, 1\}^*, \quad |H(x, \ell)| = \ell,$$

whose outputs are *prefix-consistent*: for $\ell \leq \ell'$, the string $H(x, \ell)$ is a prefix of $H(x, \ell')$, so a longer request extends a shorter one as a stream.

The canonical XOFs are SHAKE128 and SHAKE256 from the SHA-3 family. An XOF is the natural tool wherever a *controlled number* of uniform bits is needed rather than a fixed digest—above all when hashing into a finite field or onto a curve, where the output must be reduced modulo a prime; §3.6 returns to this.

3.2 Security notions: preimage, second-preimage, and collision resistance

Each of the three classical security notions is a game between a challenger and a PPT adversary $\mathcal{A}$, in the template of Definition 1.15; the keyed family H is secure in a given sense if every PPT adversary wins the corresponding game with probability negligible in the security parameter λ . All sizes are functions of λ : the output length $n = n(\lambda)$, the key length, and a polynomially bounded challenge input length $m = m(\lambda)$ from which the games sample their random challenge $x \xleftarrow{\$} \{0, 1\}^m$. Recall that a function $\varepsilon(\lambda)$ is negligible, $\varepsilon \in \text{negl}(\lambda)$, if it decays faster than the reciprocal of every polynomial (*Math Guide*, §“Polynomial, exponential, and negligible functions”; recalled in §1.2). The three notions form a strict hierarchy, and no single definition of a “good” hash exists; which notion an application needs is a per-application question that Remark 3.10 takes up.

Definition 3.5 (Preimage resistance). The game $\text{PRE}_H^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$ and $x \xleftarrow{\$} \{0, 1\}^m$, computes $y := H_s(x)$, and sends (s, y) to the adversary.
2. The adversary outputs $x' \in \{0, 1\}^*$.
3. The game outputs 1 iff $H_s(x') = y$.

The family H is *preimage resistant* (or *one-way*) if for every PPT $\mathcal{A}$,

$$\text{Adv}_H^{\text{pre}}(\mathcal{A}, \lambda) := \Pr[\text{PRE}_H^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

Note the winning condition: the adversary, given the key and the digest of a random message, must output *any* string hashing to y . Recovering the original x is not required; any preimage wins.

Definition 3.6 (Second-preimage resistance). The game $\text{SEC}_H^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$ and $x \xleftarrow{\$} \{0, 1\}^m$, and sends (s, x) to the adversary.
2. The adversary outputs $x' \in \{0, 1\}^*$.
3. The game outputs 1 iff $x' \neq x$ and $H_s(x') = H_s(x)$.

The family H is *second-preimage resistant* if for every PPT $\mathcal{A}$,

$$\text{Adv}_H^{\text{sec}}(\mathcal{A}, \lambda) := \Pr[\text{SEC}_H^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

Here the challenger gives the adversary a specific random message; the adversary must produce a different message colliding with it.

Definition 3.7 (Collision resistance). The game $\text{COL}_H^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$ and sends s to the adversary.
2. The adversary outputs a pair (x, x') .
3. The game outputs 1 iff $x \neq x'$ and $H_s(x) = H_s(x')$.

The family H is *collision resistant* if for every PPT $\mathcal{A}$,

$$\text{Adv}_H^{\text{col}}(\mathcal{A}, \lambda) := \Pr[\text{COL}_H^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

The three games differ in *who chooses what*. In the preimage game the challenger chooses the message and reveals only its digest; in the second-preimage game the challenger chooses the message and reveals it; in the collision game the adversary chooses *both* messages and need only make them collide. More adversarial choice means an easier game and hence a stronger security demand on the function; collision resistance, granting the most choice, is the strongest of the three.

Proposition 3.8 (Hierarchy of notions). *Let H be a keyed hash family that is compressing, with challenge input length $m > n$. Then:*

1. *collision resistance implies second-preimage resistance, with no loss; and*
2. *second-preimage resistance implies preimage resistance up to an additive loss of 2^{n-m} , which is negligible—so the implication holds outright—whenever H is sufficiently compressing, $m - n = \omega(\log \lambda)$.*

Neither implication reverses in general.

Proof. (1) From an adversary $\mathcal{A}$ against second-preimage resistance, construct $\mathcal{B}$ against collision resistance: on input the key s , the reduction $\mathcal{B}$ samples $x \xleftarrow{\$} \{0, 1\}^m$ itself, runs $x' \leftarrow \mathcal{A}(s, x)$ —the simulated view is exactly that of SEC, since $\mathcal{B}$ samples x from the same distribution as the challenger—and outputs the pair (x, x') . Whenever $\mathcal{A}$ succeeds, $x' \neq x$ and $H_s(x') = H_s(x)$, which is precisely a collision. Hence $\text{Adv}_H^{\text{col}}(\mathcal{B}, \lambda) \geq \text{Adv}_H^{\text{sec}}(\mathcal{A}, \lambda)$: the reduction is tight, and if the left side is negligible so is the right.

(2) Toward the contrapositive, let $\mathcal{A}$ be an inverter; we build a second-preimage finder $\mathcal{B}$. On input (s, x) , the reduction computes $y := H_s(x)$, runs $x' \leftarrow \mathcal{A}(s, y)$ —again a perfect simulation of PRE, since x is uniform and y is its digest—and outputs x' . Whenever $\mathcal{A}$ succeeds, $H_s(x') = y = H_s(x)$; what remains is to argue $x' \neq x$ with good probability. Fix $\mathcal{A}$'s coins, so that its output on each digest is a single string $x' = \mathcal{A}(s, y)$. An input x can satisfy $x = \mathcal{A}(s, H_s(x))$ only if x is *the* output assigned to its own digest, so each of the 2^n digests contributes at most one such input, and at most 2^n inputs in all are “fixed points” of the composed map. Over uniform $x \xleftarrow{\$} \{0, 1\}^m$, therefore, $\Pr[x' = x] \leq 2^n/2^m = 2^{n-m}$; averaging over $\mathcal{A}$'s coins preserves the bound. Hence

$$\text{Adv}_H^{\text{sec}}(\mathcal{B}, \lambda) \geq \text{Adv}_H^{\text{pre}}(\mathcal{A}, \lambda) - 2^{n-m}.$$

The loss is *additive*, not multiplicative: nothing bounds $\Pr[x' \neq x]$ conditioned on success, because $\mathcal{A}$'s successes might concentrate on small fibres where returning x itself is likely. Sufficient compression, $m - n = \omega(\log \lambda)$, makes 2^{n-m} negligible and the implication unconditional.

Non-reversal is witnessed by Example 3.9 below. $\square$

Example 3.9 (Preimage-resistant but not collision-resistant). Take any one-way family H and define $H'_s(x) := H_s(\bar{x})$, where $\bar{x}$ deletes the last bit of x . The family H' inherits one-wayness: a preimage challenge (s, y) for H is distributed exactly as one for H' at one bit greater challenge length, and an H' -inverter's output x'' satisfies $H_s(\overline{x''}) = y$, so deleting its last bit hands the reduction a preimage under H . Yet the pair $x \parallel 0, x \parallel 1$ is an immediate collision for every x , found without any computation. Hence preimage resistance does not imply collision resistance. The other separations are constructed similarly; the implications of Proposition 3.8 are genuinely one-directional.

Remark 3.10 (Which application needs which notion). Password storage—keeping a digest of each password so that a stolen table does not reveal them—is the natural home of *preimage* resistance: the thief holds digests, and any preimage lets her log in. (The notion alone is not the whole story there: passwords are guessable, so deployments hash each together with a unique random string, its *salt*, through a deliberately slow function, against offline guessing.) A signature scheme that signs $H(m)$, where an adversary may later try to substitute a forged document for the honestly chosen m , needs *second-preimage* resistance: the message is fixed and honest, and the attacker must collide with it. A scheme in which the *signer herself* may cheat—preparing two contracts with the same digest, signing the benign one, presenting the malicious one—needs full *collision* resistance, because there the adversary controls both messages. Commitment schemes, Merkle trees, and the Fiat–Shamir transform all fall into this last category, which is why collision resistance is the headline property of a cryptographic hash.

3.3 The birthday bound on generic collision finding

How hard can collision finding be? The generic attack—evaluate the hash on many inputs and wait for a coincidence—succeeds far sooner than intuition suggests, and its cost is governed by the birthday calculation of the *Math Guide*.

Lemma 3.11 (Birthday bound for hashing). *Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ and $N = 2^n$. Evaluate H on q distinct inputs, in the idealised model where the q digests behave as independent uniform samples from $\{0, 1\}^n$, and let C be the event that two digests coincide. Then*

$$\Pr[C] = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right),$$

with the two-sided estimate

$$1 - e^{-q(q-1)/(2N)} \leq \Pr[C] \leq \frac{q(q-1)}{2N} \leq \frac{q^2}{2N},$$

and the lower bound gives $\Pr[C] \geq 1/2$ once $q \geq 1.18\sqrt{N}$. A collision therefore appears with constant probability after about $\sqrt{N} = 2^{n/2}$ evaluations.

Proof. The exact product and both estimates are the birthday-bound proposition of the *Math Guide* (§“The union bound and a birthday calculation”), instantiated with set size $N = 2^n$ and sample count $k = q$; we do not re-derive them. Only the half-probability threshold is new. Setting the exponent of the lower bound equal to $-\ln 2$ gives $q(q-1) = 2N \ln 2$, i.e.

$$q \approx \sqrt{2 \ln 2} \sqrt{N} \approx 1.18\sqrt{N},$$

the constant being $\sqrt{2 \ln 2} = 1.1774 \dots$; for $q \geq 1.18\sqrt{N}$ the exponential falls below $1/2$ and $\Pr[C] \geq 1/2$. $\square$

Remark 3.12 (Reading the birthday bound). Three consequences deserve emphasis.

The square-root law. An n -bit digest yields only $n/2$ bits of collision security in the sense of Definition 1.32. For the conventional 128-bit collision security one must take $n = 256$ —which is why SHA-256 (and not “SHA-128”, which does not exist) is the workhorse digest size, and why the transcript hashes of Fiat–Shamir-style protocols must carry at least 256-bit output.

The attack is practical. The naive attack stores all $q \approx 2^{n/2}$ digests and sorts— $2^{n/2}$ memory, the same obstruction that made baby-step giant-step unrealistic in Remark 2.12. But collision finding, like the discrete-logarithm walk of Proposition 2.13, admits cycle finding: iterate $x \mapsto H(x)$ and detect the cycle with Floyd’s or Brent’s algorithm, in the manner of Pollard’s rho, or run the van Oorschot–Wiener parallel collision search across many machines. These achieve the same $2^{n/2}$ time in essentially constant memory, so the birthday attack is a genuine budget item, not a thought experiment.

Preimages cost the square. The best generic attack against preimage and second-preimage resistance is brute force: about 2^n evaluations to hit a prescribed digest. Those notions therefore enjoy full n -bit generic security—twice the collision level—because the attacker cannot exploit birthday coincidences among self-chosen pairs; the collision game is the only one that pays the square-root discount.

3.4 The random oracle model

The model section introduced the random oracle model as one of the two idealisations under which proofs proceed (Definition 1.26), stated its two powers informally (Remark 1.27), and fixed its status as a heuristic with known limits (Remark 1.29). With hash syntax now in hand we can define the idealised object precisely and *prove* the powers.

Definition 3.13 (Random oracle). A *random oracle* with output length n is a function $\mathcal{O} : \{0, 1\}^* \rightarrow \{0, 1\}^n$ drawn uniformly at random from the set of all such functions. Equivalently—and this is the form every proof uses—the oracle is realised *lazily*: a table, initially empty, is maintained; on query x , if x is in the table the stored value is returned, and otherwise a fresh $\mathcal{O}(x) \xleftarrow{\$} \{0, 1\}^n$ is sampled, recorded, and returned.

Three features characterise the object: outputs on distinct inputs are independent and uniform; outputs on repeated inputs are consistent; and access is *only* by querying—the oracle has no description shorter than its (exponentially long) truth table. A scheme lives in the random oracle model (ROM) when every party, honest or adversarial, interacts with a single shared $\mathcal{O}$ in place of a concrete hash, and security is proved relative to the random choice of $\mathcal{O}$; on deployment a concrete hash—SHA-256, say, or BLAKE2b—*instantiates* the oracle.

Proposition 3.14 (The two powers of the ROM). *In a ROM security proof, the reduction, which simulates $\mathcal{O}$ for the adversary, gains two capabilities that no concrete hash affords.*

1. *Observability (extractability).* The reduction sees every input the adversary queries. Since querying is the only way to learn anything about $\mathcal{O}(x)$, if the adversary’s output depends on $\mathcal{O}(x)$ then the reduction holds x .
2. *Programmability.* On a not-yet-queried input x , the reduction may choose the response $\mathcal{O}(x)$ to be any value of its liking, provided the value is distributed uniformly; the simulation remains

perfect, and the reduction can thereby plant a challenge—a value it must invert, say—inside an oracle answer.

Proof. Both capabilities are immediate from the lazy realisation of Definition 3.13, because in the simulation the reduction is the table-keeper. Every query arrives at the reduction before an answer exists, which is observability. And on a fresh input the honest table-keeper would sample the answer uniformly and independently of the adversary’s view so far; a reduction that instead inserts any value *with that same distribution* produces a view identical to the honest one, while the table keeps repeated queries consistent. That is programmability. $\square$

Remark 3.15 (Why Fiat–Shamir needs the ROM). The paradigmatic consumer of both powers at once is the Fiat–Shamir transform—the compiler that renders Halo 2’s interactive protocol non-interactive, developed in this volume under “From identification to signature via the Fiat–Shamir transform” and in full generality under “The Fiat–Shamir transform: from interactive to non-interactive”. The transform replaces the verifier’s random challenge e by $e = \mathcal{O}(\text{transcript so far})$. Soundness of the resulting non-interactive argument is proved by *rewinding*: the reduction observes (capability 1) the transcript the prover commits to, then *reprograms* (capability 2) the oracle to return a fresh independent challenge on that same transcript, extracting two accepting transcripts that share a prefix and, from them, a witness—the forking technique previewed in Remark 1.24. Neither step is available against a concrete hash, whose outputs the reduction can neither observe selectively nor rewrite. This is the proof that lives *only* in the ROM, and the reason the model is indispensable to the cryptography this monograph targets.

3.5 Domain separation and personalisation

One physical hash serves many purposes in a deployed protocol: it derives keys, commits to notes, tags spent notes, seeds Fiat–Shamir challenges. Digests, however, are interchangeable bit strings, and an output produced for one purpose can be *replayed* as another unless the purposes are cryptographically insulated. Domain separation is the discipline that insulates them.

Definition 3.16 (Domain separation). Let H be a hash modelled as a random oracle, and let $\{\tau_i\}_{i \in I}$ be a set of distinct, *prefix-free* domain tags (also called labels or personalisation strings), one per intended use: no τ_i is a prefix of another—as when all tags have equal length, or when each is length-prefixed. The *use- i hash* is

$$H^{(i)}(x) := H(\tau_i \parallel x).$$

Proposition 3.17 (Domain separation yields independent oracles). *If H is a random oracle and the tags $\{\tau_i\}$ are prefix-free, then the family $\{H^{(i)}\}_{i \in I}$ is distributed exactly as a collection of independent random oracles. Consequently a digest obtained from $H^{(i)}$ is—except with the negligible probability of a generic collision—useless against any use $j \neq i$.*

Proof. Prefix-freeness makes the domains $\tau_i \parallel \{0, 1\}^*$ pairwise disjoint: a string with prefix τ_i cannot also have prefix τ_j unless one tag is a prefix of the other. Inputs queried through different $H^{(i)}$ are therefore never equal. A random oracle assigns independent uniform values to distinct inputs, so restricting H to the disjoint slices yields, per slice, a uniformly random function, independent across slices—precisely a collection of independent random oracles. The replay conclusion follows: the value $H^{(i)}(x)$ reveals nothing about any $H^{(j)}(\cdot)$ with $j \neq i$. $\square$

Three idioms implement the discipline in deployed practice; Zcash uses all three.

1. *Context tagging.* Bind an ASCII string naming the use into every call—prepended, as in Definition 3.16, or in any framing that keeps the tag unambiguously delimited from the data. The Orchard tag inventory is of this kind: the strings `z.cash:Orchard-cv`, `z.cash:Orchard-NoteCommit`, and `z.cash:Orchard-CommitIvk` separate the value commitment, note commitment, and key commitment uses of one group-hash (orchard crate, `src/constants/fixed_bases.rs`; protocol specification §§ 5.4.8.3–5.4.8.4), and `z.cash:Orchard-gd` separates diversified key generation (`src/constants.rs`; protocol specification § 5.4.1.6). The deployed framing mirrors the definition’s rather than copying it: the curve hash of §3.7 *appends* its domain tag after the message, closed by a byte carrying the tag’s length (`pasta_curves` crate, `src/hashtocurve.rs`; protocol specification § 5.4.9.8); the length byte makes the tag self-delimiting, so distinct tags still induce disjoint effective inputs—all the proof of Proposition 3.17 uses. The tags reach that hash by two routes, and the commitment tags by both: `-cv` and `-gd` serve directly as its domain tag, while each commitment tag, suffixed `-M`, is hashed as the *message* under the fixed domain tag `z.cash:SinsemillaQ` to derive one of the commitment’s two base points and, suffixed `-r`, serves directly as the domain tag deriving the other (`sinsemilla` crate, `src/lib.rs`, `CommitDomain::new`; consumed by the orchard crate’s `src/note/commitment.rs` and `src/spec.rs`; protocol specification §§ 5.4.1.9 and 5.4.8.4).
2. *Native personalisation.* BLAKE2b’s parameter block carries a dedicated 16-byte personalisation field that is mixed into the initialisation vector, so distinct personalisations separate the primitive’s effective inputs before a single message byte is processed—the field for which Zcash favours BLAKE2 for its oracles. With BLAKE2b modelled as a random oracle of the pair (personalisation, message), the fixed-length field is a prefix-free tag and Proposition 3.17 makes the resulting domains independent; a concrete BLAKE2b instantiation does not make that independence a fact “by construction”. The deployed personalisations include `Zcash_ExpandSeed` for key expansion (`zcash_spec` crate, `src/prf_expand.rs`; protocol specification § 5.4.2), `Zcash_OrchardKDF` for the note-encryption key-derivation function (KDF, the object of §6.6; orchard crate, `src/keys.rs`; protocol specification § 5.4.5.6), and `Zcash_Orchardock` for the outgoing cipher key (orchard crate, `src/note_encryption.rs`; protocol specification § 5.4.2)—all exactly 16 bytes.
3. *Suffix and framing bits in the SHA-3 family.* FIPS 202 reserves domain-separation suffix bits appended to the message before padding: 01 for SHA-3 hashing and 1111 for the SHAKE XOFs. SHA3-256 and SHAKE256 share the same underlying permutation, yet the framed inputs differ, so their effective inputs are disjoint—though their finite digests can still coincide by chance.

The unifying requirement behind all three idioms is that distinct uses induce *disjoint effective inputs* to the underlying primitive. An ad hoc tag that happens to be a prefix of another silently merges two domains and reopens the replay attack—which is why Definition 3.16 demands prefix-freeness rather than mere distinctness. The Merkle trees of the final section apply the same discipline *within* a single structure, tagging tree layers to foreclose a forgery that confuses leaves with internal nodes; see “Layer and domain separation” there.

3.6 Hashing to a field element

The primitives ahead—commitments over the Pasta curves, Fiat–Shamir challenges, key derivations—repeatedly need to hash a message *to a field element*: a near-uniform element of $\mathbb{F}_p$ rather than a bit string. The obstacle is arithmetic: a hash emits uniform *bits*, and reducing n uniform bits modulo a prime p is not uniform on $\mathbb{F}_p$, because 2^n is not a multiple of p —some residues receive one more preimage than others. The remedy is to hash *wide* and reduce.

Construction 3.18 (Expand-then-reduce). Let p be a prime of $L = \lceil \log_2 p \rceil$ bits, and fix a XOF, or a domain-separated hash, expand, producing arbitrary-length near-uniform output. To map a message m to $\mathbb{F}_p$, with a domain tag τ and a security slack of k bits (commonly $k = 128$):

1. compute $b \leftarrow \text{expand}(\tau \parallel m, L + k)$, a string of $L + k$ near-uniform bits;
2. interpret b as an integer $B \in \{0, \dots, 2^{L+k} - 1\}$ and output $B \bmod p \in \mathbb{F}_p$.

Proposition 3.19 (Near-uniformity of expand-then-reduce). *If expand is a random oracle, the output of Construction 3.18 lies within statistical distance (Definition 1.6) at most*

$$\frac{p}{2^{L+k+1}} \leq 2^{-k}$$

of the uniform distribution on $\mathbb{F}_p$. With $k = 128$, no algorithm whatsoever—efficient or not—distinguishes the output from a uniform field element with advantage better than 2^{-128} .

Proof. The integer B is uniform on $\{0, \dots, K - 1\}$ with $K = 2^{L+k}$, and the bias of reducing a uniform B modulo p is exactly the situation of the modular-reduction proposition of the *Math Guide* (§“Uniform sampling and the bias of modular reduction”): each residue’s probability differs from $1/p$ by at most $1/K$, because the preimage counts $\lfloor K/p \rfloor$ and $\lceil K/p \rceil$ differ by one, and summing over the p residues gives statistical distance at most $p/(2K) = p/2^{L+k+1}$ —the sharper form proved there. Since $p < 2^L$, this is less than $2^{-(k+1)} \leq 2^{-k}$. The final claim is the variational characterisation of statistical distance (Proposition 1.7): no decision procedure exceeds the distance. $\square$

The slack is not optional. With $k = 0$ —a single L -bit block reduced modulo p —the bias can be macroscopic: for p just above a power of two, $K = 2^L$ is barely twice p , the residues beyond the wrap-around point receive only one preimage while the rest receive two, and each deprived residue carries as little as half its uniform mass. The k extra bits are exactly what drives this imbalance to negligibility.

Deployed practice packages the construction. The IETF hash-to-curve specification (RFC 9380) defines `hash_to_field( $m$ ,  $count$ )`, producing $count$ field elements by calling an expansion routine—`expand_message_xmd` over a fixed-output hash, or `expand_message_xof` over a XOF—with a *mandatory* domain-separation tag, and reducing each chunk of

$$L_{\text{RFC}} = \left\lceil (\lceil \log_2 p \rceil + k) / 8 \right\rceil \text{ bytes}$$

modulo p , where k is the suite’s target security level in bits. The RFC’s own suites for 255-bit fields—the `curve25519` family—set $k = 128$, hence 48-byte chunks. The Zcash suite for Pallas and Vesta is more generous: it takes $k = 256$, reducing each element from a full 64-byte BLAKE2b-512 digest, so the slack of Proposition 3.19 is realised with room to spare (`pasta_curves` crate, `src/hashtocurve.rs`: the `hash_to_field` routine expands with BLAKE2b in the XMD pattern under the suite tag `{tag}-{curve}_XMD:BLAKE2b_SSWU_RO_` and reduces two 64-byte chunks each via `from_uniform_bytes`; protocol specification § 5.4.9.8). This is the concrete recipe for mapping identities to field elements, and for producing the field elements that the next subsection feeds to a curve. Halo 2’s deployed Fiat–Shamir transcript uses the same wide-output-then-reduce principle: each challenge is the 64-byte digest of a BLAKE2b state personalised with `Halo2-Transcript`, reduced by `from_uniform_bytes` (`halo2_proofs` crate, `src/transcript.rs`); the order in which that state absorbs messages and squeezes challenges is the *Halo 2 Guide*’s, §“The complete Halo 2 protocol”.

3.7 Hashing to a curve point

The volume ahead also needs to hash *onto an elliptic curve*: given a message, produce a point of $E(\mathbb{F}_p)$ that behaves like a uniform group element—the raw material for commitment generators whose discrete logarithms nobody knows. Two requirements pull against each other. The output should be near-uniform in the relevant subgroup, so that the map can stand in for a random oracle into the group in security arguments; and the map must run in *constant time*, because its input is often secret-adjacent.

The naive approach fails the second requirement instructively. Hash the message to a candidate $x \in \mathbb{F}_p$ (Construction 3.18); test whether $g(x) = x^3 + ax + b$ is a quadratic residue, so that a y -coordinate exists; if not, increment x and retry. This *try-and-increment* loop is correct and roughly uniform, but the number of trials depends on the message: about half of all x fail the residue test, since the squares are half of $\mathbb{F}_p^\times$ (*Math Guide*, §“Quadratic residues and the Euler criterion”, whose Legendre symbol $\left(\frac{g(x)}{p}\right)$ is exactly the test), so the running time leaks information about m —an avenue for side-channel attacks. Constant-time hashing to a curve requires a map that succeeds on the *first* try, for every input.

Construction 3.20 (Simplified Shallue–van de Woestijne–Ulas map). Let $E: y^2 = g(x) = x^3 + ax + b$ over $\mathbb{F}_p$ with $ab \neq 0$, and choose, as RFC 9380 requires, a constant $Z \in \mathbb{F}_p$ such that Z is a nonsquare, $Z \neq -1$, the polynomial $g(X) - Z$ is irreducible over $\mathbb{F}_p$, and $g(b/(Za))$ is a square. (The RFC selects Z mechanically, as the candidate of smallest absolute value meeting these conditions—a nothing-up-my-sleeve constant with no hidden design freedom.) The map $\text{SWU}: \mathbb{F}_p \rightarrow E(\mathbb{F}_p)$ sends u to a point as follows. Compute the two candidate x -coordinates

$$x_1 = \frac{-b}{a} \left(1 + \frac{1}{Z^2 u^4 + Zu^2} \right), \quad x_2 = Zu^2 x_1,$$

with the fallback $x_1 = b/(Za)$ when the denominator $Z^2 u^4 + Zu^2$ vanishes (as at $u = 0$). Set $x = x_1$ if $g(x_1)$ is a square and $x = x_2$ otherwise; let y be a square root of $g(x)$ (*Math Guide*, §“Square roots modulo a prime”), its sign fixed by a convention—the deployed map matches the parity of y to that of u —and output (x, y) .

The construction contains no trial loop, hence runs in constant time; what needs proof is that one of the two candidates always works.

Proposition 3.21 (Correctness of the branch selection). *Under the parameter conditions of Construction 3.20, the map is well-defined for every $u \in \mathbb{F}_p$: the selected value $g(x)$ is a square, so $\text{SWU}(u)$ is a genuine point of $E(\mathbb{F}_p)$.*

Proof sketch. Suppose first that $D = Z^2 u^4 + Zu^2 \neq 0$. The key algebraic identity is

$$g(x_2) = Z^3 u^6 g(x_1),$$

a polynomial identity in u : substitute $x_2 = Zu^2 x_1$ into g and simplify using the formula for x_1 (we take the computation as given). If $g(x_1) = 0$, the first candidate already works, with $y = 0$. Otherwise apply multiplicativity of the Legendre symbol (*Math Guide*, §“Quadratic residues and the Euler criterion”):

$$\left(\frac{g(x_2)}{p}\right) = \left(\frac{Z^3 u^6}{p}\right) \left(\frac{g(x_1)}{p}\right) = \left(\frac{Z}{p}\right) \left(\frac{(Zu^3)^2}{p}\right) \left(\frac{g(x_1)}{p}\right) = \left(\frac{Z}{p}\right) \left(\frac{g(x_1)}{p}\right),$$

since $Z^2 u^6 = (Zu^3)^2$ is a nonzero square, contributing $+1$ (note that $D \neq 0$ forces $u \neq 0$). Because Z is a nonsquare, $\left(\frac{Z}{p}\right) = -1$, so the two symbols are opposite and the branch selection picks the square. If

$D = 0$, the prescribed fallback has $g(x_1) = g(b/(Za))$, a square by the last condition on Z . Thus every input produces a valid affine point. $\square$

A single SWU evaluation is not the end of the pipeline: the map is not surjective onto $E(\mathbb{F}_p)$, and its image is not uniformly covered—it reaches only part of the curve, with a lumpy distribution. The standardised pipeline repairs both defects.

Construction 3.22 (The hash-to-curve pipeline). The RFC 9380 map $\text{hash_to_curve} : \{0, 1\}^* \rightarrow E(\mathbb{F}_p)$ composes four steps:

1. hash to the field: $\text{hash_to_field}(m, 2)$ produces two independent near-uniform field elements u_0, u_1 (§3.6);
2. map each through SWU: $Q_0 = \text{SWU}(u_0)$, $Q_1 = \text{SWU}(u_1)$;
3. sum on the curve: $R = Q_0 + Q_1$. Summing two independent SWU images provably repairs the lumpiness: the sum lies within negligible statistical distance of uniform on $E(\mathbb{F}_p)$, the theorem on which the pipeline’s random-oracle claim rests;
4. clear the cofactor: multiply R by $h = \#E(\mathbb{F}_p)/r$ to land in the prime-order subgroup of order r where the cryptography occurs.

The result is a constant-time hash from messages to group elements which, under the hash model and suite conditions of RFC 9380, can instantiate a random oracle into the group. For the Pasta curves the last step is vacuous—their cofactor is 1 (Definition 2.25).

Remark 3.23 (The isogeny detour forced by j -invariant 0). For the Pasta curves one step needs a detour. The SWU formula for x_1 divides by a , and its numerator carries the factor $-b$: with $a = 0$ the formula is undefined, while $b = 0$ makes x_1 identically zero with $g(x_1) = 0$, collapsing the map to the constant point $(0, 0)$. The construction therefore requires $ab \neq 0$ —and Pallas and Vesta have $a = 0$ (the j -invariant-0 shape $y^2 = x^3 + 5$ of Definition 2.25; curves with $b = 0$, $j = 1728$, are excluded symmetrically). The obstruction is intrinsic, not notational: the j -invariant is an isomorphism invariant, and a Weierstrass change of variables rescales (a, b) to (u^4a, u^6b) (*Math Guide*, §“The j -invariant and the GLV endomorphism”), so no model of these curves attains $ab \neq 0$. The standardised remedy evaluates simplified SWU on an *isogenous* auxiliary curve $E' : y^2 = x^3 + a'x + b'$ with $a'b' \neq 0$, then transports the result through an *isogeny* $E' \rightarrow E$: a nonconstant map between curves given by polynomial coordinate formulas and respecting the group law. The deployed implementation applies a degree-3 isogeny, and—because an isogeny is a group homomorphism—sums $Q_0 + Q_1$ on E' first and transports the single sum (pasta_curves crate, src/hashtocurve.rs: the `iso_map` routine; src/curves.rs: the `hash_to_curve` closure; protocol specification § 5.4.9.8).

The culminating application ties together collision resistance, the random-oracle idealisation, and hashing to a curve.

Construction 3.24 (Nothing-up-my-sleeve generators). To generate public generators $G_1, \dots, G_k$ of a vector commitment (the Pedersen vector commitment of Construction 4.22 in the next section) so that no one knows any discrete-logarithm relation among them, derive each as

$$G_i = \text{hash_to_curve}(\tau \parallel i)$$

from a fixed public domain tag τ —often a human-readable protocol name—and the index i . Because the seed is a transparent, auditable string with no hidden parameters, no party could have engineered a backdoor relation in advance. This is exactly how Halo 2 produces the generators of

its Pedersen vector and inner-product commitments (the latter Construction 4.38): every generator G_i is `hash_to_curve("Halo2-Parameters")` applied to a five-byte message—a zero byte, then the four-byte little-endian encoding of i —the leading zero keeping the G_i disjoint from the two auxiliary generators the same tag derives from the one-byte messages 1 and 2 (`halo2_proofs` crate, `src/poly/commitment.rs`), and Orchard’s value commitment derives its two bases from the tag `z.cash:Orchard-cv` with the one-byte messages "v" and "r" (`orchard` crate, `src/value.rs`).

Remark 3.25 (Why NUMS generators are binding-safe). Knowledge of a discrete-logarithm relation $\sum_i a_i G_i = 0$ with not all a_i zero would let a committer open one commitment two ways, destroying the binding property that the next section defines and relies on. The NUMS derivation forecloses this: the generators are outputs of a function modelled as a random oracle into the group, so they are distributed as independent uniform group elements, and finding a discrete-logarithm relation among independent uniform elements *reduces* to computing a discrete logarithm on E , hard under the standing assumptions of Remark 2.32; the reduction, which embeds its challenge in every generator, is Theorem 4.23 of the next section. Hence no efficient party, the scheme designer included, can know such a relation.

3.8 Arithmetisation-friendly hashing: Poseidon

The hashes of this section so far are *bit-oriented*: SHA-2 and BLAKE2 shuffle and XOR 32- and 64-bit words. Inside an arithmetic circuit over a large prime field $\mathbb{F}_p$ —the setting of proof systems such as Halo 2, whose cost model, developed in the *Halo 2 Guide*, §“From computation to a grid of constraints”, prices each field multiplication as a constraint—such a hash is prohibitively expensive: every Boolean operation must be re-expressed through field arithmetic and range checks, and the tens of thousands of resulting constraints dominate the circuit. Arithmetisation-friendly hashes invert the design: the round function is a handful of *low-degree field operations*, native to the circuit, incurring few constraints. Poseidon (Grassi, Khovratovich, Rechberger, Roy, and Schofnegger, 2019) is the instance Orchard employs, and it arrives via a mode of operation worth defining in general.

Construction 3.26 (The sponge construction). Fix a permutation $f : \mathbb{F}_p^t \rightarrow \mathbb{F}_p^t$ on a state of t field elements. Split the state into a *rate* of r elements—the only part input is added into and output is read from—and a *capacity* of $c = t - r$ elements, never accessed directly: the security reserve. To hash, *absorb* the message into the rate, r elements at a time, applying f between blocks; then *squeeze* output from the rate, applying f between reads. Collision and preimage resistance hold up to approximately $\sqrt{p^c}$ work, as for a random sponge: the birthday bound of Lemma 3.11 applied to the p^c capacity states the adversary cannot see.

Construction 3.27 (The Poseidon permutation). The permutation f is a sequence of rounds, each comprising three steps:

1. *add round constants*: add a fixed vector in $\mathbb{F}_p^t$ to the state;
2. *S-box*: apply $x \mapsto x^\alpha$ to state elements, where $\alpha \geq 3$ is the smallest integer with $\gcd(\alpha, p - 1) = 1$, so that the power map is a bijection on $\mathbb{F}_p$; over the Pasta fields $\alpha = 5$, because $3 \mid p - 1$ there rules out $\alpha = 3$;
3. *mix*: multiply the state by a fixed maximum-distance-separable (MDS) matrix $M \in \mathbb{F}_p^{t \times t}$ —one whose every square submatrix is invertible, giving the fastest possible diffusion of any element’s change across the whole state.

The cost-saving principle (the HADES strategy) distinguishes two round types: a few *full* rounds at the start and end apply the S-box to all t elements, while in the many *partial* rounds between them the S-box touches a single element—the MDS mix still diffusing it across the state. Round counts are chosen to stand against the known algebraic (Gröbner-basis) attacks.

Remark 3.28 (Circuit cost). Inside a circuit, the only non-linear operation in all of Poseidon is $x \mapsto x^5$, verified by a couple of multiplication gates per S-box; the round-constant additions and the MDS multiplication are linear and almost free. A 2-to-1 Poseidon hash costs a few hundred constraints, against tens of thousands for a bit-oriented hash—the difference between a practical circuit and an impractical one.

Example 3.29 (The Orchard Poseidon instance). Orchard fixes $t = 3$ (rate 2, capacity 1), $\alpha = 5$, and 8 full plus 56 partial rounds, over the Pallas base field, at the 128-bit security level—the parametrisation named P128Pow5T3 in the deployed source (`halo2_poseidon` crate, `src/p128pow5t3.rs`; protocol specification § 5.4.1.10). It is used in *constant-input-length* mode as a two-input hash:

$$\text{PoseidonHash}(x, y) = f([x, y, 2^{65}])_0,$$

the first component of the state after a single permutation of the input padded by the fixed capacity value $2^{65} = 2 \cdot 2^{64}$, which encodes the input length 2 into the capacity so that no message padding is required (`halo2_poseidon` crate, `src/lib.rs`: the `ConstantLength` domain sets the initial capacity element to $L \cdot 2^{64}$). Keyed by its first argument, this hash serves as the pseudorandom function

$$F_{\text{nk}}(\rho) = \text{PoseidonHash}(\text{nk}, \rho)$$

—the PRF vocabulary is owned by “Pseudorandom functions and permutations” later in this volume—deployed as $\text{PRF}^{\text{nfOrchard}}$ (`orchard` crate, `src/spec.rs`, function `prf_nf`, consumed by `src/keys.rs` and `src/note/nullifier.rs`; protocol specification § 5.4.2), from which Orchard constructs the nullifier—a construction developed in the *Ironwood Guide*, §“The nullifier, and the loop back to minting”; this volume supplies only the PRF. One contrast deserves note: unlike the Pedersen and Sinsemilla hashes constructed in the next section, whose collision resistance reduces to a clean assumption (discrete log), Poseidon’s security rests on cryptanalytic confidence in its round function—the same epistemic footing as SHA-256, not the same as a reduction.

3.9 The transparent layer’s hashes

The Orchard shielded protocol hashes with BLAKE2b, Sinsemilla, and Poseidon. The transparent layer, inherited from Bitcoin, hashes with SHA-256 and two functions derived from it, none needing machinery beyond §3.2 and the birthday bound of §3.3; this register fixes their names, domains, and deployed uses for the volumes above.

- *RIPEMD-160*. A 160-bit iterated (Merkle–Damgård, the construction of §5.3) hash, $\{0, 1\}^* \rightarrow \{0, 1\}^{160}$, older than Bitcoin and inherited with its address format, whose collision resistance sits at the 80-bit birthday bound of Lemma 3.11. Zcash uses it only inside `hash160` below and the script opcodes `OP_RIPEMD160` and `OP_HASH160` (`zcash_script` crate, `src/opcode/mod.rs`).
- *The address hash* `hash160` $:= \text{RIPEMD160} \circ \text{SHA256}$. A pay-to-public-key-hash (P2PKH) address carries the 20-byte `hash160` of the 33-byte compressed encoding of an ECDSA public key (the scheme is registered in §8.10), and a pay-to-script-hash (P2SH) address the `hash160` of a *redeem script*, the spending condition the spender later reveals and satisfies (protocol specification § 5.6.1.1). The P2PKH output script—the condition a spend of the output must satisfy—is `OP_DUP OP_HASH160 <h> OP_EQUALVERIFY OP_CHECKSIG`, which recomputes the hash from the public key the spender reveals (`zcash_script` crate, `src/pattern.rs`); wallets compute it

through the `zcash_transparent` crate (`src/util/hash160.rs`), and a partially created transaction (PCZT), the multi-party construction format of the *Wallet Guide*, §“Partially created transactions (PCZT)”, carries on each transparent input preimage maps keyed by RIPEMD-160, SHA-256, hash160, and hash256 digests (`src/pczt.rs`).

- *Double SHA-256 (SHA-256d, or hash256)*. The composition $x \mapsto \text{SHA256}(\text{SHA256}(x))$ (protocol specification § 5.4.1.1), whose outer application also closes the length-extension property of plain SHA-256 (§5.3). It hashes block headers (`zebra-chain` crate, `src/block/hash.rs`); its leading four bytes are the checksum that Base58Check, the string encoding of transparent addresses, appends to the raw address bytes before converting to base 58 (protocol specification § 5.6.1.1; `zebra-chain` crate, `src/transparent/address.rs`, through the `bs58` crate’s checked mode) and the payload checksum in every peer-to-peer message header (`zebra-network` crate, `src/protocol/external/codec.rs`); and it is the legacy transaction identifier: the txid of a version-4-or-earlier transaction is SHA-256d of its serialisation (protocol specification § 7.1.1; `zcash_primitives` crate, `src/transaction/mod.rs`, hashing through the `zcash_transparent` crate’s `src/util/sha256d.rs`), whereas a version-5 txid is the BLAKE2b digest tree of ZIP 244.

The toolbox now holds its first primitive in full: a hash with three calibrated security notions, an idealisation that licenses proofs, a discipline that insulates its many uses, and maps carrying its output into fields, onto curves, and inside circuits. The next section spends this capital immediately: commitments—the sealed envelopes of the protocol—are built from exactly the hashes, generators, and hardness assumptions now on the table.

4 Commitment schemes

Two adversaries stalk this section, one on each side of a sealed envelope. Before the envelope is opened, the adversary is the *peeker*: the receiver, who holds the sealed commitment and tries to distinguish which of two candidate messages it contains—to read the bid before the auction closes, the vote before the tally. After the envelope has been delivered, the adversary is the *equivocator*: the sender, who tries to open the same commitment to a message other than the one sealed inside—to change the bid once the rival’s is public. A commitment scheme is the cryptographic sealed, tamper-evident envelope that defeats both: once delivered, the sender can no longer alter the message (*binding*), yet the recipient cannot read it until it is unsealed (*hiding*).

The primitive is the natural next step after hashing: the two constructions at the heart of this section, Pedersen and Sinsemilla, need nothing beyond the discrete-logarithm hardness of §2 and the nothing-up-my-sleeve generators that §3 showed how to derive, and nearly everything built later consumes them. Classical applications set the scene: coin-flipping over a telephone (each party commits to a bit before either reveals), zero-knowledge proofs (the prover commits to its secrets and proves statements about the sealed values), and verifiable secret sharing. Centrally for this volume, the *polynomial* commitments of §4.9 are the cryptographic engine of modern succinct argument systems, including the one deployed in Orchard: the value commitments checked by binding signatures, the note commitments accumulated in Merkle trees, and the witness polynomials of the Halo 2 proof system are all commitments in the precise sense defined here.

The plan of the section follows the two adversaries. We first fix the syntax of a commitment scheme and the three models of who may be trusted to set it up. We then define hiding and binding as games in the tradition of §1.4, each in three quantitative flavours—perfect, statistical, computational—and prove that the two perfections are jointly unattainable: every scheme must choose which adversary to defeat unconditionally. The constructions that follow are ordered by increasing algebraic richness: hash-based commitments (opaque but transparent and plausibly post-quantum), the Pedersen commitment and its vector variant (perfectly hiding, computationally binding under discrete log, and—decisively—linear), Sinsemilla (a commitment whose internal operations are curve operations, built to be cheap inside a proof), then polynomial commitment schemes,

contrasting the pairing-based KZG family with the inner-product family that Zcash deploys, and the bridge from commitments to arguments. A closing subsection returns to the classical application named above—secret sharing—and builds its verifiable form from coefficient commitments, for the threshold signatures of the *FROST Guide*.

4.1 The sealed envelope

The envelope metaphor repays being taken slowly, because each of its physical properties has an exact formal counterpart. The sender writes a message on a card, seals it in the envelope, and hands the envelope over. From that moment the receiver *possesses* the commitment: whatever the message was, it is now fixed—the sender cannot swap the card, because the envelope is in the receiver’s hands and any tampering would show. Yet the receiver *learns* nothing: the envelope is opaque. Later, at a time of the protocol’s choosing, the sender opens the envelope—in the formalism, transmits the message together with the randomness used to seal it—and the receiver checks that the revealed message is consistent with the envelope received earlier.

The two guarantees are directed at different parties and different times. Hiding protects the *sender* against a curious *receiver* during the window between commit and reveal; binding protects the *receiver* against a dishonest *sender* at reveal time. A scheme that is hiding but not binding is an envelope with a false bottom: private, but worthless as evidence. A scheme that is binding but not hiding is a glass envelope: tamper-evident, but no secret survives it. The games of §4.3 formalise exactly these two failure modes, and the theorem of §4.4 shows that against computationally unbounded adversaries one of the two protections must give way.

4.2 Syntax

Definition 4.1 (Commitment scheme). A *commitment scheme* Π for a message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is a triple of algorithms (Setup, Commit, Open):

- The algorithm $\text{Setup}(1^\lambda)$ is PPT (§1.1) and outputs *public parameters* pp , which implicitly determine the message space $\mathcal{M}_{pp}$ and randomness space $\mathcal{R}_{pp}$ and are an implicit input to the remaining algorithms.
- The algorithm $\text{Commit}(pp, m; r)$, for $m \in \mathcal{M}_{pp}$ and $r \in \mathcal{R}_{pp}$, is deterministic given r and outputs a *commitment* c . We write $\text{Commit}(pp, m)$, without an explicit r , for the probabilistic algorithm that samples $r \xleftarrow{\$} \mathcal{R}_{pp}$ uniformly and returns r alongside c .
- The algorithm $\text{Open}(pp, c, m, r)$ is deterministic and outputs a bit: 1 (accept) or 0 (reject).

Correctness requires that honestly produced commitments open: for all pp in the support of $\text{Setup}(1^\lambda)$, all $m \in \mathcal{M}_{pp}$, and all $r \in \mathcal{R}_{pp}$,

$$\text{Open}(pp, \text{Commit}(pp, m; r), m, r) = 1.$$

The pair (m, r) is called the *opening* (or *decommitment*) of c . Every scheme in this volume is *canonically opening*: $\text{Open}(pp, c, m, r)$ merely recomputes $\text{Commit}(pp, m; r)$ and checks equality with c . This is the adopted convention unless stated otherwise. For a canonically opening scheme correctness holds automatically, and a valid opening of c is precisely a pair (m, r) with $\text{Commit}(pp, m; r) = c$ —a reformulation the proofs below use constantly.

Remark 4.2 (The two phases, and a third concern). A commitment scheme is used in two temporally separated phases. In the *commit phase* the sender transmits c and nothing else; in the *reveal phase* the sender transmits the opening (m, r) and the receiver runs Open . The two security properties attach to the two phases: hiding constrains what the receiver can learn *after the commit phase*, while binding

constrains what the sender can do *in the reveal phase*. A third concern precedes both: the *setup phase* that produces pp . Who runs Setup, and whether that party can be trusted, proves decisive for some of the constructions ahead (Remark 4.3).

Remark 4.3 (Three setup models). Commitment schemes divide by how much trust their parameters demand.

1. *Common reference string (CRS)*. A trusted party runs Setup and is assumed to *erase* the randomness behind it. The parameters have internal structure, and knowledge of the discarded randomness—the *trapdoor*—would break the scheme. Of the schemes treated in this section, only KZG (§4.9) genuinely resides here.
2. *Uniform random string (URS), or transparent setup*. The parameters consist only of public coins: uniformly random group elements with no known relations among them. There is no trapdoor and no trusted party—anyone can re-derive the parameters from a public seed. This is the regime of the Pedersen and inner-product commitments below. A Pedersen generator derived by hashing to the curve (Construction 3.24) is transparent even though the textbook presentation phrases Setup as sampling and discarding a secret.
3. *Plain*. No parameters at all, or only a standardised hash function—the home of the hash-based commitment below.

Transparent setup is highly desirable: a flawed or subverted trusted setup can silently destroy binding, as the KZG trapdoor demonstrates concretely in §4.9.

4.3 Hiding and binding

Both properties are games in the sense of Definition 1.15. The hiding game arms the peeker: she chooses the two candidate messages herself, receives a commitment to one of them, and must say which.

Definition 4.4 (Hiding game and hiding). Let Π be a commitment scheme and $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ an adversary. For $b \in \{0, 1\}$, the game $\text{Hide}_{\Pi}^{\mathcal{A}}(\lambda, b)$ runs as follows.

1. The challenger runs $pp \leftarrow \text{Setup}(1^\lambda)$.
2. The adversary $\mathcal{A}_1(pp)$ outputs two messages $m_0, m_1 \in \mathcal{M}_{pp}$ and state st .
3. The challenger samples $r \xleftarrow{\$} \mathcal{R}_{pp}$ and computes $c = \text{Commit}(pp, m_b; r)$.
4. The adversary $\mathcal{A}_2(st, c)$ outputs a bit b' , which is the output of the experiment.

The *hiding advantage* of $\mathcal{A}$ is

$$\text{Adv}_{\Pi}^{\text{hide}}(\mathcal{A}, \lambda) := \left| \Pr[\text{Hide}_{\Pi}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{Hide}_{\Pi}^{\mathcal{A}}(\lambda, 1) = 1] \right|.$$

The scheme is *computationally hiding* if the advantage is negligible (§1.2) for every PPT $\mathcal{A}$; *statistically hiding* if it is negligible even for computationally unbounded $\mathcal{A}$; and *perfectly hiding* if it is exactly 0 for every adversary and every λ .

Proposition 4.5 (Distributional characterisation of hiding). *A commitment scheme Π is perfectly hiding if and only if for every pp in the support of Setup and every pair $m_0, m_1 \in \mathcal{M}_{pp}$, the random*

variables

$$\text{Commit}(pp, m_0; r) \quad \text{and} \quad \text{Commit}(pp, m_1; r), \quad r \xleftarrow{\$} \mathcal{R}_{pp},$$

are identically distributed. It is statistically hiding if the statistical distance (Definition 1.6) between them is negligible. (Only the perfect clause is an equivalence: parameters of negligible Setup probability could exhibit large distance without granting any adversary noticeable advantage.)

Proof. Fix pp and m_0, m_1 , and write D_m for the distribution of $\text{Commit}(pp, m; r)$ over uniform r . By the variational characterisation of statistical distance (Proposition 1.7), $\Delta(D_{m_0}, D_{m_1})$ equals exactly the maximum advantage, over all decision procedures—efficient or not—of telling D_{m_0} from D_{m_1} . An adversary who outputs the pair (m_0, m_1) on parameters pp and applies the optimal test therefore achieves hiding advantage $\Delta(D_{m_0}, D_{m_1})$ whenever Setup produces pp ; conversely no adversary can exceed the expectation of this quantity over pp . Advantage exactly 0 for all adversaries is thus equivalent to $\Delta = 0$ —identical distributions—for every attainable pp and every pair, and negligible statistical distance yields negligible advantage for unbounded adversaries. This coincidence of the game formulation with the distance formulation is precisely why statistical hiding was *defined* through the unbounded-adversary game. $\square$

The binding game arms the equivocator. He is not handed an honest commitment; he manufactures his own, together with two openings.

Definition 4.6 (Binding game and binding). For a commitment scheme Π and adversary $\mathcal{A}$, the game $\text{Bind}_{\Pi}^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger runs $pp \leftarrow \text{Setup}(1^\lambda)$.
2. The adversary $\mathcal{A}(pp)$ outputs a commitment c and two openings (m_0, r_0) and (m_1, r_1) .
3. The game outputs 1 iff $m_0 \neq m_1$ and $\text{Open}(pp, c, m_0, r_0) = \text{Open}(pp, c, m_1, r_1) = 1$.

The *binding advantage* of $\mathcal{A}$ is

$$\text{Adv}_{\Pi}^{\text{bind}}(\mathcal{A}, \lambda) := \Pr[\text{Bind}_{\Pi}^{\mathcal{A}}(\lambda) = 1],$$

a search game in the sense of Definition 1.15. The scheme is *computationally*, *statistically*, or *perfectly binding* according as the advantage is negligible for every PPT adversary, negligible for every unbounded adversary, or exactly 0 for every adversary and every λ .

For a canonically opening scheme the binding game has a useful collision reformulation: for fixed pp , two openings collide precisely when $\text{Commit}(pp, m_0; r_0) = \text{Commit}(pp, m_1; r_1)$ with $m_0 \neq m_1$. Perfect binding then says that distinct messages never share a commitment value at all—the message m is a deterministic function of c . Perfect binding is thus equivalent to the map $c \mapsto m$ being well defined on all attainable commitments, a formulation the counting argument of Remark 4.9 exploits.

Remark 4.7 (The asymmetry of the quantifiers). The two games quantify over different objects. Hiding quantifies over what can be *extracted from* a single honestly produced commitment: the challenger seals the envelope, and the adversary only looks. Binding quantifies over what can be *manufactured*: the adversary builds an ambiguous commitment of its own choosing, under no obligation to follow Commit honestly. A scheme can be strong in one respect and weak in the other, and the two properties actively pull in opposite directions—the more commitment values a message can take (good for hiding), the more opportunities for two messages to share one (bad for binding). The next subsection makes this tension a theorem.

4.4 Incompatibility of perfect hiding and perfect binding

Theorem 4.8 (No scheme is perfectly hiding and perfectly binding). *Let Π be a canonically opening commitment scheme with $|\mathcal{M}_{pp}| \geq 2$. If Π is perfectly hiding, then it is not perfectly binding: indeed, a computationally unbounded adversary wins the binding game with probability 1. Hence no nontrivial commitment scheme is simultaneously perfectly hiding and perfectly binding.*

Proof. Fix any pp in the support of $\text{Setup}(1^\lambda)$ and any two distinct messages $m_0, m_1 \in \mathcal{M}_{pp}$. For a message m , let

$$C_m := \{ \text{Commit}(pp, m; r) : r \in \mathcal{R}_{pp} \}$$

be the set of attainable commitments to m , and let D_m be the distribution of $\text{Commit}(pp, m; r)$ for uniform r . Perfect hiding makes D_{m_0} and D_{m_1} identical (Proposition 4.5); identical distributions have identical supports, so $C_{m_0} = C_{m_1}$. Pick any c in this common support. By definition of the supports there exist $r_0, r_1 \in \mathcal{R}_{pp}$ with

$$\text{Commit}(pp, m_0; r_0) = c = \text{Commit}(pp, m_1; r_1),$$

and then $(c, (m_0, r_0), (m_1, r_1))$ wins the binding game: both openings are valid by canonical opening, and $m_0 \neq m_1$. The double opening *exists* for every attainable pp ; an unbounded adversary *finds* it by enumerating $\mathcal{R}_{pp}$, so it wins Bind with probability 1, giving $\text{Adv}_{\Pi}^{\text{bind}} = 1 \neq 0$. $\square$

Remark 4.9 (The impossibility as a counting fact). Information-theoretically the theorem is a statement about how many bits the commitment carries. Perfect binding forces the map $c \mapsto m$ to be well defined, so c determines m completely: the commitment carries at least $\log_2 |\mathcal{M}|$ bits of information about the message. Perfect hiding forces the distribution of c to be independent of m : the commitment carries *zero* bits. The two demands are compatible only when $\log_2 |\mathcal{M}| = 0$, that is, $|\mathcal{M}| = 1$ —when there is nothing to commit to. Every real scheme therefore chooses which property to make perfect (or statistical) and settles for the other being computational. The Pedersen commitment of §4.6 is perfectly hiding and computationally binding; the hash-based commitment of §4.5, read in the random oracle model, is computationally binding (from collision resistance) and computationally hiding.

Remark 4.10 (Two regimes, two failure modes). The choice is not merely aesthetic; it decides *how* the scheme fails if its assumption falls. If binding is only computational, an adversary with enough power—one able to compute discrete logarithms, say—could *retroactively change* a committed value, and the soundness of any protocol built on top degrades to a computational assumption. If hiding is only computational, such an adversary could *retroactively learn* a committed value: privacy degrades the same way, and everlasting secrecy is lost the day the assumption breaks, even for commitments published long before. Which failure is worse is a judgement about the application: whether integrity or long-term confidentiality is the greater concern. A ledger that publishes its commitments forever has reason to care about the second failure mode; a protocol whose soundness guards live funds has reason to care about the first.

4.5 Hash-based commitments

The simplest construction seals the envelope with a hash. Recall from §3 the two properties it needs: a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ is *collision resistant* (Definition 3.7) if no PPT adversary finds $x \neq x'$ with $H(x) = H(x')$ except with negligible probability, and *preimage resistant* (one-way) if, given $H(x)$ for random x of appropriate length, no PPT adversary finds any x' with $H(x') = H(x)$ except with negligible probability.

Construction 4.11 (Hash commitment). Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ be a hash function. The *hash commitment* has message space $\mathcal{M} = \{0, 1\}^*$ and randomness space $\mathcal{R} = \{0, 1\}^\ell$ for a randomness length $\ell = \ell(\lambda)$ (for example $\ell = 2n$). Let $\text{enc}(m, r)$ be an injective, self-delimiting encoding of the pair—for example a canonical length encoding of m , followed by m , followed by the fixed-length

string r . Define

$$\text{Commit}(m; r) := H(\text{enc}(m, r)),$$

opened canonically. There are no nontrivial public parameters beyond the choice of H —the scheme lives in the plain model of Remark 4.3.

Proposition 4.12 (Binding of the hash commitment). *If H is collision resistant, the hash commitment is computationally binding.*

Proof. Suppose $\mathcal{A}$ wins the binding game: it outputs c with valid openings (m_0, r_0) , (m_1, r_1) and $m_0 \neq m_1$, so $H(\text{enc}(m_0, r_0)) = c = H(\text{enc}(m_1, r_1))$. Injectivity of enc makes the two hash inputs distinct strings—this is exactly what the self-delimiting encoding buys; without it, a boundary shift could present distinct pairs as the same string. The adversary has therefore produced an H -collision, and the reduction that runs $\mathcal{A}$ and outputs the pair $(\text{enc}(m_0, r_0), \text{enc}(m_1, r_1))$ wins the collision game with exactly the binding advantage of $\mathcal{A}$, in essentially the same time. $\square$

Proposition 4.13 (Hiding of the hash commitment, in the ROM). *Model H as a random oracle (Definition 3.13). For messages of length $\text{poly}(\lambda)$ and randomness length $\ell = \omega(\log \lambda)$, the hash commitment is computationally hiding: an adversary making at most q oracle queries has hiding advantage at most $q \cdot 2^{-\ell}$.*

Proof. The challenge commitment is $c = H(\text{enc}(m_b, r))$ for uniform $r \in \{0, 1\}^\ell$. Because the oracle’s outputs are independent and uniform, c is a fresh uniform string—carrying no information about b —unless the adversary queries the oracle at the exact point $\text{enc}(m_b, r)$. Before any such query the randomness r is information-theoretically hidden: the adversary’s view is independent of it. Each of the q queries therefore hits that point with probability at most $2^{-\ell}$, and the union bound (*Math Guide*, §“The union bound and a birthday calculation”) caps the probability of any hit by $q \cdot 2^{-\ell}$. Conditioned on no hit, the adversary’s view is identically distributed in the two worlds $b = 0$ and $b = 1$, so the advantage is at most $q \cdot 2^{-\ell}$, negligible for $\ell = \omega(\log \lambda)$ and polynomial q . $\square$

Remark 4.14 (Why the randomness is essential). Dropping r is fatal. The deterministic commitment $c = H(m)$ fails hiding for every low-entropy message space: to distinguish a commitment to m_0 from one to m_1 , the peeker simply hashes her two candidates and compares. Nothing about collision resistance or one-wayness prevents this—the attack never inverts anything. The salt r contributes ℓ uniformly random bits, so the hash input is unpredictable even when the message is one of two known values. The same phenomenon renders deterministic encryption insecure for low-entropy plaintexts, and the same cure—randomisation—applies.

Remark 4.15 (Strength profile). The hash commitment has two virtues. It is *transparent*: no trusted setup, indeed no parameters, and no algebraic structure for a subverted setup to poison. And it is *plausibly post-quantum*: generic quantum collision finding takes $\Theta(2^{n/3})$ queries against an n -bit hash (Remark 2.31), so increasing the output length compensates for the loss. Its weakness is the flip side of its opacity: the commitment is an unstructured bit string. One cannot add two hash commitments and obtain a commitment to the sum of the messages; no algebraic relation on commitments reflects any relation on the committed values. The Pedersen construction repairs exactly this defect, at the price of resting on discrete log.

4.6 The Pedersen commitment

We change algebraic scenery. Let $\mathbb{G}$ be a cyclic group of prime order q , written *additively*: for a point $P \in \mathbb{G}$ and scalar $a \in \mathbb{F}_q$, the scalar multiple is $[a]P$, with $[0]P = \mathcal{O}$ the identity, and $a \mapsto [a]P$ is

a group homomorphism $\mathbb{F}_q \rightarrow \mathbb{G}$. Concretely, $\mathbb{G}$ is the group of $\mathbb{F}_p$ -points of a prime-order elliptic curve (*Math Guide*, §“The group structure of $E(\mathbb{F}_p)$ ”), and in this volume the Pallas curve of §2.6; but the only structural facts the constructions use are that $\mathbb{G}$ is isomorphic to $\mathbb{F}_q$ as a group and that discrete logarithms in $\mathbb{G}$ are hard. Formally, a *group generator* $\mathcal{G}$ on input 1^λ outputs $(\mathbb{G}, q, G)$ with q a $\Theta(\lambda)$ -bit prime and G a generator, and the DLog assumption for $\mathcal{G}$ states that no PPT adversary, given $(\mathbb{G}, q, G, [a]G)$ for uniform $a \xleftarrow{\$} \mathbb{F}_q$, outputs a except with negligible probability—Definition 2.2, transcribed additively as in §2.6.

Construction 4.16 (Pedersen commitment). The algorithm $\text{Setup}(1^\lambda)$ runs $(\mathbb{G}, q, G) \leftarrow \mathcal{G}(1^\lambda)$, samples $s \xleftarrow{\$} \mathbb{F}_q^\times$, sets $H := [s]G$, outputs $pp = (\mathbb{G}, q, G, H)$, and discards s . The message and randomness spaces are $\mathcal{M} = \mathcal{R} = \mathbb{F}_q$, and

$$\text{Commit}(pp, m; r) := [m]G + [r]H,$$

opened canonically.

The parameters are two generators G and H whose relative discrete logarithm $s = \log_G H$ is unknown to *all* parties, the committer included. Everything hinges on this one unknown scalar: knowledge of s breaks binding (Remark 4.19), while perfect hiding, as the proof below shows, does not even require ignorance of s . In deployment the sampled-and-discarded s of the textbook presentation is replaced by deriving H through hashing to the curve (Construction 3.24): nobody ever *holds* s , and the setup is transparent (Remark 4.3).

Theorem 4.17 (Perfect hiding). *The Pedersen commitment is perfectly hiding.*

Proof. Fix any attainable $pp = (\mathbb{G}, q, G, H)$; since $s \in \mathbb{F}_q^\times$, the point H is not $\mathcal{O}$. Because $\mathbb{G}$ has prime order, the nonzero H generates $\mathbb{G}$, so $r \mapsto [r]H$ is a bijection $\mathbb{F}_q \rightarrow \mathbb{G}$. For uniform r , therefore, $[r]H$ is uniform on $\mathbb{G}$ —and so is its translate $[m]G + [r]H$, for *every* message m : adding the fixed element $[m]G$ permutes the group and leaves the uniform distribution invariant. The commitment distribution is uniform on $\mathbb{G}$ regardless of m , so for any m_0, m_1 the two distributions coincide exactly, and every adversary’s advantage is 0 by Proposition 4.5. $\square$

Theorem 4.18 (Computational binding under DLog). *If the DLog assumption holds for $\mathcal{G}$, the Pedersen commitment is computationally binding. Concretely, from any adversary $\mathcal{A}$ that wins the binding game with probability ε one constructs $\mathcal{B}$ computing discrete logarithms with probability $(1 - 1/q)\varepsilon \geq \varepsilon - 1/q$, in essentially the same running time.*

Proof. First the algebra. A double opening gives $[m_0]G + [r_0]H = [m_1]G + [r_1]H$ with $m_0 \neq m_1$, so

$$[m_0 - m_1]G = [r_1 - r_0]H.$$

The case $r_1 = r_0$ is impossible: it would force $[m_0 - m_1]G = \mathcal{O}$, and since G has order q this means $m_0 \equiv m_1 \pmod{q}$, contradicting $m_0 \neq m_1$ in $\mathbb{F}_q$. With $r_1 - r_0$ invertible modulo the prime q , the relation solves for the hidden discrete logarithm:

$$s = \log_G H = (m_0 - m_1)(r_1 - r_0)^{-1} \pmod{q}.$$

A double opening is the discrete log of H .

Now the reduction. On a DLog challenge $(\mathbb{G}, q, G, Q)$, the algorithm $\mathcal{B}$ sets $H := Q$, hands $pp = (\mathbb{G}, q, G, Q)$ to $\mathcal{A}$, and, when $\mathcal{A}$ returns a double opening, outputs $(m_0 - m_1)(r_1 - r_0)^{-1} \pmod{q} = \log_G Q$.

One distribution gap needs accounting: in the real scheme $H = [s]G$ with $s \xleftarrow{\$} \mathbb{F}_q^\times$, so H is uniform on $\mathbb{G} \setminus \{\mathcal{O}\}$, whereas the challenge $Q = [a]G$ has a uniform on all of $\mathbb{F}_q$, so Q is uniform on all of $\mathbb{G}$. The two distributions differ only on the event $Q = \mathcal{O}$, of probability $1/q$ —their statistical distance is exactly $1/q$ —and on that event $\mathcal{A}$ cannot win anyway (with $H = \mathcal{O}$ the relation above forces $m_0 = m_1$). Conditioned on $Q \neq \mathcal{O}$, the simulation is perfect, so

$$\text{Adv}^{\text{dlog}}(\mathcal{B}) = \left(1 - \frac{1}{q}\right) \text{Adv}_{\Pi}^{\text{bind}}(\mathcal{A}) \geq \varepsilon - \frac{1}{q},$$

and $\mathcal{B}$'s overhead over $\mathcal{A}$ is a constant number of field operations. $\square$

The same embed-and-extract pattern, scaled up to many generators, proves the vector generalisation (Theorem 4.23).

Remark 4.19 (Equivocation with the trapdoor). Any party who knows $s = \log_G H$ can open a Pedersen commitment to any message. Given $c = [m]G + [r]H$ and a target m' , set

$$r' := r + (m - m')s^{-1} \bmod q;$$

then $[m']G + [r']H = [m']G + [r]H + [(m - m')]G = c$. This trapdoor property is not a defect of the construction—it is precisely why discarding s in Setup is mandatory, and it is a feature the theory exploits deliberately: zero-knowledge simulators use exactly this equivocation to open commitments consistently with a transcript produced out of order. The same phenomenon priced the setup models of Remark 4.3: a commitment scheme with a trapdoor is only as binding as the claim that nobody holds it.

Theorem 4.20 (Additive homomorphism). *For all $m_0, m_1, r_0, r_1 \in \mathbb{F}_q$,*

$$\text{Commit}(m_0; r_0) + \text{Commit}(m_1; r_1) = \text{Commit}(m_0 + m_1; r_0 + r_1),$$

and more generally, for all scalars $\alpha, \beta \in \mathbb{F}_q$,

$$[\alpha] \text{Commit}(m_0; r_0) + [\beta] \text{Commit}(m_1; r_1) = \text{Commit}(\alpha m_0 + \beta m_1; \alpha r_0 + \beta r_1).$$

Proof. Expand and regroup, using commutativity of $+$ in $\mathbb{G}$, the identity $[a]P + [b]P = [a + b]P$, and the linearity $[\alpha]([m]G + [r]H) = [\alpha m]G + [\alpha r]H$ of scalar multiplication:

$$\begin{aligned} [\alpha]([m_0]G + [r_0]H) + [\beta]([m_1]G + [r_1]H) &= [\alpha m_0]G + [\beta m_1]G + [\alpha r_0]H + [\beta r_1]H \\ &= [\alpha m_0 + \beta m_1]G + [\alpha r_0 + \beta r_1]H. \end{aligned} \quad \square$$

Remark 4.21 (Value balancing: the homomorphism deployed). The homomorphism is what makes Pedersen commitments the right tool for *hidden arithmetic*, and Zcash's value balancing is the canonical deployment. Each transaction value v_i is committed as $C_i = [v_i]G + [r_i]H$; a transaction balances iff the sum of its input values equals the sum of its output values. By Theorem 4.20, the verifier can check this *without learning any v_i* : the value components cancel in $\sum C_{\text{in}} - \sum C_{\text{out}}$ precisely when the value sums agree *modulo the group order q* , in which case the difference equals $[r]H$ for the net randomness r —a commitment to 0. Cancellation alone certifies balance only modulo q —value sums differing by a multiple of q cancel too—and the deployed protocol closes the gap with range checks carried inside the proofs: the values of the notes (the shielded value records) spent and created by each Action (Orchard's combined spend-and-output unit) are proved to lie in $\{0, \dots, 2^{64}-1\}$ (the Action statement, protocol specification § 4.18.4; enforced by the note-commitment gadget's ValueCanonicity gate, orchard crate, `src/circuit/note_commit.rs`), the committed net value is the difference of two such values, and the number of summands is bounded, so the sums cannot wrap (§ 4.14). The prover does not reveal r ; it proves *knowledge* of r by using $[r]H$ as the verification key of a signature

over the transaction—the *binding signature*, treated with the completed balance argument in §8.9. In deployed Orchard the instantiation is $\text{ValueCommit}(v; \text{rcv}) = [v]V + [\text{rcv}]R$ with hashed-to-curve generators V, R (orchard crate, `src/value.rs`, `ValueCommitment::derive`; protocol specification § 5.4.8.3); each Action publishes a commitment to its *net* value change, and the verifier folds the published commitments with a commitment to the transparent value balance to obtain the binding verification key (`src/bundle.rs`, `Bundle::binding_validating_key`). The homomorphism turns an arithmetic relation on hidden values into a group equation on public commitments—a template the rest of the volume reuses repeatedly.

4.7 Pedersen vector commitments

A single group element can seal far more than a single scalar.

Construction 4.22 (Pedersen vector commitment). On input 1^λ and a length $n = \text{poly}(\lambda)$, Setup outputs $(\mathbb{G}, q)$ together with generators $G_1, \dots, G_n, H \xleftarrow{\$} \mathbb{G} \setminus \{\mathcal{O}\}$, sampled so that no nontrivial discrete-log relation among them is known to any party—in practice each generator is derived from a public seed by hashing into the curve, rejecting the identity if it occurs, a nothing-up-my-sleeve procedure (Construction 3.24) that makes the setup transparent. The message space is $\mathbb{F}_q^n$, the randomness space $\mathbb{F}_q$, and for $\mathbf{m} = (m_1, \dots, m_n)$,

$$\text{Commit}(pp, \mathbf{m}; r) := \sum_{i=1}^n [m_i]G_i + [r]H = \langle \mathbf{m}, \mathbf{G} \rangle + [r]H,$$

where $\langle \mathbf{m}, \mathbf{G} \rangle := \sum_i [m_i]G_i$ is the multiscalar (mixed) inner product of the message vector with the generator vector $\mathbf{G} = (G_1, \dots, G_n)$. Opening is canonical.

Theorem 4.23 (Properties of the vector commitment). *The Pedersen vector commitment is perfectly hiding, and computationally binding under the DLog assumption in $\mathbb{G}$.*

Proof. Hiding. Exactly as in Theorem 4.17: for uniform r the blinding term $[r]H$ is uniform on $\mathbb{G}$, so the commitment is uniform on $\mathbb{G}$ regardless of the message vector.

Binding. A double opening with $\mathbf{m} \neq \mathbf{m}'$ gives $\langle \mathbf{m}, \mathbf{G} \rangle + [r]H = \langle \mathbf{m}', \mathbf{G} \rangle + [r']H$, that is, a *nontrivial discrete-log relation*

$$\sum_{i=1}^n [m_i - m'_i]G_i + [r - r']H = \mathcal{O}$$

with not all coefficients zero. The reduction $\mathcal{B}$ must extract a single logarithm from a relation among $n+1$ generators, and it does so by embedding its challenge in *every* generator. On challenge $Q = [s]G$, it samples $u_i, w_i \xleftarrow{\$} \mathbb{F}_q$ for each i and $u_H, w_H \xleftarrow{\$} \mathbb{F}_q$, sets

$$G_i := [u_i]G + [w_i]Q, \quad H := [u_H]G + [w_H]Q,$$

and resamples any pair whose generator lands on $\mathcal{O}$. Each result is uniform on $\mathbb{G} \setminus \{\mathcal{O}\}$ —the $[u_i]G$ term alone makes the pre-rejection point uniform on $\mathbb{G}$ —so the simulated parameters follow the real distribution. Crucially, the parameters reveal nothing about the w -coordinates: for every candidate value of w_i there is exactly one u_i consistent with the observed G_i , so conditioned on the adversary’s entire view the w_i and w_H remain independent and uniform. Write the recovered relation as $\sum_i [a_i]G_i + [a_H]H = \mathcal{O}$ with coefficient vector $(a_1, \dots, a_n, a_H) \neq 0$. Substituting the embeddings and collecting the G - and Q -components gives

$$c_0 + c_1 s \equiv 0 \pmod{q}, \quad c_0 = \sum_i a_i u_i + a_H u_H, \quad c_1 = \sum_i a_i w_i + a_H w_H.$$

Since the coefficient vector is nonzero and the w -coordinates are uniform and independent of it, c_1 is uniform on $\mathbb{F}_q$, hence nonzero with probability $1 - 1/q$; in that case $s = -c_0 c_1^{-1} \bmod q$, and $\mathcal{B}$ outputs the challenge logarithm. Altogether $\text{Adv}^{\text{dlog}}(\mathcal{B}) \geq (1 - 1/q) \text{Adv}^{\text{bind}}(\mathcal{A})$. $\square$

Remark 4.24 (Compression and the cardinality trade-off). The vector commitment is *compressing*: n field elements (plus the randomness) map to a single group element, exponentially shorter than the message for large n . Compression is incompatible with statistical binding for a counting reason: there are q^n message vectors and only q commitment values, so colliding openings exist in overwhelming abundance. This is exactly why binding is only computational—an unbounded adversary finds a colliding opening by solving the underlying discrete-log relation—and it is the trade-off of Theorem 4.8 made visible at the level of cardinalities: a compressing commitment *cannot* determine its message, so it might as well hide it perfectly.

Remark 4.25 (Linearity survives). The vector commitment remains additively homomorphic in both message and randomness, componentwise: $\text{Commit}(m; r) + \text{Commit}(m'; r') = \text{Commit}(m + m'; r + r')$, by the same regrouping as Theorem 4.20. This linearity, applied to a *committed* vector against *public* coefficient vectors, is the foundation of the inner-product arguments of §4.9, which prove statements about $\langle m, b \rangle$ for public b while revealing only a single committed group element.

4.8 Sinsemilla: an algebraic hash-based commitment

Between the opaque hash commitment of §4.5 and the fully linear Pedersen commitment sits an intermediate species: *algebraic hash functions*, collision-resistant hashes whose internal operations are elliptic-curve group operations. The point of the species is proof cost. A zero-knowledge circuit that must verify a hash evaluation pays for every internal operation; when those operations are curve additions in the very field the proof system works over, the verification is cheap, while collision resistance still reduces to the discrete logarithm. Sinsemilla, deployed in Zcash Orchard, is the archetype: a Pedersen-style hash over a sequence of message chunks. We build up to it through its ancestor.

Construction 4.26 (Pedersen hash). Split the message M into k chunks $m_1, \dots, m_k$, each interpreted as a scalar in a bounded range, and fix k independent generators $P_1, \dots, P_k \in \mathbb{G}$ with unknown mutual discrete logarithms. Define

$$\text{PedersenHash}(M) := \sum_{j=1}^k [m_j]P_j \in \mathbb{G}.$$

This is the unblinded Pedersen vector commitment map used as a hash. The output is a group element; composing with a fixed encoding of group elements as bit strings yields an ordinary hash.

Proposition 4.27 (Collision resistance of the Pedersen hash). *Suppose no efficient algorithm finds a nontrivial relation $\sum_j [a_j]P_j = \mathcal{O}$ with not all $a_j = 0$ and each $|a_j|$ within the allowed chunk range—hardness that follows from DLog when the P_j are independent uniform generators. Then PedersenHash is collision resistant.*

Proof. A collision $M \neq M'$, with chunk decompositions (m_j) and (m'_j) , gives $\sum_j [m_j - m'_j]P_j = \mathcal{O}$ with some coefficient $m_j - m'_j \neq 0$ —a nontrivial relation with coefficients in the allowed range. The reduction from relation-finding to DLog is the every-generator embedding of Theorem 4.23: set each $P_j = [u_j]G + [w_j]Q$ for the challenge Q , and read the unknown logarithm off the recovered relation. $\square$

The Pedersen hash needs one independent generator per chunk, which is costly for long inputs: the generator table grows linearly with the maximum message length. Sinsemilla reorganises the computation into an iterated two-operand form that reuses a small fixed generator table—shared by all chunk positions—while preserving the discrete-log collision reduction.

Construction 4.28 (Sinsemilla hash). Split the message M into pieces $m_1, \dots, m_k$, each of fixed width $w \leq 32$ bits, and identify a piece $v \in \{0, 1\}^w$ with its integer value, bits read little-endian (the specification’s LEBS2IP). The public parameters are a base point $Q \in \mathbb{G}$ and a table $S : \{0, 1\}^w \rightarrow \mathbb{G}$ assigning to each possible piece value its own generator, both obtained by hashing to the curve. Write $\text{GroupHash}(D, M)$ for the hash-to-curve pipeline of Construction 3.22 applied to the byte string M under the domain-separation tag built from the byte string D —for Pallas the RFC 9380 suite tag $D \parallel \text{"-pallas_XMD:BLAKE2b_SSWU_RO_"} of §3.6 (protocol specification § 5.4.9.8)—and write }LE_{32}(j)$ for the four-byte little-endian encoding of an integer $0 \leq j < 2^{32}$. The table is

$$S(v) := \text{GroupHash}(\text{"z.cash:SinsemillaS"}, \text{LE}_{32}(v)), \quad v \in \{0, 1\}^w,$$

with 2^w entries shared by every instance, and the base point is personalised (§3.5) to the enclosing use—a byte string D naming it, such as a note-commitment or Merkle-hash domain—by taking that string as the message:

$$Q := \text{GroupHash}(\text{"z.cash:SinsemillaQ"}, D).$$

Both derivations are nothing-up-my-sleeve (Construction 3.24), so no party knows any discrete-log relation among $\{S(v)\}_v \cup \{Q\}$ (Remark 3.25). The deployed instantiation takes $w = 10$ (the specification’s k) over Pallas: the `sinsemilla` crate, `src/lib.rs`, fixes the two personalisation strings and derives Q in `HashDomain::new`; the table ships precomputed as `SINSEMILLA_S` in `src/sinsemilla_s.rs`, and the crate’s test `sinsemilla_s` re-derives every entry from `u32::to_le_bytes` of the index—exactly LE_{32} ; the circuit loads that table as its lookup (`halo2_gadgets` crate, `src/sinsemilla/chip/generator_table.rs`); protocol specification § 5.4.1.9. The hash iterates over the pieces with a running accumulator $\text{Acc} \in \mathbb{G}$:

$$\text{Acc}_0 := Q, \quad \text{Acc}_j := \text{Acc}_{j-1} + (\text{Acc}_{j-1} + S(m_j)) = [2]\text{Acc}_{j-1} + S(m_j),$$

and outputs the point $\text{SinsemillaHashToPoint}(M) := \text{Acc}_k$. The step costs two *incomplete* curve additions—first $\text{Acc}_{j-1} + S(m_j)$, then the result plus Acc_{j-1} ; no doubling is ever performed, and the addition formula is undefined when its operands are equal or are mutual negatives (*Math Guide*, §“Elliptic curves”)—and this pair of additions is the operation the circuit verifies in very few constraints. Unrolling the recurrence gives the closed form

$$\text{SinsemillaHashToPoint}(M) = \sum_{j=1}^k [2^{k-j}]S(m_j) + [2^k]Q :$$

the integer coefficients are powers of two determined by position alone, and the message enters only through *which* generator $S(m_j)$ each piece selects.

Proposition 4.29 (Collision resistance of Sinsemilla). *Model the table entries $S(v)$ and the base point Q as independent uniform generators (the nothing-up-my-sleeve hash-to-curve derivation covers Q as well as the table). Then any collision in $\text{SinsemillaHashToPoint}$, on inputs for which its incomplete additions are defined, yields a nontrivial discrete-log relation among $\{S(v)\}_v \cup \{Q\}$, and any input on which an incomplete addition is undefined yields one directly; either way a discrete log-*

arithm follows via the every-generator embedding of Theorem 4.23 extended to Q . The point hash is thus collision resistant under the DLog assumption in $\mathbb{G}$, the exceptional cases of incomplete addition covered by the same reduction rather than excluded.

Proof sketch. Let $M \neq M'$ collide. Suppose first that they have the same piece count k . Subtracting the unrolled forms cancels the $[2^k]Q$ terms and leaves

$$\sum_{j=1}^k [2^{k-j}](S(m_j) - S(m'_j)) = \mathcal{O}.$$

Regroup over the distinct table generators: for each value $v \in \{0, 1\}^w$, the net coefficient on $S(v)$ is

$$c_v = \sum_{j: m_j=v} 2^{k-j} - \sum_{j: m'_j=v} 2^{k-j},$$

and the relation reads $\sum_v [c_v] S(v) = \mathcal{O}$. Each c_v is a difference of two subset-sums of the distinct powers $2^{k-1}, \dots, 2^0$, hence an integer with $|c_v| < 2^k$. For *admissible* message lengths—those with 2^k below the group order q , a finite design-time check—such a coefficient vanishes modulo q only if it vanishes as an integer; and $c_v = 0$ forces, by uniqueness of binary expansions, the two subset-sums to be equal as sets of powers, that is, the positions carrying value v in M and in M' to coincide. Coincidence for every v would force $M = M'$; since $M \neq M'$, some $c_v \neq 0$ and the relation is nontrivial.

If instead the piece counts differ, say $k > k'$, the Q -constants do not cancel: the relation acquires the coefficient $2^k - 2^{k'}$ on Q , nonzero and below the group order for admissible lengths, so the relation is again nontrivial. In either case, embedding a DLog challenge in every table generator *and* in Q recovers the challenge logarithm exactly as in Proposition 4.27.

Finally, the exceptional cases. An incomplete addition is undefined only when its operands are equal or are mutual negatives, so an input reaching one presents a coincidence $[\alpha]\text{Acc}_{j-1} + S(m_j) = \mathcal{O}$ with $\alpha \in \{-1, 1, 2\}$. Substituting the unrolled form of Acc_{j-1} makes this an explicit relation among Q and the table generators, with coefficient $\alpha 2^{j-1} \neq 0$ on Q and all magnitudes at most 2^k —below the group order for admissible lengths. An input that merely triggers an exceptional case therefore hands over a nontrivial relation itself, and the same embedding extracts the challenge logarithm from it; the protocol specification proves exactly this for the deployed instantiation (§ 5.4.1.9). $\square$

The deployed hash differs from the point hash in two ways. First, the deployed `SinsemillaHash` applies the Pallas coordinate extractor to `SinsemillaHashToPoint(M)`, returning its x -coordinate rather than the point itself. Collision resistance survives the extraction: equal extracted x -coordinates mean the two points are equal or are negatives of one another, and either signed equality yields the same kind of nontrivial generator relation as in the proof above. Second, the final piece is zero-padded to the piece width—the `Pad` iterator of the `sinsemilla` crate, `src/lib.rs`, pads the message bits to a multiple of the piece width before chunking—so two bit strings of different lengths within the same final piece present identical piece sequences, and the deployed hash claims collision resistance only *between inputs of a fixed length for a given personalisation*. This is exactly the property the protocol specification requires and no more (protocol specification § 5.4.1.9: collision resistance is demanded “between inputs of fixed length, for a given personalization input”); each personalisation domain hashes inputs of a single length, as the Merkle-tree hash `MerkleCRH` of §10 illustrates. The variable-length argument in the proof above applies to the piece-aligned point presentation, where the piece sequence is the message.

The step from hash to commitment is the Pedersen blinding term, *verbatim*.

Construction 4.30 (`SinsemillaCommit`). Fix one further independent generator H' , hashed to the curve with its own domain string, so that no discrete-log relation between H' and $\{S(v)\}_v \cup \{Q\}$ is

known. Define, for $r \xleftarrow{\$} \mathbb{F}_q$,

$$\text{SinsemillaCommit}(m; r) := \text{SinsemillaHashToPoint}(m) + [r]H'.$$

Hiding is inherited from Pedersen verbatim: by the argument of Theorem 4.17, $[r]H'$ is uniform on $\mathbb{G}$ for uniform r , so the commitment distribution is independent of m , and the scheme is perfectly hiding. Binding rests on discrete log: two valid openings to distinct messages yield either a `SinsemillaHashToPoint` collision—excluded by Proposition 4.29—or a nontrivial discrete-log relation between H' and the hash generators.

Orchard’s note commitment `NoteCommit` instantiates exactly this map: the orchard crate derives it through the `sinsemilla` crate’s `CommitDomain::commit` (`src/note/commitment.rs`, `NoteCommitment::derive`; protocol specification § 5.4.8.4). Its key commitment `CommitIvk` composes the same map with x -coordinate extraction—the specification’s `SinsemillaShortCommit`—returning a base-field element rather than a point (`src/spec.rs`, `commit_ivk`, via `CommitDomain::short_commit`).

Remark 4.31 (Why Sinsemilla exists: cost inside a proof). Sinsemilla’s advantage is not speed outside a circuit—ordinary hashes are faster there—but verification cost *inside* one. In a SNARK (§9) the prover re-expresses every computational step as polynomial constraints; a bit-oriented hash such as SHA-256, all Boolean choppings and modular additions foreign to the proof field, costs tens of thousands of constraints per evaluation, whereas Sinsemilla’s per-piece cost is a handful of curve operations chosen to lie in the very field the proof system already works over. The result is a collision-resistant hash and commitment—used for note commitments and Merkle trees in Orchard—simultaneously cheap to prove and grounded in the same discrete-log hardness as the surrounding protocol. The presentation here is the conceptual core; the *Ironwood Guide* takes up the concrete Orchard instantiation—its per-use personalisation strings and the in-circuit handling of incomplete addition.

4.9 Polynomial commitment schemes

The final refinement commits not to a value or a vector but to a *function*—a polynomial—and supports proving statements about its values. This is the primitive on which the succinct argument systems of §9 stand.

Definition 4.32 (Polynomial commitment scheme). Fix a field $\mathbb{F}$ and a degree bound d . A *polynomial commitment scheme* (PCS) is a tuple (`Setup`, `Commit`, `Open`, `Eval`, `Verify`):

- The algorithm `Setup`($1^\lambda, d$) outputs public parameters pp supporting degree at most d ;
- the algorithm `Commit`($pp, f; r$) outputs a commitment C to a polynomial $f \in \mathbb{F}[X]_{\leq d}$, optionally with hiding randomness r ;
- the algorithm `Open`(pp, C, f, r) outputs a bit, verifying that C commits to the *whole* polynomial f ;
- the algorithm `Eval`(pp, f, r, z), for an evaluation point $z \in \mathbb{F}$, outputs a claimed value $y = f(z)$ together with an *evaluation proof* π ;
- the algorithm `Verify`(pp, C, z, y, π) outputs a bit, checking the evaluation proof against the commitment.

Beyond the hiding and binding of the underlying commitment (Definitions 4.4 and 4.6), a PCS must satisfy *evaluation binding*: no PPT prover can produce a commitment C , a point z , and two accepting proofs for distinct claimed values $y \neq y'$ at z , except with negligible probability. Stronger

schemes additionally satisfy *extractability* (proof of knowledge): from any successful prover one can extract an actual polynomial f of degree at most d consistent with all the evaluations it gets accepted.

Remark 4.33 (Rigidity, and why evaluation binding is separate). Why should one short commitment support probing at arbitrary points? Because polynomials are *rigid*: a nonzero polynomial of degree at most d has at most d roots (*Math Guide*, §“The Schwartz–Zippel lemma”), so two distinct polynomials of degree at most d agree on at most d points. Answering a different value at even a single point is the behaviour of a prover holding a *different polynomial*—there is no wiggle room between functions. But a warning is in order: an accepting evaluation proof is *not* an opening of C , so evaluation binding does *not* follow from the binding of Commit. It is a separate cryptographic requirement, and each construction below proves it from its own assumption—a strong Diffie–Hellman assumption for KZG, discrete log via a folding extractor for the inner-product scheme. The central design problem of a PCS is making the evaluation proof π short and fast to verify; committing to the coefficient list with any old commitment is easy, and useless.

KZG: evaluation in the exponent

The pairing-based scheme of Kate, Zaverucha, and Goldberg achieves constant-size proofs by hiding a secret evaluation point inside the parameters.

Construction 4.34 (KZG commitment). Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be groups of prime order $|\mathbb{F}|$ equipped with a nondegenerate bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ (Example 2.9), with generators $G \in \mathbb{G}_1$ and $\hat{G} \in \mathbb{G}_2$. The algorithm Setup samples a secret $\tau \xleftarrow{\$} \mathbb{F}$, publishes the *structured reference string*

$$pp = ([\tau^0]G, [\tau^1]G, \dots, [\tau^d]G, \hat{G}, [\tau]\hat{G}),$$

and *discards* τ . The commitment to $f = \sum_{i=0}^d a_i X^i$ is evaluation in the exponent:

$$\text{Commit}(pp, f) := \sum_{i=0}^d [a_i]([\tau^i]G) = [f(\tau)]G,$$

computable from pp by anyone, without knowledge of τ .

Construction 4.35 (KZG evaluation proof). To prove $f(z) = y$, the prover uses the factor theorem: z is a root of $f(X) - y$, so the quotient

$$q(X) := \frac{f(X) - y}{X - z} \in \mathbb{F}[X]_{\leq d-1}$$

is a genuine polynomial *precisely because* $f(z) - y = 0$. The proof is the commitment to the quotient, $\pi := [q(\tau)]G$, computed from the reference string. The verifier checks the single pairing equation

$$e(C - [y]G, \hat{G}) \stackrel{?}{=} e(\pi, [\tau]\hat{G} - [z]\hat{G}),$$

which, by bilinearity, is exactly the in-the-exponent form of the polynomial identity $f(\tau) - y = q(\tau)(\tau - z)$.

Proposition 4.36 (KZG evaluation binding, sketched). *Evaluation binding of KZG rests on the d-strong Diffie–Hellman assumption in the bilinear group: given the powers $[\tau^i]G$, it is infeasible to produce $[(\tau - z)^{-1}]G$ for any z of one’s choosing.*

Proof idea. Two accepting proofs π, π' for distinct values $y \neq y'$ at the same point z satisfy the two pairing equations; subtracting them (in the exponent) gives $\pi - \pi' = [(y' - y)(\tau - z)^{-1}]G$, so the forger’s output yields

$$[(\tau - z)^{-1}]G = [(y' - y)^{-1}](\pi - \pi'),$$

exactly the element the assumption declares infeasible. $\square$

Remark 4.37 (The KZG trade). What KZG buys: commitments and evaluation proofs of *constant* size (one group element each) and *constant* verification time (two pairings), all independent of the degree d . What it costs is twofold. First, the setup is trusted in the strongest sense: the secret τ is a global trapdoor. Anyone who learns τ can forge accepting evaluation proofs for *false* statements—for any commitment C , any point z , and any desired value y , the element $\pi := [(\tau - z)^{-1}](C - [y]G)$ passes the pairing check, no knowledge of any polynomial required. This is the promised demonstration that a subverted setup silently destroys binding (Remark 4.3); evaluation binding survives only if τ is provably destroyed, in practice by a multi-party *powers-of-tau* ceremony whose trapdoor stays secret as long as at least one participant is honest. Second, the construction requires pairing-friendly curves, a more delicate and less efficient setting than the ordinary curves of §2.6. Binding is computational either way, resting on the bilinear assumption—consistent with Theorem 4.8.

The inner-product alternative

The transparent alternative commits to the coefficient vector with the Pedersen vector commitment and replaces the pairing check by a recursive argument.

Construction 4.38 (IPA-style polynomial commitment). Let $G = (G_0, \dots, G_d)$ and H be transparent generators with unknown mutual discrete logs, derived from a public seed (Construction 4.22). Commit to $f = \sum_{i=0}^d a_i X^i$ by the Pedersen vector commitment of its coefficient vector $a = (a_0, \dots, a_d)$:

$$\text{Commit}(pp, f; r) := \sum_{i=0}^d [a_i]G_i + [r]H = \langle a, G \rangle + [r]H.$$

No pairings, no trusted setup.

Evaluation is an inner product: writing $z = (1, z, z^2, \dots, z^d)$ for the public vector of powers of the query point,

$$f(z) = \sum_{i=0}^d a_i z^i = \langle a, z \rangle,$$

so proving an evaluation reduces to proving an inner-product relation about a committed vector against a public vector. The *inner-product argument* (IPA) proves it recursively: each round folds the two halves of the coefficient vector—and of the generator vector—into vectors of half the dimension by taking a random linear combination dictated by a verifier challenge, sending two group elements per round (the cross-term commitments L_j and R_j) to account for the mixed terms; after $\log_2(d + 1)$ rounds a single scalar remains, and the verifier checks one final equation tying it to the folded commitment and the folded generators.

Proposition 4.39 (Properties of the IPA-based PCS). *The inner-product polynomial commitment has: transparent setup (public-coin parameters, no trapdoor); evaluation proofs of $O(\log d)$ group elements; and $O(d)$ -group-operation verification, batchable across proofs. Its binding and evaluation binding reduce to the discrete logarithm assumption in $\mathbb{G}$.*

Proof idea. Binding of the commitment itself is Theorem 4.23. For evaluation binding and extractability, one shows the folding protocol is *tree special-sound*: from a suitable tree of accepting transcripts, branching over the verifier’s challenge at each round, an extractor works back up the recursion and computes a coefficient vector consistent with the commitment and the claimed inner product—a notion this sketch uses only informally and §9 defines precisely. Two distinct extracted openings of the same commitment would yield a nontrivial discrete-log relation among G, H , contradicting DLog by the every-generator embedding. The $O(\log d)$ proof size is the halving recursion; the $O(d)$ verification is the final multiscalar multiplication over the unfolded generators. $\square$

Remark 4.40 (KZG versus IPA, and the Halo lineage). The two families mark out the design space. KZG gives constant proof size and constant verification, at the price of a structured trusted setup and pairing-friendly curves; the IPA gives transparent setup over a plain curve and rests on nothing beyond discrete log, at the price of logarithmic proofs and linear-time verification. The Halo 2 proof system deployed in Zcash Orchard builds on the inner-product family precisely to avoid trusted setup, and recovers practical verification cost through *batching*: the deployed verifier folds the final multiscalar multiplications of many proofs into one shared multiexponentiation—each proof’s deferred check is scaled by a fresh random factor and accumulated, and a single combined check is evaluated once (`halo2_proofs` crate, `src/plonk/verifier/batch.rs`, `BatchVerifier`; consumed through orchard’s `src/bundle/batch.rs`, `BatchValidator`). Batching amortises the per-proof group operations across a batch, though per-proof verifier work remains linear in d . The Halo lineage’s accumulation technique extends the same deferral across *recursively composed* proofs, one proof attesting to another’s deferred check—the capability the Pasta cycle of Remark 2.26 was designed to serve—but deployed Zcash does not use recursion: each Orchard proof is verified (in batch) directly.

Remark 4.41 (Hiding variants). As stated, the KZG commitment is binding but not hiding: it is a deterministic function of f , so committing twice to the same polynomial is visible, and a low-entropy polynomial can be found by search—the failure of Remark 4.14 in new clothing. That is acceptable when the polynomial encodes only public data. Whenever it encodes secrets—as the witness polynomials of a zero-knowledge proof do—randomness is required: KZG is augmented with a blinding summand $[r][\gamma]G$ using an extra setup element $[\gamma]G$, while the IPA commitment of Construction 4.38 already carries its $[r]H$ and is perfectly hiding by Theorem 4.23. The resulting regime is the perfect-hiding/computational-binding profile of Pedersen (Theorems 4.17 and 4.18), lifted from field elements to polynomials.

From commitments to arguments

The polynomial commitment is where the section’s main line hands over to the proof systems of §9, and the hand-over can be stated in one paragraph. A polynomial commitment scheme turns an information-theoretic proof object—a polynomial identity holding over a large field—into a succinct cryptographic argument. The prover commits to its witness polynomials; the verifier challenges at random points; evaluation proofs then convince the verifier that the committed polynomials satisfy the required identities, without the polynomials ever being revealed. Soundness is supplied by Schwartz–Zippel (*Math Guide*, §“The Schwartz–Zippel lemma”): a nonzero polynomial of low degree is almost surely nonzero at a random point, so an identity that holds at the challenge almost surely holds identically. Binding and succinctness are supplied by the commitment: the prover is locked to its polynomials *before* seeing the challenge, and the verifier handles only short commitments and proofs in place of the polynomials themselves. How this compilation is carried out in full—and what

“argument of knowledge” precisely means—is the business of §9, whose SNARK compilation subsection develops the evaluation-argument role sketched here.

4.10 Secret sharing

The section opened by naming verifiable secret sharing among the classical applications of commitments; it closes by constructing it, because the threshold signatures of the *FROST Guide* stand on it and nothing below that volume otherwise provides it. The setting is a prime field $\mathbb{F}_q$ —in the sequel the scalar field of the group $\mathbb{G}$ of §4.6—and the tool is Lagrange interpolation (*Math Guide*, §“Lagrange interpolation”): a polynomial of degree at most $t - 1$ over $\mathbb{F}_q$ is determined by its values at any t distinct points, and any t prescribed values at distinct points are attained by exactly one such polynomial.

Construction 4.42 (Shamir t -of- n secret sharing). Fix a threshold t and a party count n with $1 \leq t \leq n < q$. To share a secret $s \in \mathbb{F}_q$, the dealer samples coefficients $a_1, \dots, a_{t-1} \xleftarrow{\$} \mathbb{F}_q$ independently and uniformly and forms

$$f(X) := s + a_1X + a_2X^2 + \dots + a_{t-1}X^{t-1} \in \mathbb{F}_q[X], \quad f(0) = s,$$

a polynomial of degree at most $t - 1$ with constant term the secret. Party $i \in \{1, \dots, n\}$ receives the share $s_i := f(i)$, sent privately; the identifiers $1, \dots, n$ are distinct nonzero elements of $\mathbb{F}_q$ because $n < q$. *Reconstruction* from the shares of any set I of t parties is interpolation at 0:

$$s = f(0) = \sum_{i \in I} \lambda_{I,i} s_i, \quad \lambda_{I,i} := \prod_{j \in I, j \neq i} \frac{j}{j - i},$$

where $\lambda_{I,i} = \ell_i(0)$ is the Lagrange basis polynomial for the nodes I evaluated at 0, computable from the identifiers alone.

Correctness of reconstruction is the interpolation theorem: the t points $(i, s_i)_{i \in I}$ determine a unique polynomial of degree at most $t - 1$, which is f , and the Lagrange formula $f(X) = \sum_{i \in I} s_i \ell_i(X)$ evaluated at $X = 0$ gives the displayed sum. The coalition may be chosen after the shares are dealt, since the weights depend only on I .

Theorem 4.43 (Perfect secrecy below the threshold). *Let I be any set of at most $t - 1$ identifiers. For every secret $s \in \mathbb{F}_q$, the shares $(s_i)_{i \in I}$ of Construction 4.42 are jointly uniform on $\mathbb{F}_q^{|I|}$. In particular their distribution does not depend on s : fewer than t shares are independent of the secret, and an adversary of unbounded power holding them has advantage exactly 0 at telling any two candidate secrets apart.*

Proof. It suffices to treat $|I| = t - 1$, since fewer shares are a projection of these. Fix s and I , and consider the map

$$\Sigma_s : \mathbb{F}_q^{t-1} \rightarrow \mathbb{F}_q^{t-1}, \quad (a_1, \dots, a_{t-1}) \mapsto (f(i))_{i \in I},$$

from the dealer’s coefficient vector to the coalition’s shares. It is injective: two coefficient vectors with the same shares give two polynomials of degree at most $t - 1$ that agree at the t distinct points 0 (where both equal s) and I , hence are equal by uniqueness of interpolation, so the coefficient vectors coincide. It is surjective: for any target $(y_i)_{i \in I}$, Lagrange interpolation through the t points $(0, s)$ and $(i, y_i)_{i \in I}$ produces a polynomial of degree at most $t - 1$ with constant term s , whose remaining coefficients are a preimage. So Σ_s is a bijection—for every s —and it carries the uniform distribution on coefficient vectors to the uniform distribution on $\mathbb{F}_q^{t-1}$. The share vector is therefore uniform whatever the secret, and for any $s_0 \neq s_1$ the two share distributions coincide exactly; by the variational characterisation of statistical distance (Proposition 1.7) no procedure, efficient or not, distinguishes them. $\square$

The theorem is the perfect-hiding half of a commitment (Proposition 4.5 in different clothing): the coalition’s view is a uniformly random point of $\mathbb{F}_q^{t-1}$, consistent with every candidate secret through exactly one polynomial each. What Shamir sharing lacks is the binding half. Nothing stops a dishonest dealer from handing out values that lie on no common polynomial of degree at most $t - 1$, so that different coalitions of t parties reconstruct different “secrets”—and a party receiving a share cannot tell. Verifiable secret sharing repairs this by having the dealer commit to the polynomial.

Construction 4.44 (Feldman verifiable secret sharing). Let $(\mathbb{G}, q, G)$ be as in §4.6, so that $\mathbb{F}_q$ is the scalar field of $\mathbb{G}$. The dealer shares s as in Construction 4.42, with $f(X) = \sum_{k=0}^{t-1} a_k X^k$ and $a_0 = s$, and additionally *broadcasts* the coefficient commitments

$$\Phi_k := [a_k]G \in \mathbb{G}, \quad k = 0, \dots, t-1,$$

the coefficients in the exponent. Party i , on receiving its private share s_i , accepts it iff

$$[s_i]G \stackrel{?}{=} \sum_{k=0}^{t-1} [i^k] \Phi_k.$$

An honest share passes: since $a \mapsto [a]G$ is a homomorphism $\mathbb{F}_q \rightarrow \mathbb{G}$, $\sum_k [i^k] \Phi_k = \sum_k [a_k i^k]G = [f(i)]G$.

Proposition 4.45 (What the Feldman check certifies). *Because G generates the prime-order group $\mathbb{G}$, the map $a \mapsto [a]G$ is a bijection $\mathbb{F}_q \rightarrow \mathbb{G}$, so the broadcast vector $(\Phi_0, \dots, \Phi_{t-1})$ —however the dealer chose it—determines a unique polynomial $f^*(X) := \sum_k a_k^* X^k$ with $a_k^* = \log_G \Phi_k$, of degree at most $t - 1$. Party i ’s check passes iff $s_i = f^*(i)$. Consequently every coalition of t parties holding accepted shares reconstructs the same polynomial f^* and the same secret $s^* = f^*(0) = \log_G \Phi_0$, and a party handed an inconsistent share detects it alone and with certainty. The check certifies nothing about the value of s^* , which the dealer chooses freely.*

Proof. Both claims are injectivity of $a \mapsto [a]G$. The right-hand side of the check equals $[f^*(i)]G$ by the homomorphism, so the equation $[s_i]G = [f^*(i)]G$ holds iff $s_i = f^*(i)$. Accepted shares are thus evaluations of the one polynomial f^* , and t of them interpolate to it by uniqueness. $\square$

In the language of this section, the vector $(\Phi_k)_k$ is a *perfectly binding* commitment to the polynomial f —the coefficient list committed entry by entry under the bijection $a \mapsto [a]G$ —and the share check is an evaluation check against it, verifiable by anyone because it is linear in the coefficients. Perfect binding costs hiding, as Theorem 4.8 demands.

Remark 4.46 (The leak, and where it is harmless). Feldman sharing publishes $\Phi_0 = [s]G$. Secrecy below the threshold is no longer perfect: from the broadcast alone an unbounded adversary computes $s = \log_G \Phi_0$. For an efficient coalition of $t - 1$ parties the leak is exactly $[s]G$ and nothing more: its entire view—shares and broadcast—is computable from $[s]G$ and $t - 1$ uniformly random field elements, because the shares are uniform by Theorem 4.43 and the Φ_k are then determined by interpolating in the exponent through $(0, [s]G)$ and the points $(i, [s_i]G)$. Recovering s from that view is the discrete-logarithm problem (Definition 2.2). The leak is acceptable exactly when s is itself a discrete-logarithm secret key whose public key $[s]G$ is published anyway; then Φ_0 is the public key and nothing new escapes. When the secret must stay perfectly hidden, Pedersen verifiable secret sharing replaces each Φ_k by the Pedersen commitment $[a_k]G + [b_k]H$ of Construction 4.16, restoring perfect hiding at the price of computational binding—off this volume’s path and not constructed here. Threshold signing is the consumer: in the *FROST Guide* each signer holds a Shamir share of a signing key, the

coefficient commitments broadcast during key generation are Feldman’s, and the Lagrange weights $\lambda_{I,i}$ recombine per-signer contributions into a single signature.

5 Pseudorandom functions, block ciphers, and format-preserving encryption

The adversary of this section sits before a black box. She may submit any input she likes and read off the answer; she may let each query depend on everything the box has said so far; and when she has asked all she cares to ask, she must announce a single bit: does a short secret key drive the box, or blind chance? If no efficient strategy tells the difference, then λ bits of key have bought a perfect counterfeit of an object too large to exist in the world—a uniformly random function—and everything downstream that needs random-looking values on demand can draw them from that key. If she *can* tell, forgery follows at once. Key a Merkle–Damgård hash by naive concatenation and she extends an authentication tag she was shown onto a message nobody authenticated, without ever learning the key. Derive the diversifiers behind Orchard’s shielded addresses by any map she can predict or invert, and she links every address of one wallet to its siblings. One adversary—the *oracle distinguisher*—thus carries the whole section, and every definition in it is the same distinguishing game at a different strength: pseudorandom functions against random functions, pseudorandom permutations against random permutations, strong pseudorandom permutations against a two-sided oracle. The switching lemma prices the substitution of one ideal object for the other; AES supplies the deployed permutation; keyed BLAKE2 supplies the PRFs this section inventories; and format-preserving encryption closes the section by manufacturing, from a PRF that is not invertible, the keyed bijection on an 11-byte format that Orchard’s address derivation demands.

Notation is that of the provable-security model. Strings of length ℓ form $\{0, 1\}^\ell$, all finite strings form $\{0, 1\}^*$, and $|x|$ is the length of x ; for a finite set S , the notation $x \xleftarrow{\$} S$ draws x uniformly from S , independently of everything else. Adversaries are PPT machines in the security parameter λ (§1.1), a negligible function is $\text{negl}(\lambda)$ (recalled in §1.2), and every security notion below is a distinguishing game in the sense of Definition 1.15, scored by the difference

$$\text{Adv}(\mathcal{A}) = |\Pr[\mathcal{A} \text{ outputs 1 in Exp}_1] - \Pr[\mathcal{A} \text{ outputs 1 in Exp}_0]|$$

between the two experiments the game distinguishes. One device remains to be formalised: the black box itself.

Definition 5.1 (Oracle access). For a function f , the notation $\mathcal{A}^f$ denotes an adversary granted *oracle access* to f : a black box to which $\mathcal{A}$ may submit inputs x of its choice, receiving $f(x)$ in a single computation step, and from which it learns nothing about f beyond the answers to the queries it makes. The adversary’s running time always bounds the number of its oracle queries, which is hence polynomial in λ .

5.1 Pseudorandom functions and permutations

Pseudorandomness is measured against an ideal object, which must come first: the “truly random function” has to be made precise before anything can be said to imitate it.

Definition 5.2 (Random function). Let $\text{Func}(D, R)$ denote the set of all functions from a finite set D to a finite set R . A *random function* from D to R is an element $F \xleftarrow{\$} \text{Func}(D, R)$, chosen uniformly at random. Equivalently, the values $\{F(x)\}_{x \in D}$ are independent and uniformly distributed over R .

There are $|R|^{|D|}$ functions in $\text{Func}(D, R)$, an astronomically large set for the domains of interest: for $D = R = \{0, 1\}^{128}$ the count is $2^{128 \cdot 2^{128}}$. No such function can be stored or transmitted—its truth

table is the only description it has, and the table does not fit in the universe. The essential property of a pseudorandom function is that it replaces this object by a short key while preserving its observable behaviour. The clean mental model of the ideal object is *lazy sampling*, exactly as for the random oracle of Definition 3.13: the oracle keeps a table, initially empty; on a fresh query x it samples $F(x) \xleftarrow{\$} R$, records the pair, and returns it; on a repeated query it returns the recorded value. This produces exactly the distribution of Definition 5.2 while only ever materialising the points actually queried.

Definition 5.3 (Keyed function). A *keyed function* is a deterministic, efficiently computable map

$$F : \{0, 1\}^\lambda \times \{0, 1\}^{\ell(\lambda)} \longrightarrow \{0, 1\}^{m(\lambda)}, \quad (k, x) \longmapsto F_k(x),$$

where the input length ℓ and output length m are polynomially bounded functions of the key length λ . Fixing the key k yields the function $F_k : \{0, 1\}^{\ell(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$.

Definition 5.4 (Pseudorandom function). Let F be a keyed function. The distinguishing game $\text{PRF}_F^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples a bit $b \xleftarrow{\$} \{0, 1\}$. If $b = 1$ it samples $k \xleftarrow{\$} \{0, 1\}^\lambda$ and sets $\mathcal{O} := F_k$; if $b = 0$ it sets $\mathcal{O} := R$ for $R \xleftarrow{\$} \text{Func}(\ell(\lambda), m(\lambda))$, realised by lazy sampling, where $\text{Func}(\ell, m) := \text{Func}(\{0, 1\}^\ell, \{0, 1\}^m)$.
2. The adversary $\mathcal{A}(1^\lambda)$, receiving the security parameter in unary to convey the parameter size, queries $\mathcal{O}$ adaptively.
3. The adversary outputs a bit b' ; the game outputs 1 iff $b' = b$.

The family F is a *pseudorandom function* (PRF) if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_F^{\text{prf}}(\mathcal{A}, \lambda) := \left| \Pr_{k \xleftarrow{\$} \{0, 1\}^\lambda} [\mathcal{A}^{F_k}(1^\lambda) = 1] - \Pr_{R \xleftarrow{\$} \text{Func}(\ell(\lambda), m(\lambda))} [\mathcal{A}^R(1^\lambda) = 1] \right| \in \text{negl}(\lambda).$$

The two probabilities are the adversary's output distributions in the worlds $b = 1$ and $b = 0$, so this difference equals twice the bit-guessing advantage $|\Pr[\text{PRF}_F^{\mathcal{A}}(\lambda) = 1] - \frac{1}{2}|$ of Definition 1.15; the factor of two is immaterial to negligibility, and the difference form is the one every bound below is stated in.

Stated informally: an adversary handed a black box that is either F_k for a secret random key or a freshly sampled truly random function, and allowed to query it adaptively, cannot tell which it holds. Two features of the definition carry the weight. First, the algorithm defining F is public; only the sampled key is hidden. The adversary obtains oracle access to the resulting keyed instance F_k through Definition 5.1—never the key, only its input–output behaviour. Second, the comparison object is the gigantic, unstorable random function; the PRF compresses it into λ bits while remaining indistinguishable to every efficient observer.

That last qualifier is not removable: pseudorandomness is an inherently computational notion. Information theory cannot afford the trade. As k ranges over its 2^λ values, the keyed function F_k takes at most 2^λ distinct values in $\text{Func}(D, R)$ —a vanishing fraction of all $|R|^{|D|}$ functions—so the two distributions of Definition 5.4 are statistically almost as far apart as distributions can be, and an *unbounded* adversary could in principle distinguish them. Security holds only against computationally bounded distinguishers, in precisely the manner of the PRG separation in the proof of Proposition 1.10.

Remark 5.5 (Necessity of adaptivity). The adversary chooses its queries; it need not commit to them in advance, and later queries may depend on earlier answers. This *adaptive* access is the strongest

reasonable model and the one downstream constructions rely on. A weaker non-adaptive definition, in which the adversary fixes all queries before seeing any answer, is genuinely weaker—families exist that pass every non-adaptive test yet fall to an adaptive distinguisher—and is insufficient for many applications.

A block cipher is not merely a keyed function but a keyed *permutation*: each key must yield an invertible map, or decryption would be impossible. The relevant ideal object is therefore a random permutation.

Definition 5.6 (Keyed permutation). A keyed function $F : \{0, 1\}^\lambda \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$ with equal input and output length is a *keyed permutation* if for every key k the map F_k is a bijection of $\{0, 1\}^\ell$, and both F_k and F_k^{-1} are efficiently computable given k . The set of all permutations of $\{0, 1\}^\ell$ is $\text{Perm}(\ell)$, of cardinality $(2^\ell)!$.

Definition 5.7 (Pseudorandom permutation). Let F be a keyed permutation. The game $\text{PRP}_F^A(\lambda)$ is the game of Definition 5.4 with the ideal world replaced: for $b = 0$ the challenger sets $\mathcal{O} := P$ for $P \xleftarrow{\$} \text{Perm}(\ell)$, a uniformly random permutation. The family F is a *pseudorandom permutation* (PRP) if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_F^{\text{PRP}}(\mathcal{A}, \lambda) := \left| \Pr_k [\mathcal{A}^{F_k}(1^\lambda) = 1] - \Pr_{P \xleftarrow{\$} \text{Perm}(\ell)} [\mathcal{A}^P(1^\lambda) = 1] \right| \in \text{negl}(\lambda).$$

Definition 5.8 (Strong pseudorandom permutation). A keyed permutation F is a *strong pseudorandom permutation* (SPRP) if no PPT adversary distinguishes even when granted oracle access to *both* the permutation and its inverse: for every PPT $\mathcal{A}$,

$$\text{Adv}_F^{\text{SPRP}}(\mathcal{A}, \lambda) := \left| \Pr_k [\mathcal{A}^{F_k, F_k^{-1}}(1^\lambda) = 1] - \Pr_{P \xleftarrow{\$} \text{Perm}(\ell)} [\mathcal{A}^{P, P^{-1}}(1^\lambda) = 1] \right| \in \text{negl}(\lambda).$$

The distinction between a PRP and an SPRP is exactly the availability of a decryption oracle. A block cipher deployed where the attacker can obtain decryptions of chosen ciphertexts must be an SPRP; this is the natural security target for a block cipher, and the one the next subsection adopts.

From the outside, the two ideal objects are nearly the same thing. Query a length-preserving oracle on q distinct inputs. If the oracle is a random function $R \in \text{Func}(\ell, \ell)$, the answers are q independent uniform strings, and output collisions occur. If the oracle is a random permutation $P \in \text{Perm}(\ell)$, the answers are q *distinct* uniform strings, and no collision can occur. This is the *only* statistical difference visible to any observer, and it is the entire content of the switching lemma.

Lemma 5.9 (PRP/PRF switching lemma). Let $\ell \geq 1$ and let $\mathcal{A}$ be any adversary—even a computationally unbounded one—making at most q queries to its oracle. Then

$$\left| \Pr_{P \xleftarrow{\$} \text{Perm}(\ell)} [\mathcal{A}^P = 1] - \Pr_{R \xleftarrow{\$} \text{Func}(\ell, \ell)} [\mathcal{A}^R = 1] \right| \leq \frac{q(q-1)}{2^{\ell+1}}.$$

We use the lemma as a statement only. Its proof is a standard game-hopping argument (Remark 1.18) bounding the probability of the single distinguishing event just identified—an output collision among the q answers—and the bound is the birthday-type count $\binom{q}{2}/2^\ell$ familiar from Lemma 3.11, whose underlying calculation is the *Math Guide*’s (§“The union bound and a birthday calculation”).

The consequence is a licence: a secure PRP on a large domain is also a secure PRF whenever the number of queries stays well below the birthday bound $2^{\ell/2}$, because the adversary’s total advantage against the function world exceeds its advantage against the permutation world by at most $q(q-1)/2^{\ell+1}$. For AES, with $\ell = 128$, the switching term $q^2/2^{129}$ is negligible for any realistic query budget. This is the licence by which the FF1 construction of §5.4 treats AES—a permutation—as the PRF its round function requires.

5.2 Block ciphers and AES

Definition 5.10 (Block cipher). A *block cipher* with block length b and key length κ is a keyed permutation

$$E : \{0, 1\}^\kappa \times \{0, 1\}^b \longrightarrow \{0, 1\}^b,$$

together with its inverse $D = E^{-1}$ satisfying $D_k(E_k(x)) = x$ for all k, x . Here E_k is *encryption* and D_k *decryption* under key k . The security goal is that E be a strong pseudorandom permutation (Definition 5.8).

A block cipher is a concrete, fast, fixed-parameter instantiation of the SPRP abstraction, and the fixing of parameters changes the language of its security claims. The asymptotic PRP definition speaks of a family indexed by λ ; a real block cipher fixes b and κ once and for all— $b = 128$ and $\kappa \in \{128, 192, 256\}$ for AES—and offers *concrete* security in the resource-explicit style of Definition 1.31: an explicit function of the adversary’s resources bounds the advantage of the best known attacks.

Definition 5.11 (Concrete (t, q, ε) -security). A block cipher E is a (t, q, ε) -secure PRP if every adversary running in time at most t and making at most q oracle queries has $\text{Adv}_E^{\text{PRP}}(\mathcal{A}) \leq \varepsilon$; analogously for SPRP and PRF security.

Construction 5.12 (AES). The Rijndael cipher of Daemen and Rijmen, standardised as AES in FIPS 197, has block length $b = 128$ bits and key lengths $\kappa \in \{128, 192, 256\}$, with $N_r \in \{10, 12, 14\}$ rounds respectively. Structurally it is a *substitution-permutation network* (SPN): the 128-bit state, viewed as a 4×4 array of bytes—elements of $\mathbb{F}_{2^8}$ —is first whitened by an initial round-key XOR, then passes through N_r rounds built from four layers:

1. **SubBytes**, a nonlinear byte substitution—the *S-box*—built on field inversion $a \mapsto a^{-1}$ in $\mathbb{F}_{2^8}$ (composed with a fixed affine map), chosen so that every input difference maps to any given output difference with only small probability, and every linear approximation of it holds with probability close to one half;
2. **ShiftRows**, a transposition of bytes within the array;
3. **MixColumns**, an invertible column-wise linear mix; and
4. **AddRoundKey**, the XOR of a 128-bit round key that a key schedule expands from k .

The first $N_r - 1$ rounds compose all four layers in this order; the final round omits MixColumns. The two permutation layers together guarantee that after two rounds every output byte depends on every input byte. Each layer—the initial whitening included—is individually invertible, so E_k is a permutation as Definition 5.10 requires, and decryption applies the inverses in reverse order.

Remark 5.13 (Confusion, diffusion, and the status of AES security). The SPN design is the modern embodiment of Shannon’s two principles. The nonlinear S-box supplies *confusion*: making the relationship between key and ciphertext as complex as possible. The ShiftRows and MixColumns layers

supply *diffusion*: spreading the influence of each input bit over the whole output. Iterating the two for ten or more rounds is what gives AES its empirical resistance to all known attacks. No proof that AES is a PRP exists; like every practical block cipher, its security is a *conjecture*, corroborated by decades of failed cryptanalysis but resting on no reduction to a cleaner problem. Every theorem downstream therefore takes “ E is a PRP/SPRP” as an assumption and reasons from it—the same epistemic footing on which Remark 1.29 placed the concrete hash functions.

5.3 PRFs from hash functions: length extension and keyed BLAKE2

Block ciphers are one source of PRFs; cryptographic hash functions are another, often more convenient one, being unkeyed and ubiquitous. Here the oracle distinguisher does not merely distinguish—she forges. The naive keying of a hash H by prepending the key,

$$F_k(m) = H(k \parallel m),$$

is unsafe for the iterated Merkle–Damgård hashes (MD5, SHA-1, SHA-256)—which pad the message into blocks, fold the blocks one at a time into a fixed-width chaining state through a compression function, and emit the final state—because of the *length-extension attack*: from $H(k \parallel m)$ and $|m|$ alone, the adversary computes $H(k \parallel m \parallel \text{pad} \parallel m')$ for a suffix m' of her choosing, *without knowing* k . The reason is structural: a Merkle–Damgård digest is exactly the internal chaining state after absorbing $k \parallel m$, so the published output is a resumable snapshot of the iteration, and the attacker simply continues absorbing blocks from where the honest computation stopped. Used as a message authentication code (MAC)—a keyed function whose output tag certifies a message, defined under “Message authentication codes” in the next section—the naive construction thus surrenders a valid tag on $m \parallel \text{pad} \parallel m'$, a message the key holder never authenticated.

The classical repair is HMAC, the nested double call

$$\text{HMAC}_k(m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$$

for two fixed pad constants: the inner hash absorbs the message under one key-derived value, and the outer hash re-keys the fixed-length result, so the emitted digest is no longer a resumable chaining state. Bellare’s analysis shows that HMAC is a PRF whenever the underlying compression function is, up to a birthday term $q^2/2^\ell$ in the chaining-state length ℓ ; as a PRF it is in particular a secure message authentication code, and it is the building block of the standardised key-derivation function HKDF, noted in passing under “Key-derivation functions” in the next section. This summary is all we need, because the shielded protocol uses HMAC nowhere: its hash of choice admits direct keying.

Modern hash functions of the sponge family (Construction 3.26) and of the HAIFA family—the Merkle–Damgård variant whose compression function also takes the count of bits hashed so far, the lineage of BLAKE2—do not suffer length extension and so admit a far simpler keying. The BLAKE2 design is built to be keyed *natively*: the construction loads the key length into its parameter block and prepends the key, padded to a full block, as the first message block, and its compression function mixes in a block counter and finalisation flags. (Its successor BLAKE3 is natively keyed too, by a different mechanism: the key words replace the initial state, and a dedicated flag marks the keyed mode.) One simply writes

$$F_k(m) = \text{BLAKE2}(m; \text{key} = k)$$

and obtains a PRF directly—no nested double call. No length-extension attack remains to repair: the finalisation flag, XORed into the state for the last block, means the emitted digest is *not* the raw chaining value and cannot be used to resume the computation.

Remark 5.14 (Zcash’s use of keyed and personalised BLAKE2). The Zcash protocol uses BLAKE2b and BLAKE2s pervasively, exploiting two parameters of the BLAKE2 design: the *key*, for PRF and MAC use as above, and the *personalisation* string, a domain-separation tag baked into the initial state—16 bytes for BLAKE2b, 8 bytes for BLAKE2s. Personalisation makes the same input hashed

for two purposes—nullifier derivation versus note-commitment derivation, say—yield independent-looking outputs, so each use site is effectively an independent PRF: the practical realisation of the independent-oracle-per-context idiom that Proposition 3.17 proved for tagged random oracles, obtained here before a single message byte is processed and cheaper and cleaner than HMAC, because BLAKE2 provides keying and domain separation as first-class features. The deployed inventory spans both widths. On the BLAKE2b side, the key-expansion PRF $\text{PRF}^{\text{expand}}$ is BLAKE2b-512 personalised with `Zcash_ExpandSeed` (`zcash_spec` crate, `src/prf_expand.rs`; protocol specification §“Pseudo Random Functions”), evaluated on the spending key to derive Orchard’s key tree (`orchard` crate, `src/keys.rs`); the note-encryption KDF is personalised with `Zcash_OrchardKDF` (`orchard` crate, `src/keys.rs`; §“Orchard Key Derivation”) and the outgoing cipher key with `Zcash_Orchardock` (`orchard` crate, `src/note_encryption.rs`; §“Pseudo Random Functions”)—all 16-byte tags. On the BLAKE2s side, the nullifier PRF and fixed-base generators of Sapling—the shielded protocol preceding Orchard—use 8-byte personalisations such as `Zcash_nf` and `Zcash_PH` (`sapling-crypto` crate, `src/constants.rs`; §“Pseudo Random Functions” and §“Pedersen Hash Function”).

5.4 Format-preserving encryption and FF1

All constructions so far operate on bit strings of a length the cipher’s block size dictates. The deployed need that closes this section does not. Orchard derives the *diversifiers* behind its shielded addresses—one 11-byte value per address, many addresses per spending key—by encrypting a sequential address index with a keyed bijection on the set of 11-byte strings, so that the published diversifiers of one wallet look like unrelated random values rather than the consecutive integers they conceal. An adversary who could invert the map, or merely distinguish it from a random permutation of the format, would link a wallet’s addresses to one another. What is needed is precisely a block cipher whose “block” is an exotic finite set; the classical commercial need—encrypting a 16-digit card number into a 16-digit card number so that legacy databases, field-length constraints, and checksum formats continue to accept it—is the same problem. This is the problem *format-preserving encryption* (FPE) solves.

Definition 5.15 (Format-preserving encryption). Let $\mathcal{X}$ be a finite set, the *format*—for example $\mathcal{X} = \{0, 1, \dots, 9\}^{16}$, the 16-digit strings. A *format-preserving encryption scheme* on $\mathcal{X}$ is a keyed permutation $\mathcal{E} : \{0, 1\}^k \times \mathcal{X} \rightarrow \mathcal{X}$ such that for every key k the map $\mathcal{E}_k : \mathcal{X} \rightarrow \mathcal{X}$ is a bijection with efficiently computable inverse $\mathcal{D}_k$. Security requires $\mathcal{E}$ to be a (strong) pseudorandom permutation on $\mathcal{X}$ —Definition 5.7 with $\text{Perm}(\mathcal{X})$ in place of $\text{Perm}(\{0, 1\}^\ell)$: no efficient adversary distinguishes $\mathcal{E}_k$ from a uniformly random permutation of $\mathcal{X}$, even with access to the inverse.

The defining feature is that the domain $\mathcal{X}$ is *arbitrary*—in particular $|\mathcal{X}|$ need not be a power of two, so a plain block cipher does not apply. What is required is a keyed pseudorandom *bijection* on a set whose size is, say, 10^{16} . Determinism is forced, and it separates FPE from the randomised encryption modes of the next section: the format $\mathcal{X}$ has no room to store a per-message random value, so FPE is the deterministic, format-preserving analogue of a block cipher on the alphabet of $\mathcal{X}$. It necessarily leaks equality of plaintexts, as any deterministic cipher must; this is the unavoidable cost of preserving format with no expansion, and it is acceptable for the tokenisation use cases FPE targets—and immaterial for diversifier derivation, where each index is encrypted once.

Formats reduce to integers. Any format of the shape “strings of length ν over an alphabet of radix ρ ” is in bijection with $\{0, 1, \dots, \rho^\nu - 1\}$ by reading the string as a base- ρ numeral, so it suffices to build a keyed pseudorandom permutation of $\mathbb{Z}/M\mathbb{Z}$ for an arbitrary modulus $M = \rho^\nu$. The FF1 construction below splits $M = a \cdot b$ into two factors of nearly equal size and runs a *Feistel network* whose two halves live in $\mathbb{Z}/a\mathbb{Z}$ and $\mathbb{Z}/b\mathbb{Z}$. The Feistel network is the structural device that turns a round function—not necessarily invertible—into a permutation; we present the classical bitwise case first, then the modular generalisation FF1 uses.

Definition 5.16 (Balanced Feistel network). Let $F_1, \dots, F_r : \{0, 1\}^w \rightarrow \{0, 1\}^w$ be arbitrary *round functions*. The r -round Feistel network $\Psi[F_1, \dots, F_r]$ is the map on $\{0, 1\}^{2w}$ defined as follows: write the input as (L_0, R_0) with $L_0, R_0 \in \{0, 1\}^w$, iterate

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F_i(R_{i-1}), \quad i = 1, \dots, r,$$

and output (L_r, R_r) .

Proposition 5.17 (Feistel is a bijection regardless of the round functions). *For any round functions $F_1, \dots, F_r$ —injective or not—the map $\Psi[F_1, \dots, F_r]$ is a bijection of $\{0, 1\}^{2w}$, inverted by running the rounds backwards:*

$$R_{i-1} = L_i, \quad L_{i-1} = R_i \oplus F_i(L_i), \quad i = r, \dots, 1.$$

Proof. It suffices to show each round is invertible, the composition of bijections being a bijection. The i -th round sends $(L_{i-1}, R_{i-1}) \mapsto (R_{i-1}, L_{i-1} \oplus F_i(R_{i-1}))$. Given the output (L_i, R_i) , one recovers $R_{i-1} = L_i$ directly, and then $L_{i-1} = R_i \oplus F_i(R_{i-1}) = R_i \oplus F_i(L_i)$, using that XOR is its own inverse. The recovery uses only *forward* evaluations of F_i —at no point is an inverse of the round function required, which is the crucial point permitting a permutation to be built out of a one-way-ish PRF. The displayed backward recursion is exactly this inversion applied round by round. $\square$

This proposition is the structural heart of format-preserving encryption: it manufactures a *keyed bijection* from a keyed function that is not itself a bijection. In FF1 the round functions derive from a PRF—AES iterated in cipher-block-chaining fashion, as the construction below specifies—so the network is invertible without ever inverting the PRF: AES is only ever evaluated in the forward direction, in encryption and decryption alike.

Bijectivity is free; pseudorandomness is not. The Luby–Rackoff theorem is the classical statement that enough Feistel rounds over a PRF yield a pseudorandom permutation.

Theorem 5.18 (Luby–Rackoff). *Let the round functions $F_1, \dots, F_r$ be independent PRFs on $\{0, 1\}^w$. Against an adversary making q queries, the Feistel network Ψ is:*

1. *with $r = 3$ rounds, a pseudorandom permutation (forward queries only); and*
2. *with $r = 4$ rounds, a strong pseudorandom permutation (forward and inverse queries);*

in each case with distinguishing advantage $O(q^2/2^w)$ above the PRF advantage of the round functions.

We use the theorem as a statement, without proof, but the shape of the argument is worth recording, because it explains both the birthday term and the round counts. One first replaces each PRF round function by a truly random function, at the cost of r PRF advantage terms. For three rounds one then shows that, unless two of the at most q queries collide in an intermediate Feistel value—a birthday event of probability $O(q^2/2^w)$ —the middle round’s random function is evaluated at distinct inputs, so it produces independent uniform values that perfectly mask the relationship between input and output halves, and the network’s answers are distributed exactly as a random *function*’s—*independent and uniform on $\{0, 1\}^{2w}$* . A final application of the switching lemma (Lemma 5.9), at a further cost of $q(q-1)/2^{2w+1}$, carries the comparison from the random function to the random permutation; the term is absorbed by the same $O(q^2/2^w)$ bound. The fourth round adds the symmetry needed to resist inverse queries as well, upgrading PRP to SPRP. The full argument is a transcript analysis bounding the probability of any internal collision; the leftover term is the stated birthday bound.

Remark 5.19 (Why FF1 uses ten rounds rather than four). Theorem 5.18 gives security only up to $q \approx 2^{w/2}$ queries, and in the FPE setting w is tiny—one half of a 16-digit number—so the birthday bound $q^2/2^w$ is weak. The standardised FF1 therefore prescribes *ten* rounds rather than the minimal four, as a security margin against the more powerful attacks known for small-domain Feistel networks, which can sometimes beat the generic birthday bound. The classification of this design claim is worth making explicit: the round count itself is *specified*—NIST SP 800-38G fixes ten rounds—while a security proof matching the margin is an *open problem*; the analysis that exists (Luby–Rackoff and its small-domain refinements) does not certify the full strength the margin is hoped to buy. What the round count does *not* affect is invertibility, which Proposition 5.17 guarantees for any round count and any round function.

Construction 5.20 (The FF1 mode of NIST SP 800-38G). The FF1 mode instantiates an *unbalanced, modular, tweaked* Feistel network of ten rounds to build a PRP on $\mathbb{Z}/M\mathbb{Z}$ with $M = \rho^\nu$, the radix- ρ numeral reading of a length- ν string. It supports a *tweak* T : a non-secret string that selects, in effect, one permutation from a family indexed by T , so that the same key encrypts the same plaintext differently under different tweaks—useful for binding a token to its context. Its operation:

1. **Split.** Set $u = \lfloor \nu/2 \rfloor$ and $v = \nu - u$. Split the plaintext numeral string X into a left part A of u symbols and a right part B of v symbols, with integer values in $\{0, \dots, \rho^u - 1\}$ and $\{0, \dots, \rho^v - 1\}$ respectively. The two halves live in the moduli $a = \rho^u$ and $b = \rho^v$; the network is unbalanced when ν is odd.
2. **Round function from a PRF.** For each round $i = 0, 1, \dots, 9$, assemble a byte string Q encoding the tweak T , the round index i , and the current right half B ; prepend a fixed parameter block P encoding the radix, the lengths, and the tweak length; and apply $\text{PRF}(P \parallel Q)$, where PRF is AES in CBC-MAC mode: set $Y_0 = 0^{128}$, iterate $Y_j = \text{AES}_k(Y_{j-1} \oplus X_j)$ over the 16-byte blocks X_j of $P \parallel Q$, and output the final block Y_m (SP 800-38G, Algorithm 6). The iteration chains AES through an evolving state exactly as the Merkle–Damgård iteration of §5.3 chains a compression function; emitting the final state is safe *here* because the PRF’s inputs form a *prefix-free* set: the leading block P encodes the lengths of everything that follows, so two distinct inputs either differ already in their first block or agree on all lengths, and in neither case is one a proper prefix of the other. Prefix-free inputs are precisely the setting in which plain CBC-MAC is proved to be a PRF (a result of Petrank and Rackoff), and no length-extension query can arise. Further AES calls then expand the result to a long enough digest, which is read as an integer y .
3. **Modular Feistel step.** Update the halves by the modular analogue of Definition 5.16:

$$C = (A + y) \bmod m, \quad A \leftarrow B, \quad B \leftarrow C,$$

where the modulus m alternates between $a = \rho^u$ and $b = \rho^v$ with the parity of the round, so that the sum lands in the correct half’s domain; modular addition replaces XOR.

4. **Output.** After ten rounds, concatenate the final A and B and render the result back as a length- ν radix- ρ string.

Decryption runs the rounds in reverse, replacing modular addition by modular subtraction, $A \leftarrow (C - y) \bmod m$, exactly mirroring the backward recursion of Proposition 5.17. One quantitative detail of step 2 deserves note: the standard sizes the expanded digest at least four bytes beyond what the larger half requires, so the integer y ranges over at least 2^{32} multiples of the modulus, and the bias of the reduction $y \bmod m$ away from uniform is bounded by the modular-reduction

bias of the *Math Guide* (§“Uniform sampling and the bias of modular reduction”) at below 2^{-33} per round.

Proposition 5.21 (FF1 is a keyed bijection on $\mathcal{X}$). *For every key k and tweak T , the FF1 map $\mathcal{E}_{k,T} : \mathcal{X} \rightarrow \mathcal{X}$ is a bijection of the format set $\mathcal{X}$, and $\mathcal{D}_{k,T}$ is its two-sided inverse.*

Proof. Each round is the map $(A, B) \mapsto (B, (A + y) \bmod m)$, where $y = y(i, T, B)$ depends only on the round index, the tweak, and the right half B —not on A . Fix B , i , and T ; then y is a constant and $A \mapsto (A + y) \bmod m$ is a translation of $\mathbb{Z}/m\mathbb{Z}$, hence a bijection, inverted by subtracting the same constant. The full round is therefore invertible: from the output (B, C) recover B directly, recompute $y = y(i, T, B)$ by a *forward* PRF evaluation, and set $A = (C - y) \bmod m$. Each even-indexed round is a bijection of $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$ onto $\mathbb{Z}/b\mathbb{Z} \times \mathbb{Z}/a\mathbb{Z}$ and each odd-indexed round a bijection back, the two products coinciding when ν is even; the composition of the ten rounds, an even number, is a bijection of $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$. The numeral map $(A, B) \mapsto A \cdot b + B$ identifies $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$ with $\mathbb{Z}/M\mathbb{Z}$ as a bijection of sets only, not of groups: the factors satisfy $\gcd(a, b) = \rho^{\min(u, v)} > 1$, so the Chinese remainder theorem does not apply, and only bijectivity is used. Transporting through this map and the numeral bijection $\mathcal{X} \cong \mathbb{Z}/M\mathbb{Z}$, the composition $\mathcal{E}_{k,T}$ is a bijection of $\mathcal{X}$. Running the rounds backward with subtraction inverts it on both sides, so $\mathcal{D}_{k,T} = \mathcal{E}_{k,T}^{-1}$. $\square$

Remark 5.22 (Bijectivity from algebra, security from the PRF). The construction separates its two obligations cleanly, and the separation is why Feistel is the standard route to FPE. Invertibility comes for free from the structure: Propositions 5.17 and 5.21 hold for *any* round function whatsoever, including a broken or constant one. The PRF—AES—is responsible not for invertibility but for *pseudorandomness*: making $\mathcal{E}_{k,T}$ look like a uniformly random permutation of $\mathcal{X}$ in the sense of Definition 5.15, per the Luby–Rackoff *heuristic*—Theorem 5.18 adapted, beyond what it literally proves, to the modular, multi-round, tweaked setting.

Remark 5.23 (The domain-size caveat, and the single Orchard use). Two facts complete the picture, one cautionary and one deployed.

The domain-size caveat is real. The published SP 800-38G requires $\rho^\nu \geq 100$ and recommends $\rho^\nu \geq 10^6$ —its draft Revision 1 would make 10^6 mandatory—because for very small domains an attacker can recover or distinguish the keyed permutation by exhaustively cataloguing input–output pairs, and known message-recovery attacks on small-domain FPE (Bellare–Hoang–Tessaro and successors) erode the security margin. For the moderately large domains FPE is meant for, and with its ten rounds, FF1 is the NIST-standardised, AES-backed realisation of a keyed pseudorandom bijection on an arbitrary finite format.

The single use of FF1 in Orchard is the diversifier derivation with which this subsection opened: FF1-AES256 maps an address index to the 11-byte diversifier embedded in a shielded address, so that one spending key yields many mutually unlinkable addresses. The deployed instance (orchard crate, `src/keys.rs`: type `DiversifierKey`; protocol specification §“Pseudo Random Permutations”) fixes radix $\rho = 2$, length $\nu = 88$, and the empty tweak, and reads the 11-byte index as a little-endian bit string, so the domain is the 2^{88} strings of 88 bits—comfortably beyond the caveat above. The address built on the diversifier, and the case for a keyed permutation rather than a hash in this role, are the business of the *Ironwood Guide* (§“Diversified addresses”).

The toolbox has gained its keyed compartment: one distinguishing game at three strengths, a standard permutation conjectured to win it, a natively keyed hash that turns domain-separated digests into independent PRFs, and a Feistel construction that bends a PRF into a bijection on any finite format. The next section spends these tools on the channel itself: symmetric encryption, authentication, and the authenticated-encryption composition that carries every shielded note, each built by running a PRF in a mode that converts its indistinguishability into secrecy or integrity.

6 Symmetric encryption, AEAD, and key derivation

The adversary of this section owns the channel. Every ciphertext passes through her hands, and she attacks on two fronts at once. As an eavesdropper she tries to *distinguish*: to tell an encryption of one note plaintext from an encryption of another, or to recognise that two ciphertexts hide the same message. As a forger she tries to *modify*: a stream cipher’s ciphertext is XOR-malleable, so flipping a ciphertext bit flips the corresponding plaintext bit without any key material at all; she can also shift bytes between a packet’s public header and its encrypted body, replay a ciphertext under a different context, or manufacture a wholly new ciphertext that the receiver accepts. She is patient enough to exploit a single reused pad—two ciphertexts under one pad hand her the XOR of the plaintexts—and mathematically literate enough to exploit a key that is merely high-entropy rather than uniform. This section arms the honest parties against all of it, one guarantee at a time: confidentiality from the one-time pad through the nonce-based stream cipher ChaCha20 and the chosen-plaintext game; integrity from message authentication codes through universal hashing and Carter–Wegman; their composition as authenticated encryption with associated data, culminating in the ChaCha20-Poly1305 scheme that Zcash’s note-encryption layer deploys; and finally key derivation, the manufacture of the uniform key that every theorem in between takes as its hypothesis.

Throughout, an *adversary* is a probabilistic algorithm $\mathcal{A}$; the notation $y \leftarrow \mathcal{A}(x)$ denotes running $\mathcal{A}$ on input x and assigning its output to y , and $x \xleftarrow{\$} S$ denotes sampling x uniformly from the finite set S . Every adversary in the computational definitions is probabilistic polynomial time (PPT) in the security parameter λ , and $\text{negl}(\lambda)$ denotes a negligible function (*Math Guide*, §“Polynomial, exponential, and negligible functions”; recalled in §1.2).

The one-time pad and EAV security. The information-theoretic baseline is the *one-time pad*: to encrypt $m \in \{0, 1\}^L$ under a uniformly random key $k \in \{0, 1\}^L$ used exactly once, output $c = m \oplus k$. Since $m \oplus k$ is uniform whatever m is, the ciphertext distribution is independent of the message, and even an unbounded eavesdropper learns nothing—Shannon’s *perfect secrecy*. The price is severe on both counts the name advertises: the key must be as long as the message, and it must never be reused, for two ciphertexts under one pad reveal $c \oplus c' = m \oplus m'$ —the *two-time-pad* failure, which the adversary of the opening paragraph collects for free. The computational relaxation is *EAV security* (security against an eavesdropper): an adversary who chooses two equal-length messages m_0, m_1 and sees a single ciphertext $\text{Enc}_k(m_b)$ for a hidden uniform bit b guesses b with advantage at most $\text{negl}(\lambda)$ —the one-ciphertext, no-oracle restriction of the chosen-plaintext game formalised in §6.2. Replacing the truly random pad by the output of a pseudorandom generator stretched from a short seed—the *pseudo-one-time pad*—achieves EAV security with a fixed-length key, at the cost of resting on a pseudorandomness assumption.

6.1 Stream ciphers and ChaCha20

A *stream cipher* is the practical instantiation of the pseudo-one-time pad: a keyed, seekable generator producing a *keystream* that XORs into the plaintext. To encrypt many messages under one key without violating the one-time-pad no-reuse rule, modern stream ciphers take an additional public input called a *nonce* (number used once) and behave like a pseudorandom function of the nonce, in the sense of Section 5.

Definition 6.1 (Nonce-based stream cipher). A *nonce-based stream cipher* is a deterministic function

$$\text{KS} : \mathcal{K} \times \mathcal{N} \times \mathbb{N} \rightarrow \{0, 1\}^*$$

producing, for key k , nonce ν , and length L , a keystream $\text{KS}(k, \nu, L) \in \{0, 1\}^L$. Encryption of $m \in \{0, 1\}^L$ is $\text{Enc}_k(\nu, m) = m \oplus \text{KS}(k, \nu, |m|)$ with ciphertext (ν, c) ; decryption recomputes the

keystream and XORs it off. The security goal is that for distinct nonces the keystreams are jointly indistinguishable from independent uniform strings, provided no nonce repeats under a fixed key.

The stream cipher Zcash deploys—inside the AEAD of §6.5, in the note-encryption layer—is ChaCha20, designed by Bernstein and standardised in RFC 8439. Its core is a reversible mixing of four 32-bit words built only from modular addition, XOR, and rotation (an “ARX” design), which avoids data-dependent table lookups and hence the timing side channels that table-based ciphers must engineer around.

Construction 6.2 (ChaCha20). The ChaCha20 state is a 4×4 matrix of 32-bit words, viewed as a vector $(x_0, \dots, x_{15}) \in (\mathbb{Z}/2^{32}\mathbb{Z})^{16}$; the symbol $+$ denotes addition modulo 2^{32} , $\oplus$ XOR, and $x \lll r$ left rotation by r bits. The *quarter-round* $QR(a, b, c, d)$ updates four words by

$$\begin{aligned} a &+= b; & d &\oplus= a; & d &\lll 16; \\ c &+= d; & b &\oplus= c; & b &\lll 12; \\ a &+= b; & d &\oplus= a; & d &\lll 8; \\ c &+= d; & b &\oplus= c; & b &\lll 7. \end{aligned}$$

The state is initialised from the 128-bit constant “expand 32-byte k” (words x_0 – x_3), a 256-bit key (x_4 – x_{11}), a 32-bit block counter (x_{12}), and a 96-bit nonce (x_{13} – x_{15}). One *double round* applies QR to the four columns and then to the four diagonals; the 20 rounds of ChaCha20 are ten double rounds. Writing X for the initial state and $R(X)$ for the state after the rounds, the 64-byte keystream block is the *feed-forward* sum $R(X) + X$ (word-wise, modulo 2^{32}), serialised in little-endian order.

Remark 6.3 (Feed-forward and counter mode). The rounds R form a bijection—each quarter-round is invertible, being a composition of invertible word operations—so without the final addition the block function would be an invertible permutation, and an adversary holding a keystream block could run the rounds backwards. The feed-forward sum $R(X) + X$ destroys invertibility; it is the standard Davies–Meyer-style move for turning a permutation into a one-way mixing function. Incrementing the block counter x_{12} produces successive 64-byte keystream blocks, so $KS(k, \nu, L)$ is the concatenation of the blocks for counters $0, 1, 2, \dots$; this *counter mode* is what makes the cipher seekable and parallelisable. The security status is a conjecture, in the same heuristic position as every concrete cipher of Section 5: ChaCha20 is *conjectured* to be a secure PRF of the pair (nonce, counter), and no attack better than brute force over the 256-bit key is known. Reusing a (k, ν) pair reproduces the same keystream and re-creates the two-time-pad failure; nonce uniqueness is a hard requirement, not a hygiene recommendation.

6.2 Chosen-plaintext security

EAV security models an adversary who sees a single ciphertext. The adversary of this section is stronger: she can influence which messages get encrypted—by triggering payments, injecting plaintext into a session, or simply guessing—and she observes many ciphertexts. The corresponding notion is security against *chosen-plaintext attack* (CPA), modelled by granting her oracle access to the encryption algorithm, in the game template of Definition 1.15.

Definition 6.4 (CPA security). For a symmetric encryption scheme Π with key generator Gen , adversary $\mathcal{A}$, and bit $b \in \{0, 1\}$, the game $\text{CPA}_{\Pi}^{\mathcal{A}}(\lambda, b)$ runs as follows.

1. The challenger samples $k \leftarrow \text{Gen}(1^\lambda)$ and gives $\mathcal{A}$ oracle access to $\text{Enc}_k(\cdot)$.
2. At some point $\mathcal{A}$ submits a challenge pair (m_0, m_1) of equal-length messages and receives

$$c^* \leftarrow \text{Enc}_k(m_b).$$

3. The adversary may continue querying the oracle and finally outputs a bit b' , which is the output of the game.

The scheme Π is *CPA-secure* if for every PPT $\mathcal{A}$,

$$\text{Adv}_{\Pi}^{\text{cpa}}(\mathcal{A}, \lambda) := |\Pr[\text{CPA}_{\Pi}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{CPA}_{\Pi}^{\mathcal{A}}(\lambda, 1) = 1]| \in \text{negl}(\lambda).$$

The first consequence of giving the adversary an encryption oracle is a structural one: encryption cannot be a function of the message alone.

Proposition 6.5 (Determinism precludes CPA security). *No scheme whose Enc is deterministic and stateless is CPA-secure.*

Proof. The adversary queries the oracle on two distinct messages $m_0 \neq m_1$, recording $c_0 = \text{Enc}_k(m_0)$ and $c_1 = \text{Enc}_k(m_1)$. Decryption correctness forces $c_0 \neq c_1$: a single ciphertext cannot decrypt to both messages. She submits the challenge (m_0, m_1) , receives $c^* = \text{Enc}_k(m_b)$, and outputs 0 if $c^* = c_0$ and 1 otherwise. Since Enc is deterministic, $c^* = c_b$, and since $c_0 \neq c_1$ the comparison identifies b : the guess is always correct and the advantage is 1. $\square$

Consequently a CPA-secure scheme must be randomised or nonce-based. A nonce-based stream cipher used with a fresh nonce per encryption—random, or drawn from a counter—attains CPA security under the PRF conjecture for its keystream generator, because each encryption is then effectively a fresh one-time pad. We record the notion here and defer the routine PRF-based proof to the AEAD analysis (Theorem 6.20), where confidentiality and integrity are treated together.

Remark 6.6 (Semantic security). Goldwasser and Micali’s original notion, *semantic security*, demands that whatever an efficient adversary can compute about the plaintext from the ciphertext, she could have computed without the ciphertext, from the message length alone. Semantic security is provably equivalent to the indistinguishability notions above; we use indistinguishability throughout because it is far easier to manipulate in reductions. The content is the same: the ciphertext is computationally useless for learning anything about the message beyond its length.

6.3 Message authentication codes

CPA security protects confidentiality and says nothing about integrity, and for the XOR-based stream ciphers above the gap is not hypothetical: flipping a bit of the ciphertext flips the corresponding plaintext bit, so the channel-owning adversary can convert a payment of one amount into a payment of another without ever touching the key. Detecting tampering requires a second primitive, keyed like the cipher but aimed at the forger rather than the eavesdropper: the *message authentication code*. Here we build this symmetric, shared-key form of authentication, the form that the AEAD needs.

Definition 6.7 (Message authentication code). A *message authentication code* (MAC) is a pair $(\text{Mac}, \text{Vrfy})$ with key space $\mathcal{K}$: algorithm $\text{Mac}_k(m)$ outputs a tag t , and $\text{Vrfy}_k(m, t) \in \{0, 1\}$ accepts (1) or rejects (0). Correctness requires $\text{Vrfy}_k(m, \text{Mac}_k(m)) = 1$ for all k, m . When Mac is deterministic, canonical verification recomputes the tag and checks equality.

Definition 6.8 (Existential and strong unforgeability). For a MAC Π and adversary $\mathcal{A}$, the game $\text{Forge}_{\Pi}^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $k \xleftarrow{\$} \mathcal{K}$ and gives $\mathcal{A}$ oracle access to $\text{Mac}_k(\cdot)$; let Q be the set of messages queried.
2. The adversary outputs a pair (m^*, t^*) ; the game outputs 1 iff $\text{Vrfy}_k(m^*, t^*) = 1$ and $m^* \notin Q$.

The MAC is *existentially unforgeable under chosen-message attack* (EUF-CMA) if for every PPT $\mathcal{A}$,

$$\text{Adv}_{\Pi}^{\text{forge}}(\mathcal{A}, \lambda) := \Pr[\text{Forge}_{\Pi}^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

A *strongly unforgeable* (sUF-CMA) MAC additionally forbids producing a new valid tag on an already-queried message: the adversary wins on (m^*, t^*) as long as the oracle never returned that exact *pair*. We write $\text{Adv}_{\Pi}^{\text{s-forge}}(\mathcal{A}, \lambda)$ for the advantage in this variant.

The MAC Zcash uses, Poly1305, is not a pseudorandom-function MAC but a *one-time, universal-hash* MAC: it provides information-theoretic security per nonce and is extremely fast, at the cost of a key that authenticates only a single message. The enabling abstraction is a hash family whose *differences* are unpredictable.

Definition 6.9 (ε -almost- Δ -universal hash family). Let $(\mathcal{G}, +)$ be a finite abelian group. A family of functions $\{h_r : \mathcal{X} \rightarrow \mathcal{G}\}_{r \in \mathcal{R}}$ is ε -almost- Δ -universal (ε -A Δ U) if for all distinct $x, x' \in \mathcal{X}$ and every $\delta \in \mathcal{G}$,

$$\Pr_{r \leftarrow \mathcal{R}} [h_r(x) - h_r(x') = \delta] \leq \varepsilon.$$

Taking $\delta = 0$ recovers ordinary ε -universality—low collision probability—while the Δ condition controls *additive differences*, which is what makes the one-time MAC below secure. With $\mathcal{G} = \{0, 1\}^n$ under XOR this is the classical almost-XOR-universality; Poly1305 takes $\mathcal{G} = \mathbb{Z}/2^{128}\mathbb{Z}$ under addition modulo 2^{128} .

The Carter–Wegman paradigm turns any A Δ U family into a one-time MAC by masking the hash value with a fresh uniform pad.

Theorem 6.10 (Carter–Wegman one-time MAC). Let $\{h_r\}$ be ε -A Δ U into $(\mathcal{G}, +)$. Define a MAC with key (r, s) , where $r \xleftarrow{\$} \mathcal{R}$ and the pad $s \xleftarrow{\$} \mathcal{G}$, by $\text{Mac}_{r,s}(m) = h_r(m) + s$. If (r, s) authenticates at most one message, then any adversary—even a computationally unbounded one—who sees one message–tag pair (m, t) forges a valid pair (m^*, t^*) with $m^* \neq m$ with probability at most ε .

Proof. The adversary observes $t = h_r(m) + s$ and must output (m^*, t^*) with $m^* \neq m$ and $t^* = h_r(m^*) + s$. Subtracting the two tag equations eliminates the pad: the forgery succeeds iff

$$h_r(m^*) - h_r(m) = t^* - t.$$

Condition on the adversary’s view. The pad s is uniform and independent of r , so given the single observed tag t , every value of $h_r(m)$ is equally consistent—each candidate value determines a unique s —and the observation therefore leaks nothing about r : the posterior on r remains uniform on $\mathcal{R}$. The adversary’s difference $\delta := t^* - t$ and distinct pair (m, m^*) are thus fixed independently of r , and the A Δ U property gives $\Pr[h_r(m^*) - h_r(m) = \delta] \leq \varepsilon$. $\square$

The MAC Poly1305 instantiates the paradigm with a polynomial-evaluation hash over the prime field $\mathbb{F}_p$ for $p = 2^{130} - 5$ (prime-field arithmetic as in the *Math Guide*, §“The prime field and two inversion routes”).

Construction 6.11 (Poly1305). Fix the prime $p = 2^{130} - 5$ and work in $\mathbb{F}_p$. The key is a pair (r, s) : r is a 128-bit value subjected to a fixed bit-clamping (the top four bits of bytes 3, 7, 11, 15 and the bottom two bits of bytes 4, 8, 12 are forced to zero), and s is a 128-bit pad. To authenticate a message, split it into ℓ blocks $c_1, \dots, c_\ell$ of at most 16 bytes; encode each block as an integer in $[0, 2^{128})$ and add 2^{8b} , where b is the block's byte length—this sets a 1 bit just above the block's bytes, making every coefficient nonzero and the block encoding injective in both content and byte length. Interpreting r and each c_i as elements of $\mathbb{F}_p$, the hash is the polynomial evaluation

$$h_r(m) = \sum_{i=1}^{\ell} c_i r^{\ell-i+1} \in \mathbb{F}_p,$$

computed by Horner's rule as $(\dots((c_1 r + c_2) r + c_3) r \dots) r$. The tag is

$$\text{Mac}_{r,s}(m) = ((h_r(m) \bmod p) + s) \bmod 2^{128},$$

the field hash reduced and added to the pad modulo 2^{128} .

Proposition 6.12 (Poly1305 is almost- Δ -universal). *Restricted to messages of at most L bytes—hence at most $\ell_{\max} = \lfloor L/16 \rfloor$ blocks—the reduced Poly1305 hash family $\{m \mapsto h_r(m) \bmod p\}$, read into $\mathbb{Z}/2^{128}\mathbb{Z}$, is ε -almost- Δ -universal with respect to addition modulo 2^{128} , with*

$$\varepsilon \leq \frac{8 \ell_{\max}}{2^{106}}.$$

The loss over the ideal $\ell_{\max} \cdot p^{-1}$ accounts for the clamping of r —which restricts r to a subset of size 2^{106} —and for the final reduction modulo 2^{128} .

Proof. Sketch. Fix two distinct messages m, m' with block encodings (c_i) and (c'_i) , padded to a common length $\ell \leq \ell_{\max}$ by leading zero coefficients without changing their values. The block encoding guarantees that $m \neq m'$ yields distinct coefficient vectors: genuine coefficients are nonzero, so differing block counts set a nonzero coefficient against a zero padding one, while equal block counts leave two blocks differing in content or in final-block byte length, hence in encoding. The difference of hashes is

$$h_r(m) - h_r(m') = \sum_{i=1}^{\ell} (c_i - c'_i) r^{\ell-i+1} =: P(r) \in \mathbb{F}_p,$$

a nonzero polynomial in r of degree at most ℓ . Now count the ways a target difference $\delta \in \mathbb{Z}/2^{128}\mathbb{Z}$ can be hit. The reduced hash values lie in $[0, p)$, so their integer difference lies in the interval $(-p, p)$, which has length $2p < 8 \cdot 2^{128}$; each residue class modulo 2^{128} therefore contains at most $\lceil 2p/2^{128} \rceil = 8$ candidate integer differences u . For each such u , the equation $P(r) = u$ in $\mathbb{F}_p$ has at most ℓ roots—a polynomial of degree $\leq \ell$ over a field has at most ℓ roots (*Math Guide*, §“Roots and the factor theorem”)—so at most 8ℓ values of r produce the difference δ . The clamped r is uniform over a set of 2^{106} values (clamping zeroes 22 of the 128 bits), so it lands on one of those roots with probability at most $8\ell/2^{106} \leq 8\ell_{\max}/2^{106}$. $\square$

The bound is comfortably small in practice: for messages of 16 384 bytes ($\ell_{\max} = 1024$ blocks) it gives $\varepsilon \leq 8 \cdot 1024/2^{106} = 2^{-93}$, against the ideal $\ell_{\max}/p \approx 2^{-120}$ —the clamping and reduction losses cost 27 bits of the margin and leave it enormous. Combining Proposition 6.12 with the Carter–Wegman Theorem 6.10: Poly1305 with a fresh, uniformly random pad s per message—uniformity, not mere unpredictability, is what the theorem's pad-elimination step uses—is a one-time MAC with

forgery probability at most $8\ell_{\max}/2^{106}$ per verification attempt. In the AEAD below, ChaCha20 itself generates the one-time key (r, s) pseudorandomly from the nonce—uniform once the cipher is idealised as a random function—so the per-nonce one-time guarantee is exactly what is needed.

6.4 Authenticated encryption with associated data

We now combine the two guarantees. The composite goal is *authenticated encryption*: the adversary should be unable to learn anything about plaintexts (CPA security) *and* unable to produce any new ciphertext that decrypts to anything other than the distinguished rejection symbol $\perp$, a value distinct from every message (ciphertext integrity). Real protocols additionally require ciphertexts to bind to public context—packet headers, version bytes, an ephemeral public key—that travels in the clear but must be authenticated; this is the *associated data*.

Definition 6.13 (AEAD scheme). An *authenticated encryption with associated data* (AEAD) scheme is a pair (Enc, Dec) with key space $\mathcal{K}$, nonce space $\mathcal{N}$, and associated-data space $\mathcal{D}$: algorithm $\text{Enc}_k(\nu, a, m)$ returns a ciphertext c , and $\text{Dec}_k(\nu, a, c)$ returns a message or $\perp$. Correctness: $\text{Dec}_k(\nu, a, \text{Enc}_k(\nu, a, m)) = m$ for all k, ν, a, m . The scheme authenticates but does not encrypt the associated data a .

Definition 6.14 (Ciphertext integrity, INT-CTXT). For an AEAD scheme Π and adversary $\mathcal{A}$, the game $\text{Ctxt}_{\Pi}^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $k \xleftarrow{\$} \mathcal{K}$ and gives $\mathcal{A}$ two oracles: an encryption oracle $\text{Enc}_k(\cdot, \cdot, \cdot)$, recording the set Q of returned triples (ν, a, c) , and a verification oracle that on (ν, a, c) returns 1 if $\text{Dec}_k(\nu, a, c) \neq \perp$ and 0 otherwise. The adversary must never repeat a nonce to the *encryption* oracle (the *nonce-respecting* model); verification queries may reuse nonces freely.
2. The game outputs 1 iff some verification query $(\nu^*, a^*, c^*) \notin Q$ is answered 1.

The scheme Π has *ciphertext integrity* if for every PPT $\mathcal{A}$, $\Pr[\text{Ctxt}_{\Pi}^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda)$.

Definition 6.15 (Authenticated encryption). An AEAD scheme is *secure* (is an *AEAD*) if it is both CPA-secure and has ciphertext integrity (Definition 6.14), where the CPA game of Definition 6.4 is run with $k \xleftarrow{\$} \mathcal{K}$ and extended so that the adversary supplies the nonce and associated data of every oracle query and of the challenge, subject to the *nonce-respecting* restriction—a restriction on the adversary, not a property of the scheme—that no nonce is used twice across oracle queries and challenge. The restriction is necessary: an AEAD may encrypt deterministically given (ν, a, m) —ChaCha20-Poly1305 does—and then querying the oracle at (ν, a, m_0) and submitting the challenge (ν, a, m_0, m_1) would identify the bit by comparing ciphertexts.

The standard route from a CPA-secure cipher and a strongly unforgeable MAC to an AEAD is *encrypt-then-MAC*: encrypt the message, compute a MAC over the ciphertext together with the associated data and nonce, and append the tag; decryption verifies the tag *before* decrypting, and any tag failure returns $\perp$. The contrasting orders—MAC-then-encrypt and encrypt-and-MAC—do *not* generically achieve authenticated encryption; encrypt-then-MAC does.

Theorem 6.16 (Encrypt-then-MAC yields authenticated encryption). Let $\Pi_E = (\text{Enc}, \text{Dec})$ be a CPA-secure encryption scheme and $\Pi_M = (\text{Mac}, \text{Vrfy})$ a strongly unforgeable (sUF-CMA) MAC, with

independent keys k_E, k_M . Define the composite scheme Π' by

$$\text{Enc}'_{k_E, k_M}(\nu, a, m) = (c, t), \quad c \leftarrow \text{Enc}_{k_E}(\nu, m), \quad t \leftarrow \text{Mac}_{k_M}(\nu \parallel a \parallel c),$$

with Dec' returning $\perp$ unless $\text{Vrfy}_{k_M}(\nu \parallel a \parallel c, t) = 1$, in which case it returns $\text{Dec}_{k_E}(\nu, c)$. Then Π' is a secure AEAD: it is CPA-secure and has ciphertext integrity.

Proof. Ciphertext integrity. Suppose a PPT $\mathcal{A}$ making at most q_v verification queries wins the game: some fresh query $(\nu^*, a^*, (c^*, t^*)) \notin Q$ decrypts successfully, i.e. $\text{Vrfy}_{k_M}(\nu^* \parallel a^* \parallel c^*, t^*) = 1$. Each encryption-oracle call on (ν, a, m) returned (c, t) and corresponds to one MAC query on the string $w = \nu \parallel a \parallel c$ returning tag t . Because the framing $\nu \parallel a \parallel c$ parses uniquely (the component lengths are fixed or prepended), the freshness of (ν^*, a^*, c^*) means the MAC oracle never produced the string-tag pair (w^*, t^*) with $w^* := \nu^* \parallel a^* \parallel c^*$: either w^* was never queried, or it was queried and returned a different tag. Either way (w^*, t^*) is a fresh valid pair—a *strong* forgery against Π_M . The reduction $\mathcal{B}$ runs $\mathcal{A}$, answering encryption queries with its own MAC oracle and a self-chosen k_E . Verification queries $\mathcal{B}$ cannot answer, lacking k_M , so it guesses: it samples a uniform index $i \leq q_v$ in advance, answers each fresh verification query before the i -th with 0 (queries on triples in Q are answered 1, correctly), halts at the i -th fresh query, and outputs that query's (w^*, t^*) as its forgery. On the event that $\mathcal{A}$ wins and i hits her *first* successful fresh query—the guess is right with probability $1/q_v$ —every earlier 0 answer was correct, the simulation is perfect, and $\mathcal{B}$ wins, so

$$\Pr[\text{Ctxt}_{\Pi'}^{\mathcal{A}}(\lambda) = 1] \leq q_v \cdot \text{Adv}_{\Pi_M}^{\text{s-forge}}(\mathcal{B}, \lambda) \in \text{negl}(\lambda),$$

the factor q_v being polynomial.

CPA security. The tag is computed from c (together with k_M, ν, a , and the MAC's own coins) and so carries no information about the challenge bit beyond what c already carries. Formally, the reduction $\mathcal{B}'$ samples k_M itself, forwards $\mathcal{A}$'s oracle and challenge queries to its own Π_E oracle to obtain the c -components, and appends tags it computes with k_M . This simulates Π' perfectly, so $\text{Adv}_{\Pi'}^{\text{cpa}}(\mathcal{A}, \lambda) = \text{Adv}_{\Pi_E}^{\text{cpa}}(\mathcal{B}', \lambda) \in \text{negl}(\lambda)$. $\square$

Remark 6.17 (Why the ordering matters). Encrypt-then-MAC lets the receiver reject forged ciphertexts *without decrypting*, and this is what blocks chosen-ciphertext and padding-oracle attacks: a rejected ciphertext never reaches the decryption logic, so the decryption logic cannot leak. By contrast, MAC-then-encrypt verifies only after decrypting, exposing the decryption routine to attacker-chosen ciphertexts—the historical source of a long line of padding-oracle breaks. Ciphertext integrity is also exactly the property that upgrades CPA security to chosen-ciphertext (CCA) security, the CPA game of Definition 6.4 with a decryption oracle added that refuses only the challenge ciphertext: an authenticated-encryption scheme is automatically CCA-secure, because that oracle is useless to the adversary—every ciphertext she did not legitimately obtain decrypts to $\perp$ except with negligible probability.

6.5 The ChaCha20-Poly1305 AEAD

We now assemble the concrete scheme of RFC 8439, the AEAD by which Zcash encrypts every shielded note ciphertext and outgoing ciphertext (`zcash_note_encryption` crate, `src/lib.rs`; protocol specification, §“Symmetric Encryption”). It is an encrypt-then-MAC composition of the ChaCha20 stream cipher with the Poly1305 one-time MAC, with one economical twist: the cipher derives *both* the keystream and the one-time MAC key from the single (k, ν) input.

Construction 6.18 (ChaCha20-Poly1305). Fix a 256-bit key k and a 96-bit nonce ν . Encryption of a message m with associated data a proceeds as follows.

1. *One-time MAC key.* Run ChaCha20 with key k , nonce ν , and block counter 0; take the first

32 bytes of the keystream block as the Poly1305 one-time key (r, s) —clamp the first 16 bytes to form r ; the next 16 are s . Discard the rest of this block.

2. *Encrypt.* Run ChaCha20 with key k and nonce ν starting at block counter 1, producing the keystream KS_1 (the keystream of Remark 6.3 taken from counter 1 onward), and set $c = m \oplus \text{KS}_1(k, \nu, |m|)$.
3. *Authenticate.* Form the Poly1305 input by concatenating: a padded with zeros to a 16-byte boundary, c padded with zeros to a 16-byte boundary, the 8-byte little-endian length of a , and the 8-byte little-endian length of c . Compute the tag $t = \text{Poly1305}_{r,s}(\text{that input})$.

The ciphertext is (c, t) . Decryption recomputes (r, s) and the tag from (k, ν, a, c) , compares it to t in *constant time*, returns $\perp$ on mismatch, and otherwise outputs $m = c \oplus \text{KS}_1(k, \nu, |c|)$.

Remark 6.19 (Length encoding prevents splicing). The trailing 8-byte lengths of a and c are essential. Without them, the zero-padding that aligns a and c to 16-byte boundaries would let an adversary shift bytes between the associated data and the ciphertext, or pad-extend one of them, while leaving the Poly1305 input string unchanged—breaking the binding between a and c even though every authenticated byte is intact. Appending the explicit lengths makes the MAC input an injective encoding of the pair (a, c) , the same injectivity that the per-block length encoding supplies within one message in the AΔU analysis (Proposition 6.12).

Theorem 6.20 (Security of ChaCha20-Poly1305). *Model ChaCha20 as a pseudorandom function $F_k(\nu, j)$ from (nonce, block counter) pairs to 64-byte blocks. In the nonce-respecting model, ChaCha20-Poly1305 is a secure AEAD. Concretely, consider a PPT adversary making q encryption queries and q_v verification queries, each of whose Poly1305 inputs—padded associated data, padded ciphertext, and the length block—comprises at most $\ell_{\max}$ sixteen-byte blocks, $\ell_{\max} = \lceil |a|/16 \rceil + \lceil |c|/16 \rceil + 1$. Her CPA advantage is at most twice the best PRF-distinguishing advantage against F at comparable cost, and she forges—wins the INT-CTXT game—with probability at most*

$$\text{Adv}_F^{\text{prf}}(\mathcal{B}, \lambda) + \frac{8 q_v \ell_{\max}}{2^{106}},$$

where $\mathcal{B}$ is a PRF distinguisher of comparable cost.

Proof. Sketch. Replace F_k by a truly random function Φ from (nonce, counter) pairs to 64-byte blocks. Any noticeable change in the adversary’s success probability yields a PRF distinguisher of comparable cost—it simulates the whole game using its function oracle—so each replacement costs one $\text{Adv}_F^{\text{prf}}$ term; the CPA game is played twice (once per challenge bit), whence the factor two there. We analyse the idealised scheme.

Confidentiality. Since the adversary is nonce-respecting, every encryption query uses a fresh ν , and the keystream blocks $\Phi(\nu, 1), \Phi(\nu, 2), \dots$ are uniform and independent of everything else in her view; the ciphertext $c = m \oplus \text{KS}$ is a genuine one-time pad and reveals nothing about m beyond its length. The tag is a deterministic function of c, a, ν and the (independent) key (r, s) , and adds no information. The idealised CPA advantage is therefore zero.

Integrity. For each nonce ν , the block $\Phi(\nu, 0)[0:32]$ is uniform and independent across distinct nonces; clamping its first half samples r from Poly1305’s prescribed restricted set, while the second half s stays uniform, so the resulting one-time keys are independent across nonces. Because counter 0 is reserved for this key while encryption consumes counters ≥ 1 , each one-time key is also independent of the keystream that produced its ciphertext. Per nonce, the setting is exactly the Carter–Wegman one-time MAC of Theorem 6.10 instantiated with the ε -AΔU Poly1305 hash of Proposition 6.12, $\varepsilon \leq 8\ell_{\max}/2^{106}$. Consider any single fresh verification query (ν^*, a^*, c^*, t^*) . If ν^* was never used for encryption, the pad s for ν^* is uniform and entirely unknown, so the tag is correct with

probability 2^{-128} . If ν^* equals the nonce of some encryption query—nonce reuse is allowed in *verification* queries, only encryption is nonce-respecting—then the adversary has seen exactly one tag under that (r, s) , and freshness of (a^*, c^*) makes this a forgery on a new message under a one-time key, succeeding with probability at most ε by Theorem 6.10. The union bound over the q_v verification queries (*Math Guide*, §“The union bound and a birthday calculation”) gives forgery probability at most $8q_v\ell_{\max}/2^{106}$ in the idealised scheme; adding back the PRF-switching cost yields the stated bound. $\square$

Remark 6.21 (Single key, two roles). The scheme reuses one 256-bit key for both encryption and authentication, seemingly violating the independent-keys hypothesis of Theorem 6.16. Reserving block counter 0 for the Poly1305 key and counters ≥ 1 for the keystream is exactly what restores independence: once the cipher is idealised as a truly random function—the switch the proof of Theorem 6.20 pays for with a PRF-distinguishing term—outputs at distinct counters are independent, so (r, s) is independent of the keystream and the two-key analysis applies. For the PRF itself the independence is computational, not literal. This is a recurring design pattern—domain separation by a counter or label, the same discipline as §3.5, lets one master key safely play several roles.

Remark 6.22 (Wrong-key rejection and key commitment). The upper volumes consume this AEAD in a *trial-decryption* pattern: a receiver detects the notes addressed to it by attempting decryption of every ciphertext under each of its keys, relying on decryption under a wrong key to return $\perp$ (`zcash_note_encryption` crate, `src/lib.rs`, function `try_note_decryption`; protocol specification, §“Symmetric Encryption”). For *honestly produced* ciphertexts the scheme delivers this. Fix a ciphertext $(c, t) = \text{Enc}_k(\nu, a, m)$ and a key k' independent of k , and idealise the cipher under k and under k' as independent random functions, as in the proof of Theorem 6.20. The one-time key (r', s') that decryption under k' derives from block $(\nu, 0)$ is then uniform and independent of (c, t) : the pad s' alone makes the recomputed tag match t with probability exactly 2^{-128} , and even conditioned on one exposed tag under (r', s') , Theorem 6.10 bounds acceptance by the $\Delta\Delta$ U slack $8\ell_{\max}/2^{106}$ of Proposition 6.12, with $\ell_{\max}$ as in Theorem 6.20. Decryption under an independent key therefore rejects except with probability at most $2^{-128} + 8\ell_{\max}/2^{106}$ per attempt, plus two PRF-distinguishing terms for the two idealisations.

The guarantee stops at honest senders, and the boundary deserves care. CPA security and INT-CTXT are *single-key* notions: nothing in Definition 6.15 constrains a ciphertext crafted by a *malicious sender* who knows several keys. The scheme is in fact not *key-committing*—the separate notion demanding that no ciphertext verify under two distinct keys—and a sender who chooses keys k_1, k_2 herself can craft a single (c, t) that decrypts validly, to different plaintexts, under both: with (r_i, s_i) the one-time key each k_i derives, the hash h_{r_i} of Construction 6.11 is linear in the blocks of c , so fixing t , leaving two blocks of c free, and solving over $\mathbb{F}_p$ the two linear equations that pin the hash under r_i to $t - s_i$ (retrying until both solutions encode valid blocks) makes both keys accept (c, t) —the multi-key collision of Len, Grubbs, and Ristenpart’s partitioning-oracle attack. The upper volumes’ trial-decryption arguments therefore never rest on the tag alone against adversarial senders: their note-plaintext consistency checks—recomputing the note commitment from the decrypted plaintext and comparing it against the commitment the transaction carries—are what close this gap.

6.6 Key-derivation functions

Theorem 6.20 assumes a uniformly random 256-bit key. In practice keys may come from sources that are *high-entropy but not uniform*: a Diffie–Hellman shared secret is a structured group encoding, not a uniform bit string, and a hardware noise source may spread its unpredictability unevenly across its bits. (A passphrase is a different, usually *low-entropy* source, and needs a deliberately costly password KDF rather than the machinery of this subsection.) The adversary is owed nothing here—if the honest parties feed a non-uniform key into a scheme proved secure for uniform keys, the proof simply does not apply, and structure in the key is structure she may exploit. A *key-derivation function*

(KDF) converts an appropriate source secret into one or more pseudorandom keys. The right entropy measure is worst-case, not average-case.

Definition 6.23 (Min-entropy). The *min-entropy* of a random variable X is

$$H_\infty(X) = -\log_2 \max_x \Pr[X = x];$$

equivalently, the best chance of guessing X in one try is $2^{-H_\infty(X)}$. Given side information Z , the *conditional min-entropy* is

$$\tilde{H}_\infty(X | Z) = -\log_2 \mathbb{E}_{z \leftarrow Z} \max_x \Pr[X = x | Z = z],$$

so that $2^{-\tilde{H}_\infty(X|Z)}$ is the best chance of guessing X from Z . Recall the *statistical distance* $\Delta(X, Y) = \frac{1}{2} \sum_u |\Pr[X = u] - \Pr[Y = u]|$ of Definition 1.6 (*Math Guide*, §“Statistical distance”); we say X is ϵ -close to uniform on a set S if $\Delta(X, U_S) \leq \epsilon$.

Definition 6.24 (Key-derivation function and its security). A *key-derivation function* is a function

$$\text{KDF} : \mathcal{X} \times \mathcal{J} \times \{0, 1\}^* \times \mathbb{N} \rightarrow \{0, 1\}^*$$

taking a source secret (a sample of a random variable Σ over $\mathcal{X}$), a non-secret *salt* $\text{slt} \in \mathcal{J}$, an *info* or context string ctx , and a desired output length L . The security goal must specify the source model. For an arbitrary source of conditional min-entropy at least γ given the adversary’s side information, an extractor-style guarantee uses a salt sampled uniformly and independently of the source and requires the pair $(\text{slt}, \text{KDF}(\Sigma, \text{slt}, \text{ctx}, L))$ to be computationally indistinguishable from (slt, U_L) , even given ctx and that side information. No fixed deterministic unsalted function can provide this guarantee for *every* high-min-entropy source; an unsalted KDF therefore rests on a narrower premise, such as input keying material that is already computationally pseudorandom, or that is hard to query in a random-oracle model.

The notion a KDF targets is that of a seeded randomness extractor whose output need only *look* uniform.

Definition 6.25 (Strong computational extractor). A keyed function $\text{Ext} : \mathcal{J} \times \mathcal{X} \rightarrow \{0, 1\}^n$ is a *strong* (γ, ϵ) -computational extractor if for every source Σ over $\mathcal{X}$ of conditional min-entropy at least γ given auxiliary information Z , and a uniformly random salt $\text{slt} \xleftarrow{\$} \mathcal{J}$, the pair $(\text{slt}, \text{Ext}_{\text{slt}}(\Sigma))$ is computationally indistinguishable from (slt, U_n) , even given Z , with distinguishing advantage at most ϵ . “Strong” means the salt is revealed to the distinguisher.

Information theory can realise the notion unconditionally; we state the classical result *without proof*, since the deployed mechanism below rests on random-oracle modelling instead.

Theorem 6.26 (Leftover hash lemma; stated without proof). Let $\{g_r : \mathcal{X} \rightarrow \{0, 1\}^n\}$ be a 2^{-n} -universal hash family with a uniform public seed R . For any X with $H_\infty(X) \geq \gamma$,

$$\Delta((R, g_R(X)), (R, U_n)) \leq \frac{1}{2} \sqrt{2^{n-\gamma}}.$$

Thus investing $\gamma \geq n + 2 \log_2(1/\epsilon)$ bits of min-entropy buys n output bits that are ϵ -close to uniform—a

strong (γ, ϵ) -extractor in the statistical, not merely computational, sense.

HKDF, in passing. The standardised general-purpose KDF is Krawczyk’s HKDF (RFC 5869), built from HMAC (§5.3) in an *extract-then-expand* architecture: a salted HMAC call first distils suitable non-uniform input keying material into a short pseudorandom key (the extractor stage, justified along leftover-hash lines), and a counter-chained sequence of HMAC calls then expands that key, under a context string, up to RFC 5869’s limit of 255 hash-output-length blocks (the expander stage, justified by HMAC’s PRF security). The resulting KDF is a standard choice across many protocol source models and output lengths, though its precise guarantee still depends on assumptions about the source and about HMAC. Zcash does not use it—indeed the shielded protocol uses HMAC nowhere: neither the orchard nor the `zcash_note_encryption` crate carries an HMAC dependency (HMAC enters the wallet stack through the transparent BIP-32 derivation and BIP-39 mnemonic layers and, transitively, through the optional Tor client, never through a shielded crate)—because its requirements are narrower and a single hash call meets them in the random-oracle analysis.

The Zcash KDF. Orchard derives the note-encryption key by one personalised BLAKE2b call:

$$K_{\text{enc}} = \text{KDF}(\text{sharedSecret}, \text{epk}) = \text{BLAKE2b-256}(\text{"Zcash_OrchardKDF"; sharedSecret} \parallel \text{epk}),$$

where `sharedSecret` is the Diffie–Hellman shared point of the note-encryption key agreement, `epk` is the sender’s ephemeral public key, and the quoted string occupies BLAKE2b’s 16-byte personalisation field (orchard crate, `src/keys.rs`: the constant `KDF_ORCHARD_PERSONALIZATION` and the function `kdf_orchard`, which hashes the serialised shared point followed by the ephemeral-key bytes with a 32-byte output; protocol specification, §“Orchard Key Derivation”). The analysis models this personalised hash as a random oracle: by the domain-separation principle of §3.5 (Proposition 3.17, via the native-personalisation idiom catalogued there), the personalisation makes it an oracle independent of every other hash use in the protocol, and the derived key is uniform in the adversary’s view *unless* she queries that oracle at the hidden `sharedSecret`—a query whose argument she must first compute from the two public keys, an instance of the computational Diffie–Hellman problem of §2.2. Folding `epk` into the input binds the derived key to this particular ephemeral, the role the context string plays in HKDF. No salt is used because the claim is this narrower Diffie–Hellman-plus-random-oracle one—the unsalted premise anticipated in Definition 6.24—not extraction from every high-min-entropy source.

Remark 6.27 (The role of the KDF in the larger protocol). In the Zcash note-encryption layer, a Diffie–Hellman exchange on the Pallas curve produces a shared group element, and the single BLAKE2b call above condenses that element into the symmetric key fed to ChaCha20-Poly1305. The two halves of this section meet exactly here: key derivation manufactures the uniform key that the AEAD security theorem (Theorem 6.20) takes as its hypothesis, and the chain from a non-uniform shared secret to an authenticated ciphertext is complete. Section 7 assembles that chain end to end as hybrid public-key encryption.

7 Key agreement

Two parties who have never met share no secret, and the adversary of this section intends to keep it that way. She owns the channel between them. As an eavesdropper she records the two group elements they exchange and tries to *distinguish* the key they derive from a random string—a key she can distinguish is a key whose every later use leaks. As an active attacker she does better than listen: she intercepts each party’s contribution, substitutes her own, and runs one honest-looking session with each victim, so that both “secure channels” terminate at her desk and she reads and rewrites every message in transit—forging, as we shall prove, without violating any hardness assumption at all. And on a public ledger her seat is permanent: every note ciphertext is published for ever, and

she may scan them all at leisure, holding each against every address she can enumerate. This section builds the machinery that denies her all three postures: the Diffie–Hellman protocol atop the exponentiation asymmetry of Section 2; the static/ephemeral taxonomy that turns an interactive exchange into a one-shot flow; the key-agreement scheme abstraction that the deployed code programs against; passive security, which we prove equal to the DDH advantage *exactly*; the man-in-the-middle attack and the three external mechanisms of authentication; the elliptic-curve instantiation with its validation duties; the hybrid composition with a key-derivation function and an AEAD; key privacy, which keeps a ciphertext silent about its recipient; and the deployed endpoint, shielded-note delivery as hybrid public-key encryption carried in band on the ledger.

7.1 The Diffie–Hellman protocol

The raw material is the asymmetry established in Section 2. In a cyclic group $G = \langle g \rangle$ of order q , the map

$$\exp_g : \mathbb{Z}/q\mathbb{Z} \rightarrow G, \quad x \mapsto g^x,$$

is a group isomorphism—it carries addition of exponents to multiplication of elements—that is cheap to evaluate, at $O(\log q)$ group operations by repeated squaring, but believed infeasible to invert in the groups this volume uses: inverting it is precisely the discrete logarithm problem (Definition 2.2). Everything in this section depends only on the group structure and the hardness of certain problems in it, never on the presentation of the group; the reference instances are a large prime-order subgroup of $\mathbb{F}_p^\times$ and the elliptic-curve point groups of §7.6. Diffie and Hellman’s observation is that the asymmetry alone already lets two strangers manufacture a shared secret in public.

Construction 7.1 (Diffie–Hellman key exchange). Fix public parameters (G, g, q) with $G = \langle g \rangle$ of order q , known to all parties (and to the adversary).

1. Alice samples $a \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ uniformly, computes $A = g^a$, and sends A to Bob.
2. Bob samples $b \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ uniformly, computes $B = g^b$, and sends B to Alice.
3. Alice computes $k_A = B^a$; Bob computes $k_B = A^b$.

The value $k_A = k_B$ is the *shared secret*; the transmitted elements A and B are the *public shares* (public keys), and the exponents a and b are the *private keys*, never sent.

Proposition 7.2 (Correctness of Diffie–Hellman). *In Construction 7.1 both parties compute the same group element, $k_A = k_B = g^{ab}$.*

Proof. Unwinding the definitions,

$$k_A = B^a = (g^b)^a = g^{ba} = g^{ab} = (g^a)^b = A^b = k_B,$$

where $ba = ab$ because integer multiplication is commutative, and exponents may be read modulo q because g has order q . $\square$

The shared secret g^{ab} is thus a well-defined function of the transcript (A, B) : it can be computed by anyone who knows *one* of the two private keys, and—conjecturally—by no one who knows only the public shares. Making that conjecture precise, and seeing exactly how much it buys, is the business of §7.4. First, a structural precaution.

Remark 7.3 (Small subgroups and the prime-order rule). If G has small subgroups, an adversary can exploit them: by substituting for a public share an element of small order she confines the victim's computed secret to a small subgroup and learns the victim's exponent modulo a small factor of q —the Pohlig–Hellman leakage of Corollary 2.21 in protocol form. The remedy is the standing rule of Section 2: prefer G of *prime* order q . Then the only subgroups are $\{1\}$ and G itself, every non-identity element is a generator, and no nontrivial confinement is possible. When the natural ambient group does not have prime order— $\mathbb{F}_p^\times$ has composite order $p - 1$ —one works inside a prime-order subgroup and has each recipient *validate* incoming elements as members: check $B^q = 1$ and $B \neq 1$ before exponentiating. The elliptic-curve analogue of this validation appears in Remark 7.19.

7.2 Static and ephemeral keys

Nothing in Construction 7.1 says how long a key pair lives, and the protocol changes character with the answer.

Definition 7.4 (Static and ephemeral keys). A Diffie–Hellman key pair (x, g^x) is *static* (long-term) if it is generated once and reused across many sessions, typically published and bound to an identity; it is *ephemeral* if it is generated freshly for a single session and discarded immediately afterwards.

Three protocol shapes result, and all three matter to what follows.

- *Ephemeral–ephemeral (DHE)*. Both parties use fresh keys. Once a and b are erased, no one—the parties included—can recover g^{ab} : this is *forward secrecy*, the guarantee that a later compromise of all stored long-term state cannot decrypt recorded past sessions, because the only secrets that ever existed were the deleted ephemerals.
- *Static–ephemeral*. The recipient Bob holds a static pair $(b, B = g^b)$, with B known in advance; Alice contributes a fresh ephemeral (a, A) . The shared secret g^{ab} then requires *no interaction from Bob at agreement time*: Alice computes $g^{ab} = B^a$ from the published B , attaches A , and sends everything in a single flow; Bob recovers $g^{ab} = A^b$ whenever he next comes online. This is exactly the shape of hybrid encryption (§7.7) and of shielded-note delivery (§7.9): sender online once, recipient possibly offline, no round trip.
- *Static–static*. Both keys are long-term, so g^{ab} is a fixed function of the two identities—the same in every session. There is no forward secrecy, and the never-varying secret must be combined with fresh per-session randomness (a nonce) before it may key any symmetric scheme; using it directly would reuse one key across all time.

The static–ephemeral case is the conceptual bridge from key exchange, an interactive protocol, to public-key encryption, a one-shot algorithm. A static public key $B = g^b$ functions exactly as an encryption public key: anyone can derive a secret shared with its owner by sending a single ephemeral share. The rest of the section develops that bridge until, by §7.9, it carries deployed traffic.

7.3 The key-agreement scheme abstraction

Deployed code does not manipulate exponents; it programs against an interface. We therefore package the non-interactive (static–ephemeral) pattern as a scheme with named algorithms, so that later constructions—and the deployed note-encryption machinery—can invoke key agreement without caring which group sits underneath.

Definition 7.5 (Key-agreement scheme). A (non-interactive) *key-agreement scheme* over a public parameter set pp consists of a private-key space $\mathcal{S}$, a public-key space $\mathcal{P}$, a shared-secret space $\mathcal{K}$, and three algorithms:

- $\text{KeyGen}(\text{pp}) \rightarrow s$, randomised, outputting a private key $s \in \mathcal{S}$;
- $\text{DerivePublic}(\text{pp}, s) \rightarrow p$, deterministic, mapping a private key to its public key $p \in \mathcal{P}$;
- $\text{Agree}(\text{pp}, s, p') \rightarrow k$, deterministic, mapping one’s own private key and a counterparty’s public key to a shared secret $k \in \mathcal{K}$.

Correctness demands that for all pp and all s, t in the support of $\text{KeyGen}(\text{pp})$,

$$\text{Agree}(\text{pp}, s, \text{DerivePublic}(\text{pp}, t)) = \text{Agree}(\text{pp}, t, \text{DerivePublic}(\text{pp}, s)). \quad (1)$$

Construction 7.6 (Diffie–Hellman as a key-agreement scheme). Fix $\text{pp} = (G, g, q)$ with $G = \langle g \rangle$ of prime order q . The scheme $\text{DH}_{G,g}$ takes $\mathcal{S} = \mathbb{Z}/q\mathbb{Z}$ (or $(\mathbb{Z}/q\mathbb{Z}) \setminus \{0\}$, to avoid the degenerate case whose public key is the identity), $\mathcal{P} = \mathcal{K} = G$, and

$$\text{KeyGen} : a \xleftarrow{\$} \mathcal{S}, \quad \text{DerivePublic}(\text{pp}, a) = g^a, \quad \text{Agree}(\text{pp}, a, B) = B^a.$$

Correctness is exactly Proposition 7.2: $\text{Agree}(\text{pp}, s, g^t) = (g^t)^s = g^{ts} = g^{st} = (g^s)^t = \text{Agree}(\text{pp}, t, g^s)$.

Remark 7.7 (The deployed interface generalises the base point). The Zcash note-encryption machinery programs against precisely this abstraction, with one generalisation: the deployed DerivePublic takes the base point as an *explicit argument* rather than fixing a protocol-wide generator, so that Sapling and Orchard derive keys over per-address *diversified bases* (protocol specification §“Key Agreement”; the `zcash_note_encryption` crate, `src/lib.rs`: the `Domain` trait’s `ka_derive_public`, `ka_agree_enc`, and `ka_agree_dec` operate on abstract associated types, and Orchard’s implementation of `derive_public` in the `orchard` crate, `src/keys.rs`, receives the base g_d as a parameter). Those bases are put to work in §7.9, and the address machinery that produces them is detailed in the *Ironwood Guide* (§“Diversified addresses”). What matters here is that the interface still invokes an abstract key agreement and never inspects the underlying group—which is why the same construction has transported across three groups, from Sprout’s Curve25519 to Sapling’s Jubjub to Orchard’s Pallas.

Remark 7.8 (The abstraction boundary, and what falls outside it). Any concrete scheme satisfying equation (1) can be swapped in without changing the surrounding logic: a prime-field group, an elliptic curve, or a post-quantum non-interactive key agreement such as the CSIDH group action. A *key-encapsulation mechanism* (KEM), by contrast, cannot satisfy the correctness equation at all: encapsulation requires the recipient’s public key before it can produce its ciphertext, so no counterparty-independent DerivePublic exists. The static-ephemeral one-flow *pattern* does generalise to any KEM—the encapsulation ciphertext plays the role of the ephemeral public key—but at the cost of changing the interface the surrounding logic programs against.

7.4 Passive security and the Diffie–Hellman assumptions

The weakest adversary is the eavesdropper: she reads the channel but does not alter it, and her entire view of an honest session is the transcript (g^a, g^b) . Both formalisations of her task were laid down in §2.2. The *computational* problem is to produce the shared secret: writing $\text{Adv}_{\text{cdh}}(\mathcal{A})$ for the success

probability of $\mathcal{A}$ in the CDH game of Definition 2.4,

$$\text{Adv}_{\text{cdh}}(\mathcal{A}) = \Pr_{a,b}[\mathcal{A}(g^a, g^b) = g^{ab}].$$

The *decisional* problem is to distinguish (g^a, g^b, g^{ab}) from (g^a, g^b, g^c) with advantage $\text{Adv}_{\text{ddh}}(\mathcal{A})$ (Definition 2.5). The hierarchy $\text{DDH} \leq \text{CDH} \leq \text{DLP}$ was established in §2.3 (Theorems 2.7 and 2.8), together with the pairing-based separation of DDH from CDH (Example 2.9). The distinction is not pedantry. Hardness of CDH asserts only that the *whole* secret cannot be computed; keying a symmetric cipher requires that the secret be indistinguishable from a uniform group element, so that no individual bit or predicate of it leaks. The following classical break shows the gap is real in the most familiar group.

Remark 7.9 (A concrete DDH break: the Legendre symbol). Take $G = \mathbb{F}_p^\times$, the *full* multiplicative group, with generator g . The Legendre symbol $\left(\frac{h}{p}\right)$ is efficiently computable by Euler’s criterion $h^{(p-1)/2} \bmod p$ and is multiplicative (*Math Guide*, §“Quadratic residues and the Euler criterion”). A generator of $\mathbb{F}_p^\times$ is a nonsquare, so $\left(\frac{g^a}{p}\right) = (-1)^a$: the symbol of a public share reveals the *parity* of its exponent. The parities of a and b determine the parity of ab , hence $\left(\frac{g^{ab}}{p}\right) = (-1)^{ab}$ is determined by the two observed symbols. The distinguisher computes the predicted symbol of the shared secret from (g^a, g^b) , compares it with the symbol of the third component, and outputs “Diffie–Hellman” on a match. On a genuine triple the match is certain; on a random triple the third component’s symbol is an independent uniform sign, matching with probability $1/2$. The distinguishing advantage is the constant $1/2$ —even though CDH in $\mathbb{F}_p^\times$ is believed hard. The CDH/DDH gap therefore needs no pairings; the oldest group in the subject exhibits it. The cure is the prime-order rule of Remark 7.3: in a subgroup of odd prime order q , every element h is a square ($h = (h^{(q+1)/2})^2$), the symbol is identically $+1$, and the leak vanishes. This is one of the two reasons—the other being Pohlig–Hellman—that DDH is only ever assumed in prime-order groups.

With the assumption in place, passive security is not merely implied by DDH; it *is* DDH, advantage for advantage. We state the game in the volume’s tradition and prove the identification.

Definition 7.10 (Passive security of a key-agreement scheme). For a key-agreement scheme Σ over pp and an adversary $\mathcal{A}$, the game $\text{KA-IND}_\Sigma^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $s, t \leftarrow \text{KeyGen}(\text{pp})$ independently and computes the transcript $(\text{DerivePublic}(\text{pp}, s), \text{DerivePublic}(\text{pp}, t))$.
2. It sets $k_0 = \text{Agree}(\text{pp}, s, \text{DerivePublic}(\text{pp}, t))$, samples $k_1 \xleftarrow{\$} \mathcal{K}$ uniformly, draws a hidden bit $\beta \xleftarrow{\$} \{0, 1\}$, and sends the transcript together with k_β .
3. The adversary outputs a bit β' .

The advantage is $\text{Adv}_\Sigma^{\text{ka-ind}}(\mathcal{A}) = |\Pr[\beta' = 1 \mid \beta = 0] - \Pr[\beta' = 1 \mid \beta = 1]|$; the scheme is *passively secure* if every PPT adversary’s advantage is negligible in λ .

Theorem 7.11 (Passive security of Diffie–Hellman). *For the scheme $\text{DH}_{G,g}$ of Construction 7.6 with KeyGen uniform on $\mathbb{Z}/q\mathbb{Z}$, every adversary $\mathcal{A}$ satisfies*

$$\text{Adv}_{\text{DH}_{G,g}}^{\text{ka-ind}}(\mathcal{A}) = \text{Adv}_{\text{ddh}}(\mathcal{A}),$$

identically and with running time preserved exactly. Under the DDH assumption the shared secret

of an honest ephemeral–ephemeral session is therefore pseudorandom given the transcript: a passive adversary learns nothing about it that she could not have guessed about a random group element.

Proof. The two games are the same game. In KA-IND for $\text{DH}_{G,g}$, the transcript is (g^a, g^b) with a, b uniform and independent; the real key is $k_0 = g^{ab}$; and the uniform alternative $k_1 \xleftarrow{\$} G$ is exactly g^c for uniform independent c , because $\exp_g$ is a bijection. The challenge (g^a, g^b, k_β) is thus verbatim a sample of $\mathcal{D}_{\text{dh}}$ or $\mathcal{D}_{\text{rand}}$ according to β —the DDH distinguishing game of Definition 2.5. The identity map converts each adversary into the other, so the advantages coincide and no running time is added. $\square$

Remark 7.12 (Why pseudorandomness, not mere unpredictability). Every use of a Diffie–Hellman secret in this volume feeds it into a key-derivation function to produce a symmetric key. What should the group deliver to that function? Hardness of CDH guarantees the secret is hard to produce *in full*, but not that it lacks efficiently predictable structure: the Legendre leak of Remark 7.9 is exactly such structure. Hardness of DDH guarantees the secret is as good as a random group element (Theorem 7.11)—no predicate of it is efficiently computable from the transcript. In practice one often assumes only CDH and models the hash-based KDF as a random oracle: the derived key is then uniform unless the adversary queries the oracle at the hidden shared secret, a route made precise in Theorem 7.24. Either way the design goal is the same: a derived symmetric key indistinguishable from uniform, because that is the hypothesis every theorem of Section 6 consumes.

7.5 Active attacks and the role of authentication

Passive security is the most the bare protocol can offer, and the gap between it and what a channel needs is best exhibited as a construction. The adversary now controls the wire.

Definition 7.13 (Man-in-the-middle attack on unauthenticated Diffie–Hellman). An active adversary $\mathcal{M}$ controlling the channel between Alice and Bob runs two independent Diffie–Hellman sessions, one with each victim. She samples $m \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ and computes g^m . When Alice sends $A = g^a$ intended for Bob, $\mathcal{M}$ intercepts it and forwards g^m to Bob in its place; when Bob replies $B = g^b$, she intercepts it and forwards g^m to Alice. Alice computes $k_A = (g^m)^a = g^{am}$, which $\mathcal{M}$ also computes as A^m ; symmetrically Bob computes $k_B = (g^m)^b = g^{bm}$, which $\mathcal{M}$ computes as B^m .

Proposition 7.14 (The attack defeats unauthenticated Diffie–Hellman). *After the attack of Definition 7.13, Alice and Bob each hold a secret they believe is shared with the other but is in fact shared with $\mathcal{M}$; and $\mathcal{M}$, knowing both g^{am} and g^{bm} , can decrypt, read, modify, and re-encrypt every subsequent message in each direction, transparently to both parties. No assumption on G —not even ideal hardness of the discrete logarithm problem—prevents this.*

Proof. Correctness of Diffie–Hellman (Proposition 7.2), applied once per session, gives agreement between Alice and $\mathcal{M}$ on g^{am} and between Bob and $\mathcal{M}$ on g^{bm} . The adversary knows her own exponent m and received both A and B , so she computes both secrets directly—no problem instance is ever inverted, which is why no hardness assumption is relevant. Neither victim verifies the *origin* of the group element they received; each transcript is a syntactically valid run of Construction 7.1, so nothing in either party’s view distinguishes the attacked execution from an honest one. With k_A and k_B in hand, $\mathcal{M}$ decrypts traffic from Alice under k_A , re-encrypts it under k_B for Bob, and symmetrically in reverse, editing at will in between. $\square$

The lesson deserves stating structurally: *passive security concerns the secrecy of the shared secret; active security additionally requires the authenticity of the public shares.* Confidentiality of a channel

is worthless if the channel is established with the wrong party. The bare protocol (Construction 7.1) provides no authentication—nothing in g^a or g^b testifies to who sent it—so the remedy must bind each public share to its sender’s identity, letting the recipient reject a substituted share. Every such binding is *external* to the key exchange itself.

Remark 7.15 (Three mechanisms supply authentication). All deployed remedies fall into three families, each external to the bare exchange.

1. *Signed Diffie–Hellman*. Each party signs its ephemeral share with a long-term signing key whose verification key is certified—by a certificate authority (a third party whose signature on the key the parties already trust) or by trust-on-first-use (accepting the first key seen and rejecting later changes). The adversary of Proposition 7.14 cannot forge Alice’s signature on g^m , so her substitution is rejected. Digital signatures are the subject of Section 8; this composition is the structure of authenticated TLS.
2. *Static keys that are themselves authenticated*. If Bob’s static public key $B = g^b$ reaches Alice authentically—a trusted directory, a pinned key—then a static–ephemeral exchange toward B already authenticates Bob to Alice: only the holder of b can complete it. Note the direction: this authenticates the *recipient*, not the sender. It is the model relevant to public-key encryption, and to shielded-note delivery, where the recipient’s address is an authenticated public key (§7.9).
3. *Out-of-band verification*. The parties compare a short fingerprint of the exchanged public keys over a channel that is authentic even if low-bandwidth—digits read aloud on a call, a QR code scanned in person.

In every case the key-agreement security goal remains the passive guarantee of Theorem 7.11; authentication is a separable layer composed around the exchange, not a property of the group operation.

7.6 Elliptic-curve Diffie–Hellman

The development so far is deliberately group-agnostic; we now instantiate it on the groups the deployed protocol actually uses. Recall from the *Math Guide* (§“Elliptic curves”) that an elliptic curve E over a field F of characteristic neither 2 nor 3 is given in short Weierstrass form $y^2 = x^3 + ax + b$ with nonzero discriminant, and that its F -rational points $E(F)$ form a finite abelian group under chord-and-tangent addition with the point at infinity $\mathcal{O}$ as identity (*Math Guide*, §“The group structure of $E(\mathbb{F}_p)$ ”). The group is written additively, so exponentiation $g \mapsto g^x$ becomes *scalar multiplication* $P \mapsto [x]P$, computable in $O(\log x)$ point additions by double-and-add. Elliptic-curve Diffie–Hellman is the transliteration.

Construction 7.16 (ECDH). Fix an elliptic curve E/F , a base point $P \in E(F)$ of large prime order q generating $G = \langle P \rangle$, and cofactor $h = |E(F)|/q$. In the abstraction of Definition 7.5:

$$\text{KeyGen} : a \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}, \quad \text{DerivePublic}(a) = [a]P, \quad \text{Agree}(a, Q) = [a]Q.$$

The shared secret of parties with private keys a and b is $[a]([b]P) = [ab]P = [b]([a]P)$. The associated hard problems are the discrete-logarithm, CDH, and DDH definitions of Section 2 (Definitions 2.2, 2.4, and 2.5) read with $[x]P$ in place of g^x —the problems ECDLP (*Math Guide*, §“The elliptic-curve discrete logarithm problem”), ECCDH, and ECDDH of §2.6.

Proposition 7.17 (Correctness of ECDH). *The scheme of Construction 7.16 satisfies the key-agreement correctness condition (1).*

Proof. Scalar multiplication is cyclic-group exponentiation in additive dress: the group is abelian, and $\mathbb{Z}$ acts on $\langle P \rangle$ through $\mathbb{Z}/q\mathbb{Z}$ because $[q]P = \mathcal{O}$, so $[x]([y]P) = [xy]P$ for all integers x, y . Hence $\text{Agree}(a, [b]P) = [ab]P = [ba]P = \text{Agree}(b, [a]P)$, which is condition (1). $\square$

Remark 7.18 (Why elliptic curves). Well-chosen elliptic-curve groups admit only the generic $O(\sqrt{q})$ attacks of §2.4, whereas index calculus solves the discrete logarithm in $\mathbb{F}_p^\times$ in subexponential time (Remark 2.18); a 256-bit curve therefore delivers the security level that a multi-thousand-bit prime field requires (Example 2.17). That efficiency, together with the availability of curves engineered for cheap in-circuit arithmetic, is why the Pallas curve of §2.6 underlies Orchard.

Remark 7.19 (Subgroup and curve validation). A recipient computing $[a]Q$ on an attacker-supplied point Q must guard against two pitfalls, the elliptic-curve forms of the validation duty in Remark 7.3.

1. *Invalid-curve attack.* The received coordinates may satisfy not E 's equation but that of a different, weaker curve which shares the same addition formulas (the chord and tangent slopes of the *Math Guide*, §“Explicit affine formulas”, involve a but never b); scalar multiplication then runs happily in the wrong group, whose order may be smooth. The implementation must check the curve equation on every incoming point.
2. *Small-subgroup attack.* When the cofactor $h > 1$, an adversary submits a point Q of small order dividing h , confining $[a]Q$ to a tiny subgroup and learning a modulo a small factor. The defences are *cofactor clearing*—multiply incoming points by h —or choosing a curve with $h = 1$.

Prime-order curves such as Pallas sidestep the cofactor issue entirely: with $h = 1$ there is no small subgroup to hit, and only the on-curve check remains. The deployed arithmetic enforces exactly that check at deserialisation, as documented in Proposition 2.27, whose compressed encoding cannot even represent a point off the curve.

7.7 Hybrid public-key encryption

Three obstacles separate the Diffie–Hellman shared group element from a usable encryption scheme. First, the shared secret is a *structured group element*—a curve point with coordinates satisfying an equation—not a uniform string of key bits. Second, the exchange by itself transports no message: it manufactures a secret, and stops. Third, we want the static–ephemeral non-interactive flow of §7.2, so that the recipient need not be online. All three are resolved by one composition: key agreement for the secret, a key-derivation function for uniformity, an authenticated cipher for the message—*hybrid public-key encryption* (HPKE). The two symmetric ingredients were built in Section 6; we recall them in the specialised shape the composition needs.

Definition 7.20 (KDF, recalled and specialised). A *key-derivation function* here is a deterministic function

$$\text{KDF} : \mathcal{K} \times \{0, 1\}^* \rightarrow \{0, 1\}^\ell,$$

mapping input keying material $k \in \mathcal{K}$ together with a context string to an ℓ -bit key. This specialises Definition 6.24 (§6.6) by omitting the salt and fixing the output length ℓ . For Theorem 7.24 we model the function as a random oracle (§3.4): distinct inputs then have independent uniform outputs, and security follows provided the adversary never queries the oracle at the hidden Diffie–Hellman secret and context. This is a source-specific random-oracle guarantee, not a generic claim that a deterministic function extracts uniform bits from every high-min-entropy source.

Definition 7.21 (AEAD, recalled). An *authenticated encryption scheme with associated data* with key space $\{0, 1\}^\ell$ —matching the KDF output—is a pair (Enc, Dec) with $\text{Enc}_K(N, ad, m) = c$ and $\text{Dec}_K(N, ad, c) \in \{m\} \cup \{\perp\}$, where N is a nonce, ad is associated data authenticated but not en-

encrypted, and $\perp$ denotes rejection (Definition 6.13). Correctness: $\text{Dec}_K(N, ad, \text{Enc}_K(N, ad, m)) = m$. Security is authenticated-encryption security (Definition 6.15): encryptions of any two equal-length messages are CPA-indistinguishable (Definition 6.4), and no adversary can produce a fresh ciphertext decrypting to anything but $\perp$ (ciphertext integrity, Definition 6.14)— confidentiality and integrity simultaneously.

Construction 7.22 (DH-based hybrid public-key encryption). Fix ECDH over $G = \langle P \rangle$ of prime order q (Construction 7.16, in the interface of Definition 7.5), a secure KDF, and a secure AEAD as above. The recipient holds a static key pair $(b, B = [b]P)$ with B published. To *encrypt* a message m to B :

1. sample an ephemeral $e \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ and set $E_{\text{pk}} = [e]P$;
2. compute the shared secret $S = \text{Agree}(e, B) = [eb]P$;
3. derive the symmetric key $K = \text{KDF}(S, \text{context})$, where *context* includes E_{pk} and the protocol's labels;
4. output (E_{pk}, c) with $c = \text{Enc}_K(N, ad, m)$ for a fixed or derived nonce N —fixed is safe because each K is used once.

To *decrypt* (E_{pk}, c) with b : compute $S' = [b]E_{\text{pk}}$, derive $K' = \text{KDF}(S', \text{context})$ with E_{pk} read from the ciphertext, and output $\text{Dec}_{K'}(N, ad, c)$, which is m on an honest ciphertext; on a tampered one, ciphertext integrity makes the output $\perp$ except with negligible probability. Anyone holding B can encrypt; only the holder of b can decrypt.

Theorem 7.23 (Correctness of hybrid encryption). *For every message m and every honestly generated ciphertext (E_{pk}, c) of Construction 7.22, decryption returns m .*

Proof. Correctness of ECDH (Proposition 7.17) gives $S' = [b]([e]P) = [be]P = [eb]P = [e]B = S$. The KDF is deterministic and both sides supply the same context—the recipient recovers E_{pk} from the ciphertext itself—so $K' = K$. Correctness of the AEAD (Definition 7.21) finishes: $\text{Dec}_K(N, ad, \text{Enc}_K(N, ad, m)) = m$. $\square$

Theorem 7.24 (Passive security of hybrid encryption). *Model the KDF as a random oracle (Definition 3.13) and let the AEAD be secure in the sense of Definition 6.15. Consider a passive adversary $\mathcal{A}$ who sees the public key B and a single challenge ciphertext: she names two equal-length messages m_0, m_1 , receives the encryption of m_β for a hidden uniform bit β , and guesses β , with CPA advantage $\text{Adv}_{\text{cpa}}(\mathcal{A})$ defined as in Definition 6.4. Under CDH in G no PPT adversary has non-negligible advantage; concretely, if $\mathcal{A}$ makes at most q_H random-oracle queries, then*

$$\text{Adv}_{\text{cpa}}(\mathcal{A}) \leq 2q_H \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{ae}}(\mathcal{C}),$$

for explicit reductions $\mathcal{B}$ against CDH and $\mathcal{C}$ against the AEAD's authenticated-encryption security (of which the proof consumes only the confidentiality half, so $\text{Adv}_{\text{ae}}(\mathcal{C})$ is the CPA advantage of Definition 6.4 read against the AEAD), each running in essentially the same time as $\mathcal{A}$.

Proof. The proof is a game hop.

Game 0 is the real CPA game: the challenge ciphertext is (E_{pk}, c) with $E_{\text{pk}} = [e]P$, $S = [eb]P$, $K = \text{KDF}(S, \text{context})$, and $c = \text{Enc}_K(N, ad, m_\beta)$. Say $\mathcal{A}$ wins a game when her guess β' equals β .

Game 1 replaces K by an independent uniform ℓ -bit string, leaving everything else unchanged.

The hop. In the random oracle model the two games proceed identically unless the adversary queries the oracle at exactly the point $(S, \text{context}) = ([eb]P, \text{context})$: until that query, the oracle's value there is a uniform string she has never seen, which is precisely what Game 1 hands out. But producing $S = [eb]P$ from the view $(B, E_{pk}) = ([b]P, [e]P)$ is exactly solving CDH. The reduction $\mathcal{B}$ embeds its CDH challenge as (B, E_{pk}) and simulates Game 1 around it: it draws β and a uniform ℓ -bit key K itself, encrypts m_β under K to form the challenge ciphertext, and implements the oracle by lazy sampling—a fresh uniform string for each new query, the stored answer on a repeated one, so that the oracle remains a consistent function. The simulation is a perfect run of Game 1, and the critical query occurs in it with the same probability as in Game 0, the two games being identical until it happens. One obstruction remains: in a pairing-free group $\mathcal{B}$ cannot *recognise* which of the queries carries $[eb]P$ (Example 2.9 shows recognising it is the DDH-test a gap group would supply). So $\mathcal{B}$ picks one of the at most q_H queries uniformly at random and outputs its first component; whenever the critical query occurs, $\mathcal{B}$'s choice is correct with probability at least $1/q_H$. Hence

$$|\Pr[\mathcal{A} \text{ wins Game 0}] - \Pr[\mathcal{A} \text{ wins Game 1}]| \leq q_H \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}).$$

The factor q_H is a genuine guessing-type security loss, in the tightness taxonomy of Definition 1.22. Assuming *gap*-CDH—CDH hardness even given a DDH-decision oracle, which would let $\mathcal{B}$ test each query—restores tightness; we state the loose bound because it is what plain CDH buys, and honesty about the loss is part of the bound.

Game 1 analysed. The AEAD key is now uniform and independent of everything else in the adversary's view, which is exactly the hypothesis of the AEAD's confidentiality game. The reduction $\mathcal{C}$ forwards (m_0, m_1) to its AEAD challenger, embeds the returned ciphertext as c alongside a self-generated E_{pk} , and relays $\mathcal{A}$'s guess; the AEAD challenger's hidden bit plays β , so $|\Pr[\mathcal{A} \text{ wins Game 1}] - \frac{1}{2}| = \frac{1}{2} \text{Adv}_{\text{ae}}(\mathcal{C})$.

Accounting. The difference-form advantage of Definition 6.4 is twice the bit-guessing advantage $|\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}|$ of Definition 1.15, so the triangle inequality across the hop gives

$$\text{Adv}_{\text{cpa}}(\mathcal{A}) \leq 2(q_H \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \frac{1}{2} \text{Adv}_{\text{ae}}(\mathcal{C})) = 2q_H \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{ae}}(\mathcal{C}),$$

the stated bound.

Alternative hypothesis. The random oracle can be traded away, but not for Definition 6.24 alone: that guarantee is stated for a uniform independent salt, which the specialised KDF of Definition 7.20 omits, and the definition denies any generic unsalted guarantee. The trade needs an explicit hypothesis on the deployed function—the narrower premise Definition 6.24 allows for unsalted use: for a *uniform* group element S , the output $\text{KDF}(S, \text{context})$ is computationally indistinguishable from a uniform ℓ -bit string given the context. Under that hypothesis and DDH in place of CDH, Theorem 7.11 first replaces the shared secret by a uniform group element at cost Adv_{ddh} , and the stated indistinguishability then replaces K by a uniform string—no oracle and no q_H factor, at the price of the stronger decisional assumption on the group and an explicit pseudorandomness assumption on the KDF. $\square$

Remark 7.25 (Every ingredient is necessary). Dropping any piece of Construction 7.22 breaks the composition.

- *Without the KDF*, one would key the cipher with the raw group element S . Its bit-representation is non-uniform—curve points do not fill the encoding space—and in non-DDH groups it is partially predictable, the Legendre-symbol leakage of Remark 7.9 being the classical instance. The KDF standardises the format, extracts uniform bits, and, by folding E_{pk} into its context, ties the derived key to this particular ephemeral share.

- *Without AEAD integrity*, a passively confidential but malleable cipher would let an active adversary tamper with c undetectably—the XOR-malleability of Section 6’s stream ciphers is total. Ciphertext integrity (Definition 6.14) makes any modification decrypt to $\perp$ except with negligible probability, giving the symmetric payload its chosen-ciphertext robustness.
- *Without the ephemeral key*—a static-static variant—every message to B would begin from the same shared secret. The fresh per-message ephemeral, with E_{pk} bound into the KDF context, gives each ciphertext separate key material, so nothing relies on nonce uniqueness under a reused key. What it does *not* supply is forward secrecy against later compromise of the recipient’s static key b : from a recorded E_{pk} , an attacker who learns b recomputes $[b]E_{pk}$ and with it the ciphertext’s key. Erasing e buys only the sender-side guarantee that the key derives from no stored secret of the sender.

7.8 Key privacy

Theorem 7.24 answers the eavesdropper and leaves the scanner untouched. Her posture is different: she holds a published ciphertext against every public key she can enumerate and asks not what it says but *to whom*. Confidentiality is silent on the question—a scheme may hide the message perfectly and still stamp the recipient’s key on every ciphertext. The property that denies her is *key privacy*, due to Bellare, Boldyreva, Desai, and Pointcheval. We state it for a generic public-key scheme, then prove it for the bare static-ephemeral flow with the message folded in—ElGamal encryption, the shape whose key privacy the upper volumes consume.

Definition 7.26 (Key privacy under chosen-plaintext attack, IK-CPA). A *public-key encryption scheme* Π over pp consists of a randomised $\text{KeyGen}(pp) \rightarrow (sk, pk)$, a randomised $\text{Enc}_{pk}(m) \rightarrow c$, and a deterministic $\text{Dec}_{sk}(c)$ with $\text{Dec}_{sk}(\text{Enc}_{pk}(m)) = m$ for every key pair in the support of KeyGen and every message m . For an adversary $\mathcal{A}$ the game $\text{IK-CPA}_{\Pi}^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $(sk_0, pk_0), (sk_1, pk_1) \leftarrow \text{KeyGen}(pp)$ independently and sends (pk_0, pk_1) .
2. The adversary outputs a message m .
3. The challenger draws a hidden bit $\beta \xleftarrow{\$} \{0, 1\}$ and sends $c \leftarrow \text{Enc}_{pk_{\beta}}(m)$.
4. The adversary outputs a bit β' .

The advantage is $\text{Adv}_{\Pi}^{\text{ik-cpa}}(\mathcal{A}) = |\Pr[\beta' = 1 \mid \beta = 0] - \Pr[\beta' = 1 \mid \beta = 1]|$; the scheme is *key-private* (IK-CPA) if every PPT adversary’s advantage is negligible in λ . The message is the adversary’s own, so nothing about it is hidden; what the game hides is the key.

Construction 7.27 (ElGamal encryption). Fix $pp = (G, P, q)$ as in Construction 7.16, written additively, and take group elements as messages; write EG for the scheme. A key pair carries its own base H : in the *fixed-base* variant $H = P$; in the *per-key-base* variant KeyGen samples $H \xleftarrow{\$} G \setminus \{\mathcal{O}\}$, the generalised interface of Remark 7.7.

- **KeyGen**: choose H as above and $x \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$; output $sk = x$ and $pk = (H, Y)$ with $Y = [x]H$.
- **Enc_(H,Y)(M)**: sample $r \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$; output $(R, C) = ([r]H, [r]Y + M)$.

- $\text{Dec}_x(R, C)$: output $C - [x]R$.

Correctness is the scalar identity in the proof of Proposition 7.17, valid for any base in G : $[x]R = [x]([r]H) = [r]([x]H) = [r]Y$, so $C - [x]R = M$. The scheme is the static-ephemeral flow of §7.2 with the shared secret $[rx]H$ spent as a one-time pad on a group element; the fixed-base variant is ElGamal's original scheme transposed from $\mathbb{F}_p^\times$ to a prime-order group.

Theorem 7.28 (ElGamal is key-private under DDH). *For ElGamal over G (Construction 7.27, either base variant), every adversary $\mathcal{A}$ yields two distinguishers $\mathcal{B}_0, \mathcal{B}_1$ against DDH in G (Definition 2.5, read with $[x]P$ in place of g^x as in Construction 7.16), each running in the time of $\mathcal{A}$ plus one key generation and four scalar multiplications, with*

$$\text{Adv}_{\text{EG}}^{\text{ik-cpa}}(\mathcal{A}) \leq \text{Adv}_{\text{ddh}}(\mathcal{B}_0) + \text{Adv}_{\text{ddh}}(\mathcal{B}_1).$$

Under the DDH assumption ElGamal is therefore IK-CPA: a ciphertext is computationally independent of the public key it was made for.

Proof. The proof is a hybrid through a game that uses no key at all. Write H_0 and H_1 for the IK-CPA game with β fixed to 0 and to 1, and p_0, p_1 for the probability that $\mathcal{A}$ outputs 1 in each, so that $\text{Adv}_{\text{EG}}^{\text{ik-cpa}}(\mathcal{A}) = |p_0 - p_1|$. Let H_* be the game whose challenger, after the same key generation and the same message M from $\mathcal{A}$, answers with $(R, C) = (U_1, U_2 + M)$ for $U_1, U_2 \xleftarrow{\$} G$ independent and uniform, and let p_* be the corresponding probability. The challenge of H_* is computed without reference to either key, and the triangle inequality $|p_0 - p_1| \leq |p_0 - p_*| + |p_* - p_1|$ reduces the claim to bounding each hop by a DDH advantage.

The hop $H_0 \rightarrow H_$.* The distinguisher $\mathcal{B}_0$ receives a triple $(X, Y, Z) = ([a]P, [b]P, [c]P)$ with $c = ab$ or c uniform. It samples $s \xleftarrow{\$} (\mathbb{Z}/q\mathbb{Z})^\times$ in the per-key-base variant and sets $s = 1$ in the fixed-base variant, plants the challenge in key 0 as

$$H_0 = [s]P, \quad Y_0 = [s]X = [a]H_0,$$

generates $(x_1, (H_1, Y_1))$ honestly, and hands $\mathcal{A}$ the pair $((H_0, Y_0), (H_1, Y_1))$. For invertible s the base $[s]P$ is uniform on $G \setminus \{\mathcal{O}\}$, and $Y_0 = [a]H_0$ with a uniform is an honest public key whose private key a the distinguisher never learns—nor needs, the game offering no decryption. On receiving M it replies with

$$(R, C) = ([s]Y, [s]Z + M)$$

and outputs $\mathcal{A}$'s bit. If $c = ab$, then $R = [sb]P = [b]H_0$ and $[s]Z = [sab]P = [b]([a]H_0) = [b]Y_0$, so (R, C) is an honest encryption under key 0 with ephemeral $r = b$ uniform: the view of $\mathcal{A}$ is exactly H_0 . If c is uniform, then $[s]Z = [sc]P$ is uniform in G and independent of everything else because s is invertible, and $R = [b]H_0$ is uniform in G and independent of everything else because H_0 generates G and b is independent of a, c, s , of key 1, and of M , which $\mathcal{A}$ chose before seeing R : the view is exactly H_* . Hence $|p_0 - p_*| = \text{Adv}_{\text{ddh}}(\mathcal{B}_0)$.

The hop $H_ \rightarrow H_1$* is the same with the challenge planted in key 1 and key 0 generated honestly, giving $\mathcal{B}_1$ with $|p_* - p_1| = \text{Adv}_{\text{ddh}}(\mathcal{B}_1)$. Each distinguisher performs one honest key generation and the four scalar multiplications $[s]P, [s]X, [s]Y, [s]Z$ beyond running $\mathcal{A}$. Summing the two hops gives the bound. $\square$

Remark 7.29 (What key privacy buys upstairs). The hybrid transfers verbatim to Construction 7.22: planting the challenge in the recipient's key and the ephemeral share exactly as above makes the shared secret S uniform, and once it is, the pair (E_{pk}, c) with $c = \text{Enc}_{\text{KDF}(S, \text{context})}(N, \text{ad}, m)$ is computed without reference to the recipient's key—provided the KDF context names no recipient key,

which it must not. The deployed KDF takes $(\text{sharedSecret}, \text{epk})$ (orchard crate, `src/keys.rs`, function `kdf_orchard`; §6.6): the shared secret is the keying material, the context is `epk` under a personalisation label, and no key of the recipient enters. The protocol specification requires the composed note-encryption scheme to be key-private (§“Key Derivation”): a chain scanner holding a note ciphertext against every address she can enumerate learns nothing about which one it was sent to. The second use is the group-element share of diversified-address unlinkability. A diversified address $(G_d, \text{pk}_d) = (G_d, [\text{ivk}] G_d)$ (Remark 7.30) is a per-key-base ElGamal public key, and a fresh address of the same `ivk` over a new base $G_{d'}$ has, when that base is modelled as a uniform group element, the distribution of an encryption of $\mathcal{O}$ under that key; telling which of two authorities a third address belongs to is then the IK-CPA game, and Theorem 7.28 answers it. The protocol specification sets up that correspondence, modelling the group hash behind the base as a random oracle (§“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”); the *Wallet Guide* states the resulting unlinkability requirement (§“Diversified addresses”).

7.9 Shielded-note delivery as deployed hybrid encryption

The chain assembled in this section is not an analogy for what the deployed protocol does; it is the deployed protocol, run in band on the public ledger.

Remark 7.30 (Note delivery is HPKE on the ledger). A shielded address embeds a static public key $\text{pk}_d = [\text{ivk}] G_d$ on the protocol’s curve (Pallas, in Orchard), where the private scalar `ivk` is the recipient’s incoming viewing key. The base G_d is not a protocol-wide generator but a per-address *diversified base*, the hash-to-curve image (§3.7) of the address’s diversifier—the generalised interface of Remark 7.7 exists precisely to accommodate it (orchard crate, `src/spec.rs`, function `diversify_hash`; `src/keys.rs`, where `DiversifiedTransmissionKey` is derived as $\text{pk}_d = [\text{ivk}] g_d$); the *Ironwood Guide* develops the key hierarchy (§“Viewing keys”) and address encoding (§“Diversified addresses”) behind these bases. The sender runs Construction 7.22 over that base, with one deployed refinement: the ephemeral scalar `esk` is not sampled directly but derived by $\text{PRF}^{\text{expand}}$ (Remark 5.14) from the note’s random seed (ZIP 212; orchard crate, `src/note.rs`, where `RandomSeed::esk_inner` applies `PrfExpand::ORCHARD_ESK`), so that the uniform-ephemeral hypothesis of Theorem 7.24 is met through the PRF’s pseudorandomness and the recipient can later re-derive `esk` from the delivered seed. The sender publishes $\text{epk} = [\text{esk}] G_d$ in the transaction, derives

$$K = \text{KDF}([\text{esk} \cdot \text{ivk}] G_d, \text{epk})$$

by the personalised BLAKE2b call of §6.6, and AEAD-encrypts the note plaintext—a version byte, the recipient’s diversifier, the value, the note’s random seed, and the memo—under K with ChaCha20-Poly1305 (Construction 6.18), attaching the ciphertext to the transaction. The nonce is fixed to zero and the associated data is empty, safe because each K keys exactly one encryption (protocol specification §“Encryption (Sapling and Orchard)”); the `zcash_note_encryption` crate, `src/lib.rs`: `encrypt_note_plaintext`, which chains `ka_agree_enc`, `kdf`, and `ChaCha20Poly1305` with the zero nonce; the Orchard instantiation of the `Domain` trait is orchard crate, `src/note_encryption.rs`.

Chain scanners see (epk, c) in every shielded output, but lacking `ivk` they learn nothing (Theorem 7.24); the recipient *trial-decrypts*, deriving $K' = \text{KDF}([\text{ivk}] \text{epk}, \text{epk})$ for each incoming transaction and keeping those outputs whose AEAD tag verifies *and* whose plaintext passes two consistency checks: the note commitment recomputed from it must match the commitment on chain, and the `esk` re-derived from its random seed must reproduce the published `epk`—the notes addressed to them (protocol specification §“Decryption using an Incoming Viewing Key (Sapling and Orchard)”); `zcash_note_encryption` crate, `src/lib.rs`, `try_note_decryption` and `check_note_validity`). Non-interactivity is essential, and it is exactly the static-ephemeral flow of §7.2: the recipient need never have been online when the note was sent. Authenticity of the static key—the concern of §7.5—is supplied by mechanism (2) of Remark 7.15: the address is part of the recipient’s verified payment credential, so the man-in-the-middle substitution has nowhere to stand; and, as noted there, this

authenticates the recipient rather than the sender. The complementary guarantees on the note’s *contents*—that the value is well-formed and conserved, that the commitment on chain matches the plain-text delivered—come not from the encryption but from the accompanying zero-knowledge proof.

The primitive toolbox now covers the delivery of a secret to a party who was never online to negotiate it. What travels alongside that ciphertext—the authorising signatures over the transaction that carries it—is the business of the next section.

8 Digital signatures

The adversary of this section is the *forger*. It watches the network accept spend after spend, each authorised by a signature; it may coax the legitimate signer into signing any messages of its choosing; and it wins if it can then exhibit one signature the signer never made—a spend of coins it does not control, an authorisation conjured from public data alone. Every verifier in a consensus network checks every signature, so the forger’s target is public and its victory is total: a single forged spend authorisation steals, and a single forged balance certificate mints value from nothing. The primitive that defeats it, the digital signature, is the last cryptographic object of this volume built directly on the discrete logarithm; it will also turn out, in its Schnorr form, to be the first *proof of knowledge* we construct—the bridge to the proof systems of Section 9.

8.1 Signatures versus MACs

The message authentication codes of §6.3 already defeat a forger, but only between two parties who share a secret key: verification runs Vrfy_k , so whoever can *check* a tag can also *mint* one. Authentication by MAC is therefore private and repudiable—a verifier cannot exhibit the tag to a third party as evidence, since the verifier could have produced it. A blockchain inverts every one of these constraints. A transaction’s authorisation must be checked by every full node, none of whom the signer has met and none of whom may be capable of forging it; and the check must be transferable, so that a block, once validated, convinces everyone else.

A *digital signature* meets these demands by splitting the key. Signing uses a secret key sk ; verification uses a matching *public* key pk that may be published to the world. Anybody can verify; only the holder of sk can sign; and because verification needs no secret, a valid signature is *non-repudiable* evidence that the holder of sk signed. The price is computational: where the Carter–Wegman MAC of §6.3 was information-theoretically secure, every signature scheme in this section rests on the hardness of the discrete logarithm.

Zcash’s Orchard protocol uses two closely related signature schemes, both Schnorr-type schemes over an elliptic curve, and both derived from the one identification protocol this section develops (reddsa crate, `src/orchard.rs`; protocol specification § 5.4.7):

- the *spend authorisation signature*, whose key pair can be *re-randomised* so that successive uses are unlinkable (orchard crate, `src/keys.rs`, `SpendAuthorizingKey::randomize`; protocol specification § 4.15), and
- the *binding signature*, which simultaneously certifies the consistency of a transaction’s value commitments and proves knowledge of a discrete logarithm (orchard crate, `src/bundle.rs`, `Bundle::binding_validating_key`; protocol specification § 4.14).

Conventions for this section. Throughout, $\mathbb{G}$ is a cyclic group of prime order q , written additively, with a fixed generator G ; scalar multiplication is $[x]P$ for $x \in \mathbb{F}_q$ and $P \in \mathbb{G}$. Concretely $\mathbb{G}$ is the Pallas group of §2.6, whose order is the 255-bit prime q , so scalars live in the field $\mathbb{F}_q$. Because $\mathbb{G}$ has prime order and G generates it, the map $x \mapsto [x]G$ is a bijection $\mathbb{F}_q \rightarrow \mathbb{G}$: the discrete logarithm of any $P \in \mathbb{G}$ to base G always *exists* and is *unique*. Hardness of computing it is the DLog assumption of §2.1, transcribed additively.

8.2 Syntax and security goal

Definition 8.1 (Digital signature scheme). A *digital signature scheme* is a triple of PPT algorithms $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ with, for each security parameter λ , a message space $\mathcal{M}_\lambda$:

- $\text{KeyGen}(1^\lambda)$ outputs a key pair (pk, sk) ;
- $\text{Sign}(sk, m)$, for $m \in \mathcal{M}_\lambda$, outputs a signature σ ;
- $\text{Verify}(pk, m, \sigma)$ is deterministic and outputs a bit.

Correctness requires that for every λ , every (pk, sk) in the support of $\text{KeyGen}(1^\lambda)$, and every $m \in \mathcal{M}_\lambda$,

$$\Pr[\text{Verify}(pk, m, \text{Sign}(sk, m)) = 1] = 1,$$

the probability over the coins of Sign . Probability-1 correctness is called *perfect*; a scheme achieving only overwhelming-probability correctness tolerates a $\text{negl}(\lambda)$ failure rate. Every scheme in this section is perfectly correct.

The security goal transcribes the MAC forgery game of Definition 6.8 into the public-key setting: the adversary now *receives* the verification key, and its oracle access models a signer who can be induced to sign arbitrary messages.

Definition 8.2 (Existential unforgeability under chosen-message attack). For a signature scheme Π and adversary $\mathcal{A}$, the game $\text{Forge}_\Pi^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger runs $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$.
2. The adversary $\mathcal{A}$ runs on input pk with oracle access to $\text{Sign}(sk, \cdot)$; let Q be the set of messages it submits to the oracle, and let (m^*, σ^*) be its output.
3. The game outputs 1 iff $\text{Verify}(pk, m^*, \sigma^*) = 1$ and $m^* \notin Q$.

Scheme Π is *existentially unforgeable under chosen-message attack* (EUF-CMA) if for every PPT $\mathcal{A}$,

$$\text{Adv}_\Pi^{\text{forge}}(\mathcal{A}, \lambda) := \Pr[\text{Forge}_\Pi^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

Two features of the definition deserve emphasis. The oracle access is *adaptive* and bounded only by the adversary's polynomial running time: it may choose each query after seeing all previous signatures. And the forgery is *existential*: the adversary wins with a valid signature on *any* message never submitted to the oracle, however meaningless—the definition does not ask the forged message to be useful, so security under it protects even messages the signer would consider absurd.

Definition 8.3 (Strong unforgeability). The *strong* variant, sUF-CMA, replaces the winning condition $m^* \notin Q$ by: the pair (m^*, σ^*) was not among the message–signature pairs returned by the oracle.

Strong unforgeability forbids producing even a *new* signature on an *already-signed* message. Plain EUF-CMA permits a scheme in which, given one valid signature, anyone can cheaply compute a second, different valid signature on the same message; such *malleability* is harmless when a signature only certifies origin, but fatal whenever a signature doubles as a unique token—certain anti-replay mechanisms, for instance, identify a message by its signature and break if signatures are malleable.

8.3 The hash-and-sign paradigm

The textbook mechanisms—RSA, Schnorr, ECDSA—sign only fixed-size inputs: an element of $\mathbb{Z}/N\mathbb{Z}$ for RSA, a scalar in $\mathbb{F}_q$ for Schnorr and ECDSA. Arbitrary messages are accommodated by hashing first, and the composition is provably safe.

Theorem 8.4 (Hash-and-sign). *Let Π' be a signature scheme with message space $\mathcal{Y}_\lambda$, and let $H : \{0, 1\}^* \rightarrow \mathcal{Y}_\lambda$ be drawn from a collision-resistant family (Definition 3.7). Define Π with message space $\{0, 1\}^*$ by*

$$\text{Sign}(sk, m) := \text{Sign}'(sk, H(m)), \quad \text{Verify}(pk, m, \sigma) := \text{Verify}'(pk, H(m), \sigma).$$

If Π' is EUF-CMA and H is collision resistant, then Π is EUF-CMA; concretely, for every adversary $\mathcal{A}$ against Π there are comparably efficient $\mathcal{B}_1$ against H and $\mathcal{B}_2$ against Π' with

$$\Pr[\mathcal{A} \text{ wins}] \leq \text{Adv}_H^{\text{col}}(\mathcal{B}_1) + \text{Adv}_{\Pi'}^{\text{forge}}(\mathcal{B}_2).$$

Proof. Let (m^*, σ^*) be a valid forgery of $\mathcal{A}$ on a fresh message $m^* \notin Q$, and split on the digest. *Case 1: the digest is reused*, i.e. $H(m^*) = H(m)$ for some queried $m \in Q$. Then $m^* \neq m$ but their digests agree, so the pair (m^*, m) is a collision in H ; the reduction $\mathcal{B}_1$, which generates a key pair with KeyGen' itself and answers $\mathcal{A}$'s signing queries by signing digests under that secret key—the collision game grants no signing oracle—outputs it. *Case 2: the digest is fresh*, i.e. $H(m^*)$ differs from the digest of every queried message. Then $(H(m^*), \sigma^*)$ is a valid forgery against Π' on a message never submitted to its signing oracle, and $\mathcal{B}_2$ outputs it. The two cases are exhaustive, so the union bound on the winning event gives the stated inequality. $\square$

Remark 8.5 (What “hash-and-sign” hides). In all the Schnorr-type schemes below, the hash absorbs more than the message: the public key and the per-signature commitment value enter the hash alongside m . Hashing the public key prevents key-substitution (duplicate-signature) attacks, in which a signature valid under one key is exhibited as valid under another; hashing the commitment is exactly what the Fiat–Shamir transform of §8.5 requires. “Hash-and-sign” in practice therefore means hashing more than the message; Theorem 8.4 captures only the message-compression part of the discipline.

8.4 The Schnorr identification protocol

Every scheme in the rest of this section is a costume worn by a single interactive protocol, in which a prover convinces a verifier that it *knows* the discrete logarithm of a public point—without revealing anything about it. We develop the protocol self-contained, with concrete statements and proofs; the general vocabulary it exemplifies (*Sigma-protocols*, of which it is the canonical instance: three-move, public-coin, honest-verifier zero-knowledge (HVZK) proofs of knowledge) is deliberately deferred to Section 9, where it is developed for arbitrary relations.

The object of the protocol is the discrete-log relation

$$R_{\text{dl}} = \{ (X, x) \in \mathbb{G} \times \mathbb{F}_q : X = [x]G \} :$$

the *statement* is a public point X , the *witness* its discrete logarithm x .

Construction 8.6 (Schnorr identification). Fix $\mathbb{G} = \langle G \rangle$ of prime order q . The prover holds a secret key $x \in \mathbb{F}_q$; both parties hold the public key $X = [x]G$. The protocol has three moves.

1. *Commitment.* The prover samples $r \xleftarrow{\$} \mathbb{F}_q$ uniformly and sends $R = [r]G$. We call R the

commitment (or nonce point) and r the nonce.

2. *Challenge*. The verifier samples $c \xleftarrow{\$} \mathbb{F}_q$ uniformly and sends c .
3. *Response*. The prover sends $s = r + cx$ in $\mathbb{F}_q$.

The verifier accepts iff

$$[s]G = R + [c]X. \quad (2)$$

A triple (R, c, s) satisfying (2) is an *accepting transcript* for X . The verifier's only message is a uniformly random value chosen independently of R , so the protocol is *public-coin*: the verifier keeps no secrets and exercises no discretion.

Three properties make this protocol what it is: honest runs always accept; answering two distinct challenges on one commitment is possible only for a party who knows x ; and a transcript reveals nothing about x . We prove each in turn.

Proposition 8.7 (Perfect completeness). *If prover and verifier are honest, the verifier accepts with probability 1.*

Proof. With $R = [r]G$, $X = [x]G$ and $s = r + cx$, linearity of scalar multiplication gives

$$[s]G = [r + cx]G = [r]G + [cx]G = R + [c]([x]G) = R + [c]X,$$

which is exactly (2). The computation holds identically for every choice of r and c . $\square$

Theorem 8.8 (2-special soundness). *There is an efficient extractor that, given two accepting transcripts (R, c, s) and (R, c', s') for the same statement X with the same commitment R but distinct challenges $c \neq c'$, outputs the discrete logarithm x of X .*

Proof. Both transcripts satisfy (2); subtracting the two equations in $\mathbb{G}$ cancels R :

$$[s - s']G = [c - c']X = [(c - c')x]G.$$

Since G generates a group of prime order q , equality of the two multiples forces the equality of scalars $s - s' = (c - c')x$ in $\mathbb{F}_q$. The hypothesis $c \neq c'$ makes $c - c'$ a nonzero element of the field $\mathbb{F}_q$, hence invertible, so the extractor outputs

$$x = (s - s')(c - c')^{-1} \in \mathbb{F}_q.$$

Verification that $X = [x]G$ is direct, and the whole computation costs a single field inversion. $\square$

Theorem 8.8 is the central mechanism of this section. It makes the protocol a *proof of knowledge*: below, it converts a convincing prover into an extractable witness; after the Fiat–Shamir transform it is the source of unforgeability; and in §8.6 it reappears with its sign reversed, as the reason a signer must never reuse a nonce.

Remark 8.9 (From special soundness to knowledge, by rewinding). Suppose a (possibly dishonest) prover makes the verifier accept with probability ε noticeably larger than $1/q$ —the probability of simply guessing the single challenge for which a prepared response works. Run the prover once: record its commitment R , feed it a uniform challenge c , and note whether its response accepts. Now *rewind* it to the moment just after it sent R —restore its internal state, which is possible because we run the prover as a subroutine—and feed a fresh independent challenge c' . A standard averaging argument shows that with probability roughly ε^2 , both runs accept and $c \neq c'$; the argument is the *heavy-row*

lemma, an instance of Markov’s inequality applied to the matrix whose rows are the prover’s random coins and whose columns are challenges (*Math Guide*, §“Random variables and expectation”). The two accepting transcripts share R , so the extractor of Theorem 8.8 recovers x . Making the verifier accept with non-trivial probability is therefore, up to rewinding, the same as *knowing* x . The same rewinding argument, sharpened into the forking lemma, carries the signature analysis of §8.6.

Theorem 8.10 (Special honest-verifier zero knowledge). *There is an efficient simulator Sim that, on input a statement $X \in \mathbb{G}$ and a challenge $c \in \mathbb{F}_q$, outputs an accepting transcript (R, c, s) distributed identically to an honest execution conditioned on the challenge being c . Consequently, for a uniform challenge, the distribution of simulated transcripts coincides exactly with the distribution of real ones.*

Proof. The simulator works *backwards*: it samples $s \xleftarrow{\$} \mathbb{F}_q$ uniformly, sets

$$R := [s]G - [c]X,$$

and outputs (R, c, s) —accepting by construction, since (2) rearranges to exactly this definition of R .

Compare distributions with the challenge fixed to c . In a real honest transcript, r is uniform on $\mathbb{F}_q$, $R = [r]G$, and $s = r + cx$; the map $r \mapsto r + cx$ is a translation of $\mathbb{F}_q$, hence a bijection, so s is uniform on $\mathbb{F}_q$, and $R = [s - cx]G = [s]G - [c]X$ is the same deterministic function of s that the simulator uses. The real transcript *is*: sample s uniform, set $R = [s]G - [c]X$ —precisely the simulated distribution. The two distributions are identical, not merely statistically close, and averaging over a uniform c preserves the identity. $\square$

Remark 8.11 (Why honest-verifier zero knowledge suffices). The simulator of Theorem 8.10 matches the real distribution only when the challenge is sampled independently of R . A cheating verifier could instead choose c as a function of R , and against such a verifier the simulator—which commits to R only *after* seeing c —no longer obviously works. For our purposes honest-verifier zero knowledge is exactly enough: after the Fiat–Shamir transform the challenge is a *hash* of R , and in the random oracle model that hash behaves like an honest, R -independent coin. The deeper point of the theorem is what the simulator’s existence *proves*: since anyone, without x , can manufacture transcripts with the true distribution, a transcript carries no information about x beyond what the statement X already reveals.

8.5 From identification to signature via the Fiat–Shamir transform

An identification protocol convinces one verifier, once, interactively. A signature must convince everyone, forever, on its own. The *Fiat–Shamir transform* removes the interaction by making the prover compute the challenge itself—as a hash of everything the verifier would have seen before issuing it, with the message folded in so that the resulting object certifies m . The transform is generic: it converts *any* Sigma-protocol into a non-interactive proof, and, with a message in the hash, into a signature. This subsection owns only the Schnorr instance; the general transform, its subtleties and its failure modes are the business of Section 9 (§“The Fiat–Shamir transform: from interactive to non-interactive”).

Construction 8.12 (Schnorr signature). Fix $\mathbb{G} = \langle G \rangle$ of prime order q and a hash function $H : \{0, 1\}^* \rightarrow \mathbb{F}_q$. Messages are arbitrary bit strings.

- $\text{KeyGen}(1^\lambda)$: sample $x \xleftarrow{\$} \mathbb{F}_q$, set $X = [x]G$, output $(pk, sk) = (X, x)$.
- $\text{Sign}(x, m)$: sample $r \xleftarrow{\$} \mathbb{F}_q$, set $R = [r]G$, compute the challenge

$$c = H(R \parallel X \parallel m),$$

set $s = r + cx$ in $\mathbb{F}_q$, and output $\sigma = (R, s)$.

- $\text{Verify}(X, m, (R, s))$: recompute $c = H(R \parallel X \parallel m)$ and accept iff $[s]G = R + [c]X$.

Proposition 8.13 (Perfect correctness). *The Schnorr signature scheme is perfectly correct.*

Proof. For an honest signature (R, s) with $R = [r]G$, $c = H(R \parallel X \parallel m)$ and $s = r + cx$, the verifier recomputes the same c —the challenge is a deterministic function of (R, X, m) , all of which the verifier holds—and the check $[s]G = R + [c]X$ holds by the completeness computation of Proposition 8.7. $\square$

Remark 8.14 (Two conventions). Two presentational choices in Construction 8.12 deserve note.

1. The signature transmits (R, s) and omits c , which the verifier recomputes. The space-saving variant transmits (c, s) instead: the verifier recovers $R = [s]G - [c]X$ (the simulator’s equation, read as reconstruction) and checks $H(R \parallel X \parallel m) = c$. The two are interchangeable; the deployed schemes below use the (R, s) form.
2. The hash takes the public key X alongside R and m —*key-prefixing*. The single-key unforgeability proof of §8.6 does not require it, but it forecloses attacks that relate signatures under different keys (Remark 8.5), it is standard in modern instantiations such as EdDSA, and it earns its keep in the re-randomised setting of §8.8.

8.6 Security in the random oracle model and the forking lemma

What must a forger of Schnorr signatures accomplish? A valid pair (R^*, s^*) on a fresh message is an accepting transcript of the identification protocol whose challenge is $H(R^* \parallel X \parallel m^*)$ —a value the forger cannot choose freely if the hash is honest. The security argument makes “honest” precise by idealising H , then turns the rewinding of Remark 8.9 into a quantitative tool.

Definition 8.15 (Random oracle, general range). The random oracle of Definition 3.13 generalises verbatim to an arbitrary finite range: a random oracle with range Y is a function $\mathcal{O} : \{0, 1\}^* \rightarrow Y$ drawn uniformly from the set of all such functions, realised lazily by sampling each new answer $\mathcal{O}(z) \xleftarrow{\$} Y$ on first query. For Schnorr signatures the range is the challenge space $Y = \mathbb{F}_q$, and the ROM for signatures models $H(\cdot) = \mathcal{O}(\cdot)$, with every party—signer, verifier, forger, and reduction—sharing oracle access (Definition 1.26).

The model remains a heuristic, with the uninstantiability caveat and the working stance of Remark 1.29: a ROM proof rules out all attacks that treat the hash as a black box—the overwhelming majority—and it is the standard yardstick for Fiat–Shamir signatures. The reductions below exploit both powers of Proposition 3.14: *programmability* (the reduction chooses oracle answers on the fly, provided they appear uniform) and *observability* (the reduction sees every query the adversary makes).

The first tool disposes of the signing oracle: in the ROM, the zero-knowledge simulator of Theorem 8.10 answers signing queries without the secret key.

Lemma 8.16 (Signature simulation). *In the ROM there is an efficient simulator that, given only the public key X and the ability to program H , answers signing queries with signatures distributed identically to genuine ones, except that it aborts with probability at most $q_s(q_s + q_h)/q$ over an attack making q_s signing and q_h hash queries.*

Proof. To sign m without x : sample $c \xleftarrow{\$} \mathbb{F}_q$ and $s \xleftarrow{\$} \mathbb{F}_q$ uniformly, set $R := [s]G - [c]X$ —the HVZK simulator of Theorem 8.10—then *program*

$$H(R \parallel X \parallel m) := c,$$

and output (R, s) . The signature verifies by construction, and by Theorem 8.10 its distribution matches a real signature’s exactly: in both cases s is uniform, and c is uniform because a fresh oracle answer is.

Programming fails only if the entry $H(R \parallel X \parallel m)$ is already defined—by an earlier adversary hash query or an earlier simulated signature. The point $R = [s]G - [c]X$ is a fixed translate of $[s]G$ for uniform s , hence uniform over the q elements of $\mathbb{G}$; at most $q_h + q_s$ oracle entries exist at any point of the attack, so a given signing query collides with probability at most $(q_h + q_s)/q$, and the union bound over q_s signing queries gives the stated abort probability. Conditioned on no abort, the adversary’s view is identical to the real game. $\square$

Lemma 8.16 reduces forgery to a pure discrete-log problem: the reduction can play the whole EUF-CMA game knowing only X . What remains is to convert one forgery into *two* accepting transcripts on the same commitment—the input that Theorem 8.8 demands. The quantitative engine is the forking lemma, in the game-based form due to Bellare and Neven.

Theorem 8.17 (General forking lemma). *Fix an integer $Q \geq 1$, a finite set C with $|C| = h \geq 2$, and a randomised input generator Gen . Let B be a randomised algorithm that on input y and challenges $c_1, \dots, c_Q \in C$ returns a pair (I, out) with $I \in \{0, 1, \dots, Q\}$, where $I = 0$ means failure, and let*

$$\text{acc} := \Pr[I \geq 1 : y \leftarrow \text{Gen}; c_1, \dots, c_Q \xleftarrow{\$} C; (I, \text{out}) \leftarrow B(y, c_1, \dots, c_Q)].$$

Define the forking algorithm F_B on input y : pick coins ρ for B and $c_1, \dots, c_Q \xleftarrow{\$} C$; run $(I, \text{out}) \leftarrow B(y, c_1, \dots, c_Q; \rho)$; if $I = 0$, return $\perp$; otherwise resample $c'_1, \dots, c'_Q \xleftarrow{\$} C$ afresh and run $(I', \text{out}') \leftarrow B(y, c_1, \dots, c_{I-1}, c'_I, \dots, c'_Q; \rho)$ with the same coins ρ ; if $I' = I$ and $c_I \neq c'_I$, return $(I, \text{out}, \text{out}')$, else $\perp$. Then

$$\text{frk} := \Pr[F_B(y) \neq \perp : y \leftarrow \text{Gen}] \geq \text{acc} \left(\frac{\text{acc}}{Q} - \frac{1}{h} \right),$$

the probability also over the coins of F_B .

Proof idea. Condition on the coins ρ and the prefix $c_1, \dots, c_{I-1}$ up to the forking point. For each fixed prefix there is (averaging over the suffix) a set S of good values of c_I on which B succeeds with index I ; write $p = |S|/h$. The fork succeeds at this prefix iff the first draw c_I and the independent re-draw c'_I both land in S and differ: two independent uniform draws land in S together with probability p^2 , and subtracting the diagonal of equal draws costs at most $p \cdot 1/h$, so the per-prefix fork probability is at least $p(p - 1/h)$. Taking expectations over prefixes, $\mathbb{E}[p^2] \geq (\mathbb{E}[p])^2$, since $\mathbb{E}[p^2] - (\mathbb{E}[p])^2 = \mathbb{E}[(p - \mathbb{E}[p])^2] \geq 0$ by linearity of expectation (*Math Guide*, §“Random variables and expectation”); this squares the average success probability. The index I ranging over Q possible positions introduces the factor $1/Q$, and collecting terms yields $\text{frk} \geq \text{acc}^2/Q - \text{acc}/h$. The squaring step is where the squared probability—and hence the tightness loss examined below—enters. $\square$

Theorem 8.18 (EUF-CMA security of Schnorr signatures in the ROM). *Model H as a random oracle with range $\mathbb{F}_q$. If an adversary $\mathcal{A}$ runs in time t , makes at most q_h hash queries and q_s signing queries, and forges with probability ϵ , then there is an algorithm $\mathcal{D}$ running in time approximately $2t$*

that computes discrete logarithms in $\mathbb{G}$ with probability

$$\epsilon' \geq \frac{\epsilon^2}{q_h + q_s + 1} - \frac{q_h + q_s + 1}{q} - \frac{q_s(q_h + q_s)}{q}.$$

In particular, if DLog is hard in $\mathbb{G}$, the Schnorr signature scheme is EUF-CMA in the ROM.

Proof structure. Wrap $\mathcal{A}$ as the algorithm B of Theorem 8.17, with input $y = X$ —the DLog challenge, so Gen outputs $X = [x]G$ for uniform x —challenge set $C = \mathbb{F}_q$ (so $h = q$), and $Q = q_h + q_s + 1$ prepared challenges. Algorithm B runs $\mathcal{A}$ on $pk = X$, answering the j -th *distinct* hash query with c_j from its challenge list and answering signing queries by the simulator of Lemma 8.16, whose programming aborts cost the $q_s(q_h + q_s)/q$ term.

Suppose $\mathcal{A}$ outputs a valid fresh forgery $(m^*, (R^*, s^*))$, and let $c^* = H(R^* || X || m^*)$. The critical hash input $R^* || X || m^*$ must actually have been queried during the run: otherwise c^* is a uniform value $\mathcal{A}$ has never seen, and the verification equation $[s^*]G = R^* + [c^*]X$ then holds only with probability $1/q$ over the oracle's choice—the $(q_h + q_s + 1)/q$ term collects these guessing events (the $+1$ covering the query B itself makes to verify the forgery). So B outputs the index I of the critical query together with $out = (R^*, c^*, s^*, m^*)$, and $acc \geq \epsilon$ minus the loss terms above.

Now fork. Both runs of F_B share the coins ρ and the challenge prefix $c_1, \dots, c_{I-1}$, which together determine $\mathcal{A}$'s behaviour up to the moment it produces the critical query; both runs therefore query the *same* critical input $R^* || X || m^*$ at index I , but receive different answers $c^* \neq c'^*$. Success of the fork yields two accepting transcripts (R^*, c^*, s^*) and (R^*, c'^*, s'^*) with the same commitment and distinct challenges, whereupon the extractor of Theorem 8.8 computes

$$x = (s^* - s'^*)(c^* - c'^*)^{-1},$$

the discrete logarithm of X . Substituting acc , $h = q$ and $Q = q_h + q_s + 1$ into $frk \geq acc^2/Q - acc/h$ gives the stated bound; the running time is two runs of $\mathcal{A}$ plus bookkeeping. $\square$

Example 8.19 (The tightness loss, in numbers). The dominant term of Theorem 8.18 is $\epsilon' \gtrsim \epsilon^2/q_h$: the reduction runs the forger twice and *squares* its success probability, divided by the number of hash queries. The reduction is therefore far from tight (Definition 1.22). Concretely, a forger succeeding with probability $\epsilon = 2^{-40}$ after $q_h = 2^{60}$ hash queries yields a guaranteed DLog-solver advantage of only about

$$\epsilon^2/q_h = 2^{-80}/2^{60} = 2^{-140},$$

a factor 2^{100} weaker than ϵ itself. The guarantee runs in the useful direction—any forger yields *some* DLog solver, so DLog hardness still implies security—but the quantitative content is feeble. Two lessons follow, and they must be kept apart. The conservative lesson (Remark 1.23): a designer who wants *this theorem* to underwrite a target signature-security level must provision $\mathbb{G}$ with roughly twice as many bits of DLog security as the target, since the reduction hands back only the square of the forger's advantage; and the squaring is intrinsic to the rewinding route—it enters at the squaring step of Theorem 8.17, so any analysis that runs the forger twice and extracts by special soundness pays it. What the theorem does *not* establish is a matching attack: the loss belongs to the reduction, not provably to the scheme, so doubled parameters are the price of this proof, not a demonstrated necessity, and parameter selection in practice weighs concrete attack models and query bounds as well. Tighter reductions exist once the rewinding route is abandoned—under the *one-more* discrete logarithm assumption (that no efficient adversary, given $\ell + 1$ random elements of $\mathbb{G}$ and ℓ queries to a discrete-logarithm oracle, outputs all $\ell + 1$ logarithms), under DDH (Definition 2.5) together with key-prefixing, or in the idealised algebraic models of Remark 1.30.

Proposition 8.20 (Nonce reuse is fatal). Suppose a signer produces two Schnorr signatures (R, s) and (R, s') on distinct messages $m \neq m'$ with the same commitment R —a nonce r reused outright, or

drawn twice from a faulty source. Then, except with probability $1/q$ over the challenge hash, anyone can compute the secret key from the two signatures.

Proof. The challenges $c = H(R \parallel X \parallel m)$ and $c' = H(R \parallel X \parallel m')$ differ except with probability $1/q$, the chance that the random oracle answers the two distinct inputs identically. Given $c \neq c'$, the pairs (R, c, s) and (R, c', s') are two accepting transcripts sharing a commitment with distinct challenges, and the extractor of Theorem 8.8—run now by the *adversary*—outputs $x = (s - s')(c - c')^{-1}$. $\square$

Special soundness, the source of unforgeability, is thus also its dark mirror: the security of the scheme rests entirely on the nonce being fresh and unpredictable for every signature. This is not a theoretical worry—it is the classic operational failure of deployed Schnorr-family schemes—and it motivates deriving nonces *deterministically* from the secret key and message, the defining idea of EdDSA, to which we now turn.

8.7 RedDSA: re-randomisable Schnorr for Orchard

Construction 8.21 (EdDSA, schematically). EdDSA retains the Schnorr signature algebra of Construction 8.12 unchanged and hardens two implementation surfaces.

1. *Deterministic nonces.* The nonce is derived as a hash of a secret prefix (part of the expanded secret key) and the message, rather than sampled fresh: no faulty random-number generator can repeat a nonce across distinct messages and trigger Proposition 8.20.
2. *Key-prefixing.* The challenge hash absorbs the public key alongside R and m , as in Construction 8.12 and for the reasons of Remark 8.14.

In the ROM the deterministic nonce is, to anyone ignorant of the secret prefix, a fresh uniform value per message, so the unforgeability proof of Theorem 8.18 carries over essentially unchanged.

Zcash’s signature scheme generalises EdDSA in two directions, and the generalisations are the whole point: the base point becomes a parameter, and the key becomes shiftable.

Construction 8.22 (RedDSA). RedDSA is the Schnorr/EdDSA scheme with the base point $P \in \mathbb{G}$ a *parameter* of the scheme: different uses sign with respect to different generators.

- **KeyGen:** sample $x \xleftarrow{\$} \mathbb{F}_q$, output secret key x and public key $X = [x]P$.
- **Sign(x, m):** sample a fresh random byte string T and derive the nonce $r = H(T \parallel X \parallel m)$. This is *randomised*, hash-based nonce generation: unlike EdDSA’s derivation, it is not deterministic from the signing key and message, so its safety rests on T being fresh and unpredictable, the hashing ensuring only that a repeated T cannot repeat a nonce across distinct messages. Set $R = [r]P$, $c = H(R \parallel X \parallel m) \in \mathbb{F}_q$, $s = r + cx$, and output $\sigma = (R, s)$.
- **Verify($X, m, (R, s)$):** accept iff $[s]P = R + [c]X$ with $c = H(R \parallel X \parallel m)$.

For a fixed base point, RedDSA is Schnorr/EdDSA, and its EUF-CMA security in the ROM is Theorem 8.18 with G replaced by P .

Remark 8.23 (RedDSA as deployed). The deployed scheme follows Construction 8.22 line for line (reddsa crate, src/signing_key.rs, SigningKey::sign; protocol specification § 5.4.7). The randomiser T is 80 uniform bytes, and both hashes are the function H° : BLAKE2b-512 personalised with Zcash_RedPallasH, its 64-byte output reduced to $\mathbb{F}_q$ by wide reduction (src/hash.rs, HStar)—the expand-then-reduce discipline of Construction 3.18, with the 512-bit width making the modular bias

negligible. The nonce hash absorbs $T \parallel X \parallel m$ and the challenge hash $R \parallel X \parallel m$, key-prefixed in both cases.

Orchard instantiates RedDSA *twice*, both times as *RedPallas*—RedDSA over the Pallas curve—and the two instantiations differ *only* in the base point (`reddsa crate, src/orchard.rs`):

- the *spend authorisation* base point, derived by hashing to the curve with domain `z.cash:Orchard` and input `G`, for the re-randomisable signature of §8.8; and
- the *value-commitment randomness* base point, derived with domain `z.cash:Orchard-cv` and input `r`, for the binding signature of §8.9. This is byte-for-byte the randomness base of Orchard’s value commitment (`orchard crate, src/value.rs, ValueCommitment::derive`, with the domain strings in `src/constants/fixed_bases.rs`)—an identity that §8.9 shows is the entire design.

The curve varies only across protocol generations: Sapling’s *RedJubjub* is the same scheme over the Jubjub curve (`reddsa crate, src/lib.rs`, the Sealed impls for `sapling::SpendAuth` and `sapling::Binding`); the curve, its two base points and the $H^\circledast$ personalisation (`Zcash_RedJubjubH` against `Zcash_RedPallasH`) are all that distinguish the two.

8.8 Key re-randomisation and unlinkability

A shielded protocol faces a linkage problem that no ordinary signature scheme addresses. Spending a note requires proving spend authority, and the obvious design—publish the spender’s public key and a signature under it—would stamp every spend by the same wallet with the same key, linking them all. Orchard’s answer exploits the algebraic structure of the RedDSA public key: because the key is a homomorphic image of the secret, it can be *shifted* into a fresh disguise for every spend.

Definition 8.24 (Key re-randomisation). Fix a base point P . Given a secret key $x \in \mathbb{F}_q$ with public key $X = [x]P$, and a *randomiser* $\alpha \in \mathbb{F}_q$, the re-randomised signing and verification keys are

$$rsk := x + \alpha \in \mathbb{F}_q, \quad rk := X + [\alpha]P \in \mathbb{G}.$$

Orchard publishes a fresh rk per spend, signs with the shifted secret key rsk , and proves in zero knowledge that rk is a valid re-randomisation of an authorised public key (`orchard crate, src/keys.rs`, `SpendAuthorizingKey::randomize` and `SpendValidatingKey::randomize`, delegating to `reddsa’s src/signing_key.rs` and `src/verification_key.rs`).

Proposition 8.25 (Re-randomisation consistency). *The pair (rsk, rk) is a valid RedDSA key pair: $rk = [rsk]P$, so a RedDSA signature under rsk verifies under rk .*

Proof. Linearity of scalar multiplication gives

$$[rsk]P = [x + \alpha]P = [x]P + [\alpha]P = X + [\alpha]P = rk.$$

The pair is thus exactly KeyGen-shaped, and the correctness of signing and verification (Proposition 8.13) applies verbatim. $\square$

The enabling structure deserves a sentence of its own. The public key is a homomorphic image $[\cdot]P$ of the scalar, so shifting the secret by α shifts the public key by $[\alpha]P$ —a quantity computable from α and P alone, *without* the secret key. A verifier, or a circuit, can therefore check the relationship between X and rk given α , while only the spender who knows x can sign under the shifted key.

Theorem 8.26 (Perfect unlinkability of re-randomised keys). *Fix a base point P . For any public key $X \in \mathbb{G}$, if $\alpha \xleftarrow{\$} \mathbb{F}_q$ is uniform then $rk = X + [\alpha]P$ is uniformly distributed over $\mathbb{G}$. Consequently, in the game where an adversary—of unbounded computational power, and permitted to choose two public keys X_0, X_1 itself—receives $rk = X_b + [\alpha]P$ for a challenger’s uniform bit b and fresh uniform α , and must guess b : the distributions of rk for $b = 0$ and $b = 1$ are identical, and no adversary guesses b with probability better than $1/2$.*

Proof. The point P generates $\mathbb{G}$, which has prime order q , so $\alpha \mapsto [\alpha]P$ is a bijection $\mathbb{F}_q \rightarrow \mathbb{G}$; for uniform α , the point $[\alpha]P$ is uniform over $\mathbb{G}$. Translation by the fixed element X is a bijection of $\mathbb{G}$ and carries the uniform distribution to itself, so $rk = X + [\alpha]P$ is uniform over $\mathbb{G}$ —for every X , hence independently of X . The two conditional distributions of rk coincide exactly, so the adversary’s view is statistically independent of b ; no test distinguishes identical distributions, and the best any distinguisher can do is guess, succeeding with probability exactly $1/2$. $\square$

The guarantee is information-theoretic—perfect, resting on no computational assumption—but its *precondition* matters: α must be sampled uniformly and freshly per use, and must be kept secret. In Orchard the transaction builder samples $\alpha \xleftarrow{\$} \mathbb{F}_q$ afresh for each spend (orchard crate, `src/builder.rs`, `SpendInfo::build`) and reveals it only to the proving circuit and the signer.

Remark 8.27 (Scope of unlinkability). Theorem 8.26 covers only the re-randomised key. Full unlinkability of a spend further requires that the accompanying signature and zero-knowledge proof leak nothing beyond their statements, that the randomiser α stays secret, and that no other transaction field correlates two spends. In Orchard the protocol publishes rk with each Action (the unit that spends one note and creates one) while proving, inside the Halo 2 circuit, knowledge of a witness for $rk = X + [\alpha]P$ with X a public key whose holder is authorised to spend the note (Orchard’s *spend validating key*)—the circuit constrains rk as a public input (orchard crate, `src/circuit.rs`)—and the network verifies the spend authorisation signature against the public rk . Signing under rk requires $rsk = x + \alpha$, that is, knowledge of *both* x and α ; only the legitimate key-holder in possession of this spend’s α can sign.

Remark 8.28 (Unforgeability under re-randomised keys, and the key-prefixing subtlety). Does unforgeability survive re-randomisation? A forgery (R, s) under rk satisfies $[s]P = R + [c]rk$ with $c = H(R \parallel rk \parallel m)$. One is tempted to subtract $[\alpha]P$ from the verification equation: $[s - c\alpha]P = R + [c]X$, apparently a forgery under the original key X . But the identity holds with $c = H(R \parallel rk \parallel m)$, while the base verifier recomputes $H(R \parallel X \parallel m)$ —the plain subtraction converts forgeries only for the *non-key-prefixed* variant of Schnorr. For key-prefixed RedDSA one argues in the ROM by either of two routes. (i) The pair $(rsk, rk) = (x + \alpha, X + [\alpha]P)$ is itself a base-scheme key pair (Proposition 8.25) whose public key is uniform (Theorem 8.26), so a forger against rk is *literally* a forger against the base scheme at the key rk ; the reduction, which knows α , recovers $x = rsk - \alpha$ from the extracted rsk . (ii) When the adversary sees signatures under several randomisers α_i , one re-runs the forking argument of Theorem 8.18 on the critical query $H(R^* \parallel rk \parallel m^*)$, simulating signing under the remaining keys by oracle programming (Lemma 8.16). A pitfall marks the boundary: a naive translation that programs $H(R \parallel rk \parallel m) := H_{\text{base}}(R \parallel X \parallel m)$ must be a carefully injective domain swap, and it fails outright with multiple randomisers—distinct rk_i -prefixed inputs would collide on a single X -prefixed point, a detectable deviation from a random oracle.

8.9 Binding signatures as a proof of knowledge of a discrete logarithm

The second RedPallas instantiation guards a different treasure: not who may spend, but whether value is conserved. A shielded transaction hides its amounts inside commitments; the forger of this subsection is a counterfeiter who would arrange commitments whose hidden values do not balance—minting money—while still producing whatever certificate the protocol demands. The defence con-

sumes the homomorphism established in §4.6 and turns this section’s central lesson—a Schnorr signature is a proof of knowledge of a discrete logarithm—from analysis into design.

The value-commitment setup. Shielded protocols commit to value with the homomorphic Pedersen commitment

$$\text{cm}(v, \rho) = [v]V + [\rho]P,$$

where V and P are *independent* generators of $\mathbb{G}$ —independent meaning that no party knows the discrete logarithm of one to the base of the other, arranged by deriving both by hashing to the curve (Construction 3.24)—and ρ is a random blinding scalar. (In the notation of the orchard crate and of Remark 4.21, the randomness base here called P is the generator R ; this section reserves R for the signature nonce point. The base P is exactly the binding base point of Remark 8.23.) By Theorem 4.20, the group sum of commitments commits to the sum of values with the sum of blindings.

Sapling commits *per note*: a transaction carries input commitments $\text{cm}(v_i^{\text{in}}, \rho_i^{\text{in}})$ and output commitments $\text{cm}(v_j^{\text{out}}, \rho_j^{\text{out}})$, and the net commitment is

$$B = \sum_i \text{cm}(v_i^{\text{in}}, \rho_i^{\text{in}}) - \sum_j \text{cm}(v_j^{\text{out}}, \rho_j^{\text{out}}).$$

Orchard has *no* per-note value commitments: each Action—the unit that simultaneously spends a note of value v^{old} and creates a note of value v^{new} —commits once to its *net* value change, publishing

$$\text{cv}^{\text{net}} = \text{cm}(v^{\text{old}} - v^{\text{new}}, \rho) = [v^{\text{old}} - v^{\text{new}}]V + [\rho]P,$$

and $B = \sum_i \text{cv}^{\text{net}, i}$ over the transaction’s Actions (orchard crate, `src/value.rs`, `ValueCommitment::derive`; protocol specification § 5.4.8.3). The Action structure itself—how spends and outputs are fused into these units and assembled into a transaction—is the business of the *Ironwood Guide*. Either way, the homomorphism collapses the pile:

$$B = [\Delta_v]V + [\bar{\rho}]P,$$

where Δ_v is the net value— $\sum_i v_i^{\text{in}} - \sum_j v_j^{\text{out}}$ in Sapling, $\sum_i (v_i^{\text{old}} - v_i^{\text{new}})$ in Orchard—and $\bar{\rho}$ is the net blinding. The transaction balances precisely when Δ_v equals the public balancing value v^{bal} ; for a transaction with internal balance $\Delta_v = 0$, the V -component vanishes and $B = [\bar{\rho}]P$. Three observations set up the construction. Every element of $\mathbb{G}$ is some multiple of P , since P generates the prime-order group, so what distinguishes a balanced transaction is not the *existence* of a representation $B = [\beta]P$ but the ability to compute one. When the values balance, the signer knows the multiplier: it is the net blinding $\bar{\rho}$, assembled from the individual blindings. When they do not, computing any P -multiple representation of B from the published commitments would require knowing $\log_P V$, assumed hard.

Definition 8.29 (Binding signature). With B and $\bar{\rho}$ as above and the transaction balanced, so that $B = [\bar{\rho}]P$, set the *binding verification key* $\text{bvk} := B$ and the *binding signing key* $\text{bsk} := \bar{\rho}$. The binding signature is a RedDSA signature over the base point P , with signing key $\bar{\rho}$ and verification key B , on a message that is the hash of the transaction (the *sighash*). The verifier *recomputes* B from the publicly listed value commitments and the public balancing value, then checks $[s]P = R + [c]B$ with $c = H(R \parallel B \parallel \text{sighash})$.

Theorem 8.30 (The binding signature is a proof of knowledge of $\log_P B$). *In the ROM, a valid binding signature on a transaction is a proof of knowledge of the discrete logarithm of the net commitment B to the base P ; combined with the soundness of the accompanying zero-knowledge proofs (per*

Action in Orchard, per note in Sapling), which fix and range-check the committed values, it forces the transaction to balance.

Proof. Part 1: proof of knowledge. The binding signature is exactly a Fiat–Shamir Schnorr proof for the relation R_{dl} with base P and statement B . By special soundness (Theorem 8.8) together with the forking argument of Theorem 8.18—specialised to a single statement and *no* signing oracle—any algorithm producing a valid signature under B with non-negligible probability can be rewound to two accepting transcripts (R, c, s) , (R, c', s') with $c \neq c'$, from which the extractor recovers $\bar{\rho} = (s - s')(c - c')^{-1}$ with $B = [\bar{\rho}]P$. Producing a valid binding signature is therefore equivalent, up to the standard rewinding loss, to *knowing* $\log_p B$.

Part 2: knowledge forces balance. Write $B = [\Delta_v]V + [\bar{\rho}']P$ for the *true* net value and net blinding that the commitments imply; these are well defined once the accompanying proofs fix the committed openings—in Orchard the per-Action circuit fixes v^{old} , v^{new} and the opening of cv^{net} , in Sapling the per-note proofs fix each v_i, ρ_i . If the signer knows $\bar{\rho}$ with $B = [\bar{\rho}]P$, subtracting the two expressions for B gives

$$[\Delta_v]V = [\bar{\rho} - \bar{\rho}']P,$$

where Δ_v enters as a scalar, that is, as its residue in $\mathbb{F}_q$. Whenever $\Delta_v \not\equiv 0 \pmod{q}$, that residue is a nonzero element of $\mathbb{F}_q$, hence invertible, so

$$V = [(\bar{\rho} - \bar{\rho}')\Delta_v^{-1}]P$$

—exhibiting $\log_p V$ and contradicting the independence of V and P . Therefore $\Delta_v \equiv 0 \pmod{q}$.

Part 3: from congruence to equality. Balance in $\mathbb{F}_q$ alone is not yet economic balance: an integer net value equal to a nonzero multiple of q would pass the test above while minting q units at a stroke. The gap is closed by the *range checks* in the same accompanying proofs: in Orchard the per-Action circuit constrains v^{old} and v^{new} to $\{0, \dots, 2^{64} - 1\}$ (the Action statement, protocol specification § 4.18.4), and in Sapling the per-note proofs constrain each v_i likewise. A transaction with n Actions (count notes instead for Sapling) therefore has $|\Delta_v| \leq n(2^{64} - 1)$ as an integer, vastly below the 255-bit q for any transaction that can be serialised, and the only multiple of q in that range is 0. Hence $\Delta_v = 0$ over $\mathbb{Z}$, not merely in $\mathbb{F}_q$. $\square$

In deployment the transaction is not internally balanced: it declares a public balancing value v^{bal} , and the verifier signs off against $B - [v^{\text{bal}}]V$, folding the public value into the verification key—the orchard crate’s `Bundle::binding_validating_key` subtracts a commitment to v^{bal} with zero blinding from the sum of the Actions’ `cvnet` (`src/bundle.rs`; protocol specification § 4.14). The argument of Theorem 8.30 applies verbatim to the folded key and forces $\Delta_v \equiv v^{\text{bal}} \pmod{q}$; the balancing value is encoded as a signed 64-bit integer, so the range checks promote the congruence to the integer equality $\Delta_v = v^{\text{bal}}$ exactly as in Part 3.

Remark 8.31 (Why a signature, and not just a proof). The single pair (R, s) accomplishes three things at once. First, it proves knowledge of $\bar{\rho}$, certifying balance, by Theorem 8.30. Second, because the Fiat–Shamir challenge hashes the transaction sighash, the same signature *authenticates* the transaction: altering any signed field changes c and invalidates (R, s) , so the binding signature doubles as an integrity check binding the shielded components together—the origin of the name. Third, the construction needs no trusted setup and no separate circuit for the balance check: it is pure elliptic-curve arithmetic that anyone can verify. The homomorphism of the commitment turns the arithmetic statement “the values balance” into the algebraic statement “ B is a known multiple of P ”, and the Schnorr signature is the off-the-shelf proof of knowledge for precisely that statement—the purest illustration in the protocol that a digital signature is a proof of knowledge of a discrete logarithm.

Remark 8.32 (One scheme, two meanings). Set the two RedPallas uses side by side. The spend authorisation signature verifies under a re-randomised key $rk = X + [\alpha]P$, chosen to *hide* which long-term key authorised the spend: the signing key $rsk = x + \alpha$ encodes authority, and α buys unlinkability (Theorem 8.26). The binding signature verifies under a key $B = [\bar{\rho}]P$ that the spender does *not*

choose: the transaction’s own commitments force it, the “secret key” $\bar{\rho}$ is the net blinding factor, and signing proves balance (Theorem 8.30). Both are the same three lines of Schnorr arithmetic; what differs is the *meaning* assigned to the discrete logarithm—authority in one case, value-conservation in the other. The unifying lesson of this section: once a public key is the image of a secret under $x \mapsto [x]P$, a Schnorr signature is a compact, non-interactive proof of knowing that secret, and the designer chooses what knowing it *means*.

8.10 The transparent layer’s schemes: ECDSA and BIP-340 over secp256k1

Two signature schemes in the series live outside the Pallas group: the transparent layer authorises spends with ECDSA, and the issuance bundle of Zcash Shielded Assets is authorised with a BIP-340 Schnorr signature. Both run over secp256k1, and both are registered here at the level a consumer needs—syntax, deployed conventions, and where their security stands—rather than constructed from scratch.

The curve secp256k1 is the short Weierstrass curve $y^2 = x^3 + 7$ over the prime field $\mathbb{F}_p$ with $p = 2^{256} - 2^{32} - 977$, with a fixed base point G of prime order n (a 256-bit prime) and cofactor 1; every constant is a named parameter of SEC 2. In SEC 2’s terminology it is a *Koblitz curve*, one with $a = 0$, a choice that buys implementation speed and plays no role in what follows. The group law, point encodings, and the discrete-logarithm problem on such a curve are the general machinery of the *Math Guide*, §“Elliptic curves” and §“The elliptic-curve discrete logarithm problem”; the hardness assumption is Definition 2.1 in the group $\langle G \rangle$.

Construction 8.33 (ECDSA over secp256k1, schematically). Let $e \in \mathbb{Z}/n\mathbb{Z}$ be the integer read from the 32-byte digest of the message, and write $x(R)$ for the x -coordinate of a point, read as an integer; the secret scalar is written d , since x is taken.

- KeyGen: sample $d \xleftarrow{\$} [1, n - 1]$, set $X = [d]G$, output (X, d) .
- Sign(d, e): sample $k \xleftarrow{\$} [1, n - 1]$, set $R = [k]G$ and $r = x(R) \bmod n$, then $s = k^{-1}(e + rd) \bmod n$; restart if $r = 0$ or $s = 0$; output (r, s) .
- Verify($X, e, (r, s)$): require $r, s \in [1, n - 1]$, compute $R' = [es^{-1}]G + [rs^{-1}]X$, and accept iff R' is not the identity and $x(R') \equiv r \pmod{n}$.

Correctness: $R' = [s^{-1}(e + rd)]G = [k]G = R$. The nonce k must be fresh and secret, for the reason of Proposition 8.20 in different algebra: two signatures (r, s) and (r, s') under one k on distinct digests $e \neq e'$ give $s - s' = k^{-1}(e - e')$, hence $k = (e - e')(s - s')^{-1}$ and then $d = (sk - e)r^{-1}$.

Where Schnorr’s challenge is a hash of R , ECDSA’s is the coordinate r itself, so the scheme is not a Fiat–Shamir transform and the forking-lemma route to Theorem 8.18 does not apply. Unforgeability analyses exist—in the generic group model, and in the ROM under additional assumptions on the map $R \mapsto x(R) \bmod n$ —but they lie outside this volume’s path: the transparent layer inherits ECDSA’s security from Bitcoin’s track record rather than from a reduction.

Remark 8.34 (Deployment conventions: the sighash, DER, and low s). Three conventions fix how the transparent layer uses Construction 8.33, which protocol specification § 4.1.7 names as the script signature scheme.

1. *The message is the sighash.* The digest e is the 32-byte SIGHASH transaction hash of protocol specification § 4.10 (ZIP 244 for version-5 transactions), so the sighash algorithm plays the role of the hash in Theorem 8.4. Signer and verifier both feed it in as a digest (`zcash_transparent crate, src/builder.rs`; `zcash_script crate, src/opcode/mod.rs`).

2. *DER encoding and the sighash-type byte.* On the wire a signature is the DER encoding of (r, s) —an ASN.1 SEQUENCE of two minimally encoded INTEGERS—followed by one trailing byte, the *sighash type*, selecting which parts of the transaction the sighash commits to (SIGHASH_ALL for every signature the transaction builder emits; a signer working from a partially created transaction, `src/pczt/signer.rs`, honours whichever valid type the input records). The signer appends the byte (`zcash_transparent crate, src/builder.rs, construct_script_sig`); the verifier splits it off and enforces strict DER unconditionally, the BIP-66 rule, so a malformed encoding fails script evaluation (`zcash_script crate, src/signature.rs, Decoded::from_bytes`). The public key travels as its 33-byte compressed encoding, the bytes that `hash160` commits to (§3.9).
3. *Low- s normalisation.* If (r, s) verifies then so does $(r, n - s)$, so a third party can re-encode a valid signature and change a pre-ZIP-244 transaction id; the normalisation takes the representative $s \leq (n - 1)/2$ (BIP-62), which the deployed signer always emits (`secp256k1 crate, sign_ecdsa`, whose `libsecp256k1` backend documents the created signature as always in lower- s form; called from the `zcash_transparent crate, src/builder.rs` and `src/pczt/signer.rs`) and which script evaluation checks only under the `LowS` flag, one the consensus check does not set (`zcash_script crate, src/external/pubkey.rs, check_low_s`; `zebra-script crate, src/lib.rs`).

Construction 8.35 (BIP-340 Schnorr over `secp256k1`, schematically). Write $H_\tau(m) = \text{SHA256}(\text{SHA256}(\tau) \parallel \text{SHA256}(\tau) \parallel m)$ for the *tagged hash* with ASCII tag τ —domain separation in the sense of Definition 3.16, the doubled tag digest making the prefix a whole 64-byte SHA-256 block—and read digests as integers modulo n .

- *Keys.* Sample $d \xleftarrow{\$} [1, n - 1]$ and set $X = [d]G$. The public key is the 32-byte coordinate $x(X)$ alone (*x-only*): the verifier reconstructs X as the point with that abscissa and *even* y , and the signer replaces d by $n - d$ when $y(X)$ is odd, keeping the pair consistent.
- *Sign(d, m):* derive $k = H_{\text{BIP0340/nonce}}(t \parallel x(X) \parallel m) \bmod n$ with $t = d \oplus H_{\text{BIP0340/aux}}(a)$ for auxiliary randomness a ; set $R = [k]G$, negating k when $y(R)$ is odd; compute $e = H_{\text{BIP0340/challenge}}(x(R) \parallel x(X) \parallel m) \bmod n$ and $s = k + ed \bmod n$; output the 64 bytes $(x(R), s)$.
- *Verify($x(X), m, (r, s)$):* lift $x(X)$ to the even- y point X , recompute e , set $R' = [s]G - [e]X$, and accept iff R' is not the identity, $y(R')$ is even, and $x(R') = r$.

Remark 8.36 (BIP-340 against Construction 8.12; the ZSA consumer). The algebra is the Schnorr signature of Construction 8.12 in its (R, s) form with key-prefixing (Remark 8.14), under $\mathbb{G} = \langle G \rangle$ of order n and $H = H_{\text{BIP0340/challenge}}$; the unforgeability proof of §8.6 carries over with that tagged hash modelled as the random oracle. What differs is convention: points travel by abscissa alone with parity fixed by the even- y rule, saving a byte per key and the parity of R per signature and letting the same 32 bytes serve as key encoding and as hash input; the challenge hash is domain-separated by tag; and the nonce is deterministic with optional auxiliary randomness, as in Construction 8.21. The in-series consumer is the issuance authorisation signature of Zcash Shielded Assets: ZIP 227 (*specified*, in a draft ZIP) instantiates `IssueAuthSig` as BIP-340 over `secp256k1`, prefixes both the validating-key encoding and the signature with a scheme byte `0x00`, and signs the transaction’s sighash. The reference implementation, not yet deployed, is the QED-it fork of the `orchard` crate (commit `cf801a5`), `src/issuance/auth.rs`, whose `ZSASchnorr` signs through the `secp256k1` crate’s BIP-340 routine with zeroed auxiliary randomness. The issuance protocol itself is the business of the *ZSA Guide*.

9 Interactive proofs, zero knowledge, and SNARKs

The adversary of this section wears two faces, one on each side of a conversation. Facing the verifier stands the *cheating prover*: a party with no witness—no spendable note, no authorisation, no satisfying assignment—who would nevertheless talk the verifier into accepting, and thereby mint value, spend another’s coins, or certify a false computation to a network in which every validator takes the proof’s word for the hidden data. Facing the prover stands the *curious verifier*: a party who follows the protocol, or deviates from it arbitrarily, in order to squeeze from the interaction more than the single bit it is owed—to distinguish which witness the prover holds, and so unmask the spender, link the transactions, or read the amounts a shielded protocol exists to hide. The two threats pull in opposite directions: defeating the first means the transcript must pin the prover down; defeating the second means the transcript must reveal nothing. That both can be defeated at once—and, more remarkably still, by a proof *shorter than the statement it certifies*—is the subject of this section, and the reason a private transaction can be checked by millions of verifiers who learn nothing from it but its validity.

9.1 Three questions

Three questions organise everything that follows.

Can a prover convince a sceptical verifier that a statement is true? Not by assertion—the verifier trusts nothing—but by a protocol at whose end the verifier is justified in accepting, because no cheating strategy could have survived it. The answer is the theory of *interactive proofs* and *arguments*, refined into *proofs of knowledge*: protocols after which the verifier is entitled to conclude not merely that a witness exists but that the prover *possesses* one.

Can the prover convince without revealing why the statement is true? The witness—the factorisation, the colouring, the discrete logarithm, the transcript of the computation—must stay hidden; the verifier must finish the protocol knowing the statement holds and nothing else. The answer is *zero knowledge*, made precise by the simulation paradigm.

Can the proof be small—dwarfed by the statement it certifies—and cheap to check? A blockchain verifier cannot afford to re-execute every computation it accepts, nor even to read a proof as long as the computation’s description. The answer is the *SNARK*: the succinct non-interactive argument of knowledge.

The development answers the questions in order and culminates in the architecture underlying the Halo 2 proving system deployed in Orchard: a *polynomial interactive oracle proof*, compiled into an argument by a *polynomial commitment scheme* (§4.9) and made non-interactive by the *Fiat–Shamir transform*. Every earlier section of this volume feeds in: the games and reductions of §1, the discrete logarithm of §2, the random oracle of §3, the commitments of §4, and the Schnorr protocol of §8.4, which will turn out to have been this section’s canonical example all along.

Conventions are those of §1 throughout: λ is the security parameter, adversaries are PPT (non-uniform where it matters), and negl denotes a negligible function. Proof systems introduce one re-indexing: the objects of study are indexed by an *instance* x rather than by 1^λ , and asymptotic statements are measured in the instance length $|x|$ —for the instances this volume cares about, $|x|$ and λ are polynomially related, so the two scales agree up to the polynomial slop the definitions already absorb.

9.2 Languages, relations, and witnesses

What is a “statement”, and what does it mean to hold the “why” of one? The vocabulary is that of languages and relations over bit strings.

Definition 9.1 (Language, relation, witness). Fix the alphabet $\{0, 1\}$. A *language* is a set $L \subseteq \{0, 1\}^*$. A *binary relation* is a set $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ of pairs (x, w) , where x is called the *instance* (or

statement) and w a witness for x . The relation induces the language

$$L_R := \{x : \exists w \text{ with } (x, w) \in R\},$$

and for each instance x the (possibly empty) witness set is $R(x) := \{w : (x, w) \in R\}$. The relation R is an *NP-relation* if it is

1. *polynomially balanced*: there is a polynomial p with $|w| \leq p(|x|)$ for every $(x, w) \in R$; and
2. *polynomial-time decidable*: a deterministic algorithm decides, in time polynomial in $|x|$, whether a given pair (x, w) belongs to R .

The class NP is exactly the class of languages induced this way: a language L lies in NP if and only if $L = L_R$ for some NP-relation R . This certificate characterisation of NP we take from complexity theory without proof; it is the bridge over which every “statement with a short checkable reason” enters the theory. Membership in L_R is existentially quantified—*some* witness exists—while holding an element of $R(x)$ is possession of the reason itself. The gap between the two quantifiers is where this section lives.

Example 9.2 (Three running relations). Three relations recur through the section, in increasing order of expressive power.

1. *Discrete logarithm*. In the setting of §8.4—a cyclic group $\mathbb{G} = \langle G \rangle$ of prime order q , written additively—the relation is

$$R_{\text{dl}} = \{(X, x) \in \mathbb{G} \times \mathbb{F}_q : X = [x]G\}.$$

Because $x \mapsto [x]G$ is a bijection $\mathbb{F}_q \rightarrow \mathbb{G}$, every instance $X \in \mathbb{G}$ has exactly one witness: the induced language is *trivial*—all of $\mathbb{G}$ —and membership is vacuous. What is hard, under the DLP assumption of §2.1, is to *produce* the witness. This is the prototype of why proving membership differs from proving knowledge.

2. *Graph three-colouring*. With instances encoding finite graphs $G = (V, E)$,

$$R_{\text{3col}} = \{(G, c) : c : V \rightarrow \{1, 2, 3\} \text{ and } c(u) \neq c(v) \text{ for all } \{u, v\} \in E\},$$

the relation of *proper* three-colourings. The induced language L_{3col} is NP-complete—every NP language reduces to it in polynomial time, a notion and fact we take from complexity theory without proof—so a zero-knowledge protocol for it yields one for *every* NP language by reduction (§9.8).

3. *Circuit satisfiability*. With instances encoding circuits,

$$R_{\text{sat}} = \{(C, w) : C \text{ a Boolean (or arithmetic) circuit and } C(w) = 1\}.$$

This is the universal target of practical proof systems: a computation is first *arithmetised* into the satisfiability of an arithmetic circuit over a finite field $\mathbb{F}$, and the prover proves knowledge of a satisfying assignment (§9.8).

The discrete-logarithm example already shows that “ $x \in L$ ” is too weak a target for cryptography: every group element is some multiple of G , so a protocol establishing mere membership establishes nothing. The content of the Schnorr protocol is that the prover *knows* the scalar—and formalising “knows” for an algorithm, an object with no mental states, is the central conceptual move of this section (§9.4).

9.3 Interactive protocols, proofs, and arguments

Definition 9.3 (Interactive protocol). An *interactive protocol* is a pair (P, V) of algorithms, the *prover* and the *verifier*. Both receive a common input x ; the prover additionally receives a private input w (a witness, or an arbitrary auxiliary string); each party holds its own private random tape. The parties alternate sending messages $a_1, a_2, \dots$ over a number of rounds polynomial in $|x|$; after the last message the verifier outputs a single bit, written

$$\langle P(w), V \rangle(x) \in \{0, 1\},$$

with 1 for *accept*—a random variable over both parties’ coins. The *transcript* of an execution is the full sequence of exchanged messages (together with x , and with the verifier’s coins when relevant). The protocol is *public-coin* if every message the verifier sends is a fresh uniformly random string that it also reveals—the verifier’s messages *are* its coins—and its final decision is a deterministic function of the transcript. The verifier is required to run in probabilistic polynomial time, always.

The last clause is the asymmetry on which everything turns. The verifier is the party being *protected*, so its power is fixed at PPT once and for all: a guarantee that only holds against verifiers with super-polynomial resources would protect nobody. The prover is the party whose power we *vary*, and that variation is exactly the distinction between proofs and arguments.

Definition 9.4 (Interactive proof and interactive argument). Let $L \subseteq \{0, 1\}^*$ be a language, and let $c, s : \mathbb{N} \rightarrow [0, 1]$. The protocol (P, V) is an *interactive proof system* for L with *completeness error* c and *soundness error* s if:

1. *Completeness*. For every $x \in L$ there is a prover input w with

$$\Pr[\langle P(w), V \rangle(x) = 1] \geq 1 - c(|x|).$$

2. *Soundness*. For every $x \notin L$ and every prover strategy P^* —of *unbounded* computational power, with arbitrary auxiliary input z —

$$\Pr[\langle P^*(z), V \rangle(x) = 1] \leq s(|x|).$$

The soundness clause is a game in the sense of §1.4: the honest verifier is the challenger, the prover strategy is the adversary, and the win condition is an accept on a false instance. The two thresholds must be bounded apart,

$$c(|x|) + s(|x|) \leq 1 - \frac{1}{\text{poly}(|x|)},$$

and in the protocols of this volume both errors are negligible (often $c = 0$). The class of languages admitting such proof systems—with the honest prover unbounded—is IP. Every protocol this volume constructs is for a language $L = L_R$ induced by an NP-relation R , with the honest prover required to run in polynomial time given a witness $w \in R(x)$ as its private input; the unbounded honest prover matters only for the complexity-theoretic statements below.

An *interactive argument system* (or *computationally sound proof*) relaxes only the soundness quantifier: the guarantee holds against every PPT cheating prover P^* , that is, for $x \notin L$ and PPT P^* with arbitrary auxiliary input z , $\Pr[\langle P^*(z), V \rangle(x) = 1] \leq s(|x|)$, where s is required to be negligible. Soundness of an argument typically rests on a computational hardness assumption.

Remark 9.5 (Proofs versus arguments: neither dominates). A proof resists an unbounded cheating prover: its soundness is information-theoretic and unconditional. An argument resists only efficient adversaries, and an unbounded prover can typically break one outright—by breaking the binding of a commitment the protocol relies on (Theorem 4.8 showed perfectly hiding commitments have this exposure), or simply by brute-forcing a discrete logarithm. In exchange, arguments achieve properties

provably impossible for proofs, chief among them *succinctness*: a sound interactive proof for an NP-complete language whose communication is much smaller than the witness would let an unbounded prover compress NP certificates below what complexity theory is believed to allow, collapsing classes believed distinct, whereas succinct *arguments* exist under standard assumptions (§9.8). For cryptographic applications—all of the SNARK literature included—the prover is a real machine with real resource bounds, and the argument notion is exactly right: bounded-prover soundness is the price of succinctness, and its gateway.

Why interaction, and why randomness? Strip both away and see what is left. With a deterministic verifier and a single prover message, the system proves exactly the class NP: the one message is a certificate, the verifier’s check is the relation’s decision procedure—and there is no hope of zero knowledge, since the certificate is, in spirit, the witness itself, and the verifier could replay the prover’s analysis at leisure. Verifier randomness changes the game: a cheating prover must commit to its messages *before* knowing the challenge, so a random spot-check catches what a predictable one never would. Interaction lets the verifier probe adaptively, round after round. Together they buy real power: the equality $IP = PSPACE$ —interactive proofs capture exactly the languages decidable with polynomial memory and unrestricted running time, a class believed far larger than NP—is a landmark of complexity theory (Lund, Fortnow, Karloff, and Nisan; Shamir) that we cite without proof. For this section the sharper point is qualitative: interaction and randomness are what make zero knowledge and knowledge extraction possible at all. A single static certificate can be neither hidden nor rewound.

Proposition 9.6 (Soundness amplification by repetition). *Let (P, V) be an interactive proof (or argument) for L with completeness error c and soundness error s , and let $k = k(|x|)$ be polynomially bounded. The protocol that runs k independent instances and accepts only if all k accept has completeness error at most $k \cdot c$ and soundness error at most s^k . Sequential repetition achieves this always; parallel repetition achieves it for proofs and, with care, for many arguments. In particular, if $s \leq 1 - \delta$ for a noticeable function δ , then choosing k with $\delta(|x|) \cdot k(|x|) = \omega(\log|x|)$ —always possible with k polynomial—drives the soundness error below every inverse polynomial, so a base protocol needs only a noticeable soundness gap.*

Proof. For completeness, the honest prover fails at least one of the k runs with probability at most $k \cdot c$ by the union bound (*Math Guide*, §“The union bound and a birthday calculation”). For soundness under sequential repetition, fix $x \notin L$ and any cheating strategy; conditioned on the transcripts of the first $i - 1$ runs, the i -th run is an execution against a fresh independently-random verifier, so it accepts with probability at most s whatever state the prover carries forward; multiplying the conditional bounds gives s^k . (For parallel repetition of proofs the same bound holds by a more careful argument, which we omit; for arguments parallel repetition can fail to amplify in general, and we use only the sequential form.) Finally $s^k \leq (1 - \delta)^k \leq e^{-\delta k}$, which is negligible once $\delta k = \omega(\log|x|)$. $\square$

9.4 Knowledge soundness and extractors

Soundness, as defined, is silent exactly where cryptography speaks. For the relation R_{dl} every instance $X \in \mathbb{G}$ lies in the induced language, so the soundness clause of Definition 9.4—quantified over $x \notin L$ —holds *vacuously*: a verifier that always accepts is “sound” for the trivial language. The assurance actually wanted is that a convincing prover *possesses* the discrete logarithm. But possession is not a property an algorithm wears on its sleeve; the definition reads it operationally. A prover knows a witness if an efficient procedure, granted full control over the prover—the ability to run it, feed it messages, and *rewind* it to an earlier state to try different continuations—can compute the witness from it. The procedure is called the *extractor*, and what it extracts is the meaning of “knows”.

Definition 9.7 (Knowledge soundness; proof of knowledge). *Let (P, V) be an interactive protocol*

and R an NP-relation. The protocol is a *proof of knowledge* for R with *knowledge error* $\kappa : \mathbb{N} \rightarrow [0, 1]$ if there exist a probabilistic oracle algorithm $\mathcal{E}$ (the *extractor*) and a polynomial q such that for every prover strategy P^* —possibly cheating, possibly computationally unbounded—and every instance x , writing

$$\varepsilon(x) := \Pr[\langle P^*, V \rangle(x) = 1]$$

for the probability that P^* convinces the honest verifier, the following holds whenever $\varepsilon(x) > \kappa(x)$: the extractor $\mathcal{E}$, given x and oracle access to P^* *with the ability to rewind*—to run P^* from any point with the same coins, feeding verifier messages of $\mathcal{E}$ ’s choosing—outputs a witness $w \in R(x)$ within expected running time

$$\frac{q(|x|)}{\varepsilon(x) - \kappa(x)}.$$

If the guarantee is required only for *PPT* prover strategies P^* , the protocol is an *argument of knowledge*.

Three consequences can be read straight off the definition. The extraction cost may blow up as ε approaches κ : the bound $q(|x|)/(\varepsilon - \kappa)$ is useful precisely when the prover’s success is noticeably above the knowledge error. A prover succeeding with probability at most κ need yield nothing at all— κ is the “free” success rate, typically the probability of blind guessing, below which conviction carries no evidence of knowledge. And any prover noticeably above κ surrenders the witness: whatever code and coins produce conviction can be mined, efficiently, for the very object the conviction asserts. Knowledge is thereby defined as *efficient computability relative to the prover’s own code and coins*—an algorithm knows what can be efficiently computed from it. Note the expected-time clause: extractors are the standing reason this volume’s adversary class admits expected polynomial time (Remark 1.4).

Remark 9.8 (The rewinding justification). No live counterparty could rewind the prover, and none ever does: the extractor is a *thought experiment*, never executed during an honest run of the protocol. It exists to give the security claim its meaning—and, in security proofs, its teeth. In a reduction the extractor *is* the reduction: the cheating prover is a subroutine the reduction runs through its input/output interface alone (Definition 1.20), and black-box access already includes choosing the subroutine’s random tape and restarting it from the top with the same coins—so replaying it under different challenges is not science fiction but the ordinary right of a machine over its own subroutines, no reading of its code required. The same device runs in both directions across this section: the *simulator* of §9.5 rewinds a verifier to fake transcripts without a witness, and the extractor rewinds a prover to mine real witnesses from conviction. Rewinding is the single most important technical device in the theory.

Proposition 9.9 (Knowledge soundness implies soundness). *A proof of knowledge for R with knowledge error κ is an interactive proof for L_R with soundness error $s(x) \leq \kappa(x)$. The converse fails: a protocol can be sound without being a proof of knowledge.*

Proof. Fix $x \notin L_R$, so that $R(x) = \emptyset$. If some prover strategy P^* achieved $\varepsilon(x) > \kappa(x)$, the extractor run on P^* would halt in finite expected time and output some $w \in R(x)$ —impossible, as the witness set is empty. Hence $\varepsilon(x) \leq \kappa(x)$ for every P^* , which is the soundness bound. For the failure of the converse: soundness constrains only false instances, so a protocol for a trivial language—such as R_{dl} ’s, where no false instances exist—is automatically sound however it behaves, while nothing forces a witness to be extractable from it. One may be convinced of a true statement by a prover who does not know why it is true; knowledge soundness is strictly stronger, and it—not mere soundness—is the notion cryptographic applications actually rely on (§9.8). $\square$

9.5 The simulation paradigm and zero knowledge

What should “the verifier learned nothing” mean? The definitional move—often called the most influential idea in modern cryptography—is the *simulation paradigm*: the verifier learned nothing beyond the truth of the statement if it could have produced, entirely on its own and without ever talking to the prover, a transcript indistinguishable from the real one. Whatever a party can manufacture for itself it did not learn from the conversation; if the whole conversation is reproducible without the witness, the conversation taught nothing about the witness. The three grades of indistinguishability—perfect, statistical, computational—carry over verbatim from §1.2 (Definitions 1.6 and 1.9, with the unbounded-distinguisher bound of Proposition 1.7 and the hierarchy of Proposition 1.10) under one re-indexing: ensembles are indexed by instances x (and auxiliary inputs) rather than by the security parameter alone, and negligibility is measured in $|x|$.

Definition 9.10 (Zero knowledge). Let (P, V) be an interactive proof or argument for $L = L_R$. For a verifier strategy V^* , an instance x , a witness $w \in R(x)$, and an auxiliary input z , the *view*

$$\text{View}_{V^*}(P(w), V^*(z))(x)$$

is the random variable consisting of x , the random tape of V^* , and every message V^* receives—everything the verifier sees. The protocol is *zero knowledge* if there exists a PPT simulator Sim such that for every $x \in L$, every witness $w \in R(x)$, and every auxiliary input z , the ensembles

$$\{\text{Sim}(x, z)\} \quad \text{and} \quad \{\text{View}_{V^*}(P(w), V^*(z))(x)\}$$

are indistinguishable; the grade of indistinguishability—perfect, statistical, computational—gives *perfect* (PZK), *statistical* (SZK), or *computational* (CZK) zero knowledge. Two regimes for the quantifier over V^* :

1. *Honest-verifier zero knowledge* (HVZK): the guarantee is required only for $V^* = V$, the verifier that follows the protocol, with z empty; the simulator must reproduce the distribution of honest-execution transcripts.
2. *Malicious-verifier (auxiliary-input) zero knowledge*: the guarantee holds for *every* PPT verifier strategy V^* , deviating arbitrarily, with arbitrary auxiliary input z . Here Sim may use V^* as a subroutine *and rewind it*, and is permitted expected polynomial running time (Remark 1.4).

Why does the simulator’s existence suffice? The simulator receives x —and z —but *not* the witness w . If its output is nevertheless indistinguishable from the real view, then the real view carries no efficiently extractable information about w beyond the fact $x \in L$: any property of w an efficient verifier could compute from the interaction, it could compute equally well from the simulator’s output, which was manufactured without w —otherwise the computation would itself be a distinguisher. The simulator wields a power the prover lacks: it may rewind V^* , retrying until the verifier’s challenges happen to align with a transcript preparable without the witness. That power is legitimate for exactly the reason rewinding always is (Remark 9.8): the simulator is the analyst’s tool, holding the verifier as a subroutine, not a participant in any real execution.

The auxiliary input z models the prior knowledge a malicious verifier may bring to the conversation—the history of earlier protocol runs, partial information about the witness, anything. Quantifying over all z is what makes zero knowledge *compose*: a protocol that is zero knowledge for every auxiliary input remains zero knowledge when run as a subroutine of a larger system, whose surrounding state simply becomes the z of the analysis. Practice therefore adopts the auxiliary-input formulation, and so does this volume.

9.6 Sigma-protocols

The general definitions are now in place, and one protocol shape realises them with remarkable economy: three moves, public coins, and algebra doing all the work. The shape is named for the capital Σ its message flow traces on the page.

Definition 9.11 (Sigma-protocol). Let R be an NP-relation. A *Sigma-protocol* for R with *challenge space* C is a three-move public-coin protocol on common input x , the prover holding w with $(x, w) \in R$:

1. *Commitment*. The prover sends a first message a (the *commitment*, or announcement), computed from (x, w) and fresh randomness.
2. *Challenge*. The verifier sends $e \xleftarrow{\$} C$, uniformly random.
3. *Response*. The prover sends z , computed from (x, w, a, e) and its randomness.

The verifier decides by a deterministic predicate $\text{Verify}(x, a, e, z) \in \{0, 1\}$. Three properties are required.

- *Completeness*. An honest prover holding $(x, w) \in R$ convinces the honest verifier with probability 1.
- *Special soundness*. There is a PPT algorithm that, given x and any two accepting transcripts (a, e, z) and (a, e', z') with the *same* commitment a but *distinct* challenges $e \neq e'$, outputs a witness $w \in R(x)$.
- *Special honest-verifier zero knowledge* (sHVZK). There is a PPT simulator that, on input x and any challenge $e \in C$, outputs a pair (a, z) such that $\text{Verify}(x, a, e, z) = 1$ and the triple (a, e, z) is distributed exactly—or statistically, or computationally, close to—a real honest transcript conditioned on the challenge being e . The simulator chooses a *after* fixing e ; reversing the honest order is what makes simulation possible without w .

Two accepting transcripts sharing a commitment but differing in challenge are called a *collision*. Special soundness says a collision is a witness: it is the local, two-transcript reformulation of knowledge extraction, and the engine of the theorem that follows.

Theorem 9.12 (Special soundness yields knowledge soundness). *Let (P, V) be a Sigma-protocol for R with challenge space C of size $N := |C|$. Then (P, V) is a proof of knowledge for R with knowledge error $\kappa = 1/N$.*

Proof. We construct an extractor establishing the slightly weaker knowledge error $2/N$, with expected running time $O(1/(\varepsilon - 2/N))$ prover invocations whenever the prover's success probability satisfies $\varepsilon > 2/N$; the closing paragraph addresses the gap to the optimal $1/N$.

The prover as a matrix. Fix x and a prover strategy P^* ; fold any auxiliary input into its code. The behaviour of P^* is a deterministic function of its random tape ρ : fixing ρ fixes the commitment $a(\rho)$, and then each challenge e determines the response $z(\rho, e)$. Form the Boolean matrix

$$H[\rho, e] := \begin{cases} 1 & \text{if } \text{Verify}(x, a(\rho), e, z(\rho, e)) = 1, \\ 0 & \text{otherwise,} \end{cases}$$

with one row per random tape and one column per challenge; then $\varepsilon = \Pr_{\rho, e}[H[\rho, e] = 1]$, and a collision is two 1-entries in a single row.

The extractor. Step 1. Run P^* with fresh ρ and a uniform challenge e , repeatedly, until an accepting transcript (a, e, z) appears. The number of trials is geometric with success probability ε , so this costs $1/\varepsilon$ invocations in expectation—the geometric waiting time (*Math Guide*, §“Beyond finite spaces: countable additivity, limits, and densities”). *Step 2.* Alternate two kinds of trial: (i) rewind P^* to the moment just after it emitted a —the tape ρ stays fixed—and feed one fresh uniform challenge $e' \neq e$; (ii) make one entirely fresh probe, as in Step 1. If a kind-(i) trial accepts, a collision $(a, e, z), (a, e', z')$ is in hand: go to Step 3. If a kind-(ii) trial accepts, *abandon* the current row and continue Step 2 from the new accepting transcript (a fresh *phase*). *Step 3.* Apply the special-soundness algorithm of Definition 9.11 to the collision and output the witness.

Analysis. Call a row ρ *heavy* if its acceptance density $\delta_\rho := \Pr_e[H[\rho, e] = 1]$ is at least $\varepsilon/2$. The heavy-row lemma—a Markov-style averaging—states that an accepting entry lies in a heavy row with probability at least $1/2$: the total accepting mass contributed by light rows is less than $\varepsilon/2$ (each light row contributes less than $\varepsilon/2$ of its own row measure), so conditioned on acceptance the light rows carry less than half the mass. Every phase opens with a fresh accepting transcript distributed exactly as Step 1’s (the opening transcript is either Step 1’s or a kind-(ii) success, both conditioned draws from the accepting entries), so the lemma applies at every phase, not only the first.

Each alternation of Step 2 contains a kind-(ii) trial, which accepts with probability ε ; the number of alternations in a phase is therefore dominated by a geometric variable with mean $1/\varepsilon$, and a phase costs $O(1/\varepsilon)$ invocations in expectation. This interleaving is not an optimisation but a necessity: a row whose only accepting challenge is the already-used e —any row with $\delta_\rho \leq 1/N$ —contains no collision at all, and an extractor that only rewound would sit in such a row for unbounded expected time. The fresh probes cap every phase.

Now condition on the phase’s row being heavy. A kind-(i) trial draws e' uniformly from $C \setminus \{e\}$ and accepts with probability at least

$$\frac{\delta_\rho N - 1}{N - 1} \geq \delta_\rho - \frac{1}{N} \geq \frac{\varepsilon}{2} - \frac{1}{N},$$

which is positive precisely when $\varepsilon > 2/N$. The phase ends at its first success of either kind, and the chance that this first success is a collision rather than a row-switch is at least

$$\frac{\varepsilon/2 - 1/N}{(\varepsilon/2 - 1/N) + \varepsilon} = \Omega\left(\frac{\varepsilon - 2/N}{\varepsilon}\right).$$

Un-conditioning costs the heavy-row factor $1/2$, so each phase yields a collision with probability $\Omega((\varepsilon - 2/N)/\varepsilon)$, and the number of phases is dominated by a geometric variable with mean $O(\varepsilon/(\varepsilon - 2/N))$. By Wald’s identity—the expected total is the expected number of phases times the expected cost per phase (*Math Guide*, same section)—the extractor’s expected number of prover invocations is

$$O\left(\frac{\varepsilon}{\varepsilon - 2/N}\right) \cdot O\left(\frac{1}{\varepsilon}\right) = O\left(\frac{1}{\varepsilon - 2/N}\right),$$

of the form $q(|x|)/(\varepsilon - 2/N)$ demanded by Definition 9.7 with $\kappa = 2/N$.

Closing the factor of two. The threshold $\delta_\rho \geq \varepsilon/2$ defining heaviness is an artefact of this exposition, and it is exactly what costs the extra $1/N$ in the knowledge error. Sharper accounting removes it: Damgård’s refined heavy-row analysis in his Sigma-protocol lecture notes, and, in full generality, the k -special-soundness extractor of Attema, Cramer, and Kohl, which achieves knowledge error exactly $(k - 1)/N$ —here $k = 2$, giving the optimal $\kappa = 1/N$ that matches the soundness error. We cite these rather than reproduce them; every use in this volume tolerates the factor of two. $\square$

Remark 9.13 (Knowledge error equals soundness error, and how to size the challenge space). The knowledge error $1/N$ of Theorem 9.12 is no accident of the analysis: an explicit attack meets it. A prover with no witness guesses the challenge in advance: it picks $e^* \in C$, runs the sHVZK simulator on (x, e^*) to obtain an accepting pair (a, z) —for a protocol such as Schnorr’s the simulator’s algebra works for *every* instance—and sends a . If the verifier’s challenge happens to equal e^* , which occurs

with probability exactly $1/N$, the prepared z is accepted. One challenge is thus always answerable without a witness, so $1/N$ is also the protocol's soundness error, and Theorem 9.12 says the two coincide. One caveat keeps the lower bound honest: under Definition 9.7 an extractor facing this guessing prover ($\varepsilon = 1/N$) is granted expected time $q(|x|)/(1/N - \kappa)$, so for exponential N a claimed κ below $1/N$ is not formally refuted—the definition would hand the extractor a super-polynomial budget, enough to brute-force a witness unaided. The attack does rule out any noticeably smaller κ once the extractor is required to run in strict polynomial time and witnesses are genuinely hard to compute: such an extractor, run on this witness-free prover, would otherwise mine a witness from nothing—for R_{dl} , solving the discrete-logarithm problem itself. To make the error negligible there are two routes: take C exponentially large—for instance $C = \mathbb{F}_q$ for a λ -bit prime q , so $N = q$ —or repeat a small-challenge protocol $\omega(\log \lambda)$ times (Proposition 9.6). Large challenge spaces are why a *single* round of Schnorr already has negligible knowledge error, while a protocol with a polynomially small challenge space needs many repetitions.

Multi-round protocols generalise the collision to a tree. The succinct arguments of §9.8 run many challenge rounds, and their extractors need more than two transcripts.

Definition 9.14 (Tree special soundness). Let (P, V) be a $(2\mu + 1)$ -move public-coin protocol for R in which the prover speaks first and the verifier issues μ uniform challenges from a space C , and let $k_1, \dots, k_\mu$ be positive integers. A $(k_1, \dots, k_\mu)$ -tree of accepting transcripts for x is a tree of depth μ in which every node at level i has k_i children labelled by pairwise distinct challenges for round i , every root-to-leaf path carries the prover messages of one accepting transcript, and transcripts passing through a common node share their prefix up to that node. The protocol is $(k_1, \dots, k_\mu)$ -special-sound if a PPT algorithm computes a witness $w \in R(x)$ from any such tree. A Sigma-protocol is the case $\mu = 1, k_1 = 2$.

Extraction generalises phase by phase: under the hypotheses of a tree-extraction theorem—the Attema–Cramer–Kohl analysis cited in the proof of Theorem 9.12 is the general form—rewinding the prover round by round assembles the required tree of $\prod_i k_i$ accepting transcripts: the protocol is a proof of knowledge with knowledge error growing from $1/|C|$ to the order of $(\sum_i (k_i - 1))/|C|$, and with extraction cost polynomial provided the tree size $\prod_i k_i$ is. Both degradations are benign in the regime the compiled protocols of this section occupy: C exponentially large, μ logarithmically bounded, and the k_i constant, so the error stays negligible and $\prod_i k_i$ polynomial. This is the notion under which the inner-product argument's extractor operates (Proposition 4.39): each of the $\log_2 d$ folding rounds contributes one level of branching to the tree, three accepting children at every node. Each round's accepting relation is linear in three unknown parent terms with coefficients that are powers of the challenge, so three suitably distinct challenges make that system invertible. The *Halo 2 Guide* carries out that extraction for the deployed protocol, whose IPA normalisation involves each round's challenge through the coefficients $u^{-1}, 1, u$: the extractor accordingly needs pairwise distinct *nonzero* challenges at every node—a negligible-fraction exclusion absorbed into the knowledge error—and not the pairwise distinct squares ($u \neq \pm u'$) that the alternative $u^{-2}, 1, u^2$ normalisation would demand.

Proposition 9.15 (sHVZK implies HVZK). A Sigma-protocol satisfying special honest-verifier zero knowledge is honest-verifier zero knowledge in the sense of Definition 9.10, with the same grade (perfect, statistical, or computational).

Proof. The honest view is a triple (a, e, z) in which e is uniform on C . The HVZK simulator draws $e \xleftarrow{\$} C$ itself, invokes the sHVZK simulator on (x, e) to obtain (a, z) , and outputs (a, e, z) . For each fixed e , the simulated pair (a, z) is distributed as a real run conditioned on that challenge—exactly, or up to the stated grade of closeness—and the simulator's e has the same uniform marginal as the honest

verifier's; averaging the conditional distributions over uniform e therefore reproduces the honest view, with any statistical or computational slack preserved under the averaging. $\square$

Example 9.16 (Schnorr: the canonical Sigma-protocol). The Schnorr identification protocol of §8.4 is a Sigma-protocol for the discrete-logarithm relation R_{dl} of Example 9.2, and the properties proved there, self-contained, are exactly the clauses of Definition 9.11. The commitment is $R = [r]G$; the challenge is a uniform $c \in \mathbb{F}_q$; the response is $s = r + cx$; the predicate is $[s]G = R + [c]X$. Perfect completeness is Proposition 8.7; 2-special soundness is Theorem 8.8, whose collision algebra yields the witness $x = (s - s')(c - c')^{-1}$; special HVZK is Theorem 8.10, whose simulator samples s and solves $R = [s]G - [c]X$ —choosing the commitment after the challenge, as the definition prescribes. The challenge space is $\mathbb{F}_q$, so $N = q$ and the knowledge error $1/q$ is negligible in a single round (Remark 9.13). By Theorem 9.12 and Proposition 9.15, Schnorr's protocol is a perfect-HVZK proof of knowledge of a discrete logarithm.

Remark 9.17 (From honest to malicious verifiers: three remedies). Sigma-protocols are honest-verifier zero knowledge only. A malicious V^* may choose e as a function of the commitment a , and the sHVZK simulator—which fixes e before producing a —does not obviously cope: nothing lets it predict which challenge the verifier's function will return for a commitment it has yet to manufacture. Three standard remedies close the gap. (1) *Guess and rewind*, for small challenge spaces: the simulator guesses $e^* \xleftarrow{\$} C$, runs the sHVZK simulator on (x, e^*) , and feeds the resulting a to V^* ; if the verifier's challenge equals e^* , the round is simulated, and otherwise the simulator rewinds V^* and guesses afresh. The honest prover fixes a before any challenge exists, so the simulated a carries (almost) no information about the guess and each attempt succeeds with probability close to $1/N$: for polynomially large N the expected $O(N)$ attempts stay polynomial, and sequential repetition of the small-challenge protocol—simulated round by round, the auxiliary input carrying the history—is full malicious-verifier zero knowledge, at the price of many rounds. For exponentially large N the guess never lands, and repetition does not generically upgrade honest-verifier to malicious-verifier zero knowledge. (2) *Challenge commitment*: the verifier commits to e before seeing a (a coin-tossing step using the commitments of §4), so e cannot depend on a ; under the additional conditions such a compiler imposes for simulation, the simulator rewinds past the opening. (3) *The Fiat–Shamir transform*: remove the verifier's choice altogether by deriving e from a hash of the transcript—under the hypotheses of Theorem 9.20, in the programmable random oracle model, the resulting *non-interactive* protocol is zero knowledge against *all* verifiers, since there is no live challenge left to skew. For the SNARK pipeline route (3) is the one that matters, and honest-verifier zero knowledge is exactly the property the transform requires. It is next.

9.7 The Fiat–Shamir transform: from interactive to non-interactive

In a public-coin protocol the verifier contributes nothing but fresh randomness: its messages are coin tosses, its decision a deterministic function of the transcript (Definition 9.3). Every such message can therefore be replaced by a value that *anyone* can compute—a hash of everything sent so far—and the interaction collapses into a single proof string, verifiable by recomputing the hashes. The prover no longer talks to a live, possibly malicious verifier but to a fixed public function, which is why honest-verifier zero knowledge upgrades to full non-interactive zero knowledge. The cost is that the hash must be idealised: the analysis lives in the random oracle model, with $H : \{0, 1\}^* \rightarrow C$ a random oracle whose range is the challenge space (Definition 8.15), and it exploits both powers established in Proposition 3.14: *observability*—the reduction sees every query the adversary makes—and *programmability*—the reduction may fix the oracle's output at a freshly queried point to any value, provided answers stay consistent and uniformly distributed.

Definition 9.18 (The Fiat–Shamir transform). Let (P, V) be a public-coin protocol for R with chal-

challenge space C , and let H be a random oracle with range C . The *Fiat–Shamir transform* $\text{FS}[(P, V), H]$ is the non-interactive system defined, in the Sigma case, by:

- *Prover*. Compute the commitment a as in (P, V) ; set $e := H(x, a)$; compute the response z ; output the proof $\pi := (a, z)$ (equivalently (a, e, z)).
- *Verifier*. Given x and $\pi = (a, z)$, recompute $e := H(x, a)$ and accept iff $\text{Verify}(x, a, e, z) = 1$.

A multi-round public-coin protocol applies the rule per round: the i -th challenge is

$$e_i := H(x, a_1, e_1, \dots, a_i),$$

each challenge hashing the *entire* transcript prefix. To bind a message m to the proof—turning it into a signature—one includes m in the hash: $e := H(x, a, m)$.

Remark 9.19 (The pitfall: weak Fiat–Shamir). The definition’s insistence that each challenge hash *all* prior messages, the instance x included, is load-bearing, and omitting any part of it has broken deployed systems. Hashing $e := H(a)$ without x —“weak Fiat–Shamir”—lets an attacker fix (a, e, z) first and choose the instance *afterwards*, searching for an x that the frozen challenge happens to validate: the interactive protocol’s soundness quantified over provers who commit before the challenge, and an unhashed x re-opens exactly that order of quantifiers, yielding proofs of false statements. In multi-round protocols, hashing only the latest message rather than the running prefix breaks soundness the same way, one round at a time. The maxim: *the challenge must be a hash of everything the verifier would have seen up to the moment it speaks*.

Theorem 9.20 (Fiat–Shamir security in the ROM). *Let (P, V) be a Sigma-protocol for R with challenge space C of size N , special soundness, and special HVZK; assume further that, conditioned on (x, e) , the first message output by the sHVZK simulator has min-entropy $\omega(\log \lambda)$ (min-entropy H_∞ : Definition 6.23). In the random oracle model, $\Pi = \text{FS}[(P, V), H]$ is a non-interactive argument for R that is:*

1. perfectly complete, inheriting the protocol’s completeness;
2. knowledge sound: *from any PPT prover P^* that makes at most Q random-oracle queries and outputs an accepting proof for x with probability ϵ , an extractor obtains a witness $w \in R(x)$ with probability at least $\epsilon(\epsilon/Q - 1/N)$ up to constant factors—non-negligible whenever ϵ is non-negligible and N is super-polynomial; and*
3. non-interactive zero knowledge: *a PPT simulator that controls the random oracle produces, without the witness, proofs indistinguishable from real ones—for all verifiers, including malicious ones.*

Proof. Completeness. The honest prover computes $e = H(x, a)$ and an honest response; the verifier recomputes the very same e from the same inputs and runs the same predicate, so the protocol’s probability-1 completeness transfers verbatim.

Zero knowledge, by programming. The simulator, on input x , draws $e \xleftarrow{\$} C$, runs the sHVZK simulator on (x, e) to obtain an accepting pair (a, z) , and then *programs* the oracle:

$$H(x, a) := e.$$

Programming fails only if (x, a) was queried before this moment; by hypothesis the simulated first message has, conditioned on (x, e) , min-entropy $\omega(\log \lambda)$, so a given prior query collides with a with probability $2^{-H_\infty(a)}$, and a union bound over the adversary’s prior queries makes the failure probability negligible. Conditioned on no failure, the programmed answer is a fresh uniform value—exactly

what an unprogrammed oracle would have returned—so the oracle’s joint distribution is untouched, and by sHVZK the output (a, z) is distributed as a real proof. The programming is invisible to every verifier, and there is no live challenge for a malicious verifier to skew: its malice is moot, and honest-verifier simulation delivers full non-interactive zero knowledge.

Knowledge soundness, by forking. Let P^* output an accepting proof (a, z) for x with probability ε . With overwhelming probability a successful P^* actually queried H at (x, a) : otherwise the challenge $e = H(x, a)$ used by the verifier is a uniform value the prover never saw, and the proof verifies with probability at most $1/N$. Say the critical query is the j -th of the at most Q queries. The extractor runs P^* once, recording the transcript of oracle answers and the index j of the critical query; it then *rewinds* P^* to just before the j -th query and re-runs it with the same coins, answering the first $j - 1$ queries identically—so the commitment a is unchanged—and the j -th and subsequent queries with fresh uniform values, the j -th being $e' \xleftarrow{\$} C$. This is precisely the experiment of the general forking lemma (Theorem 8.17, with $h = N$): if P^* succeeds with probability ε over Q query slots, then with probability at least $\varepsilon(Q - 1/N)$ both runs succeed at the same index *and* $e \neq e'$. Conditioned on that event, the two runs yield accepting transcripts (a, e, z) and (a, e', z') with a common commitment and distinct challenges—a collision—and the special-soundness algorithm of Definition 9.11 outputs $w \in R(x)$. $\square$

Remark 9.21 (Caveats, and the refinements SNARKs need). The random oracle is an idealisation: a real hash function is fixed, public, and not random, and there exist contrived protocols secure in the ROM yet insecure under *every* concrete instantiation. The transform is nonetheless trusted in practice, with the working stance of Remark 1.29, and it underlies essentially every deployed non-interactive proof and signature—Fiat–Shamir-compiled SNARKs included. Two refinements matter for the compiled arguments of §9.8. First, Theorem 9.20 covers the one-challenge Sigma case only, and does not by itself prove security for a multi-round protocol such as Halo 2’s: there the extractor must fork the transcript at *each* round, its success probability degrading with the round count and with the protocol’s soundness structure, and special soundness must generalise to tree special soundness (Definition 9.14)—a separate multi-round theorem, whose hypotheses must match the protocol at hand. Second, when the underlying interactive protocol is itself only computationally sound—an argument—the ROM extraction *layers on top of* the computational soundness reduction rather than replacing it. There is also a gap between proofs that merely observe the oracle and proofs that program it (the non-programmable ROM is strictly weaker as a proof technique); deployed systems adopt the programmable model—the zero-knowledge clause above, which programs the oracle outright, already shows why—and an analysis must state which programmable-oracle (and, where used, algebraic) model it works in.

Example 9.22 (Schnorr signatures, recovered). Applying Definition 9.18 to Schnorr’s Sigma-protocol (Example 9.16) with a message m folded into the challenge hash yields, symbol for symbol, the Schnorr signature scheme of Construction 8.12; and its EUF-CMA security in the ROM (Theorem 8.18) is established by exactly the forking argument in the proof of Theorem 9.20, augmented with a signing-oracle simulation. The signatures section proved the instance; this section supplies the general statement it instantiates. The slogan: *proof of knowledge + Fiat–Shamir = signature*.

9.8 SNARKs: succinct non-interactive arguments of knowledge

The Sigma-protocol paradigm reaches all of NP: a commit–challenge–reveal proof of knowledge for the NP-complete relation $R_{3\text{col}}$ of Example 9.2—zero knowledge under one-way functions alone, a result of Goldreich, Micali, and Wigderson taken without proof—extends to every NP language by reduction. The generic reduction is hopelessly inefficient, however, its proofs polynomially *longer* than the computation they certify, so practical systems abandon it and arithmetise the computation directly.

A blockchain verifier operates under constraints the classical theory never contemplated. It cannot interact—a transaction is broadcast once and verified by parties who will never speak to its author,

years later, from a proof string sitting in a block. It cannot afford to re-execute—every full node checks every proof, so verification must cost less than the computation being certified, by orders of magnitude. And it must resist a prover with every incentive to cheat: an accepting proof *is* money. The object meeting all three constraints at once is the SNARK.

Definition 9.23 (Non-interactive argument). A *non-interactive argument* for an NP-relation R is a triple of PPT algorithms:

- $\text{Setup}(1^\lambda, R) \rightarrow srs$, producing a *structured reference string*—the public parameters. In the random oracle model all three algorithms additionally access one globally sampled oracle; the oracle is not part of the srs —a PPT setup cannot output an infinite object—though public parameters may be derived through it. A setup holding no secret trapdoor is *transparent*: no secret is ever held, so none must be trusted to have been discarded (Remark 4.3).
- $\text{Prove}(srs, x, w) \rightarrow \pi$, for $(x, w) \in R$.
- $\text{Verify}(srs, x, \pi) \rightarrow \{0, 1\}$.

The system is *complete* if honestly generated proofs always verify. A *preprocessing* argument lets Setup depend on the circuit or relation and output, alongside the proving parameters, a short *verification key*, amortising the verifier’s per-statement work across all statements of that shape.

Definition 9.24 (SNARK). A *succinct non-interactive argument of knowledge* (SNARK) for an NP-relation R is a non-interactive argument that is additionally:

1. *Knowledge sound*. For every PPT prover P^* there is a PPT extractor $\mathcal{E}$ —in the ROM, observing P^* ’s oracle queries and rewinding or forking it—such that, on the same setup,

$$\Pr[\text{Verify}(srs, x, \pi) = 1 \wedge (x, \mathcal{E}^{P^*}) \notin R] \leq \text{negl}(\lambda),$$

the probability over the setup and both algorithms’ coins: an accepting proof implies an extractable witness, except with negligible probability.

2. *Succinct*. The proof size $|\pi|$ and the verifier’s running time are *sublinear* in the size of the computation proved—typically $|\pi| = O(\text{poly}(\lambda))$ or $O(\text{poly}(\lambda) \log|C|)$, and verification $O(\text{poly}(\lambda) \log|C|)$ for a circuit C , independent of, or polylogarithmic in, the witness and circuit sizes. (Some of the literature reserves “succinct” for polylogarithmic proofs; the constant-size proofs of pairing-based schemes are the strongest form.)

With zero knowledge—the non-interactive, ROM form of Definition 9.10, as delivered in the Sigma case by Theorem 9.20, clause 3—it is a *zk-SNARK*.

Remark 9.25 (Why these systems are arguments). The “AR” in SNARK is earned by necessity, not by preference. Remark 9.5 showed that a succinct *proof* for an NP-complete language is essentially ruled out: a transcript shorter than the witness cannot information-theoretically pin the witness down, and an unbounded prover could otherwise compress certificates below what complexity theory is believed to allow. Succinct *arguments* escape because a computationally bounded prover cannot *find* a convincing-but-false short proof even though one may exist—succinctness and computational soundness arise together, by necessity. The constructions below realise the necessity at a specific spot: their polynomial commitments are binding only against *efficient* adversaries, under cryptographic assumptions, and an unbounded prover could equivocate on a commitment and defeat extraction. It is at that commitment layer that the information-theoretic core turns into an argument.

What does *knowledge* soundness buy an application? Consider a private transaction whose state-

ment reads: *there exists a note I am authorised to spend, and input and output values balance*. Plain soundness guarantees only that *some* satisfying witness exists—and for such existential statements about a large anonymity set, witnesses exist in abundance: other people’s notes satisfy the existential quantifier perfectly well. Soundness alone would not prevent a prover holding no spendable note from proving the existential statement, should she somehow come by a proof. Knowledge soundness closes the gap: whoever produced the accepting proof *actually holds* a concrete witness—a real spendable note, a real authorisation—that the extractor could recover from her. The application may then act—release funds, grant access—exactly as though the witness had been presented in the clear, while it stays hidden. Soundness protects against false statements; knowledge soundness protects against provers who do not know what they claim. Almost every cryptographic use, and certainly every monetary one, requires the latter (Proposition 9.9).

The pairing-based route

The first SNARKs to reach deployment, and the ones the deployed Sapling protocol still uses, verify with a bilinear pairing.

Definition 9.26 (Bilinear pairing). Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be cyclic groups of prime order r , the first two written additively with generators G and $\hat{G}$. A *bilinear pairing* is a map $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ that is

1. *bilinear*: $e([a]P, [b]Q) = e(P, Q)^{ab}$ for all $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$, and $a, b \in \mathbb{F}_r$;
2. *non-degenerate*: $e(G, \hat{G}) \neq 1$, so that $e(G, \hat{G})$ generates $\mathbb{G}_T$; and
3. *efficiently computable*.

Bilinearity lets a verifier test one multiplication of hidden scalars—whether $c = ab$, given $[a]G$, $[b]\hat{G}$, and $[c]G$ —without learning any of them (Example 2.9); the KZG check of Construction 4.35 is such a test. The groups come from *pairing-friendly* elliptic curves, chosen with a small embedding degree (the quantity of Definition 2.24, there required large) so that $\mathbb{G}_T$ lives in a manageable extension field; the curve of the deployed Sapling protocol is BLS12-381, a Barreto–Lynn–Scott curve of embedding degree twelve (protocol specification § 5.4.9.2). The deployed system treats the pairing as a black box, and so does this volume.

The lineage is short. The BCTV14 family (Ben-Sasson, Chiesa, Tromer, and Virza) is a preprocessing argument in the sense of Definition 9.23: the setup encodes one fixed arithmetic circuit into a structured reference string of group elements, the prover’s work is a handful of multiscalar multiplications over that string, and the proof is a constant number of group elements—eight, in the deployed encoding (protocol specification § 5.4.10.1)—checked by a constant number of pairing equations, whatever the circuit’s size. Groth’s refinement (Groth16) cuts the proof to *three* group elements, $\pi_A, \pi_C \in \mathbb{G}_1$ and $\pi_B \in \mathbb{G}_2$, and verification to a single pairing-product equation: three pairings— π_A against π_B , π_C against a fixed verification-key element, and a public-input-weighted combination of verification-key elements against another—compared with one pairing precomputed once per circuit. Deployed Zcash uses Groth16 over BLS12-381 for every Sapling spend and output proof (protocol specification § 5.4.10.2; proofs produced and verified by the `bellman` crate’s `groth16` module, `src/groth16/prover.rs` and `src/groth16/verifier.rs`, consumed through the `zcash_proofs` crate, `src/prover.rs`). PLONK, the third generation, is the recipe of Theorem 9.28 below with the KZG scheme of Construction 4.34 as its commitment: the gate-and-permutation arithmetisation Halo 2 also uses, each polynomial opening checked by two pairings, so proof size and verification cost stay constant. Constant-size proofs and constant-time verification, independent of the circuit, are the family’s signature—the strongest form of the succinctness clause of Definition 9.24.

The price is the setup. The reference string is produced from secret scalars—the evaluation point τ of Construction 4.34 and, in Groth16, further secrets that tie the proof elements to one circuit—which

must be *destroyed* once the parameters are published: they are the *toxic waste*. Whoever retains them forges accepting proofs of false statements at will, exactly as the trapdoor τ forges KZG openings (Remark 4.37), and no verifier can tell: knowledge soundness collapses silently. For Groth16 the waste is *per circuit*—every new statement shape needs a fresh setup—whereas PLONK’s KZG string is universal, one powers-of- τ string serving every circuit up to a size bound, but a trusted one still. The mitigation is a *multiparty ceremony*: participants contribute secret randomness in turn, each re-randomising the running parameters and destroying its own share, so that the waste stays unknown as long as *one* participant was honest. Zcash ran such ceremonies for the parameters of its earlier proof systems—the BCTV14 Sprout circuit and the Groth16 Sapling circuits (protocol specification §§ 5.7–5.8)—and each remains sound only on that one-honest-participant assumption.

The other route through the design space keeps the discrete logarithm and nothing else. The inner-product commitment of Construction 4.38 needs no pairing and no trusted setup—its generators are hashed into existence, with no secret behind them—at the price of logarithmic proofs and a linear-time opening check (Proposition 4.39); it is the road the *Halo 2 Guide* takes, and the one the rest of this subsection paves.

Polynomial interactive oracle proofs

Modern SNARKs are built in two layers—one information-theoretic, one cryptographic—and the information-theoretic layer speaks the language of polynomials.

Definition 9.27 (Polynomial interactive oracle proof). Fix a finite field $\mathbb{F}$. A *polynomial interactive oracle proof* (Poly-IOP) for a relation R over $\mathbb{F}$ is a public-coin interactive protocol in which, in place of ordinary messages, the prover in each round sends an *oracle* for a polynomial $p_i \in \mathbb{F}[X]$ of some bounded degree; the verifier sends uniformly random field-element challenges, and may *query* each polynomial oracle at points of its choice, receiving the evaluation $p_i(\zeta)$. After the interaction the verifier accepts or rejects as a function of its challenges and the few evaluations it queried. Completeness and (knowledge) soundness are as for interactive proofs, with the soundness analysis purely algebraic—resting, invariably, on the Schwartz–Zippel lemma: a nonzero polynomial of degree at most d over $\mathbb{F}$ vanishes at a uniformly random point with probability at most $d/|\mathbb{F}|$ (*Math Guide*, §“The Schwartz–Zippel lemma”). A good Poly-IOP verifier makes only $O(1)$ or $O(\log)$ queries, each a single evaluation.

The oracle is an idealisation—no real prover can hand over an opaque box answering evaluation queries honestly—but a productive one: it isolates the algebra from the cryptography. Soundness of the Poly-IOP layer is *unconditional*, resting on no assumption beyond the rigidity of low-degree polynomials; the cryptography enters only when the oracles are instantiated.

Arithmetisation is how a computation becomes a Poly-IOP statement. The claim “I know w with $C(w) = 1$ ” (the relation R_{sat} of Example 9.2) is encoded as a small set of polynomial identities over $\mathbb{F}$ that hold if and only if the computation is correct. The computation is first expressed as a *constraint system*—the customisable *PLONKish* gate-and-permutation form used by Halo 2—whose satisfaction is equivalent to a handful of polynomials satisfying identities on a chosen evaluation domain, a fixed finite set of points with one point per constraint row. In the PLONKish form, polynomials encoding the wire values a, b, c and fixed selector polynomials q_L, q_R, q_M, q_O, q_C chosen at circuit-definition time must satisfy the gate identity

$$q_L \cdot a + q_R \cdot b + q_M \cdot ab + q_O \cdot c + q_C = 0$$

at every point of the domain—selectors switching each row among addition, multiplication, and custom behaviours—while a *permutation argument* enforces the copy constraints wiring one gate’s output to another’s input. One more step turns the on-domain requirement into a spot-checkable one: the identity is demanded only on the domain H , and a polynomial vanishes on all of H exactly when

it is divisible by the vanishing polynomial $Z_H := \prod_{h \in H} (X - h)$ (*Math Guide*, §“Roots and the factor theorem”), so the prover also sends an oracle for the quotient t with

$$q_L \cdot a + q_R \cdot b + q_M \cdot ab + q_O \cdot c + q_C = t \cdot Z_H$$

as an identity of polynomials—one that must now hold at *every* point of $\mathbb{F}$. Checking it at a single random point is the Poly-IOP: the verifier queries the oracles at a random ζ , checks the displayed equation there, and by Schwartz–Zippel a false statement survives with probability at most $d/|\mathbb{F}|$, with d bounding the degree of either side.

The polynomial commitment scheme is the cryptographic half. The PCS of Definition 4.32 supplies exactly what the idealised oracle promised: a short, binding commitment to a polynomial, with short proofs—verifiable against the commitment alone—that $p(\zeta) = v$; evaluation binding forbids two accepted values at one point, and the *extractability* the definition names is what SNARK compilation consumes: a prover whose evaluation proofs verify must “know” an underlying polynomial. Three instantiations mark out the design space: KZG (Construction 4.34)—pairing-based, constant-size, trusted setup; the inner-product-argument commitment over a single elliptic curve (Construction 4.38)—transparent, no trusted setup, logarithmic-size openings, the choice made by Halo 2; and the hash-based transparent schemes, outside this volume’s deployed path and mentioned only to bound the territory.

Theorem 9.28 (The compilation recipe, informal). *Let PIOP be a public-coin polynomial IOP for R , complete and knowledge-sound, and let PC be a polynomial commitment scheme that is correct, evaluation-binding, and extractable. Compile as follows: wherever the prover would send a polynomial oracle p_i , it sends $\text{Commit}(p_i)$ instead; wherever the verifier would query $p_i(\zeta)$, the prover sends the claimed value together with a PC evaluation proof, which the verifier checks against the commitment. Under compatible composition hypotheses the result is a public-coin interactive argument of knowledge for R , and a suitable multi-round Fiat–Shamir theorem in the ROM (Definition 9.18 gives the transform: every challenge a hash of the entire transcript so far) makes it non-interactive. Its proof is succinct; under the strict Definition 9.24 it is a SNARK only if verification is also sublinear in the proved computation. If, additionally, the Poly-IOP is properly blinded—random masking terms on the prover’s polynomials, hiding commitments (Remark 4.41)—and the corresponding simulation theorem applies, the result is zero knowledge.*

Proof sketch: layered extraction and the succinctness account. Completeness transfers verbatim: honest commitments open to honest evaluations, and the honest Poly-IOP verifier accepts.

Knowledge soundness is layered, one extractor feeding another. Given a successful prover against the compiled argument, the SNARK extractor first invokes PC *extractability* on each commitment, recovering actual polynomials $\hat{p}_i$ of the right degrees consistent with every evaluation the prover got accepted; this step is where the cryptographic assumption lives, and why the compiled object is an *argument*—a prover that could equivocate on its commitments would defeat it, and only computational binding rules that out. The extractor then hands the $\hat{p}_i$ to the Poly-IOP’s *unconditional* knowledge extractor, which is sound against *any* polynomials—in particular the extracted ones—and outputs $w \in R(x)$; evaluation binding guarantees the prover could not have answered queries inconsistently with the committed $\hat{p}_i$, so the algebraic soundness analysis applies to the extracted polynomials as run. Fiat–Shamir then converts the public-coin interactive argument to a non-interactive one in the ROM by the multi-round generalisation of Theorem 9.20: challenges become transcript hashes (Remark 9.19 governing what they must absorb), observability and forking supply the rewinding that the layered extractor needs—tree-style, per Definition 9.14—and the honest-verifier zero knowledge of the blinded protocol becomes full non-interactive zero knowledge by oracle programming.

Succinctness is an accounting exercise, and the account is paid entirely by PC. The proof consists of a constant or logarithmic number of commitments and evaluation proofs—short by construction, sublinear in the circuit, independent of the witness—so the compiled *proof* is succinct whenever

the commitment scheme’s objects are. The *verifier* is sublinear only when the evaluation checks are themselves succinct, as for KZG’s two pairings; with the inner-product commitment each evaluation check costs $O(d)$ group operations (Proposition 4.39), so the compiled verifier runs in time linear in the circuit while the proof stays succinct. The compiled object therefore satisfies every clause of Definition 9.24 when the PC openings verify succinctly, and otherwise yields succinct proofs with linear-time verification—the trade Halo 2 accepts and then recovers through batching. $\square$

Remark 9.29 (Halo 2 is the recipe, instantiated). The proving system deployed in Orchard (protocol specification § 5.4.10.3) instantiates the recipe of Theorem 9.28, and each ingredient can be pointed to in the deployed source. The Poly-IOP is a PLONKish arithmetisation—custom gates declared as polynomial expressions, lookup arguments asserting that witness values lie in a declared table, and a permutation argument enforcing copy constraints (`halo2_proofs` crate, `src/plonk/circuit.rs` for the gate expressions, `src/plonk/lookup.rs` and `src/plonk/permutation.rs` for the two argument types). The polynomial commitment is the inner-product-argument scheme over a single elliptic curve, *transparent*—the generators are derived by hashing to the curve from the fixed domain string `Halo2-Parameters`, so the setup holds no trapdoor—with logarithmic-size openings (`src/poly/commitment.rs`, `Params::new` for the parameter derivation; `src/poly/commitment/prover.rs` for the folding rounds of the opening argument). Fiat–Shamir over a concrete hash makes the argument non-interactive: the transcript is a running BLAKE2b state into which every prover message is absorbed, domain-separated by type, and out of which every challenge is squeezed—the entire-prefix discipline of Definition 9.18 enforced by construction (`src/transcript.rs`, `Blake2bWrite` and `Blake2bRead`).

The linear cost of each IPA opening check (Proposition 4.39) makes the deployed verifier’s work linear in the commitment dimension: Halo 2 has succinct *proofs* but does not meet the strict sublinear-verifier clause of Definition 9.24, and it is called a zk-SNARK under the broader convention that asks succinctness of the proof alone. The system’s signature innovation—the technique that named *Halo*—is *accumulation*: the one expensive step, the $O(d)$ final check of each IPA opening, is deferred and passed along rather than paid inside a circuit, one proof attesting to the verification of another—which is what makes *recursive* composition possible without a trusted setup. Deployed Zcash does not use recursion: each Orchard proof is verified directly, and what the deployed verifier takes from the Halo idea is the deferral alone, exploited for *batching*—many proofs’ deferred final checks collapsing into one multiscalar multiplication (`src/poly/commitment/verifier.rs`, the `Accumulator` and `Guard` types; `src/plonk/verifier/batch.rs`, `BatchVerifier`; consumed for Orchard bundles as described in Remark 4.40). These claims rest on model and composition hypotheses beyond the generic recipe (Remark 9.21).

Three load-bearing ideas from this section carry forward into everything the upper volumes build. *Rewinding* (Remark 9.8) is the universal device: the same cloning of an algorithm’s state powers the extractor that defines knowledge and the simulator that defines zero knowledge. *The layered split* of Theorem 9.28—an unconditionally sound information-theoretic core, wrapped in a cryptographically enforced commitment layer—is the architecture of every modern proof system, and the right frame for auditing one: algebraic bugs and cryptographic bugs live in different layers. And *knowledge soundness*, not mere soundness, is what an application stakes its security on: a shielded protocol does not care that a spendable note exists; it cares that the prover has one.

10 Merkle trees and commitments to sets

The adversary of this final section tells lies about a set. Her first lie is *inclusion*: she claims that a value sits in a committed collection it never entered—a payment out of thin air, spent against a ledger that never recorded it. Her second lie is *equivocation*: she publishes one short commitment and later produces two different collections that both explain it, so that the commitment pinned down nothing. Her third role is not a lie but a gaze: as an *observer* she watches an honest member prove that it belongs,

and learns *which* member spoke—enough, in a payment system, to trace every coin. This section defeats all three with a single construction, built from nothing beyond the collision-resistant hash of §3, and closes the volume by spending everything the earlier sections banked: the commitments of §4 become the leaves, the algebraic hashes of §3 and §4 become the internal nodes, and the zero-knowledge arguments of §9 hide the path from leaf to root.

The plan runs from public to private. We first fix the two statements a verifier might want proved about a large collection, and recall the compression function and its collision game. The binary Merkle tree follows, with its root as a binding commitment to the whole list; a worked depth-two tree keeps every index honest. Authentication paths then give logarithmic-size membership proofs, complete and sound; the soundness proof introduces the one technique this section reuses until the end—extracting a hash collision from the first level at which two disagreeing computations agree. Layer and domain separation keep that extraction honest against structural confusion between leaves and nodes, with a deployed failure as the cautionary tale. Append-only trees and the frontier make the structure grow in logarithmic time; the vector-commitment abstraction names exactly what the root achieves; and zero-knowledge paths convert the public proof into a private one. The capstone assembles an Orchard shielded spend end to end.

10.1 Two statements about a set

The setting is asymmetric. A prover holds a large collection of values—transactions in a block, public keys in a directory, records in a log; for this volume, the ever-growing set of note commitments in a shielded payment system. A verifier wants conviction about that collection without storing it, indeed without ever seeing most of it. Two statements cover the uses that matter here.

- *Membership*. “Value v occurs at position i of the list”—or, forgetting the position, “ v belongs to the set.”
- *Integrity*. “The collection is exactly the one committed to earlier, unaltered.”

What is wanted is a single short *commitment* that determines the whole collection, together with short *membership proofs*: “short” meaning of size independent of the number of elements, or at worst logarithmic in it. A commitment of that shape makes integrity a one-comparison check, and membership a small certificate that travels with the value it certifies. The construction achieving this—the *Merkle tree*, due to Ralph Merkle—needs nothing more than a collision-resistant hash function, and it is the membership backbone of Orchard: every shielded spend proves, against a published root, that some note commitment lies in the tree.

Notation is as fixed in §3.1 and carried throughout the volume: $\{0, 1\}^*$ is the set of finite binary strings, $\{0, 1\}^n$ the strings of length exactly n , $x \parallel y$ concatenation, and λ the security parameter. All algorithms are PPT in λ , and $\text{negl}(\lambda)$ denotes a negligible function (*Math Guide*, §“Polynomial, exponential, and negligible functions”).

10.2 The compression function and collision resistance

The tree consumes one ingredient, which we recall in the keyed form that makes its security a meaningful asymptotic statement.

Definition 10.1 (Compression function). Let $H : \mathcal{K} \times \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ be a keyed hash family exactly as in Definition 3.2: a key $s \in \mathcal{K}$ is sampled publicly at random and fixed, the digest length is $\ell = \ell(\lambda)$, and $H_s := H(s, \cdot)$ is the family member in force. When the input length is restricted to a fixed multiple of the output length, the family is a *compression function*; a *two-to-one* compression function has signature

$$h_s : \{0, 1\}^{2\ell} \longrightarrow \{0, 1\}^\ell,$$

taking two digests to one.

The key s models the choice of a concrete function from a family, which is what rescues collision resistance from the hardcoded-collision triviality discussed in §3.1: for a single fixed function a colliding pair exists and some small program outputs it, so only the keyed formulation supports a clean quantifier. In practice one deploys a fixed standardised function—or, inside arithmetic circuits, an algebraic hash in the sense of §3.8 and §4.8—and we suppress s from the notation, writing h and H plain.

The security property the tree needs is collision resistance, exactly as in Definition 3.7 (§3.2): the game COL_H^A hands the adversary the key s , the adversary outputs a pair (x, x') , and it wins if $x \neq x'$ while $H_s(x) = H_s(x')$; the family is collision resistant if every PPT adversary wins with probability $\text{negl}(\lambda)$. This is the strongest of the three classical hash notions (Proposition 3.8), and the generic birthday attack caps an ℓ -bit digest at $\ell/2$ bits of collision security (Lemma 3.11)—whence the 256-bit outputs used at the 128-bit level here. Merkle trees require *full* collision resistance, not merely second-preimage resistance, for a reason the soundness theorem (Theorem 10.12) makes exact: the party who assembles the tree may itself be the adversary, choosing every leaf, which is precisely the both-inputs-adversarial regime that Remark 3.10 assigns to collision resistance.

10.3 Binary Merkle trees

The construction hashes the collection pairwise, level by level, until one digest remains.

Definition 10.2 (Binary Merkle tree). Fix a two-to-one compression function $h : \{0, 1\}^{2\ell} \rightarrow \{0, 1\}^\ell$, a depth $d \in \mathbb{N}$, and an ordered list $L = (L_0, \dots, L_{2^d-1})$ of 2^d leaf values in $\{0, 1\}^\ell$. The *Merkle tree of depth d over L* is the complete binary tree with 2^d leaves whose nodes carry labels in $\{0, 1\}^\ell$, defined level by level from the leaves (level 0) to the root (level d) by

$$N_{0,j} = L_j \quad (0 \leq j < 2^d), \quad N_{t+1,j} = h(N_{t,2j} \parallel N_{t,2j+1}) \quad (0 \leq t < d, 0 \leq j < 2^{d-t-1}). \quad (3)$$

Level t has 2^{d-t} nodes, and $N_{t,j}$ is the j -th node from the left at level t . The unique level- d node is the *root*: $\text{root}(L) := N_{d,0}$.

Example 10.3 (A depth-two tree). With $d = 2$ and leaves L_0, L_1, L_2, L_3 , the tree has seven nodes: the four leaves, the two level-one nodes

$$N_{1,0} = h(L_0 \parallel L_1), \quad N_{1,1} = h(L_2 \parallel L_3),$$

and the root

$$\text{root}(L) = N_{2,0} = h(h(L_0 \parallel L_1) \parallel h(L_2 \parallel L_3)).$$

The single ℓ -bit root summarises the whole list of four ℓ -bit strings; the example carries through the next two subsections.

A list whose length m is not a power of two is handled by fixing the depth d with $2^d \geq m$ and padding the remaining $2^d - m$ leaf slots with a canonical value—the all-zero string, say, or the hash of the empty string. Padding must not create ambiguity: two different lists must never yield the same padded leaf sequence, else the root fails to determine the list before any cryptography is consulted. Binding the true length m into the root’s computation—one further compression of the top node with an encoding of m —removes any such ambiguity. An alternative is the *unbalanced* tree, in which a node with a single child passes its child’s label upward unchanged; it is convenient but subtle, and we return to it alongside second-preimage attacks in Remark 10.16.

The root is short; the claim that it *commits* to the list is a security property, stated as a game in the tradition of Definition 1.15.

Definition 10.4 (Merkle commitment; binding). The *Merkle commitment* to a list L of 2^d leaves is $\text{root}(L) \in \{0, 1\}^\ell$. The game $\text{BIND}_{H,d}^A(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$, fixing the compression function $h := H_s$, and sends s to the adversary.
2. The adversary outputs two lists L, L' , each of 2^d leaves in $\{0, 1\}^\ell$.
3. The game outputs 1 iff $L \neq L'$ and $\text{root}(L) = \text{root}(L')$.

The commitment is *binding* if every PPT adversary wins with probability $\text{negl}(\lambda)$.

Proposition 10.5 (Binding of the root). *If h is collision resistant, the Merkle commitment is binding. Concretely, there is a deterministic procedure that, from any two distinct equal-length lists $L \neq L'$ with $\text{root}(L) = \text{root}(L')$, extracts a collision for h in time linear in the tree size—one root-to-leaf walk of $O(d)$ hash comparisons. The reduction is tight: an adversary winning BIND with probability ε yields a collision finder succeeding with the same probability ε .*

Proof. Build both trees. The lists differ at some leaf index j_0 , so $N_{0,j_0} \neq N'_{0,j_0}$, writing N and N' for the labels of the two trees. Follow the leaf-to-root path

$$N_{0,j_0}, N_{1,\lfloor j_0/2 \rfloor}, N_{2,\lfloor j_0/4 \rfloor}, \dots, N_{d,0}$$

in both trees. The labels differ at level 0 and agree at level d —the common root—so there is a *smallest* level t at which they first agree: with $j = \lfloor j_0/2^t \rfloor$,

$$N_{t,j} = N'_{t,j}, \quad \text{yet} \quad N_{t-1,2j} \neq N'_{t-1,2j} \text{ or } N_{t-1,2j+1} \neq N'_{t-1,2j+1},$$

since one of the two children at level $t-1$ is the path node, at which the labels still disagree by minimality of t . By the recurrence (3),

$$h(N_{t-1,2j} \parallel N_{t-1,2j+1}) = N_{t,j} = N'_{t,j} = h(N'_{t-1,2j} \parallel N'_{t-1,2j+1}),$$

while the two 2ℓ -bit inputs differ in at least one half: a collision for h , located after at most d comparisons along one path. The reduction wrapping this extractor runs the BIND adversary once and extracts whenever it wins, so collision resistance forces the winning probability to be negligible. $\square$

The proof's engine deserves its name now, because the section reuses it almost verbatim twice more: *a discrepancy at the leaves and agreement at the root must, by this pigeonhole-like descent, manifest a collision somewhere on the path between them.* There is a smallest level at which two disagreeing computations agree; the compression call producing that first agreement has two differing preimages. All three security theorems of this section are this one sentence wearing different inputs.

10.4 Authentication paths and membership proofs

The root alone certifies integrity; membership needs a certificate relating one leaf to the root. The certificate is the list of siblings along the leaf's path upward—everything the verifier needs to recompute the root from the leaf, and nothing more.

Definition 10.6 (Authentication path). Let a depth- d tree over L be given, and let i be a leaf index, $0 \leq i < 2^d$, with binary expansion $i = b_{d-1}b_{d-2} \cdots b_0$, so that bit b_t records whether the level- t node on the leaf-to-root path is a left child ($b_t = 0$) or a right child ($b_t = 1$). The *authentication*

path (also *Merkle proof* or *membership witness*) for index i is

$$\pi_i = ((s_0, b_0), (s_1, b_1), \dots, (s_{d-1}, b_{d-1})),$$

where s_t is the label of the *sibling* of the level- t node on the path. Explicitly, with $j_t = \lfloor i/2^t \rfloor$ the path node's index at level t ,

$$s_t = \begin{cases} N_{t, j_t+1} & \text{if } b_t = 0 \quad (\text{sibling on the right}), \\ N_{t, j_t-1} & \text{if } b_t = 1 \quad (\text{sibling on the left}). \end{cases}$$

The path has exactly d entries.

Definition 10.7 (Path verification). The verifier $\text{Verify}(rt, i, v, \pi_i)$, on a root rt , an index i , a claimed leaf value $v \in \{0, 1\}^\ell$, and a path $\pi_i = ((s_0, b_0), \dots, (s_{d-1}, b_{d-1}))$, proceeds in two stages. First the *index-consistency check*: reject unless, for every t , the bit b_t carried in π_i equals bit t of the binary expansion of i . Then set $a_0 := v$ and fold the path upward: for $t = 0, \dots, d-1$,

$$a_{t+1} := \begin{cases} h(a_t \parallel s_t) & \text{if } b_t = 0, \\ h(s_t \parallel a_t) & \text{if } b_t = 1. \end{cases} \quad (4)$$

Accept iff $a_d = rt$. We call a_d *the root recomputed from* (i, v, π_i) . The bit b_t places the running value on the side matching the argument order of the tree-building recurrence: a left child is hashed on the left.

Proposition 10.8 (Completeness). *For every list L , every index i , and π_i the genuine authentication path for i in the tree over L ,*

$$\text{Verify}(\text{root}(L), i, L_i, \pi_i) = \text{accept}.$$

Proof. The index-consistency check passes by construction, the genuine path carrying exactly the bits of i . We show by induction on t that $a_t = N_{t, \lfloor i/2^t \rfloor}$, the label of the level- t node on the leaf-to-root path. The base is $a_0 = L_i = N_{0, i}$. For the step, the level- t path node and its sibling s_t are precisely the two children of the level- $(t+1)$ path node, and the fold (4) places a_t on the side dictated by b_t with s_t on the other side—reproducing exactly the two arguments of h in the recurrence (3), so $a_{t+1} = N_{t+1, \lfloor i/2^{t+1} \rfloor}$. At $t = d$ this reads $a_d = N_{d, 0} = \text{root}(L)$, and the verifier accepts. $\square$

Proposition 10.9 (Logarithmic proof size). *An authentication path in a depth- d tree consists of d sibling labels and d direction bits, $d\ell + d$ bits in all, and verification performs exactly d evaluations of h . Choosing the minimal sufficient depth $d = \lceil \log_2 m \rceil$ for a list of $m \geq 2$ leaves (any $2^d \geq m$ suffices) yields proofs of $\lceil \log_2 m \rceil (\ell + 1)$ bits—logarithmic in the length of the list.*

Proof. One sibling label, one direction bit, and one compression per level, over the d levels above the leaves. $\square$

The concrete scale is worth pausing on. For a set of a billion elements— $m = 2^{30} = 1\,073\,741\,824$ —a membership proof consists of 30 hash values and 30 direction bits. With a 256-bit hash the sibling labels occupy $30 \cdot 256 = 7680$ bits, which is 960 bytes; direction bits included, 964 bytes—under a kilobyte, however the billion elements are arranged. (The byte counts are the output of the volume's companion script.) This logarithmic succinctness is the central objective: the verifier's storage is one digest, the certificate fits in a packet, and neither grows meaningfully as the set does.

Example 10.10 (A depth-two membership proof). Continue Example 10.3, and prove membership of L_2 at index $i = 2 = (10)_2$, so $b_0 = 0$ and $b_1 = 1$. At level 0 the path node $N_{0,2} = L_2$ is a left child, with sibling $s_0 = N_{0,3} = L_3$. At level 1 the path node $N_{1,1} = h(L_2 \parallel L_3)$ is a right child, with sibling $s_1 = N_{1,0} = h(L_0 \parallel L_1)$. The path is

$$\pi_2 = ((L_3, 0), (h(L_0 \parallel L_1), 1)).$$

Verification folds upward: $a_1 = h(L_2 \parallel L_3)$, then $a_2 = h(h(L_0 \parallel L_1) \parallel a_1) = \text{root}(L)$, and the verifier accepts. Note what the verifier never saw: neither L_0 nor L_1 themselves, only their digest $h(L_0 \parallel L_1)$ —yet it is convinced that L_2 sits at index 2 of the committed list.

10.5 Soundness: forging a path implies a collision

Completeness says honest paths verify; the security question is whether dishonest ones can. The threat is the inclusion lie of the section’s opening: an adversary who convinces the verifier, against the genuine root, that position i holds a value it does not.

Definition 10.11 (Path forgery). Fix a list L of 2^d leaves with root $rt = \text{root}(L)$. The game $\text{PATHFORGE}_{H,L}^A(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$ to fix $h := H_s$, builds the tree over L , and sends s together with L —equivalently, the whole tree—to the adversary.
2. The adversary outputs an index i and a pair (v^*, π^*) .
3. The game outputs 1 iff $\text{Verify}(rt, i, v^*, \pi^*) = \text{accept}$ and $v^* \neq L_i$.

A winning output is a *path forgery at index i* : the verifier accepts, against the honestly computed root, a leaf value different from the genuine one at position i .

Theorem 10.12 (Soundness of Merkle membership proofs). *Let h be collision resistant. Then no PPT adversary, given the list L and the whole tree, produces a path forgery except with probability $\text{negl}(\lambda)$. In constructive form: there is a deterministic extractor that from any list L , index i , and accepting forgery (v^*, π^*) with $v^* \neq L_i$ outputs a collision for h , in time linear in the tree size—recomputing the genuine leaf-to-root labels from L dominates; given the built tree, one verifier run plus a level-by-level comparison, $O(d)$ work in all, completes the extraction.*

Proof. Run the verifier’s fold (4) on (i, v^*, π^*) , obtaining the chain $a_0^* = v^*, a_1^*, \dots, a_d^* = rt$ (the final equality because the forgery is accepting). Alongside it place the genuine chain $a_t = N_{t, \lfloor i/2^t \rfloor}$, the labels of the leaf-to-root path in the honest tree; by the induction of Proposition 10.8 this is the chain the fold produces on the genuine inputs, and $a_d = rt$.

The two chains disagree at the bottom, $a_0^* = v^* \neq L_i = a_0$, and agree at the top, $a_d^* = a_d = rt$. Hence

$$t^* := \min \{ t : a_t^* = a_t \} \in \{1, \dots, d\}$$

exists, and $a_{t^*-1}^* \neq a_{t^*-1}$ by minimality. Because the forgery is accepting, the index-consistency check of Definition 10.7 forces the direction bits carried in π^* to equal the bits of i —the same bits the genuine path carries—so at level $t^* - 1$ both chains place their running value on the same side of the compression call: both compute $h(\cdot \parallel \text{sibling})$ if $b_{t^*-1} = 0$, both $h(\text{sibling} \parallel \cdot)$ if $b_{t^*-1} = 1$. Form the two 2ℓ -bit preimages accordingly,

$$X^* = \begin{cases} a_{t^*-1}^* \parallel s_{t^*-1}^* & \text{if } b_{t^*-1} = 0, \\ s_{t^*-1}^* \parallel a_{t^*-1}^* & \text{if } b_{t^*-1} = 1, \end{cases} \quad X = \begin{cases} a_{t^*-1} \parallel s_{t^*-1} & \text{if } b_{t^*-1} = 0, \\ s_{t^*-1} \parallel a_{t^*-1} & \text{if } b_{t^*-1} = 1, \end{cases}$$

where $s_{t^*-1}^*$ is the sibling claimed in π^* and s_{t^*-1} the genuine sibling. Then

$$h(X^*) = a_{t^*}^* = a_{t^*} = h(X), \quad X^* \neq X,$$

the inequality because the two preimages differ in the half occupied by the running value. The pair (X^*, X) is a collision for h , found deterministically in $O(d)$ work once the genuine chain is at hand—a read-off from the built tree, or $2^d - 1$ hashes from the bare list L . $\square$

The reduction wrapping the extractor is immediate. If a PPT adversary $\mathcal{A}$ produces path forgeries with probability $\varepsilon(\lambda)$, the collision finder $\mathcal{B}$ that receives the key, samples or receives L , runs $\mathcal{A}$, and applies the extractor outputs a collision whenever $\mathcal{A}$ succeeds—hence with probability $\varepsilon(\lambda)$, at an overhead of building the tree over L , which $\mathcal{B}$ does in any case to play the challenger, plus one verifier run and $O(d)$ comparisons. Collision resistance of h therefore forces $\varepsilon(\lambda) \in \text{negl}(\lambda)$, and the reduction is tight: no advantage is lost in the translation.

Remark 10.13 (Scope: a fixed honest root). Theorem 10.12 is a statement about a *fixed honest root*: relative to a root that genuinely is $\text{root}(L)$, the only leaf value admitting an accepting path at position i is L_i , up to a collision in h . It does not by itself prevent an adversary who is free to *choose* the root from committing to a maliciously crafted tree; that threat is what binding (Proposition 10.5) rules out. With an untrusted party supplying the root, both properties are necessary and they divide the labour cleanly: binding ensures the root determines at most one list, and path soundness ensures openings are faithful to it. Together they make the root a sound, binding commitment with succinct openings—the abstraction §10.8 names, where a single argument will deliver both faces at once.

10.6 Layer and domain separation

The soundness theorem has a structural blind spot, inherited from an innocent-looking convenience. Suppose the leaves are arbitrary-length data, hashed into $\{0, 1\}^\ell$ by the *same* function used internally. Then an internal label $h(A \parallel B)$ is itself a well-formed leaf value—it has the right length—and one string can play both roles. An adversary exploits the ambiguity directly: she presents the internal node $N_{1,0} = h(L_0 \parallel L_1)$ as if it were a *leaf* near the top of a shallower tree, equipped with the correspondingly shorter authentication path, and a verifier who does not know the depth accepts. The root now has a second list explaining it—a genuine second-preimage attack, with no collision of h anywhere in sight. Absent a way for a label to assert “I am a leaf” or “I am an internal node”, the leaf-to-root structure that Theorem 10.12 relies on collapses.

The repair is the domain-separation discipline of §3.5, applied *within* a single structure: distinct structural positions get distinct tags.

Definition 10.14 (Layer and domain separation). A Merkle construction has *domain separation* if the function applied at distinct structural positions is forced to differ, by prepending a position-dependent tag to every hash input. Concretely, from a variable-input-length hash H define

$$\text{LeafHash}(v) := H(0x00 \parallel v), \quad \text{NodeHash}_t(A, B) := H(0x01 \parallel \langle t \rangle \parallel A \parallel B),$$

where the byte tags $0x00$ and $0x01$ distinguish the *layer kind*—leaf versus internal node—and $\langle t \rangle$ is a fixed-length encoding of the level index, distinguishing internal nodes by depth. The tree recurrence becomes

$$N_{0,j} = \text{LeafHash}(L_j), \quad N_{t+1,j} = \text{NodeHash}_{t+1}(N_{t,2j}, N_{t,2j+1}),$$

and the verifier’s fold applies the same tagged functions at the same positions.

Proposition 10.15 (Separation forces within-domain collisions). *With the tagging of Definition 10.14, any collision extracted by the binding proof (Proposition 10.5) or the soundness proof (Theorem 10.12) is a collision for H between two inputs sharing the same leading tag: a leaf preimage against a leaf preimage, or a level- t node preimage against a level- t node preimage. In particular, no accepting forgery can substitute an internal node for a leaf, or a node at one level for a node at another, without exhibiting a within-domain collision of H —which collision resistance forbids.*

Proof sketch. The extractors of both proofs locate the smallest level at which two disagreeing computations agree, and collide the two preimages of the compression call at that position. Under the tagged recurrence, the verifier’s recomputation applies the position’s prescribed function exactly as the honest tree does: at the leaves both preimages pass through `LeafHash`, at level $t \geq 1$ both pass through `NodeHasht`. Both colliding preimages therefore begin with the same tag—`0x00`, or `0x01 || <t>`—and the collision lies within a single domain. A cross-domain confusion, by contrast, would require two inputs with *different* leading tags and equal H -images, which is again an H -collision; either way the adversary has broken collision resistance, and the shallower-tree attack above is foreclosed because a leaf preimage and a node preimage can no longer be the same string. $\square$

Remark 10.16 (Unbalanced trees, padding, and a deployed failure). The subtleties are not hypothetical. Bitcoin’s transaction Merkle tree handles an odd level by *duplicating* its last node before hashing pairwise, and the duplication rule is not injective: a list of transactions and the same list with its tail duplicated can yield the same root. This is CVE-2012-2459, a consensus-splitting vulnerability—an attacker could present two *different* transaction lists sharing a root, one valid and one invalid, and nodes could be made to cache the invalid one as permanently rejected (documented at length in Zcash’s inherited validator: `zebra-chain crate, src/block/merkle.rs`, which reproduces the upstream warning that the flawed algorithm must not be reused; the deployed defence—rejecting any block whose transaction hashes are not unique—lives in `zebra-consensus, src/block/check.rs`). The lesson generalises to three rules. First, layer-separate leaves from internal nodes, as Definition 10.14 prescribes. Second, make every padding or duplication rule *injective* on admissible lists—for instance by binding the true leaf count into the root’s computation. Third, feed `NodeHash` only fixed-length inputs, so that no length-extension structure of the underlying hash (§3.5) can intervene between layers.

Remark 10.17 (Algebraic hashes in the tree; Orchard’s MerkleCRH). When the Merkle path is verified inside a zero-knowledge circuit (§10.9), arithmetic gates over $\mathbb{F}_p$ evaluate the compression function, and the choice of h dominates the cost. A bit-oriented hash such as SHA-256 costs tens of thousands of constraints per call; one therefore uses an *algebraic* hash with compression $h : \mathbb{F}_p^2 \rightarrow \mathbb{F}_p$. Poseidon (§3.8) is a low-degree permutation-based map computable in few constraints; Sinsemilla (§4.8) is cheap in constraints through lookups and incomplete curve additions, *not* through low degree. For Poseidon, layer and domain separation come not from byte tags but from distinct field constants—initial capacity values or round constants—fed into the permutation; the tree’s security argument is identical, with “collision” read as a collision of the field-valued compression function.

Orchard’s Merkle hash MerkleCRH retains the tag mechanism of Definition 10.14 in encoded form. Each internal call prepends the 10-bit encoded layer index to the two 255-bit child encodings—a single fixed input length of 520 bits (a total the volume’s companion script checks), exactly the fixed-length-per-personalisation regime in which the deployed Sinsemilla hash is collision resistant (§4.8)—with *all* layers sharing one Sinsemilla instance (`orchard crate, src/tree.rs, MerkleHashOrchard::combine`, prepending `i2lebsp_k(level)` to two `L_ORCHARD_MERKLE = 255`-bit child encodings; constants in `src/constants/sinsemilla.rs`; protocol specification § 5.4.1.3). Separation between *distinct* Sinsemilla instances comes from the personalisation string—here `z.cash:Orchard-MerkleCRH`—fed to `hash-to-curve` to derive the instance-specific base point Q , while the generator table S is shared across all instances (`sinsemilla crate, src/lib.rs: HashDomain::new` derives Q from the personalisation via the `z.cash:SinsemillaQ` hash-to-curve

domain; the table derives from the fixed `z.cash:SinsemillaS`). Sinsemilla is collision resistant under a discrete-logarithm assumption (Proposition 4.29), so the tree’s hashing and its commitment role fuse into one Pedersen-like map: the same structure that compresses the children also binds them.

10.7 Append-only and incrementally updatable trees

A shielded pool’s set of note commitments only ever grows, and it grows one leaf at a time. Rebuilding a depth-32 tree from scratch on every insertion would cost 2^{32} hashes; the structure of the tree permits the update in d hashes and $O(d)$ state, and the state has a name.

Definition 10.18 (Append-only Merkle tree). Fix a maximum depth d , giving capacity 2^d , and the level-indexed `NodeHash` functions of Definition 10.14. Leaf values here enter as level-0 labels directly, with no `LeafHash`: the leaves are already digests, and the fixed depth together with the level tags (Remark 10.17) carries the layer separation—the convention of deployed Orchard, whose leaf is the extracted coordinate itself (Example 10.31). An *append-only* (or *incremental*) Merkle tree maintains a position counter m —the number of leaves inserted so far, $0 \leq m \leq 2^d$ —and supports one mutating operation `Append( $v$ )`, which places v at leaf index m and increments m . Empty leaf slots $m, \dots, 2^d - 1$ carry a fixed *empty-leaf* value $\perp_0$, and the labels of entirely empty subtrees follow by precomputation:

$$\perp_{t+1} := \text{NodeHash}_{t+1}(\perp_t, \perp_t), \quad 0 \leq t < d.$$

The *root after m appends*, written root_m , is the root of the depth- d tree, under this recurrence, whose first m level-0 labels are the inserted values and whose remaining ones are $\perp_0$.

Definition 10.19 (Frontier). The *frontier* of an append-only tree holding m leaves is the minimal state needed to (i) compute the current root and (ii) update incrementally on the next `Append`. For each level $t \in \{0, \dots, d-1\}$ it holds at most one *carry*: the label of the left child of the next node to be completed at level $t+1$, retained precisely when leaf index m has a 1 bit at position t —a completed left sibling awaiting its right neighbour. Equivalently, writing m in binary as $b_{d-1} \dots b_0$, the frontier stores, for each t with $b_t = 1$, the already-finalised left-sibling label f_t at level t along the rightmost filled path. Its size is at most d labels—one per set bit of m , hence $O(\log m)$ state.

The right picture is arithmetic. The binary representation of m decomposes the inserted leaves into completed perfect subtrees of distinct heights—one of height t for each set bit b_t —and the frontier holds exactly the roots of those subtrees: the pieces that are “complete and waiting for a right neighbour”. An append is then the Merkle analogue of incrementing a binary counter: a carry at level t corresponds to combining two height- t subtrees into one of height $t+1$, and the chain of carries is as long as the run of trailing 1-bits of m .

Proposition 10.20 (Incremental append in logarithmic time). *From the frontier of an append-only tree holding $m < 2^d$ leaves, `Append( $v$ )` computes the new frontier and the new root root_{m+1} using at most d evaluations of `NodeHash` and $O(d)$ additional work, while storing only $O(d)$ labels throughout.*

Proof. The algorithm is carry propagation followed by an upward fold. Set the running label $c \leftarrow v$ at level $t \leftarrow 0$. While bit t of m is 1, the new path node at level t is a right child whose left sibling is the stored carry f_t : set $c \leftarrow \text{NodeHash}_{t+1}(f_t, c)$, clear the carry f_t , and increment t . When bit t of m is 0, store c as the new carry f_t and stop. Writing z for the number of trailing 1-bits of m , the carry phase costs exactly z hashes and deposits f_z at level z —the root of the just-completed height- z subtree

containing the new leaf, matching the binary increment $m \mapsto m + 1$, which clears z trailing ones and sets bit z .

To obtain root_{m+1} , fold upward from level z with the just-deposited f_z as the running value—it is the path label at level z , not a sibling. For $t = z, \dots, d - 1$, combine the running value with the stored carry f_t as left sibling when bit t of m is 1—those carries lie above the cleared run and survive the append untouched—and with the precomputed empty-subtree label $\perp_t$ as right sibling otherwise; at level z itself bit z of m is 0, so the first combination is with $\perp_z$, never with the deposited carry. One hash per level makes $d - z$ hashes in all (zero if $z = d$, when the tree has just filled). The total is exactly $z + (d - z) = d$ evaluations of NodeHash , and the retained state is the at-most- d carries: precisely the frontier for $m + 1$ leaves. $\square$

Example 10.21 (The frontier as a binary counter). Take depth $d = 3$ (capacity 8) and insert v_0, v_1, v_2 in turn; the frontier evolves exactly as the binary counter of the leaf count m .

- After v_0 , $m = 1 = (001)_2$: one completed height-0 subtree; the frontier is $\{f_0 = v_0\}$.
- After v_1 , $m = 2 = (010)_2$: leaf v_1 at index 1 is a right child, triggering one carry $c = \text{NodeHash}_1(v_0, v_1)$ —a completed height-1 subtree; the frontier is $\{f_1 = c\}$, with f_0 cleared.
- After v_2 , $m = 3 = (011)_2$: leaf v_2 at index 2 is a left child and becomes a new height-0 carry; the frontier is $\{f_1 = \text{NodeHash}_1(v_0, v_1), f_0 = v_2\}$ —exactly the two set bits of 3.

Each append touched only the carry chain to the new leaf, never the whole tree. (The trace involves no numeric hash values and follows mechanically from the algorithm of Proposition 10.20; the volume’s companion script replays it symbolically and checks the occupied frontier levels against the set bits of m at every step.)

Appending changes the root, which raises a question the definition of soundness so far does not answer: which root is a proof valid against?

Definition 10.22 (History binding). An append-only tree is *history-binding* when the surrounding protocol treats every historical root as an acceptable commitment in its own right: each membership proof verifies against the root it was formed for, and appending further leaves never invalidates a previously valid $(\text{root}, \text{leaf}, \text{path})$ triple for that root. The soundness of each individual proof is exactly Theorem 10.12 applied to the corresponding fixed root.

Deployed Orchard is history-binding by construction. The chain records each block’s note-commitment-tree root—its *anchor*—and a spend may prove membership against *any* recorded historical anchor: consensus requires the anchor named by a transaction to be the root of the Orchard note-commitment tree as it stood at the end of some earlier block, checked by membership in the set of recorded anchors (zebra-state crate, `src/service/check/anchors.rs`, `sapling_orchard_anchors_refer_to_final_treestates`; protocol specification § 3.7, ‘Action Transfers and their Descriptions’). Per-proof soundness is then the fixed-root soundness theorem, once per anchor. The frontier is likewise the deployed representation of the growing tree (`incrementalmerkletree` crate, `src/frontier.rs`, types `Frontier` and `NonEmptyFrontier`; the empty-subtree labels are the `EMPTY_ROOTS` table of orchard’s `src/tree.rs`).

Remark 10.23 (Why append-only, and what it costs). Deletion or in-place update could be supported at $O(\log m)$ per operation, given the relevant siblings; the restriction to appends is a design choice, made for a monotonicity it buys. Once a leaf is inserted it is never removed, so a membership proof valid against the root at insertion time remains valid against *that historical root* forever. This is exactly what a shielded pool requires—a note, once added, can always be spent later by proving against some past anchor—and it spares the spender from tracking a moving target as the tree grows under everyone else’s insertions. The cost falls on the verifier, who must accept a window of historical roots rather than a single current one; the previous paragraph is that window, deployed.

10.8 Vector commitments

The section has been proving properties of one construction; it pays to name the abstraction the construction inhabits, both to state precisely what the root achieves and to mark the boundary with its relatives. The abstraction is a commitment to an *ordered vector* that opens one position at a time. (Its unordered sibling—the *cryptographic accumulator*, which certifies set membership with no notion of position—exists in the literature, but Orchard never needs it: notes occupy positions, and the position is part of what the spend circuit witnesses. We do not develop it.)

Definition 10.24 (Vector commitment). A *vector commitment* is a tuple of PPT algorithms (Setup, Commit, Open, Vf):

- $\text{Setup}(1^\lambda, n) \rightarrow pp$ produces public parameters for vectors of length n ;
- $\text{Commit}(pp, (v_0, \dots, v_{n-1})) \rightarrow (C, aux)$ outputs a short commitment and auxiliary state;
- $\text{Open}(pp, i, aux) \rightarrow \pi_i$ produces a proof that position i holds v_i ;
- $\text{Vf}(pp, C, i, v, \pi) \rightarrow \{\text{accept, reject}\}$ verifies an opening.

The scheme is *correct* if honest openings always verify, and *succinct* if $|C|$ and $|\pi_i|$ are bounded by a fixed polynomial in λ and $\log n$. It is *position-binding* if every PPT adversary wins the following game with probability $\text{negl}(\lambda)$: given pp , the adversary outputs (C, i, v, v', π, π') and wins iff $v \neq v'$ while both $\text{Vf}(pp, C, i, v, \pi)$ and $\text{Vf}(pp, C, i, v', \pi')$ accept. Note that the adversary chooses C itself: binding must hold even for maliciously formed commitments, with no honest committer anywhere in the game.

Theorem 10.25 (Merkle trees are succinct position-binding vector commitments). *Instantiate the syntax by: Setup samples h (and fixes d with $n = 2^d$); Commit computes the tree and outputs $C = \text{root}$ with aux the full tree; Open returns the authentication path; and $\text{Vf} = \text{Verify}$. If h is collision resistant, this is a correct, succinct, position-binding vector commitment, with commitment size ℓ and opening size $O(\ell \log n)$.*

Proof. Correctness is completeness (Proposition 10.8), and the size bounds are Proposition 10.9. For position-binding, suppose an adversary outputs two accepting openings (v, π) and (v', π') at the same index i under the same root C , with $v \neq v'$. Run the verifier's fold (4) on each, obtaining two re-computed chains that disagree at the bottom ($v \neq v'$) and agree at the top (both reach C , since both openings accept). The index-consistency check forces both paths to carry the direction bits of i , so at every level both chains place their running value on the same side of the compression call. The first-level-of-agreement extraction of Theorem 10.12 now applies verbatim to this pair of chains and yields two distinct 2ℓ -bit inputs with the same h -image: a collision. The argument needed *no honest reference tree*—it pits the two forged openings against each other—so it binds even a maliciously chosen root, which is the strong form of position-binding the definition demands. $\square$

Remark 10.26 (Position-binding versus path soundness). The two theorems answer different verifiers. Path soundness (Theorem 10.12) fixes an *honest* root and forbids opening a position to anything but its true value; it speaks to a verifier who trusts where the root came from. The vector-commitment theorem (Theorem 10.25) forbids opening a single position two *different* ways even under an *adversarial* root; it speaks to a verifier who received the root from an untrusted source, and it is what “commitment” should mean— C determines at most one value per position, whoever made C . Both reduce to the same first-level-of-agreement collision extraction; the only difference is which pair of chains is collided, forged-against-genuine in the one case and forged-against-forged in the other.

10.9 Privacy-preserving membership via zero-knowledge paths

The Merkle membership proof is succinct but not private—and for a shielded payment system, not private is fatal. Revealing the authentication path of §10.4 discloses the leaf value itself (here, the note commitment being spent) and the index i at which it sits, and even the sibling labels can leak structural information about the neighbourhood of the leaf. An observer correlating spends with the appends that created their leaves links payments end to end: spending a note must not reveal *which* note is spent. The remedy is to prove the *existence* of a valid path in zero knowledge, treating the entire path as a private witness and publishing only the root.

Definition 10.27 (The membership relation). For the domain-separated Merkle construction of depth d , with leaves entering as level-0 labels under the convention of Definition 10.18, define

$$R_{\text{mem}} := \{ (rt; (v, i, \pi)) : \text{Verify}(rt, i, v, \pi) = \text{accept} \},$$

with the *statement* the public root rt and the *witness* comprising the leaf v , its index i , and the authentication path π . A zero-knowledge argument for R_{mem} lets a prover convince a verifier that *some* leaf lies in the tree with root rt while revealing none of v , i , or π . We assume a zk-SNARK as §9 develops it (Definition 9.24): *completeness* (honest provers convince), *knowledge soundness* (an accepting prover “knows” a witness, formalised by an extractor), and *zero knowledge* (the proof reveals nothing beyond the truth of the statement, formalised by a simulator).

Construction 10.28 (In-circuit path verification). The Merkle-path circuit C_{mem} of depth d takes as public input the root rt , and as private (witness) inputs the leaf v , the direction bits $b_0, \dots, b_{d-1}$, and the siblings $s_0, \dots, s_{d-1}$. It implements the d compression calls of the fold (4)—the level- t call being the position’s prescribed function, NodeHash_{t+1} in the domain-separated construction, with no LeafHash call since the leaf enters as the level-0 label (Definition 10.18)—each preceded by the conditional swap that selects the argument order (a_t, s_t) or (s_t, a_t) according to b_t , and enforces the single output constraint $a_d = rt$. Its size is d times the cost of one compression call plus $O(d)$ constraints for the swaps. The position enters *only* through the bits $b_0, \dots, b_{d-1}$ —the binary decomposition of i —so the index-consistency check of Definition 10.7 is enforced structurally: there is no separate index to disagree with its bits. Deployed Orchard matches this shape exactly: the circuit’s Merkle gadget decomposes the witnessed position into little-endian bits and applies a conditional swap per layer (`halo2_gadgets crate, src/sinsemilla/merkle.rs, gadget MerklePath`; consumed by the orchard crate’s `src/circuit.rs`; protocol specification § 4.9, ‘Merkle Path Validity’).

Theorem 10.29 (Privacy-preserving membership). Let h be collision resistant and let (P, V) be a zk-SNARK for R_{mem} —equivalently, for satisfiability of C_{mem} . Then the protocol “the prover sends a SNARK proof for the statement rt ; the verifier checks it” is a membership argument that is:

1. complete: a prover holding a genuine leaf and its path produces an accepting proof;
2. sound: any prover producing an accepting proof for an honest root $rt = \text{root}(L)$ knows—via the SNARK extractor—a witness (v, i, π) with $\text{Verify}(rt, i, v, \pi) = \text{accept}$, and v equals the genuine leaf L_i unless a collision in h is thereby found: the proven leaf is a real member of the committed list;

3. *private*: the proof reveals nothing about v , i , or π beyond the fact that some valid leaf exists under rt .

Proof idea. Completeness composes SNARK completeness with tree completeness (Proposition 10.8): the genuine (L_i, i, π_i) satisfies C_{mem} , and the honest prover proves it. Soundness composes the SNARK knowledge extractor with the path-forgery-to-collision extractor of Theorem 10.12: from an accepting prover, the SNARK extractor produces (v, i, π) accepted by `Verify`; if $v \neq L_i$, that output is precisely a path forgery, hence yields a collision for h , hence occurs with probability $\text{negl}(\lambda)$. Zero knowledge carries over verbatim from the SNARK simulator, which produces from rt alone a proof indistinguishable from the real one—so the real proof cannot be conveying v , i , or π . $\square$

Remark 10.30 (Public root, private path; deployment trade-offs). The asymmetry in R_{mem} is the essential design point. The root must be *public* so that the verifier checks the proof against a state everyone agrees on—a previously published note-commitment-tree anchor—and soundness is relative to that public root, exactly as Remark 10.13 delimits. The path must be *private* so that observers cannot link the spend to a particular leaf. SNARK succinctness adds the final ingredient: the verifier checks a proof whose size is essentially independent of d —polylogarithmic—even though the witness contains d siblings, so the construction *compresses* the membership proof while simultaneously *hiding* it. This is the precise sense in which a Merkle root over note commitments, opened in zero knowledge, is a privacy-preserving membership primitive.

Deployment chooses where to pay for that verification. Pairing-based schemes such as KZG (§4.9) achieve constant-size proofs with fast verification, at the price of a trusted setup. Halo 2’s IPA-based proofs grow logarithmically in size, but their verification is *linear* in the circuit size; the deployment amortises it by batch-verifying many proofs in a single multiexponentiation (`halo2_proofs` crate, `src/plonk/verifier/batch.rs`, `BatchVerifier`; consumed through the `orchard` crate’s `src/bundle/batch.rs`, `BatchValidator`). The accumulation technique of the Halo lineage—which would make per-proof verifier work sublinear by deferring the linear check across recursively composed proofs—is not deployed: each Orchard proof is verified, in batch, directly.

10.10 A shielded spend, end to end

The volume closes by assembling the pieces. Every primitive below has been constructed in a previous section or in this one; the shielded spend is their composition, and the description follows the deployed Orchard sources.

Example 10.31 (An Orchard shielded spend). Each shielded output of a transaction appends a *note commitment* cm —itself a hiding, binding Sinsemilla commitment to the note’s value, recipient, and randomness (§4.8)—as a leaf of an append-only Merkle tree built with the Sinsemilla compression function of Remark 10.17. Strictly, the leaf is the extracted x -coordinate

$$\text{cm}_x = \text{ExtractP}(\text{cm}),$$

a 255-bit Pallas base-field element (*Math Guide*, §“Pallas and Vesta assembled”) matching the child encodings of Remark 10.17 (`orchard` crate, `src/note/commitment.rs`, `ExtractedNoteCommitment`, via `spec::extract_p`; appended through `src/tree.rs`, `MerkleHashOrchard::from_cm_x`). The protocol periodically publishes the current root—the *anchor*—on chain, one per block.

To spend a note, its owner:

1. locates the leaf cm_x in the public tree and reads off its authentication path π and position i (`src/tree.rs`, `MerklePath::root` recomputes the anchor from exactly these data);
2. chooses a recent anchor $rt = \text{root}_m$ whose tree already contains the leaf—any recorded historical anchor serves, per §10.7;

3. proves in zero knowledge, per Theorem 10.29, the statement rt with witness (cm_x, i, π) : membership of the note commitment under the anchor, well-formedness of the note opening, and correct derivation of a *nullifier*—a deterministic tag whose on-chain uniqueness prevents double-spending, computed from the note and the spender’s nullifier key on $\text{PRF}^{\text{nfOrchard}}$ of §3.8 by a construction of the *Ironwood Guide*, §“The nullifier, and the loop back to minting” (orchard crate, `src/circuit.rs`: the recomputed root is constrained to equal the public anchor, and the instance carries the anchor and the nullifier; `src/note/nullifier.rs`, `Nullifier::derive`; protocol specification § 4.9, ‘Merkle Path Validity’, and § 4.18.4, ‘Action Statement’);
4. publishes the proof and the nullifier—but neither cm , nor i , nor π .

By Theorem 10.29, the spend convinces the network that the spender owns *some* note genuinely added to the tree—soundness resting on the collision resistance of Sinsemilla (Proposition 4.29) and the knowledge soundness of the proof system—while the network learns nothing about *which* note. (One refinement of the deployed circuit is worth recording: the root-equals-anchor constraint is enforced for notes of nonzero value, while zero-valued dummy spends may witness an arbitrary path—by design, so that every action carries a spend whether or not a real note backs it.) The Merkle root is thus a succinct, binding, privacy-preserving commitment to the set of all notes ever created.

The spend consumes the whole volume. The note commitment is the hiding and binding envelope of §4; the tree hash and its personalised domains are the discipline of §3 instantiated by Sinsemilla; the nullifier is derived under the spender’s nullifier key from a PRF of §5, instantiated by Poseidon (§3.8), by the construction of the *Ironwood Guide*, §“The nullifier, and the loop back to minting”; the proof is a zk-SNARK of §9 for the relation R_{mem} extended with the note’s opening; and the authorising signatures over the spend—re-randomised RedDSA for spend authority, the binding signature for value balance—are those of §8. How the pieces sit inside a transaction, per Action, with the key hierarchy and note-delivery encryption around them, is the business of the *Ironwood Guide*.